Models in Parallel Programming

Virginia Niculescu
Babes-Bolyai University
Department of Computer Science

Course Content

Theoretical orientation

- Models of parallel architectures
- Models of performance evaluations
- PRAM model
- BSP/LogP Models
- BMF
- Task-Based Models
- Parallel design methodologies
- Parallel programming patterns
- Parallel Skeletons
- Concurrency- Software transactional memory
- Theoretical models for CUDA programming

Practical orientation

- Multithreading + OpenMP
  - C++
- MPI
  - C++
- CUDA
  - C++

Infrastructure

1) High Performance Computing system - cluster structure

hybrid architecture: private cloud + HPC cluster

HPC (1360 cores) :

- Performance: 60 Tflops (Rpeak) and 40Tflops (sustained performance).
- Computation nodes 50 (each with 2 proc. X 10 cores);
- Computation nodes 12(each with 2 proc. X 10 cores + GPU);
- Computation nodes 12(each with 2 proc. X 10 cores + Xeon Phi coprocessor)
- Management nodes: 4

- Connectivity: InfiniBand FDR (54Gbps)

Accessible by web – user accounts are needed.

Requests are necessary!

- upload the **Account Request Form** in the "HPC" folder in Files

- write the username in 'Planification' file

- details: http://hpc.cs.ubbcluj.ro

References

- Structured Parallel Programming: Patterns for Efficient Computation," Michael McCool, Arch Robinson, James Reinders, Morgan Kaufmann,2012 .
- A Pattern Language for Parallel Programming. Berna L. Massingill, Timothy G. Mattson, and Beverly A. Sanders, Addison Wesley Software Patterns Series, 2004.
- Grama, A. Gupta, G. Karypis, V. Kumar. Introduction to Parallel Computing, Addison Wesley, 2003.
- Ian Foster. Designing and Building Parallel Programs, Addison-Wesley 1995.
- K.M. Chandy, J. Misra, Parallel Program Design: A Foundation, Addison-Wesley, 1988.
- M. J. Quinn. Parallel Programming in C with MPI and OpenMP, McGraw Hill, 2004.
- J. Misra. PowerList: A structure for parallel recursion.ACM Transactions on Programming Languages and Systems, 16(6):1737-1767, November 1994.
- V. Niculescu. Calcul Paralel. Proiectare si dezvolare formala a programelor paralele. Presa Univ. Clujana, 2006.
- Selim Akl, Parallel Computation: Models and Methods. Prentice Hall, 1996.
- D. Skillicorn. Foundations of Parallel Programming, Cambridge International Series on Parallel Computations, 1994
- D.B. Skillicorn, D. Talia. Models and Languages for Parallel Computation. ACM Computer Surveys, 30(2) pg.123-136, June 1998.
- B. Wilkinson, C.M. Allen. Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers, Prentice Hall, 1999.

Introduction to Parallel Computing

Lecture 1

Parallel programming

- Broad/Old field of computer science concerned with:
  - Architecture, HW/SW systems, languages, programming paradigms, algorithms, and theoretical models
  - Computing in parallel
- Performance is the *raison d'être* for parallelism
  - High-performance computing
  - Drives computational science revolution
- Topics of study
  - Parallel architectures
  - Parallel programming
  - Parallel algorithms

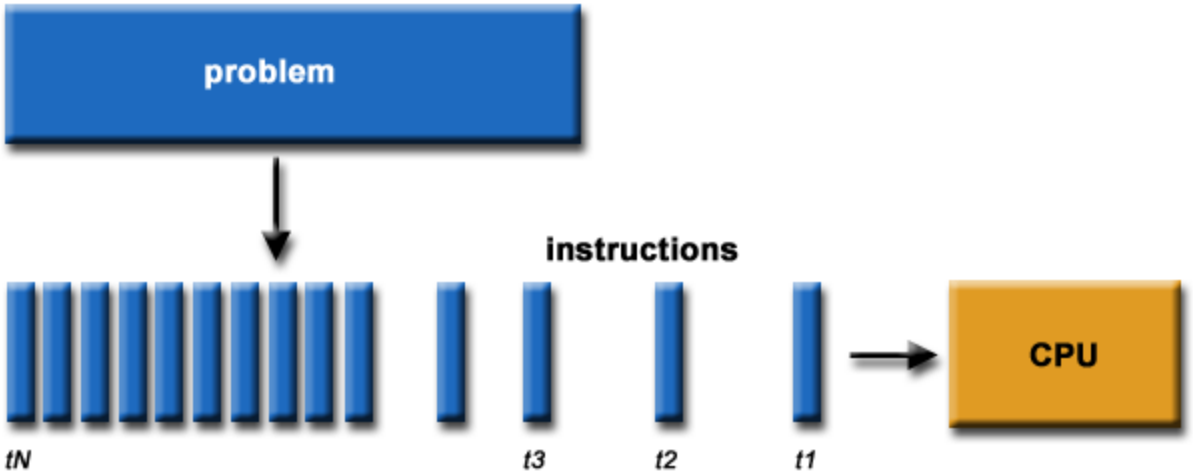
- Parallel performance models and tools
- Parallel applications

Parallel Processing – What is it?

- A *parallel computer* is a computer system that uses multiple processing elements simultaneously in a cooperative manner to solve a computational problem
- *Parallel processing* includes techniques and technologies that make it possible to compute in parallel
  - Hardware, networks, operating systems, parallel libraries, languages, compilers, algorithms, tools, …
- Parallel computing is an evolution of serial computing
  - Parallelism is natural
  - Computing problems differ in level / type of parallelism
- Is Parallelism is all about performance?

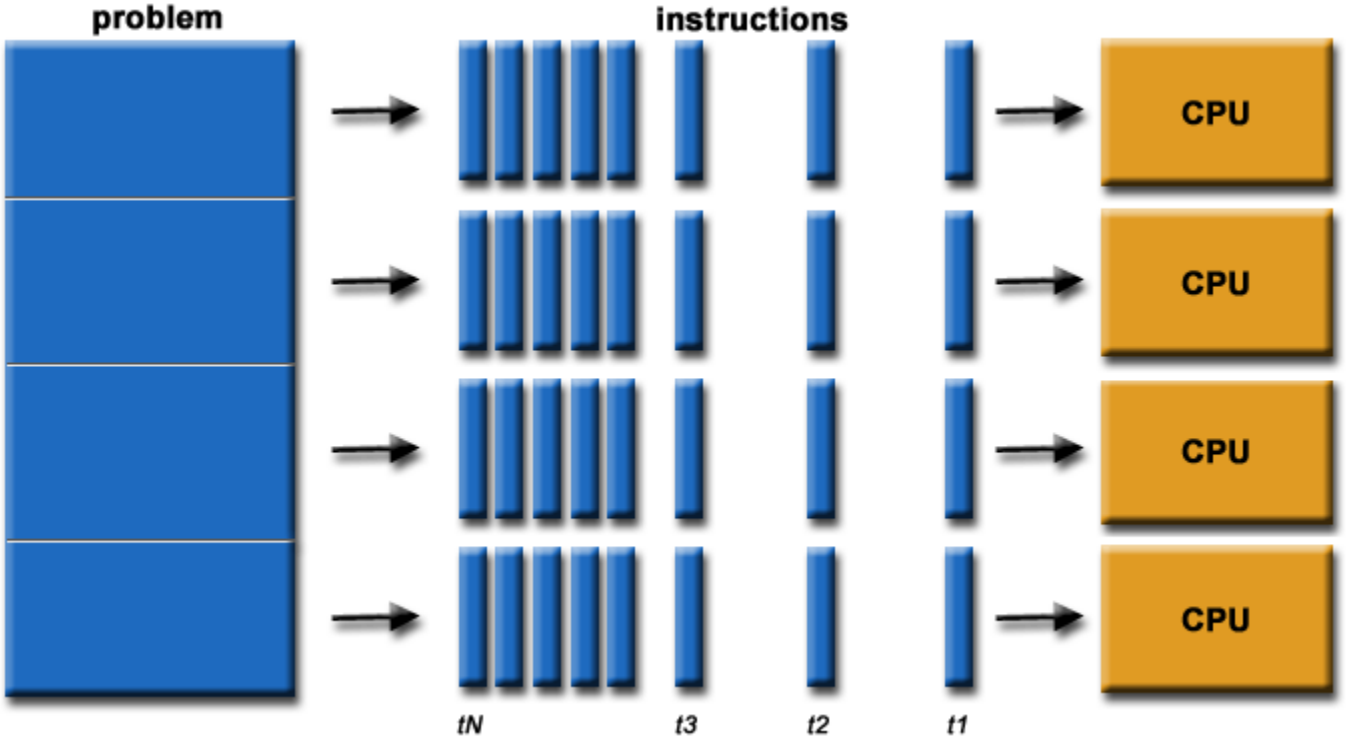
Serial computation [*Blaise Barney, Lawrence Livermore National Laboratory*]

- Traditionally, software has been written for ***serial*** computation:
- To be run on a single computer having a single Central Processing Unit (CPU);
- A problem is broken into a discrete series of instructions.
- Instructions are executed one after another.
- Only one instruction may execute at any moment in time.



Parallel computation

- In the simplest sense, ***parallel computing*** is the simultaneous use of multiple compute resources to solve a computational problem:
- To be run using multiple CPUs /cores/ processing elements
- A problem is broken into discrete parts that can be solved concurrently
- Each part is further broken down to a series of instructions
- Instructions from each part execute simultaneously on different CPUs


problem
instructions
CPU
CPU
CPU
CPU
tN
t3
t2
t1

The compute resources can include:

- A single computer with multiple processors;
- An arbitrary number of computers connected by a network;
- A combination of both.

The computational problem usually demonstrates characteristics such as the ability to be:

- Broken apart into discrete pieces of work that can be solved simultaneously;
- Execute multiple program instructions at any moment in time;
- Solved in less time with multiple compute resources than with a single compute resource.

Why Do Parallel Computing

- Time: Reduce the turnaround time of applications
- Performance: Parallel computing is the only way to extend performance toward the TFLOP(teraflop - 10^{12}) PFLOP(petaflop - 10^{15})realms
- Cost/Performance: Traditional vector computers become too expensive
- Memory: Applications often require memory that goes beyond that addressable by a single processor

- **Save time and/or money:** In theory, throwing more resources at a task will shorten its time to completion, with potential cost savings. Parallel clusters can be built from cheap, commodity components.

- **Solve larger problems:** Many problems are so large and/or complex that it is impractical or impossible to solve them on a single computer, especially given limited computer memory. For example:
  - "Grand Challenge" (en.wikipedia.org/wiki/Grand_Challenge) problems requiring PetaFLOPS and PetaBytes of computing resources.
  - Web search engines/databases processing millions of transactions per second

- **Provide concurrency/parallelism:** A single compute resource can only do one thing at a time. Multiple computing resources can be doing many things simultaneously. For example, the Access Grid (http://www.accessgrid.org/) provides a global collaboration network where people from around the world can meet and conduct work "virtually".

- **Use of non-local resources:** Using compute resources on a wide area network, or even the Internet when local compute resources are scarce. For example:
  - SETI@home (setiathome.berkeley.edu) uses over 330,000 computers for a compute power over 528 TeraFLOPS (as of August 04, 2008)
  - Folding@home (folding.stanford.edu) uses over 340,000 computers for a compute power of 4.2 PetaFLOPS (as of November 4, 2008)

Limits to serial computing

- Both physical and practical reasons pose significant constraints to simply building ever faster serial computers:
  - ***Transmission speeds*** - the speed of a serial computer is directly dependent upon how fast data can move through hardware. Absolute limits are the speed of light (30 cm/nanosecond) and the transmission limit of copper wire (9 cm/nanosecond). Increasing speeds necessitate increasing proximity of processing elements.
  - ***Limits to miniaturization*** - processor technology is allowing an increasing number of transistors to be placed on a chip. However, even with molecular or atomic-level components, a limit will be reached on how small components can be.
  - ***Economic limitations*** - it is increasingly expensive to make a single processor faster. Using a larger number of moderately fast commodity processors to achieve the same (or better) performance is less expensive.

Hardware level parallelism

- Current computer architectures are increasingly relying upon

hardware level parallelism

to improve performance:

  - Multiple execution units
  - Pipelined instructions
  - Multi-core

Why use parallel processing?

Parallelism = concurrency + parallel HW + performance

- Two primary reasons (both performance related)
  - Faster time to solution (response time)
  - Solve bigger computing problems (in same time)
- Other factors motivate parallel processing
  - Effective use of machine resources
  - Cost efficiencies
  - Overcoming memory constraints
- Serial machines have inherent limitations
  - Processor speed, memory bottlenecks, …

Advantages of Parallel Programming

- Needs to solve larger problems
  - more memory intensive
  - more computation
  - more data intensive
- Parallel programming provides
  - more efficient use of CPU resources
  - more efficient use of memory resources
  - solve problems that were not possible with serial program
  - solve problems more quickly

Cont…

- Many classes of important algorithms are ideal for parallel execution. Most algorithms can benefit from parallel processing such as Laplace equation, Monte Carlo, FFT (signal processing), image processing

- Life itself is a set of concurrent processes
  - Scientists use modeling

    => why not model the systems in a way closer to nature

Some Misconceptions

- Requires new parallel languages?
  - Not always. Uses standard languages and compilers (Fortran, C, C++, Java, Occam, Python).
  - However, there are some specific parallel languages.
- Requires new code?
  - Not always. Most existing code can be used. Many production installations use the same code base for serial and parallel code.
- Requires confusing parallel extensions?
  - Not always. They are not that bad. Depends on how complex you want to make it. From nothing at all (letting the compiler do the parallelism) to installing semaphores yourself
- Parallel computing is difficult:
  - Not always. Just different and subtle.
  - Depends on the level of abstraction.

Who and What?

- Top500.org provides statistics on parallel computing users - the charts below are just a sample.

The Future:

- During the past 20 years, the trends indicated by:
  - ever faster networks,
  - distributed systems, and
  - multi-processor computer architectures (even at the desktop level)

  clearly show that

  parallelism is the future of computing.

Concurrency

- Consider multiple tasks to be executed in a computer
- Tasks are concurrent with respect to each others if
  - They *can* execute at the same time (*concurrent execution*)
    - implies that there are no dependencies between the tasks
- Dependencies
  - If a task requires results produced by other tasks in order to execute correctly, the task's execution is *dependent*
  - If two tasks are dependent, they are not concurrent
  - Some form of synchronization must be used to enforce (satisfy) dependencies
- Concurrency is fundamental to computer science
  - Operating systems, databases, networking, …

Concurrency and Parallelism

- Concurrent is not the same as parallel! Why?
- Parallel execution
  - Concurrent tasks *actually* execute at the same time
  - Multiple (processing) resources have to be available
- **Parallelism = concurrency + "parallel" hardware**
  - Both are required
  - Find concurrent execution opportunities
  - Develop application to execute in parallel
  - Run application on parallel hardware
- Is a parallel application a concurrent application?
- Is a parallel application run with one processor parallel? Why or why not?

Parallelism

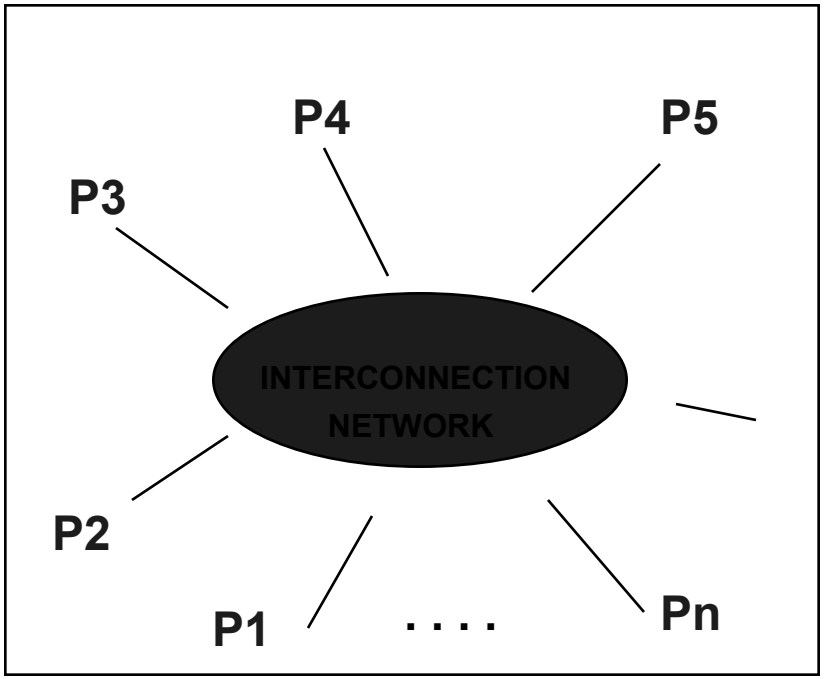
- There are different granularities of parallelism (parallel execution) in programs
  - Processes, threads, routines, statements, instructions, …
  - Think about what are the software elements that execute concurrently
- These must be supported by hardware resources
  - Processors, cores, … (execution of instructions)
  - Memory, DMA, networks, … (other associated operations)
  - All aspects of computer architecture offer opportunities for parallel hardware execution
- Concurrency is a necessary condition for parallelism
  - Where can you find concurrency?
  - How is concurrency expressed to exploit parallel systems?

Perspectives on Parallel Processing

- Parallel computer architecture
  - Hardware needed for parallel execution?
  - Computer system design
- (Parallel) Operating system
  - How to manage systems aspects in a parallel computer
- Parallel programming
  - Libraries (low-level, high-level)
  - Languages
  - Software development environments
- Parallel algorithms
- Parallel performance evaluation
- Parallel tools
  - Performance, analytics, visualization, …

PARALLEL AND DISTRIBUTED COMPUTATION
...a comparison...

- **MANY INTERCONNECTED PROCESSORS WORKING CONCURRENTLY**



- **CONNECTION MACHINE**
- **INTERNET** **Connects all the computers of the world**

TYPES OF MULTIPROCESSING FRAMEWORKS
PARALLEL DISTRIBUTED

TECHNICAL ASPECTS

•PARALLEL COMPUTERS (USUALLY) WORK IN TIGHT SYNCRONY, SHARE MEMORY TO A LARGE EXTENT AND HAVE A VERY FAST AND RELIABLE COMMUNICATION MECHANISM BETWEEN THEM.

• DISTRIBUTED COMPUTERS ARE MORE INDEPENDENT, COMMUNICATION IS LESS FREQUENT AND LESS SYNCRONOUS, AND THE COOPERATION IS LIMITED.

PURPOSES

• PARALLEL COMPUTERS COOPERATE TO SOLVE MORE EFFICIENTLY (POSSIBLY) DIFFICULT PROBLEMS

• DISTRIBUTED COMPUTERS HAVE INDIVIDUAL GOALS AND PRIVATE ACTIVITIES. *SOMETIME* COMMUNICATIONS WITH OTHER ONES ARE NEEDED. (E. G. DISTRIBUTED DATA BASE OPERATIONS).

PARALLEL COMPUTERS: COOPERATION IN A *POSITIVE* SENSE

DISTRIBUTED COMPUTERS: COOPERATION IN A *NEGATIVE* SENSE, ONLY WHEN IT IS NECESSARY

FOR PARALLEL SYSTEMS

WE ARE INTERESTED TO SOLVE ANY PROBLEM IN PARALLEL

FOR DISTRIBUTED SYSTEMS

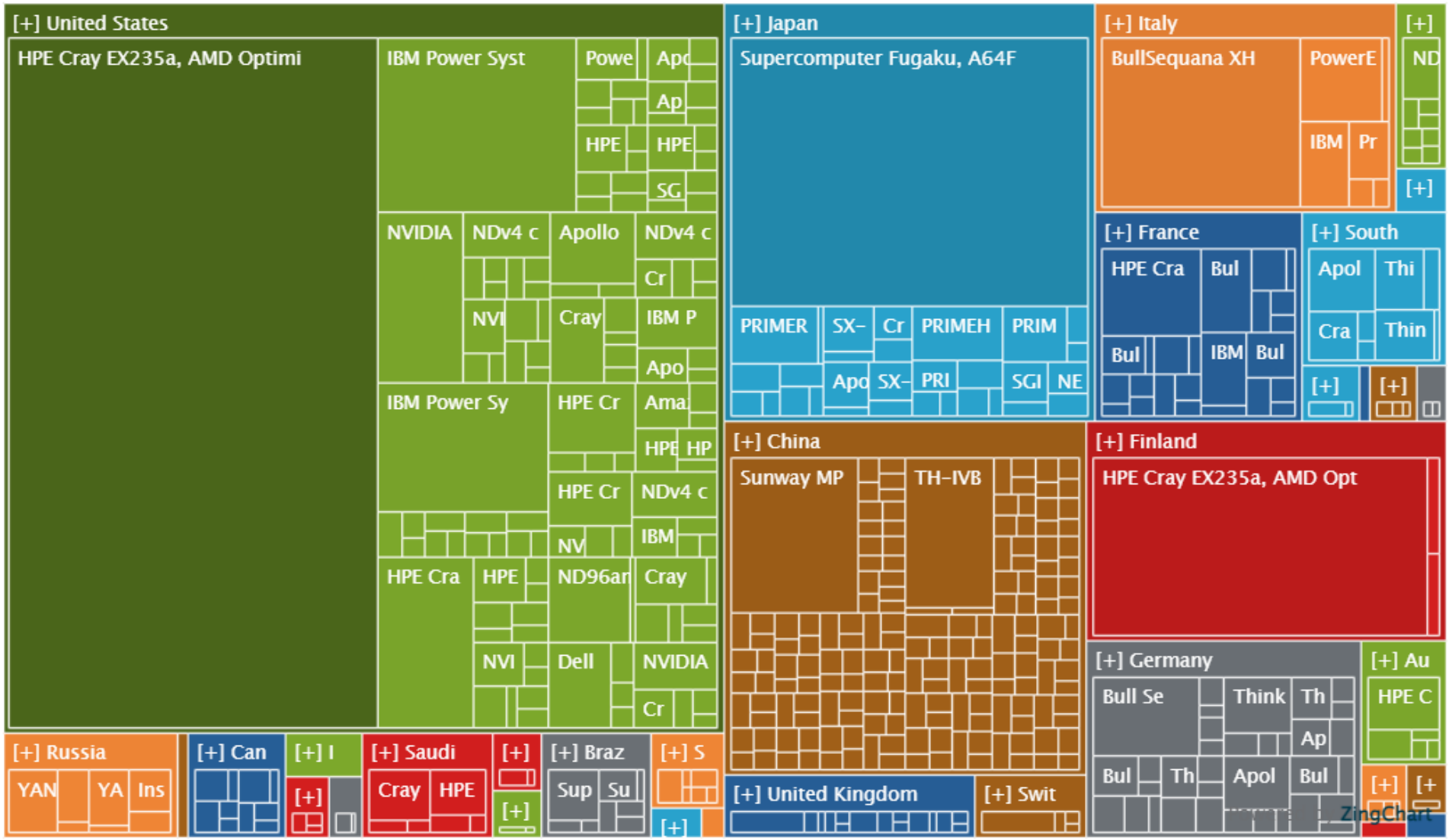
WE ARE INTERESTED TO SOLVE IN PARALLEL PARTICULAR PROBLEMS ONLY, TYPICAL EXAMPLES ARE:

- **COMMUNICATION SERVICES (client –service applications)**
  ROUTING
  BROADCASTING

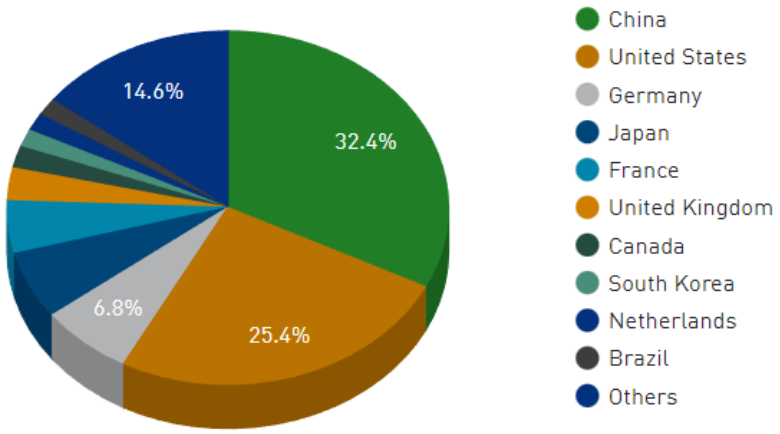
- **MAINTENANCE OF CONTROL STUCTURE**
  SPANNING TREE CONSTRUCTION
  TOPOLOGY UPDATE
  LEADER ELECTION

- **RESOURCE CONTROL ACTIVITIES**
  LOAD BALANCING
  MANAGING GLOBAL DIRECTORIES

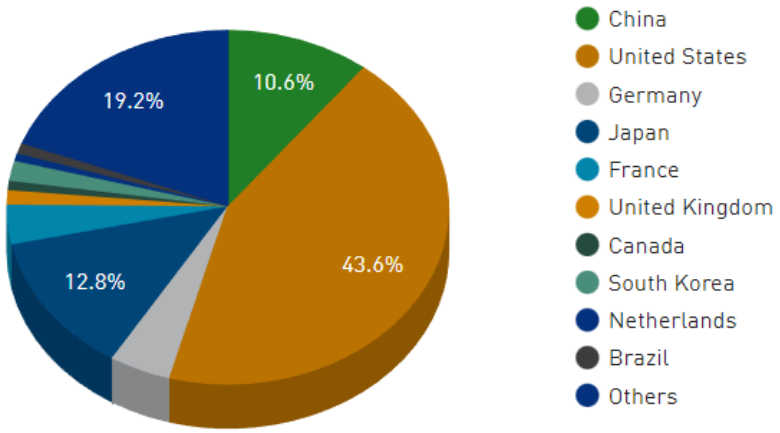
2022 – Top500


[+] United States
HPE Cray EX235a, AMD Optimi
IBM Power Syst
Powe
Apc
Ap
HPE
HPE
SG
NVIDIA
NDv4 c
Apollo
NDv4 c
Cr
NVI
Cray
IBM P
Apo
IBM Power Sy
HPE Cr
Ama
HPE
HP
HPE Cr
NDv4 c
NV
IBM
HPE Cra
HPE
ND96ar
Cray
NVI
Dell
NVIDIA
Cr
[+] Japan
Supercomputer Fugaku, A64F
PRIMER
SX-
Cr
PRIMEH
PRIM
Apo
SX-
PRI
SGI
NE
[+] Italy
BullSequana XH
PowerE
IBM
Pr
[+]
ND
[+]
[+] France
HPE Cra
Bul
Bul
IBM
Bul
[+] South
Apol
Thi
Cra
Thin
[+]
[+]
[+] China
Sunway MP
TH-IVB
[+] United Kingdom
[+] Swit
[+] Finland
HPE Cray EX235a, AMD Opt
[+] Germany
Bull Se
Think
Th
Ap
Bul
Th
Apol
Bul
[+] Au
HPE C
[+]
[+]
[+] Russia
YAN
YA
Ins
[+] Can
[+] I
[+]
[+] Saudi
Cray
HPE
[+]
[+]
[+] Braz
Sup
Su
[+] S
[+]
ZingChart

Countries System Share



Countries Performance Share



TOP10 System - November 2023

R_{max} and R_{peak} values are in PFlop/s. For more details about other fields, check the TOP500 description.

R_{peak} values are calculated using the advertised clock rate of the CPU. For the efficiency of the systems you should take into account the Turbo CPU clock rate where it applies.

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	1,679.82	22,703
2	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	4,742,808	585.34	1,059.33	24,687
3	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Microsoft Azure United States	1,123,200	561.20	846.84	

Lecture 2

Parallel Architectures

~different models of architectures that impose different models of computation~

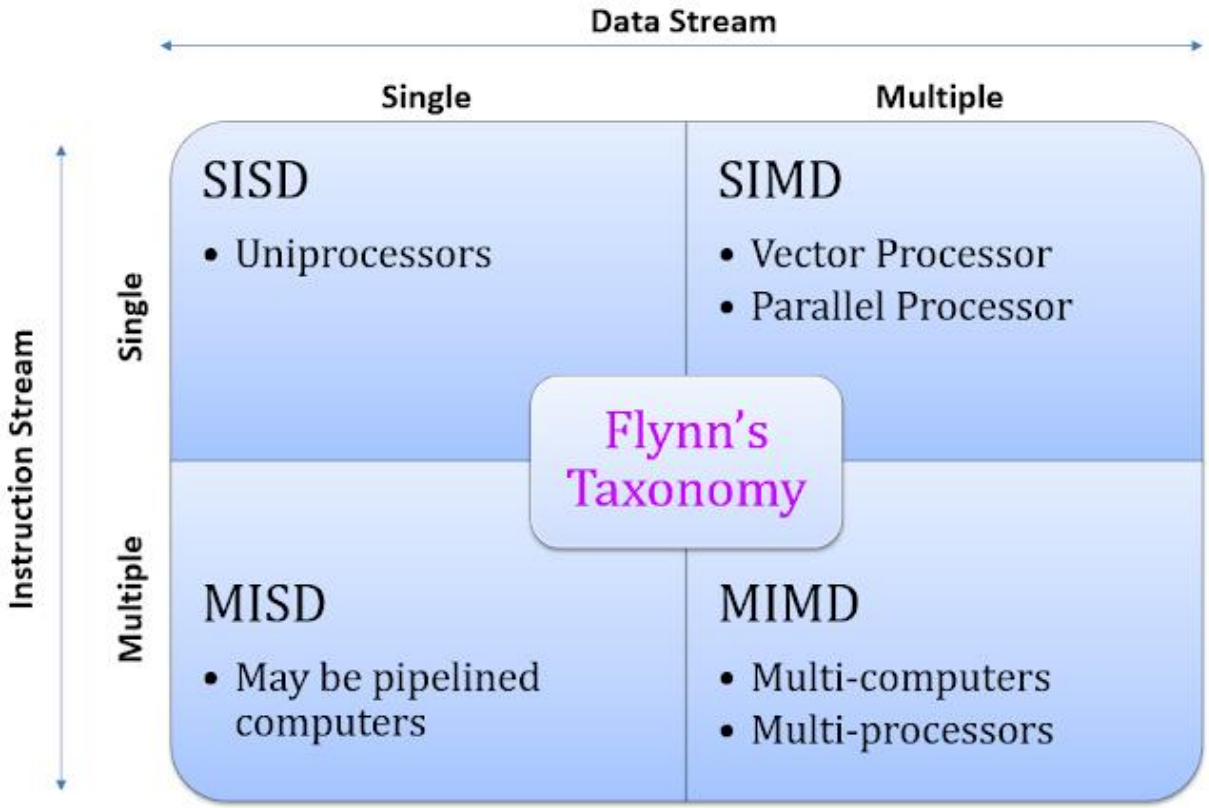
Reference: Introduction to Parallel Computing, Univ. of Oregon

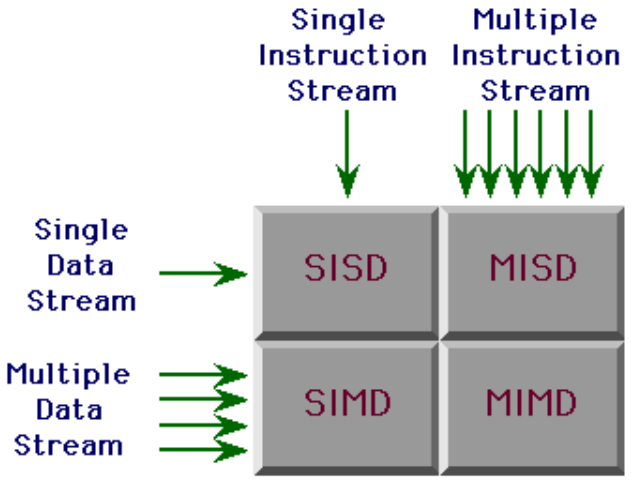
Argonne National Laboratory: papers and presentations

Outline

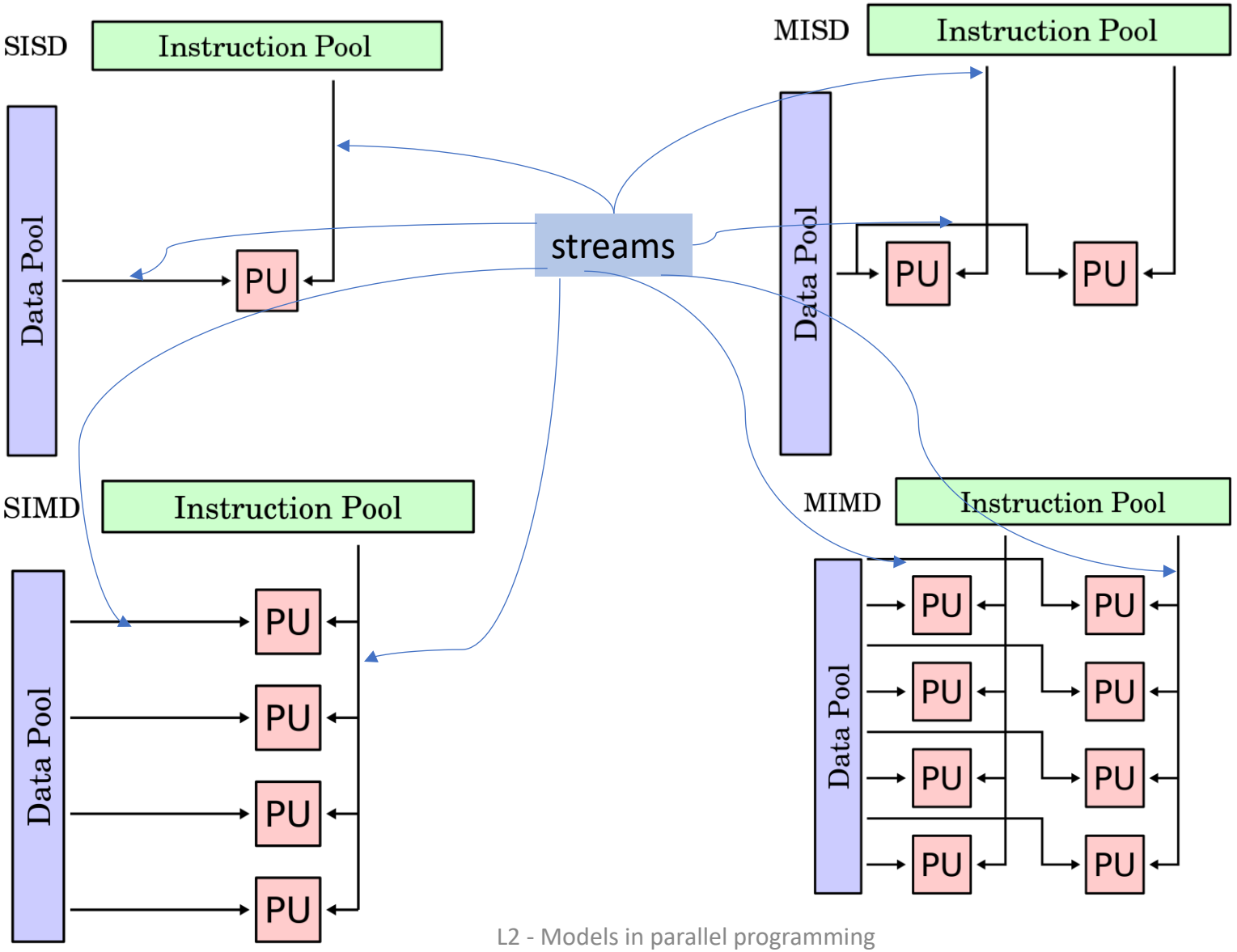
- Parallel architecture types – Flynn taxonomy
- Instruction-level parallelism
- SIMD
- DataFlow architectures
- Shared memory architectures
  - Memory organization: UMA, NUMA
  - Coherency: CC-UMA, CC-NUMA
- Distributed memory architectures
  - Interconnection networks
  - Clusters
  - Clusters of SMPs
  - Heterogeneous clusters of SMPs
- Examples

Flynn Taxonomy (M.J. Flynn -1966)

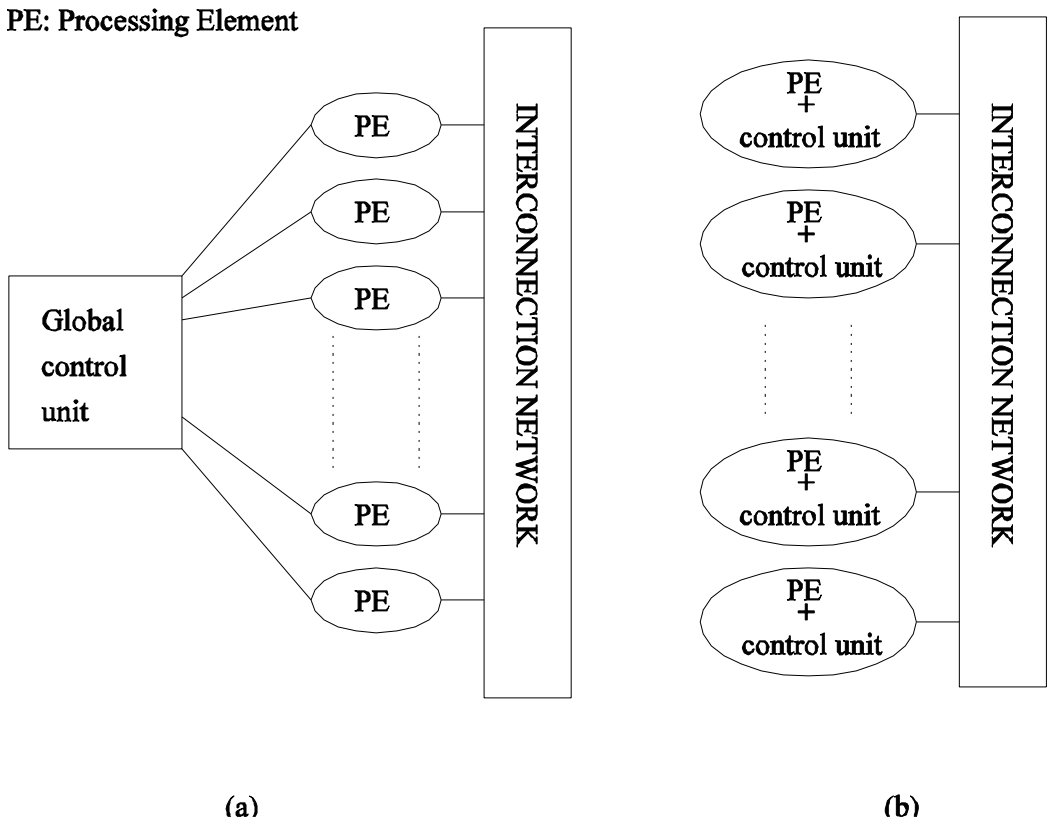




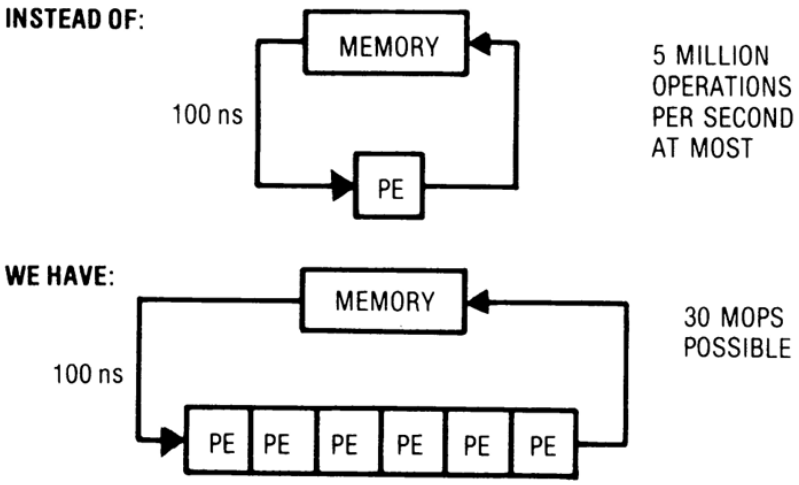
Flynn Taxonomy



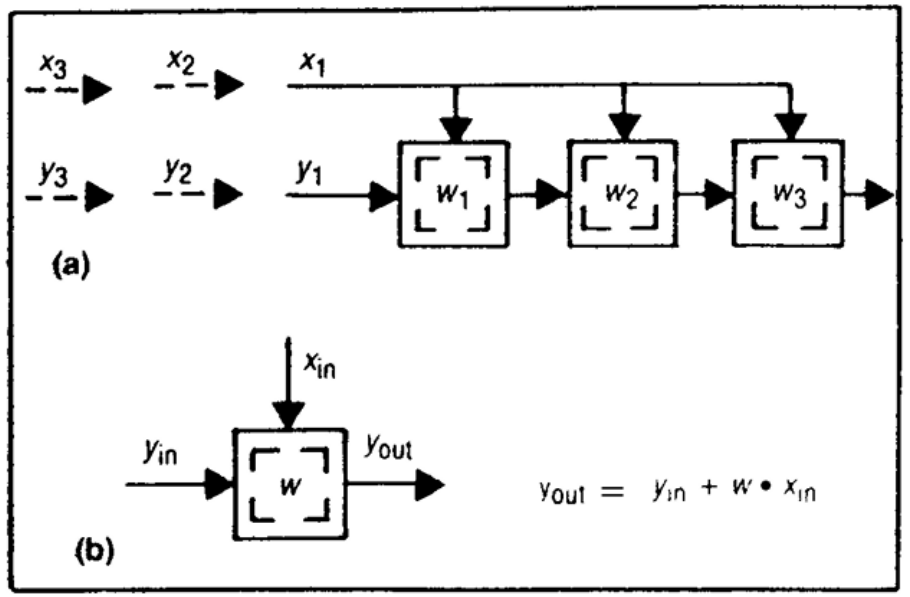
SIMD versus MIMD



MISD relaxation –Pipeline architecture



Example for 1D convolutional array where xi's are broadcast, wi's stay, and yi's move systolically.



MISD relaxation -> Systolic Architecture

Orchestrate data flow for high throughput with less memory access

Different from pipelining

Nonlinear array structure, multidirection data flow, each PE may have (small) local instruction and data memory

Different from SIMD

Each PE may do something different

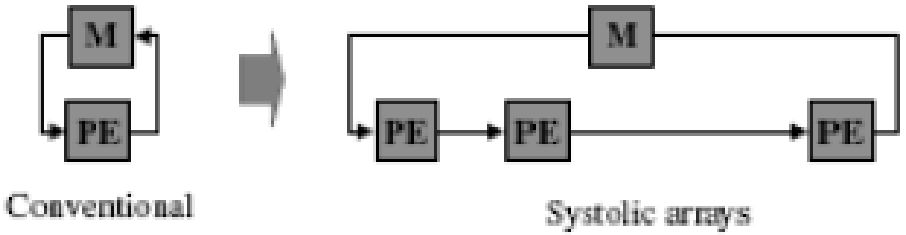
Initial motivation

VLSI enables inexpensive special-purpose chips

Represent algorithms directly by chips connected in regular pattern

Very-large-scale integration

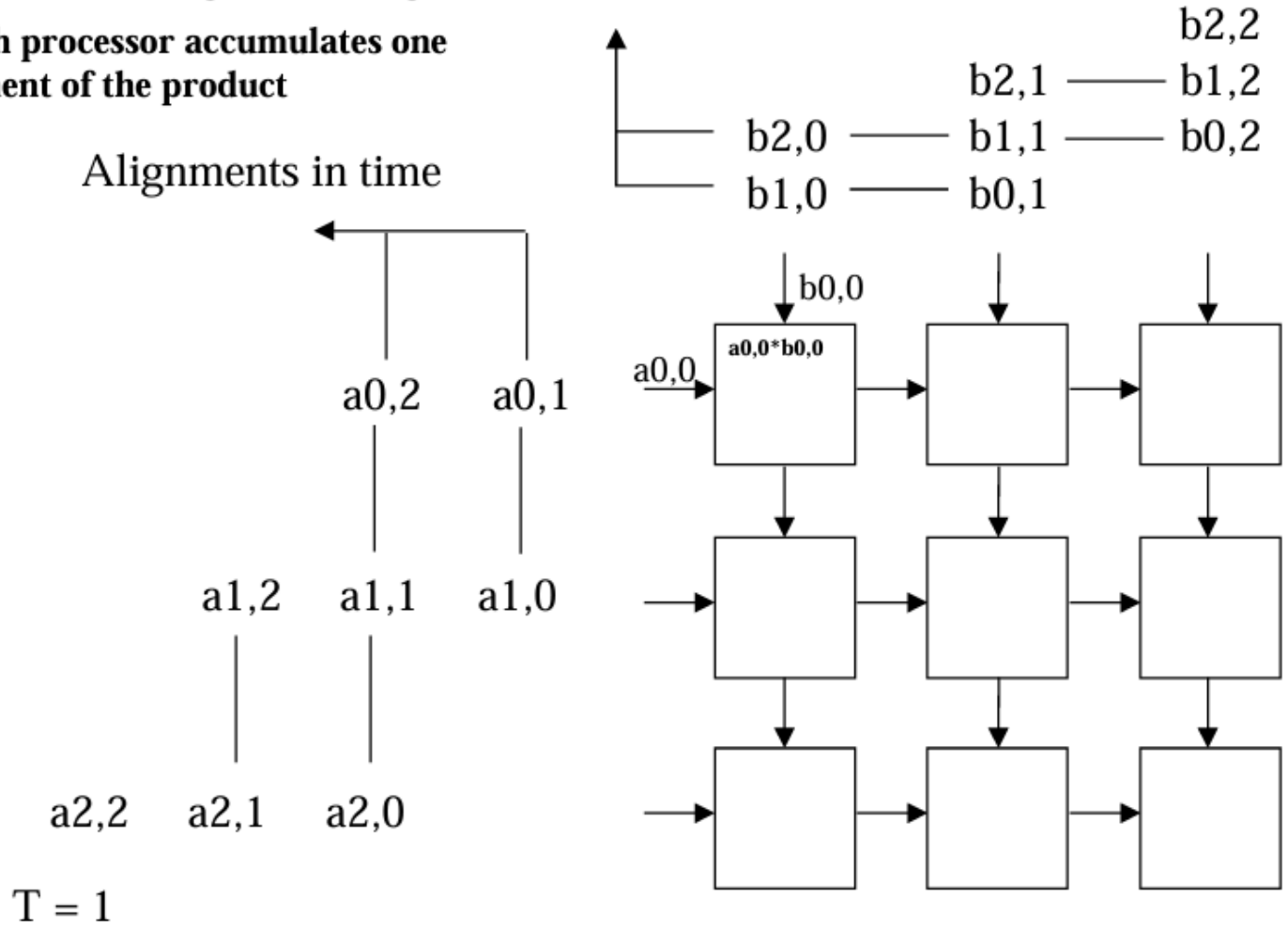
Systolic Architectures



Replace a processing element(PE) with an array of PE's without increasing I/O bandwidth

Systolic Array Example:
3x3 Systolic Array Matrix Multiplication

- **Processors arranged in a 2-D grid**
- **Each processor accumulates one element of the product**



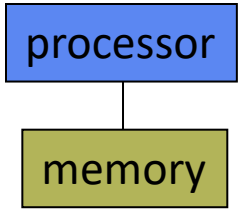
Example source: http://www.cs.hmc.edu/courses/2001/spring/cs156/

Parallel Architecture Types

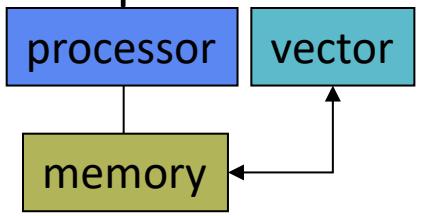
- Uniprocessor
  - Scalar processor

a scalar processor is a processor whose instructions operate on a single data

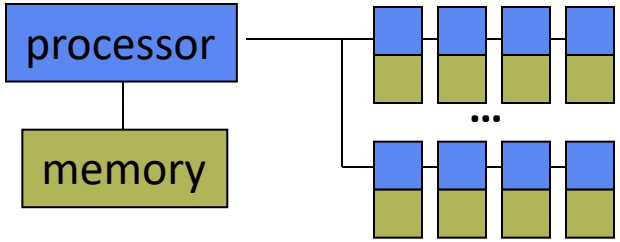
a vector processor refers to a processor that implements instructions operating on an array of data.



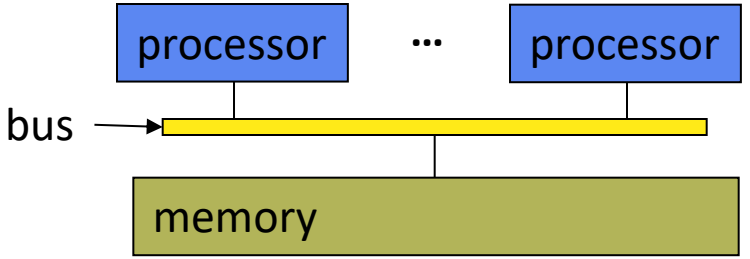
  - Vector processor



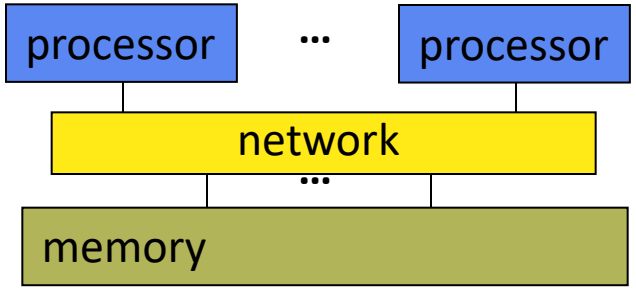
  - Single Instruction Multiple Data (SIMD) Processor



- Shared Memory Multiprocessor (SMP)
  - Shared memory address space
  - Bus-based memory system

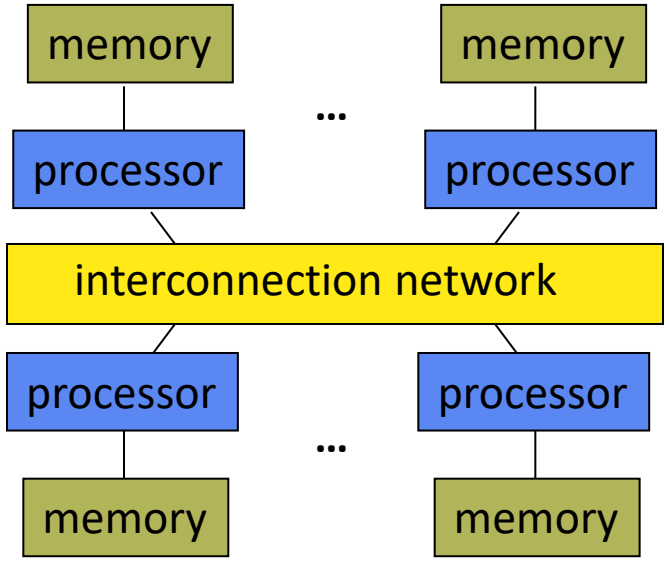


  - Interconnection network



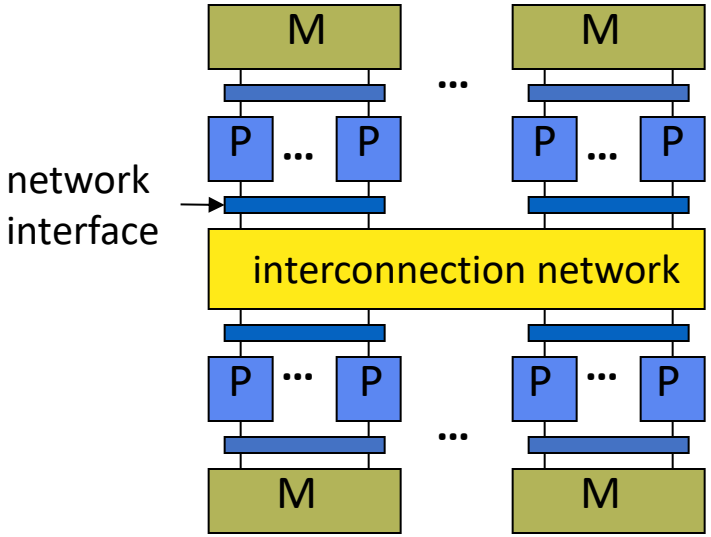
Parallel Architecture Types (2)

- Distributed Memory Multiprocessor
  - Message passing between nodes



  - Massively Parallel Processor (MPP)
    - Many, many processors

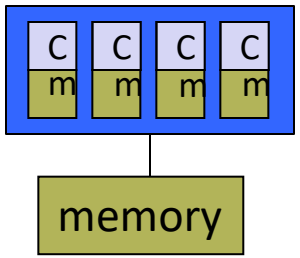
- Cluster of SMPs
  - Shared memory addressing within SMP node
  - Message passing between SMP nodes



  - Can also be regarded as MPP if processor number is large

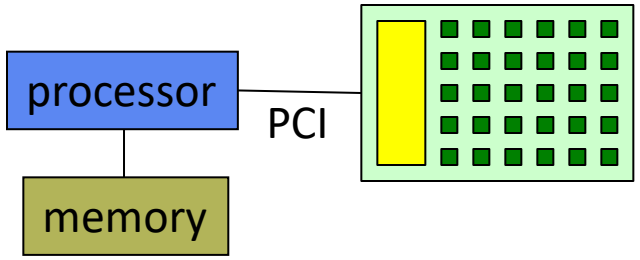
Parallel Architecture Types (3)

- Multicore
  - Multicore processor



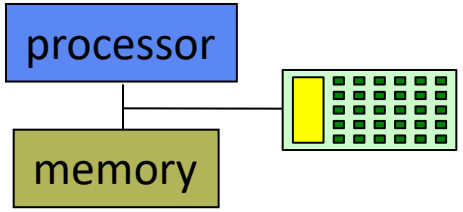
cores can be hardware multithreaded (hyperthread)

  - GPU accelerator

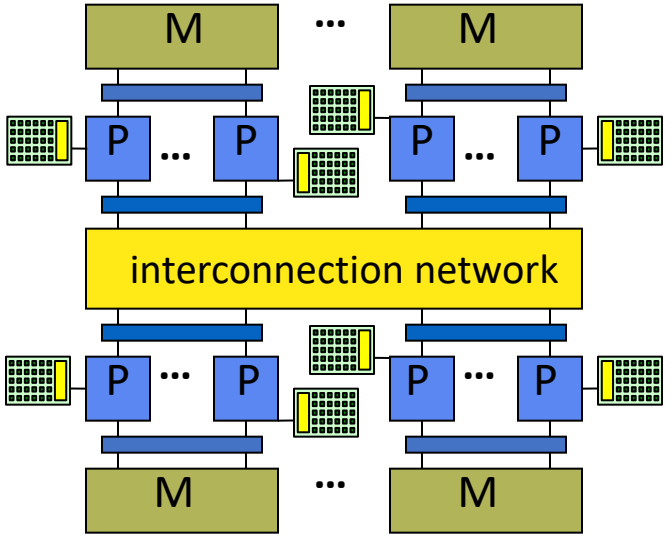


  - "Fused" processor accelerator

reduce data transfer overheads



- Multicore SMP+GPU Cluster
  - Shared memory addressing within SMP node
  - Message passing between SMP nodes
  - GPU accelerators attached



How do we get parallelism?

- Instruction-Level Parallelism (ILP)
- Data parallelism
  - Increase amount of data to be operated at the same time
- Processor parallelism
  - Increase number of processors
- Memory system parallelism
  - Increase number of memory units
  - Increase bandwidth to memory
- Communication parallelism
  - Increase amount of interconnection between elements
  - Increase communication bandwidth

Instruction-Level Parallelism (ILP)

- Opportunities for splitting up instruction processing
- Pipelining within instruction
- Pipelining between instructions
- Overlapped execution
- Multiple functional units
- Out of order execution
- Multi-issue execution
- Superscalar processing - can execute more than one instruction during a clock cycle
- Superpipelining - a technique of splitting instructions into many separate "pipelines"
- Very Long Instruction Word (VLIW) - instruction set architectures designed to allow instructions to execute in parallel.
- Hardware multithreading (hyperthreading)

Parallelism inside the CPU

(ref: https://www.tutorialspoint.com/cuda/cuda_tutorial.pdf)

- The five essential steps required for an instruction to finish:
  - Instruction fetch (IF)
  - Instruction decode (ID)
  - Instruction execute (Ex)
  - Memory access (Mem)
  - Register write-back (WB)
- This is a basic five-stage RISC architecture.
- There are multiple ways to achieve parallelism in the CPU.
  - one is ILP (Instruction Level Parallelism), also known as pipelining.

Instruction Level Parallelism

- how
  Instruction Level Parallelism works:

Instr. No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

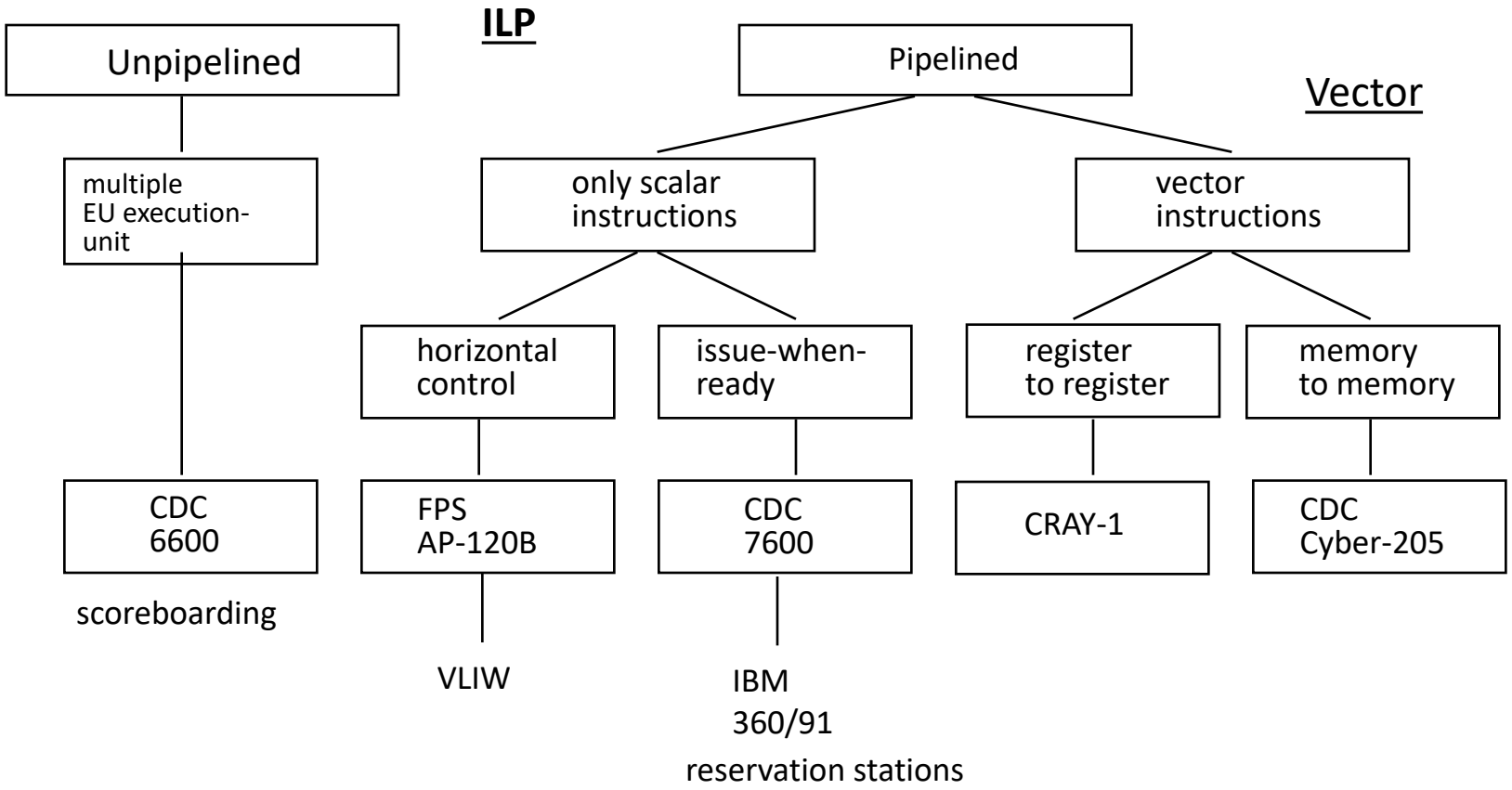
- **Using instruction pipelining, the instruction throughput increase (instructions per seconds).**
- **This way, we can process many instructions in one-clock cycle.**

ILP in a superscalar architecture

- The primary difference between a **superscalar** and a **pipelined** processor is (a superscalar processor is also pipelined) that the former uses **multiple execution units -EUs**(on the same chip) to achieve ILP whereas the latter divides the EU in multiple phases to do that.
  - This means that in superscalar, several instructions can simultaneously be in the same stage of the execution cycle. This is not possible in a simple pipelined chip.
  - Superscalar microprocessors can execute two or more instructions at the same time. They typically have at least 2 ALUs.
- **Superscalar processors can dispatch multiple instructions in the same clock cycle.**
  - This means that multiple instructions can be started in the same clock cycle.
  - In a pipeline architecture at any clock cycle, only one instruction is dispatched.
  - This is not the case with superscalars. But we have only one instruction counter (in-flight, multiple instructions are tracked). There is still just one process !

Parallelism in Single Processor Computers

- History of processor architecture innovation



Data-Parallel Architecture

Many variants of DLP (Data Level Parallelism) architectures over decades

– Vector processors
  - Cray processors: Cray-1, Cray-2, …, Cray X1

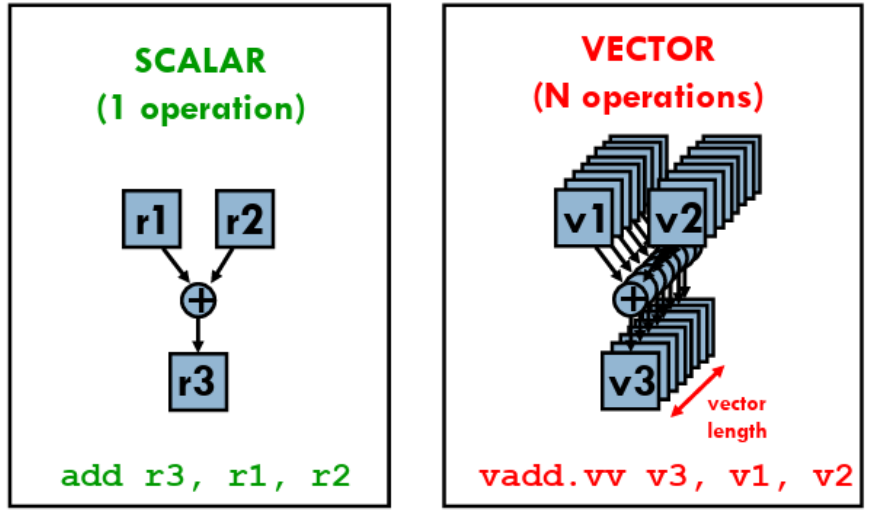
– SIMD extensions
  - Intel MMX, SSE and AVX units

– Old massively parallel computers
  - Connection Machines
  - MasPar machines

– Modern GPUs -> from SIMD to SIMT
  - NVIDIA, AMD, Qualcomm, …
  - Focus on throughput rather than latency

Vector Processing

- Scalar processing
  - Processor instructions operate on scalar values
  - integer registers and floating-point registers
- Vectors
  - Set of scalar data
  - ***Vector registers***
    - integer, floating point (typically)
  - *Vector instructions operate on vector registers* (SIMD)
- Vector unit pipelining
- Multiple vector units
- Vector chaining

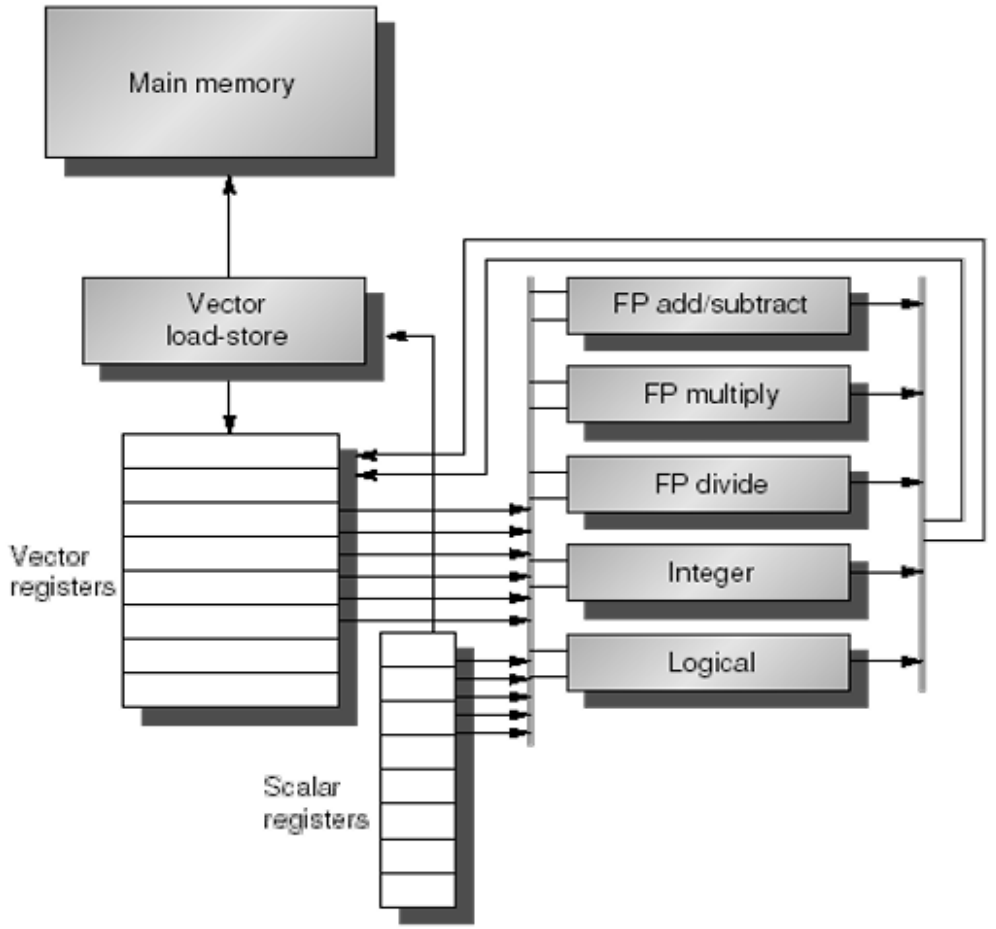


Liquid-cooled with inert fluorocarbon.
(That's a waterfall fountain!!!)



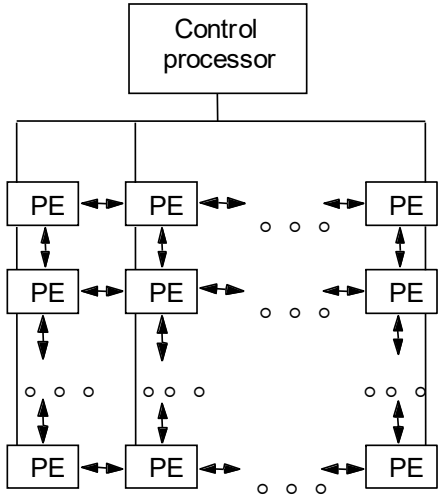
Cray 2

Example of a Vector Processor

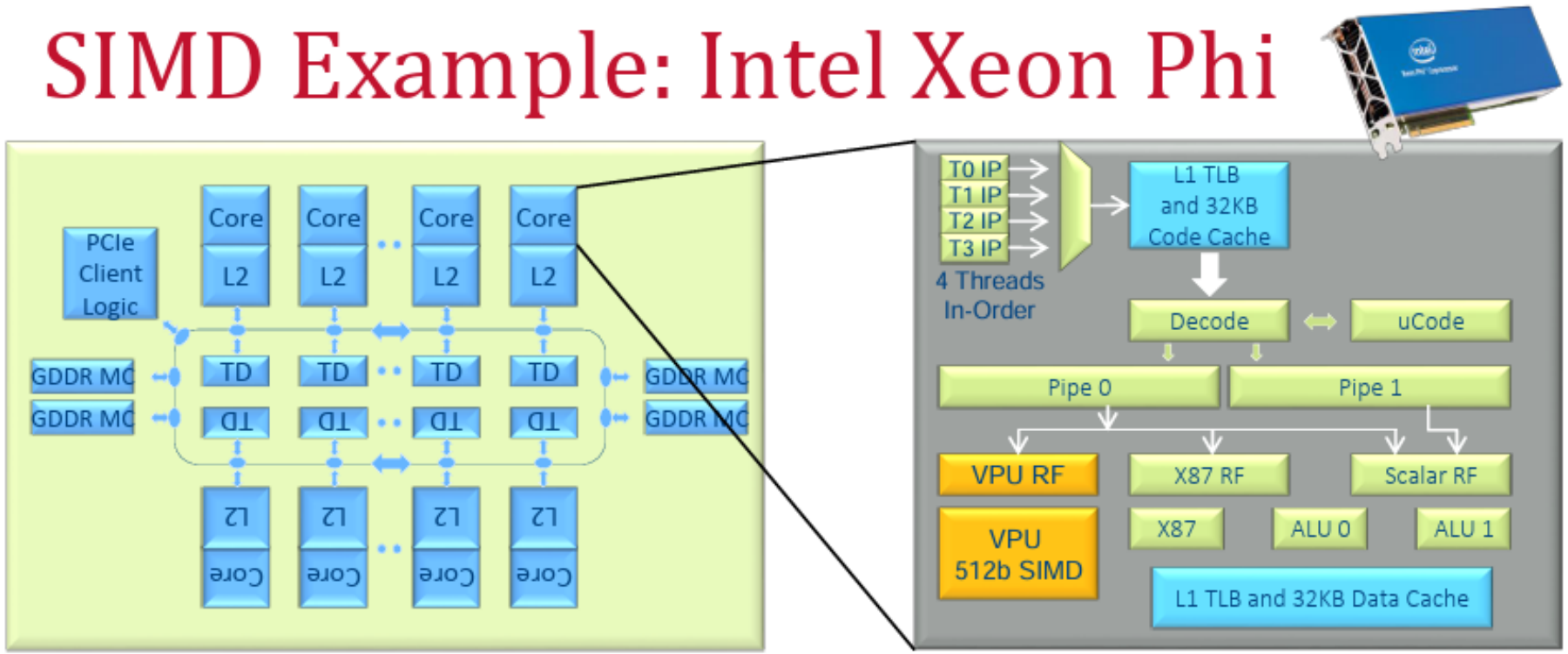


Data Parallel Architectures

- SIMD (Single Instruction Multiple Data)
  - Logical single thread (instruction) of control
  - Processor associated with data elements
- Architecture
  - Array of simple processors with memory
  - Processors arranged in a regular topology
  - Control processor issues instructions
    - **All processors execute same instruction** (maybe disabled)
  - Specialized synchronization and communication
  - Specialized reduction operations
  - Array processing



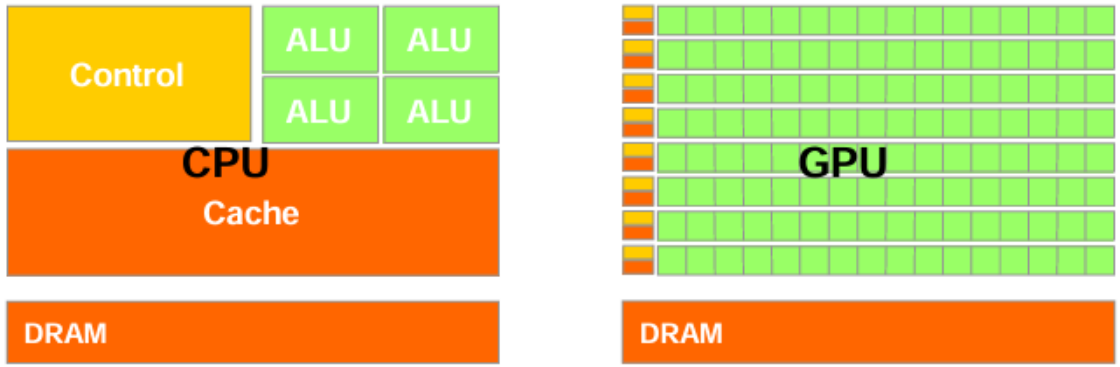
SIMD Example: Intel Xeon Phi



- Multi-core chip with Pentium-based SIMD processors
  - Targeting HPC market (Goal: high GFLOPS, GFLOPS/Watt)
- 4 hardware threads + wide SIMD units
  - Vector ISA: 32 vector registers (512b), 8 mask registers, scatter/gather
- In-order, short pipeline

Graphics Processing Unit (GPU)

- An architecture for compute-intensive, highly data-parallel computation
  - Exactly what graphics rendering is about
  - Transistors devoted to data processing rather than caching and flow control

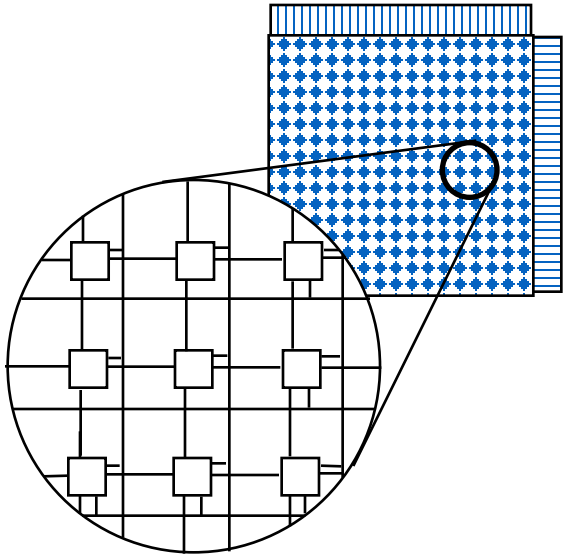


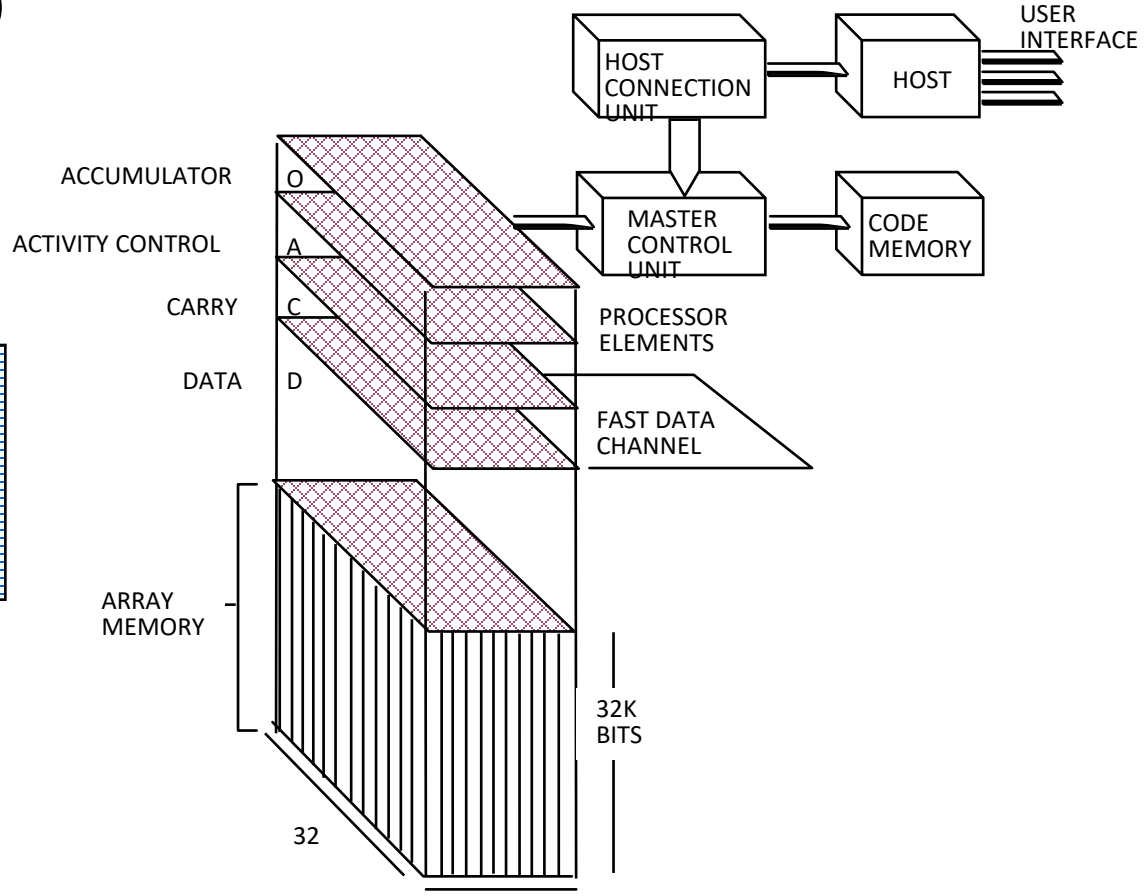
Data Parallelism in GPUs

- GPUs take advantage of massive DLP to provide very high FLOP rates
  - More than 1 Tera DP FLOP in NVIDIA GK110
- **SIMT** execution model
  - **Single instruction multiple threads**
  - Trying to distinguish itself from both “vectors” and “SIMD”
  - A key difference: better support for conditional control flow
- Program it with CUDA or OpenCL (among other things)
  - Extensions to C
  - Perform a “shader task” (a snippet of scalar computation) over many elements
  - Internally, GPU uses scatter/gather and vector-mask-like operations

AMT DAP 500 - Distributed Array Processor.

- Applied Memory Technology (AMT)
- Distributed Array Processor (DAP)

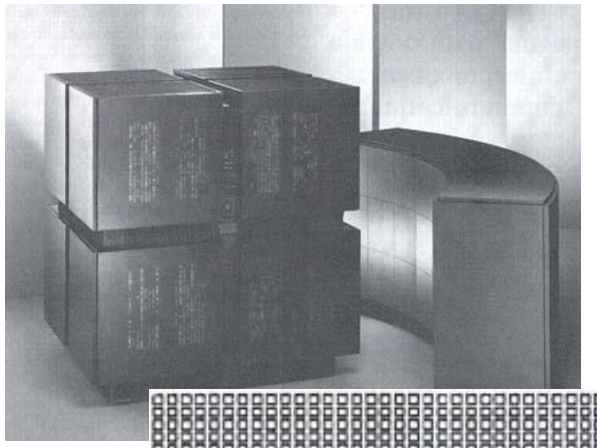


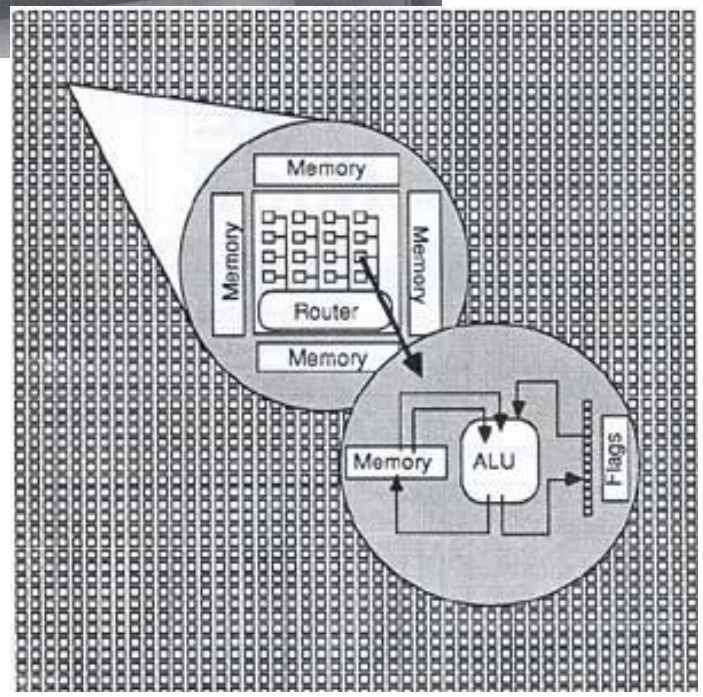


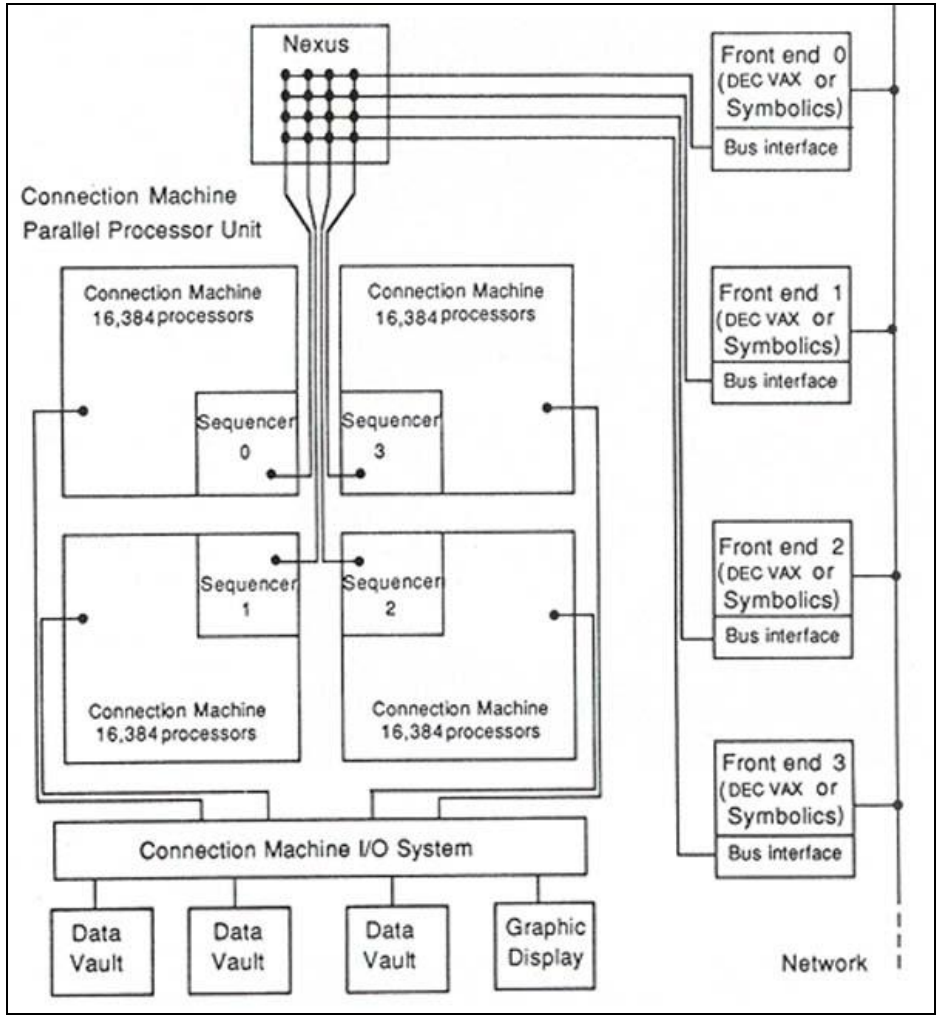
Thinking Machines Connection Machine

16,000 processors!!!

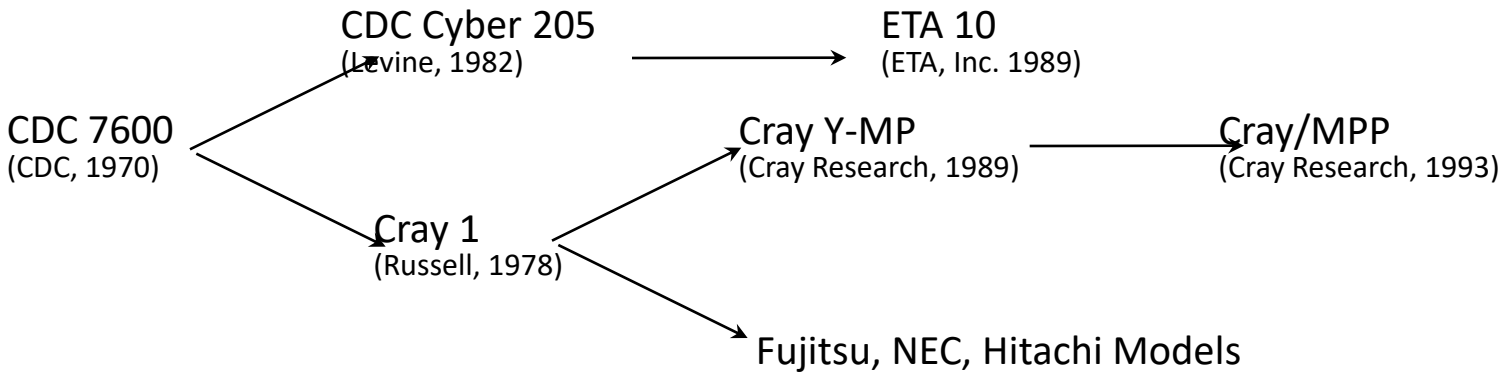
(Tucker, IEEE Computer, Aug. 1988)



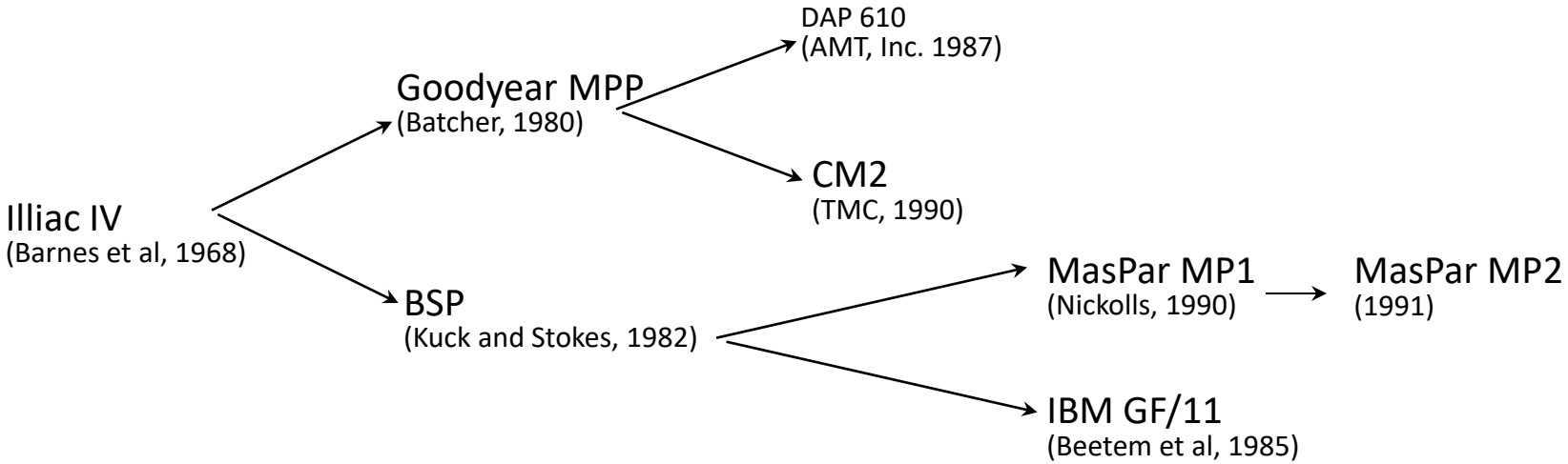




Vector and SIMD Processing Timeline



(a) Multivector track



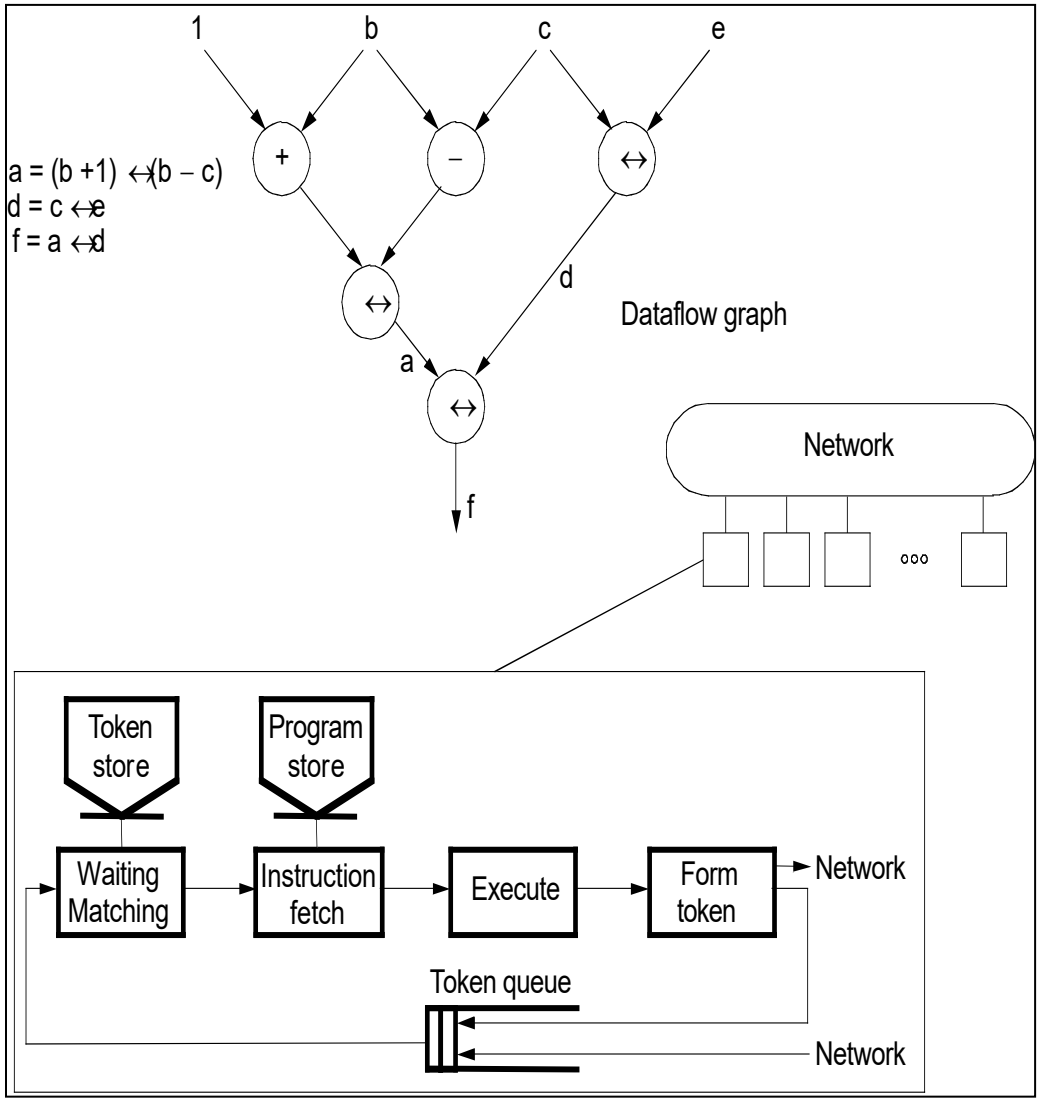
(b) SIMD track

DataFlow Architectures

Dataflow Architectures

- ***Represent computation as graph of dependencies***
- Nodes are operations.
- Arcs are operands that contain tokens.

- Operations are stored in memory until operands are ready
- ***Operations can be dispatched to processors***
- **Tokens** carry tags of next instruction to processor
- Tags compared in matching store
- ***A match fires execution***
- Machine does the hard parallelization work
- Hard to build correctly !!!!



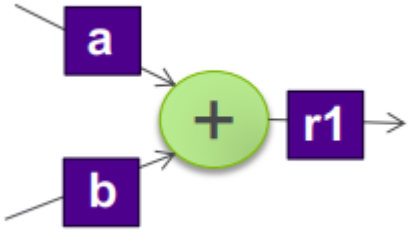
DATAFLOW OPERATIONAL SEMANTICS – actor based

- Tokens -> Data values

Firing Rules -> All tokens are present in the input arcs

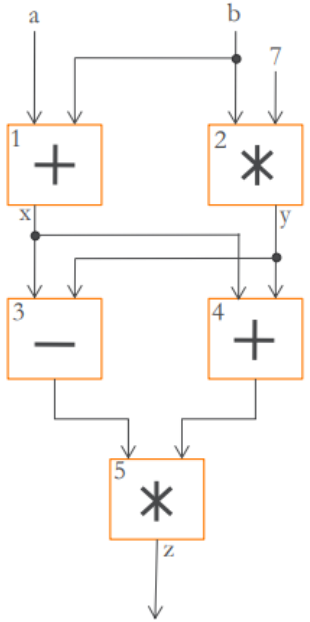
  - Actor removes tokens from each of its input arcs
  - Actor executes operation
  - Actor places tokens on each of its output arcs

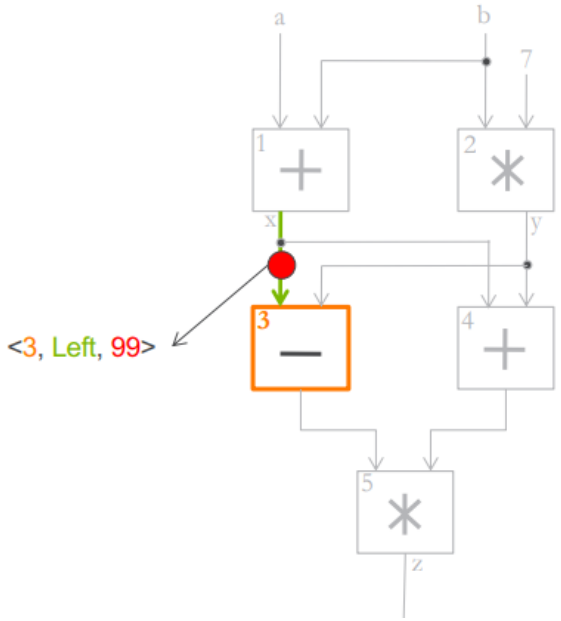
- Assignment operation -> Placing a token in the output arc



Example

```
x = a + b;
y = b * 7;
 z = (x-y) * (x+y);
```



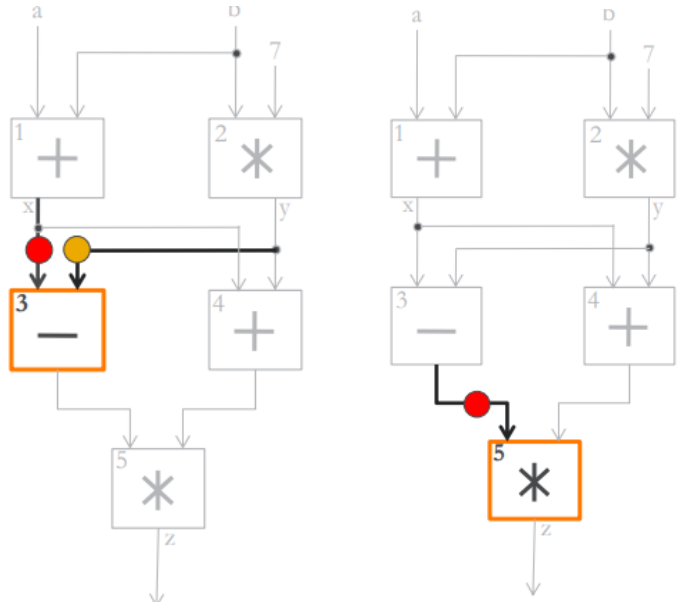


- Values in dataflow graphs

are represented as tokens

- An operator executes when all its input tokens are present
- Copies of the result token are distributed to the destination operators

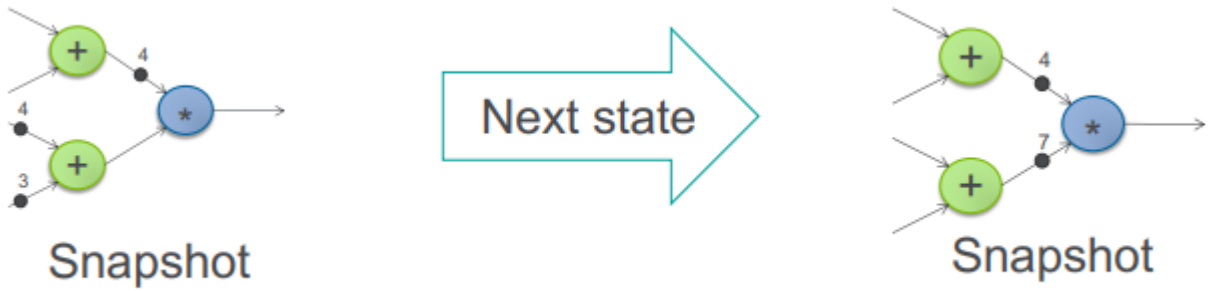
No separate control flow



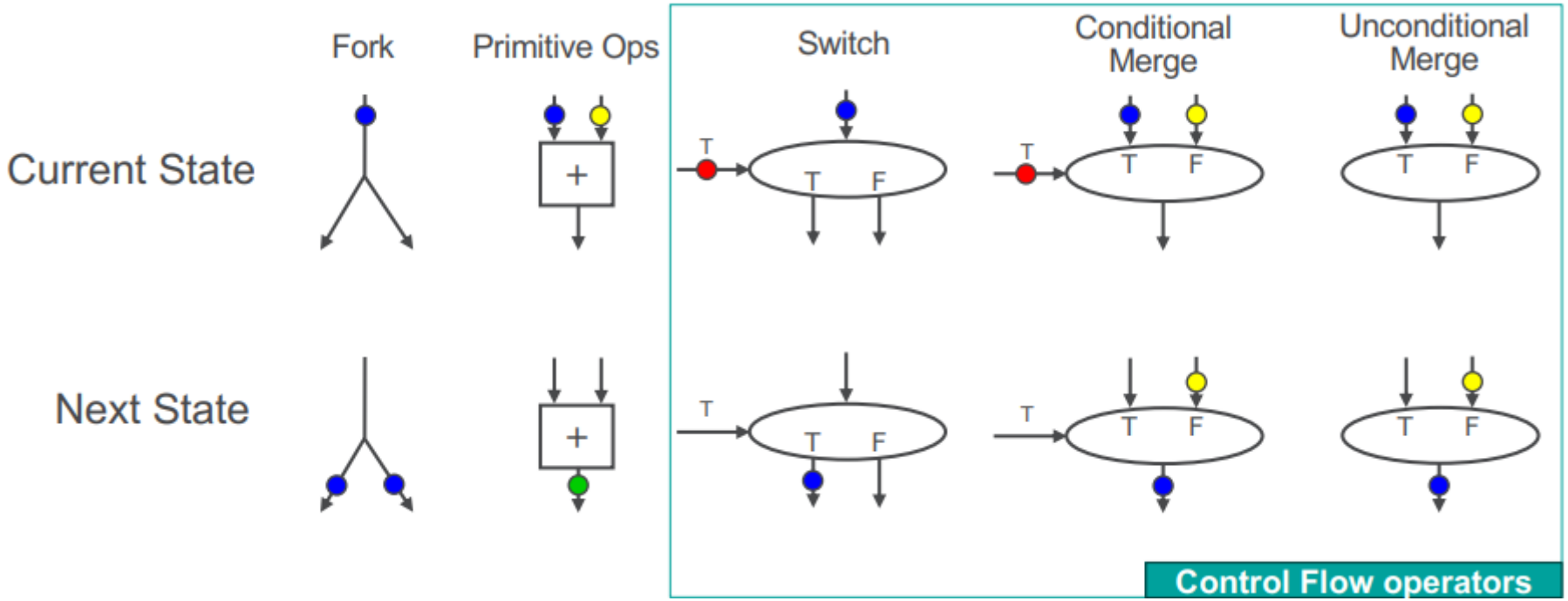
Configurations

- Snapshot/Configuration -> State of the program

- Next state -> Any enabled actor fired defines the “next state” of the computation



Operators

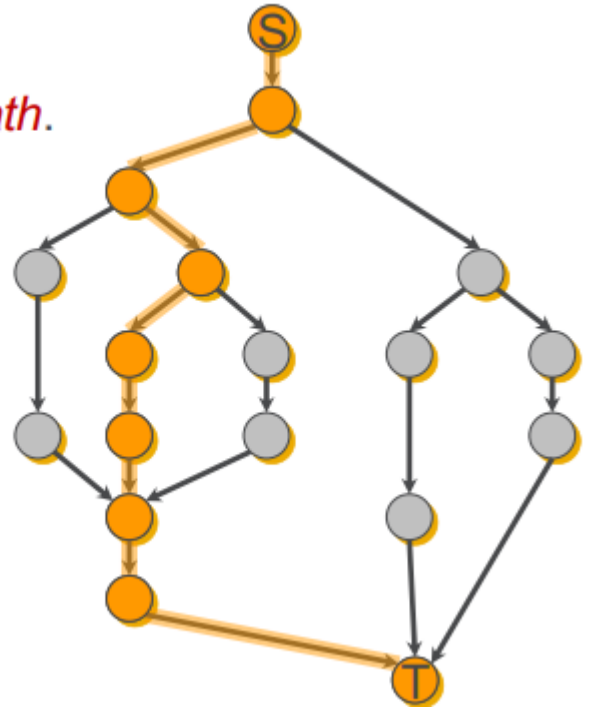

Fork
Primitive Ops
Switch
Conditional Merge
Unconditional Merge
Current State
Next State
T
F
+
Control Flow operators

Dataflow performance

Critical path

longest path from S to T is known as the *critical path*.

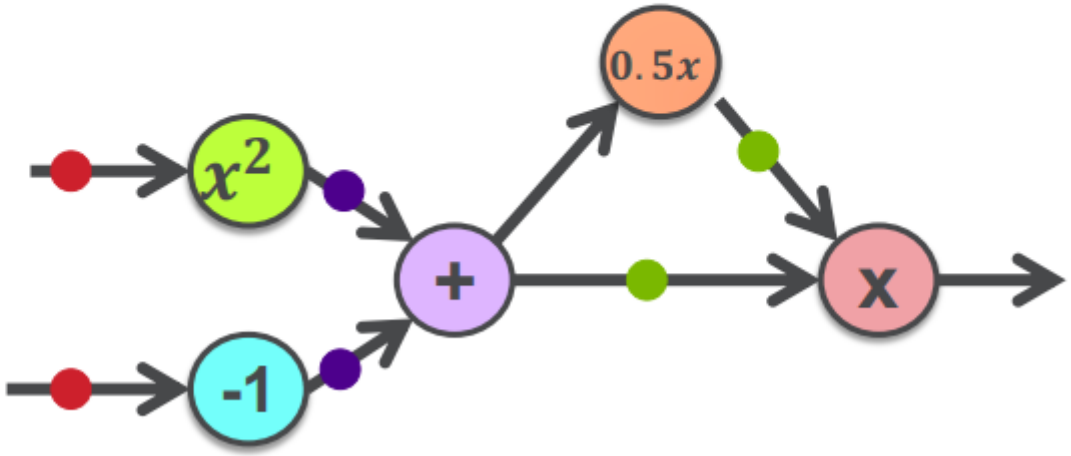
Dataflow Minimum execution time is determined by the critical path and scheduling of ready tokens.

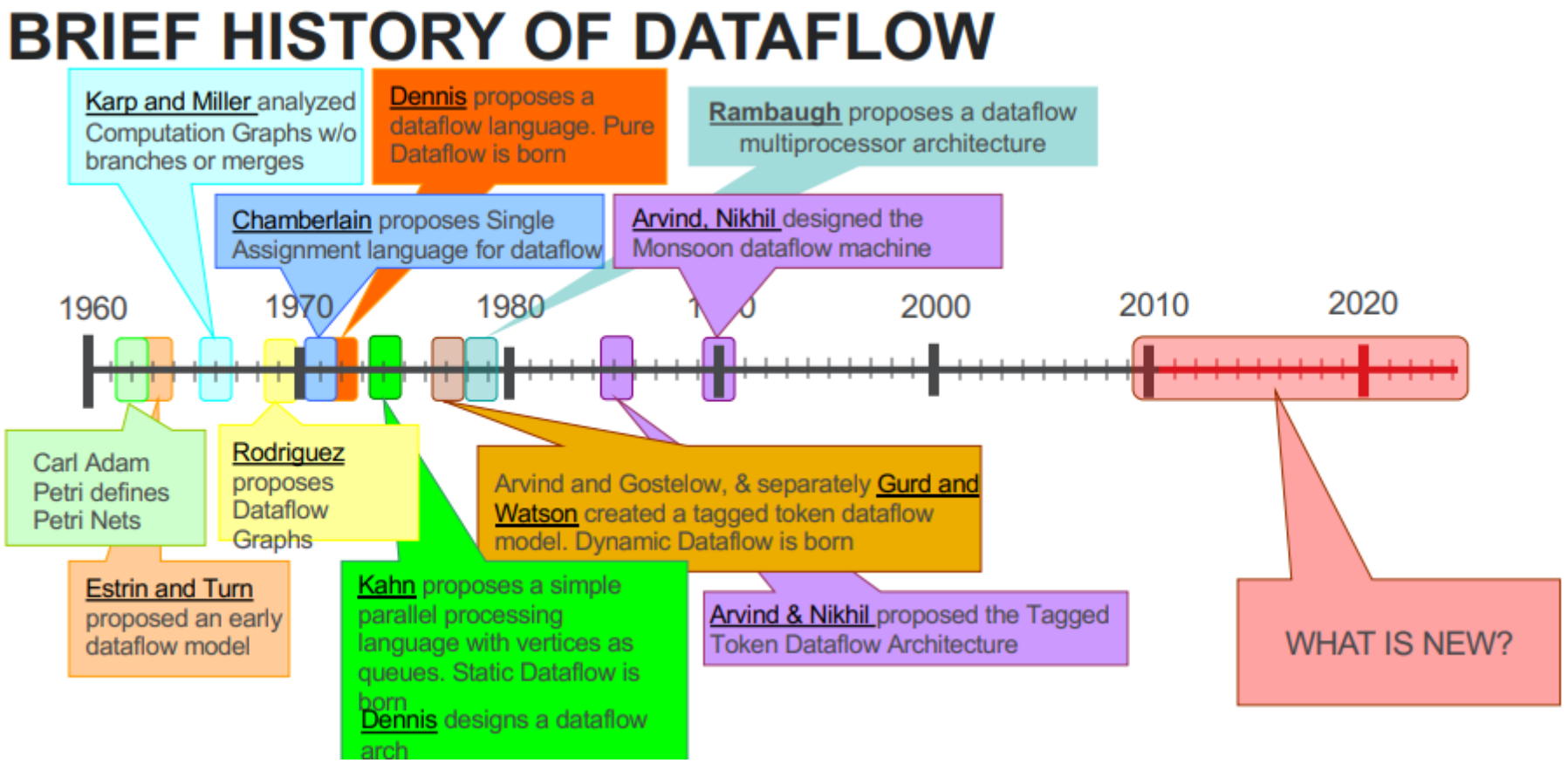


Dataflow Performance

DATAFLOW PIPELINING

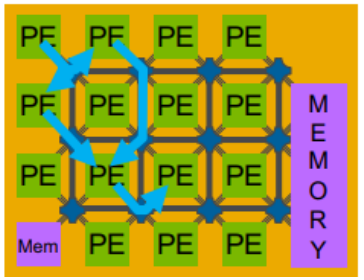
Allow multiple executions of the same dataflow graph

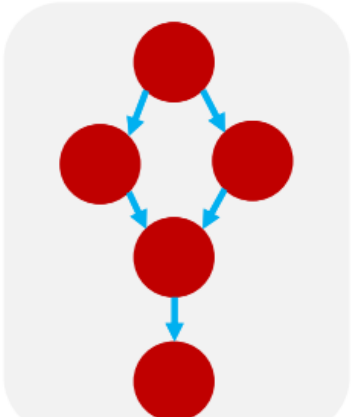


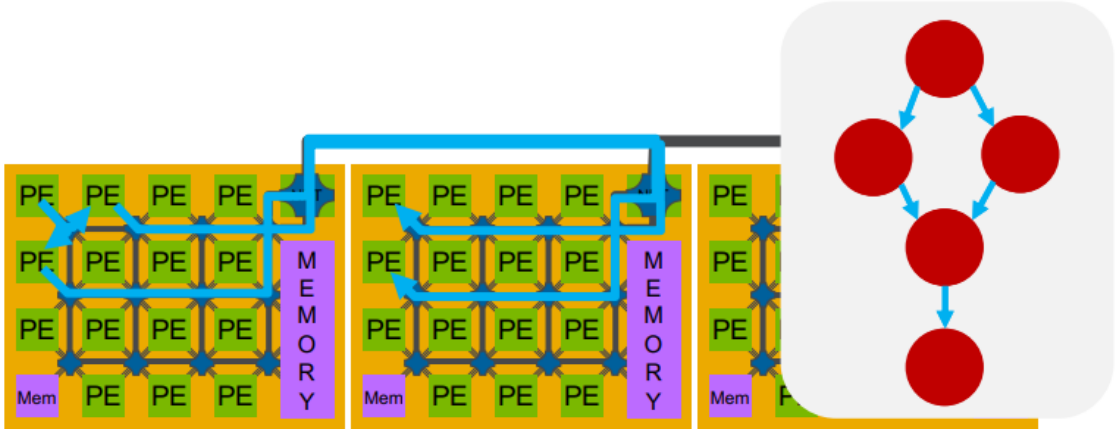

BRIEF HISTORY OF DATAFLOW
Karp and Miller analyzed Computation Graphs w/o branches or merges
Dennis proposes a dataflow language. Pure Dataflow is born
Rambaugh proposes a dataflow multiprocessor architecture
Chamberlain proposes Single Assignment language for dataflow
Arvind, Nikhil designed the Monsoon dataflow machine
1960
1970
1980
2000
2010
2020
Carl Adam Petri defines Petri Nets
Rodriguez proposes Dataflow Graphs
Arvind and Gostelow, & separately Gurd and Watson created a tagged token dataflow model. Dynamic Dataflow is born
Estrin and Turn proposed an early dataflow model
Kahn proposes a simple parallel processing language with vertices as queues. Static Dataflow is born
Dennis designs a dataflow arch
Arvind & Nikhil proposed the Tagged Token Dataflow Architecture
WHAT IS NEW?

Spatial Reconfigurable Architectures

1. Granularity of dataflow operations
2. Spatial reconfigurable architectures



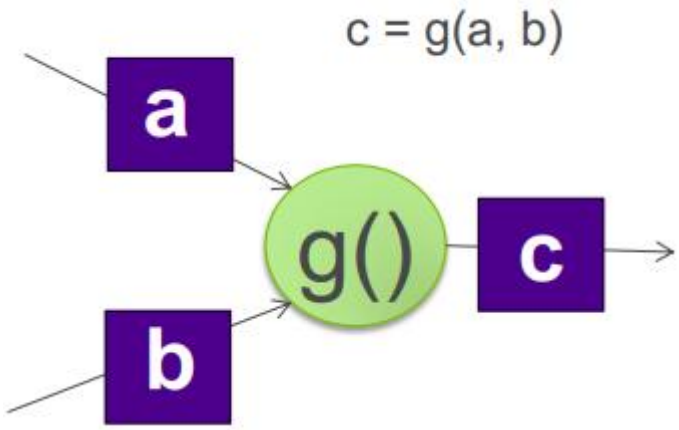




COARSE GRAIN DATAFLOW

- Dataflow graph node executes more complex mathematical operations

- Hybrid execution models
  - Von Neumann + Dataflow
  - Node **g()** is defined in terms of multiple instructions
- Data locations (i.e., tokens) **a, b**, and **c** have a larger memory footprint



Moder Spatial Architectures

Cerebras CS-2



SambaNova DataScale SN30



Habana Gaudi1



GroqRack



	Cerebras CS2	SambaNova Cardinal SN30	Groq GroqRack	Habana Gaudi1
Compute Units	850,000 Cores	640 PCUs	5120 vector ALUs	8 TPC + GEMM engine
On-Chip Memory	40 GB L1, 1TB+ MemoryX	>300MB L1 1TB	230MB L1	24 MB L1 32GB
Process	7nm	7nm	7 nm	7nm
System Size	2 Nodes including Memory-X and Swarm-X	8 nodes (8 cards per node)	9 nodes (8 cards per node)	2 nodes (8 cards per node)
Estimated Performance of a card (TFlops)	>5780 (FP16)	>660 (BF16)	>250 (FP16) >1000 (INT8)	>150 (FP16)
Software Stack Support	Tensorflow, Pytorch, CSLang	SambaFlow, Pytorch, C++ SDK	GroqAPI, ONNX, C/C++ Groq Runtime	Synapse AI, TensorFlow and PyTorch, TPC
Interconnect	Ethernet-based	Ethernet-based	RealScale ™	Ethernet-based

HANDS ON WITH AI ACCELERATORS

CHALLENGES of DATAFLOW ARCH.

- Memory management
- Resource allocation and scheduling
- Parallelism control vs locality
- Balancing “size” of each dataflow node
- Programmability

Shared Physical Memory

Shared Physical Memory

- Add processors to single processor computer system
- Processors *share* computer system resources
  - Memory, storage, …
- Sharing physical memory
  - Any processor can reference any memory location
  - Any I/O controller can reference any memory address
  - ***Single physical memory address space***
- Operating system runs on any processor, or all
  - OS see single memory address space
  - Uses shared memory to coordinate
- ***Communication*** occurs as a result of ***loads and stores***

Memory

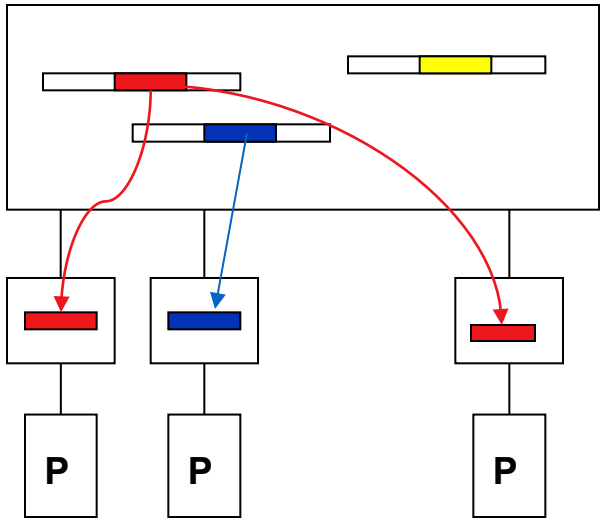
Latency vs. Bandwidth

- *Latency* = time needed for one date(byte or word) to arrived at processor
  (after the request was sent)

- *Bandwidth* = the rate at which data can be read from or stored into memory by a processor.
  - expressed in units of *bytes/second*

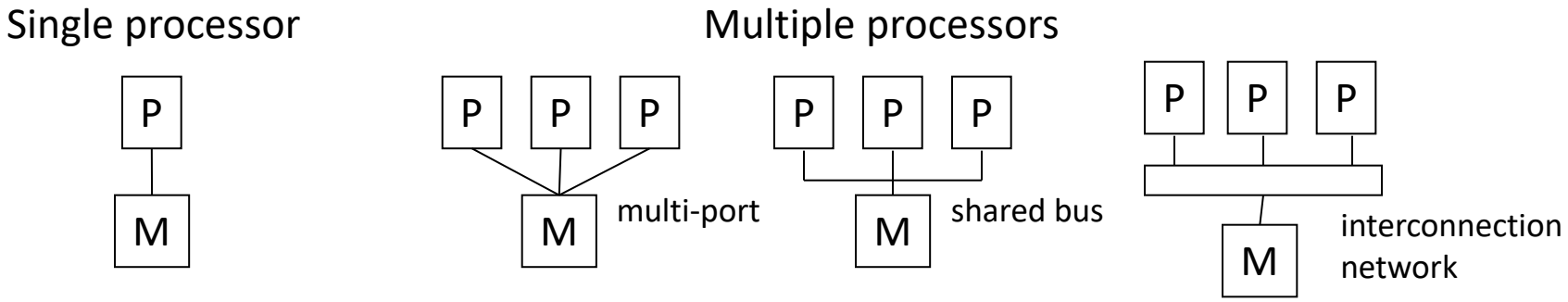
Caching in Shared Memory Systems

- Reduce average latency
  - automatic replication closer to processor
- Reduce average bandwidth
- Data is logically transferred from producer to consumer
  - store reg $\rightarrow$ mem
  - load reg $\leftarrow$ mem
- Processors can share data efficiently
- What happens when store and load are executed on different processors?
- Cache coherence problems

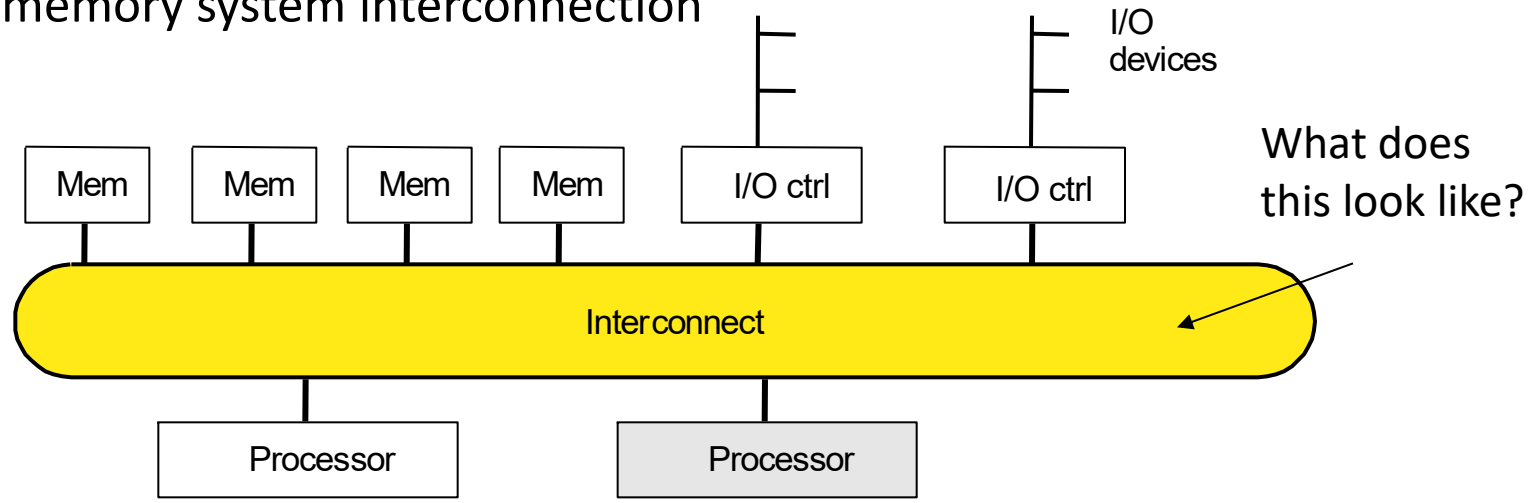


Shared Memory Multiprocessors (SMP)

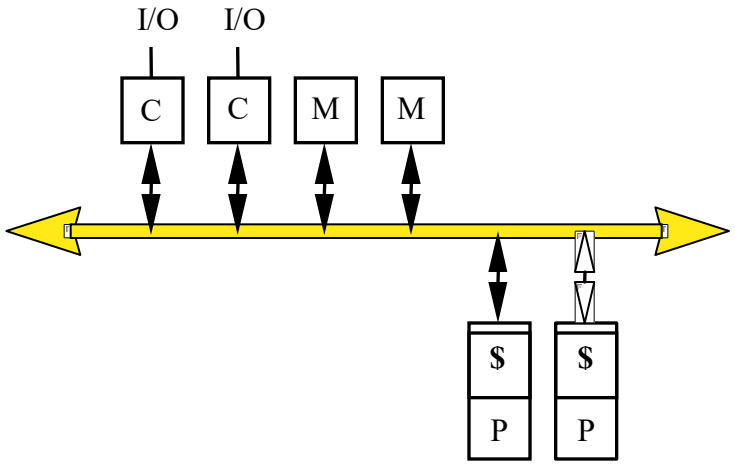
- Architecture types



- Differences lie in memory system interconnection



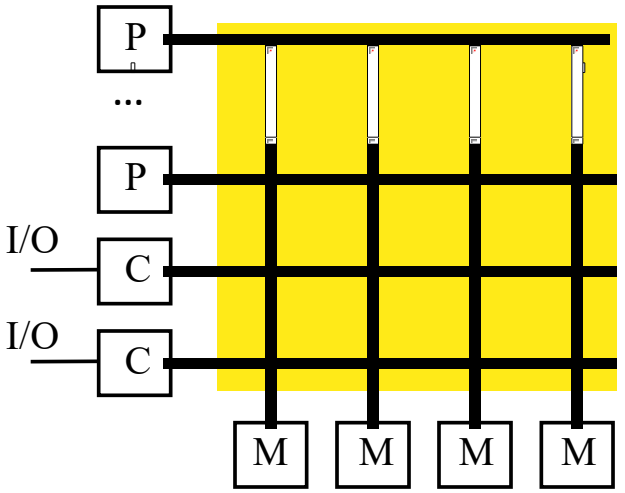
Bus-based SMP



- Memory bus handles all memory read/write traffic
- Processors share bus
- *Uniform Memory Access* (*UMA*)
  - Memory (not cache) uniformly equidistant
  - Take same amount of time (generally) to complete accesses
- May have multiple memory modules
  - Interleaving of physical address space
- Caches introduce memory hierarchy
  - Lead to data consistency problems
  - Cache coherency hardware necessary (*CC-UMA*)

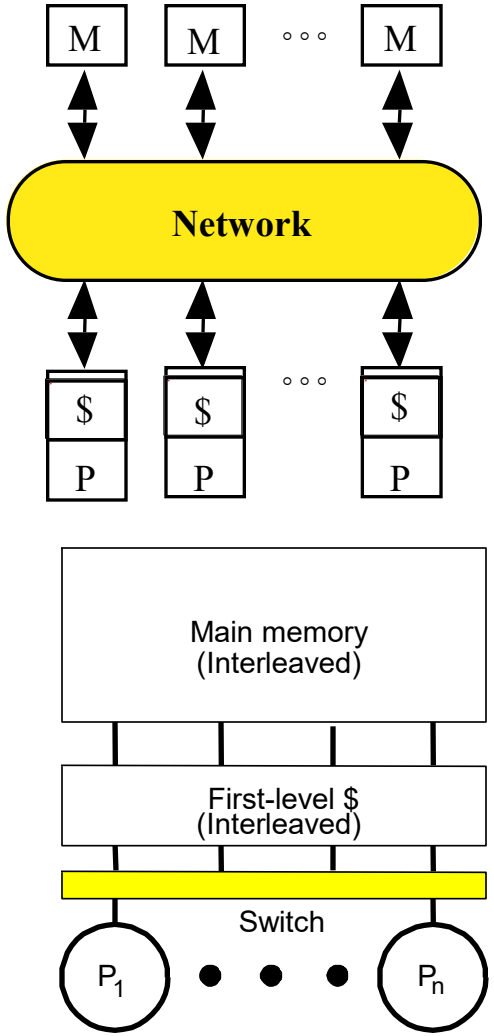
Crossbar SMP

- Replicates memory bus for every processor and I/O controller
  - Every processor has direct path
- UMA SMP architecture
- Can still have cache coherency issues
- Multi-bank memory or interleaved memory
- Advantages
  - Bandwidth scales linearly (no shared links)
- Problems
  - High incremental cost (cannot afford for many processors)
  - Use switched multi-stage interconnection network

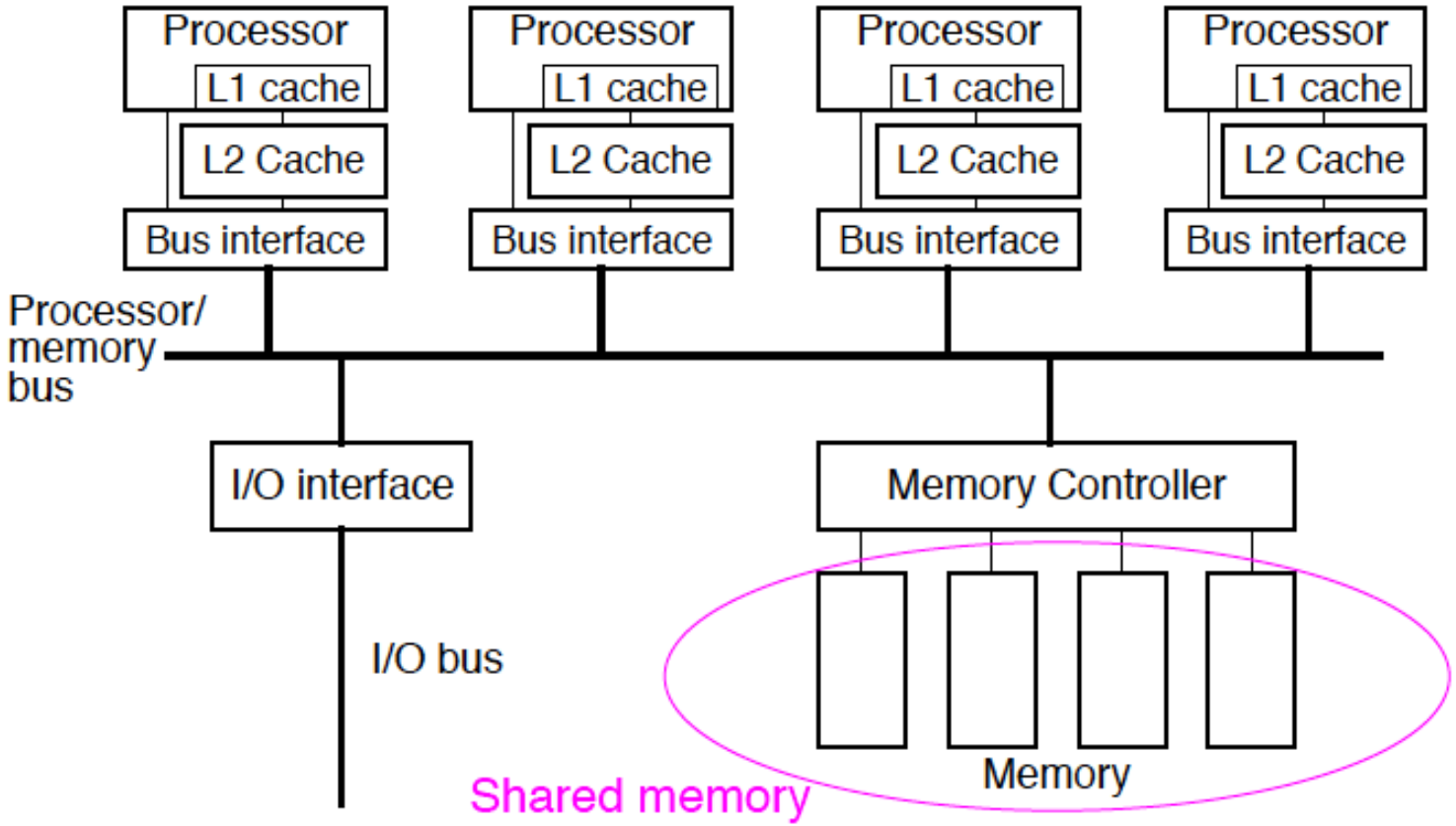


Symmetric Multiprocessors and Shared Cache

- *Interconnection network connects processors to memory*
- Centralized memory (UMA – Uniformed Memory Access)
- Network determines performance
  - Continuum from bus to crossbar
  - Scalable memory bandwidth
- Memory is physically separated from processors
- *Could have cache coherence problems*
- *Shared cache* reduces coherence problem and provides fine grained data sharing

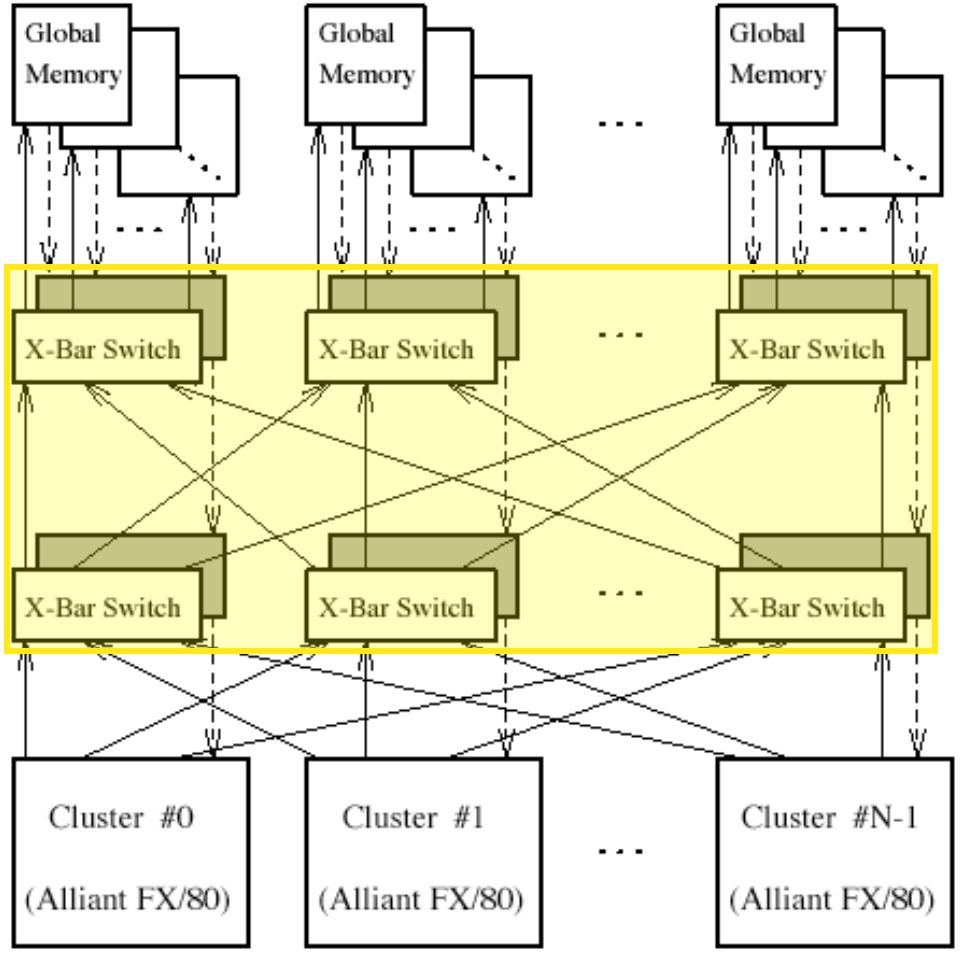


Levels of caching

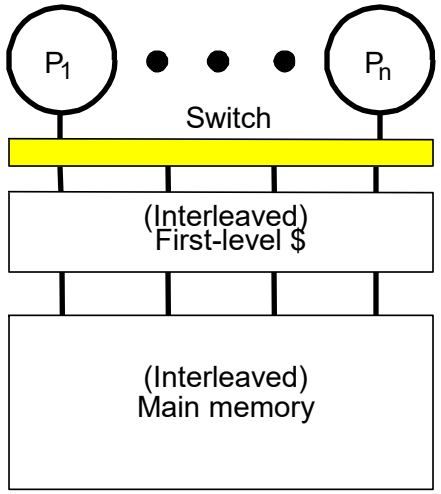


Example: University of Illinios CSRD Cedar Machine

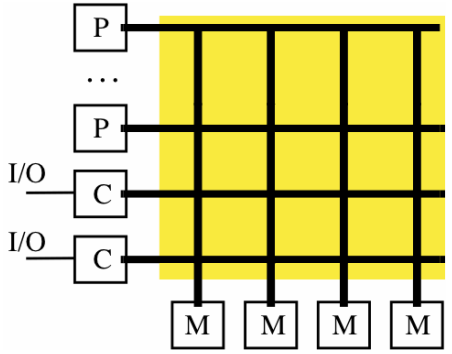
- Center for Supercomputing Research and Development
- Multi-cluster scalable parallel computer
- Alliant FX/80
  - 8 processors w/ vectors
  - Shared cache
  - HW synchronization
- Omega switching network
- Shared global memory
- SW-based global memory coherency



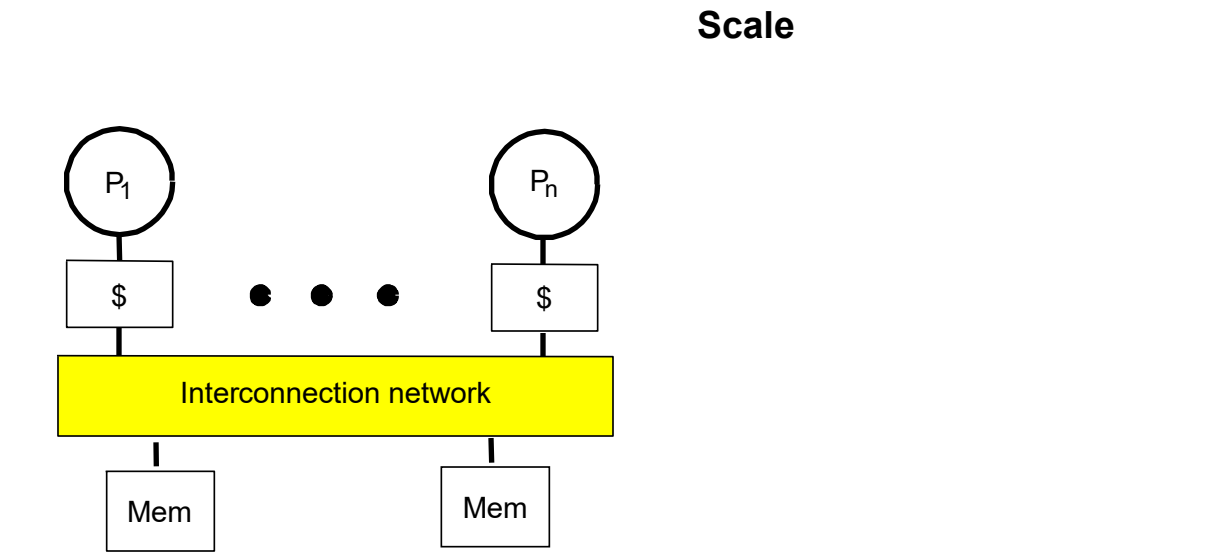
Extensions of the Memory System



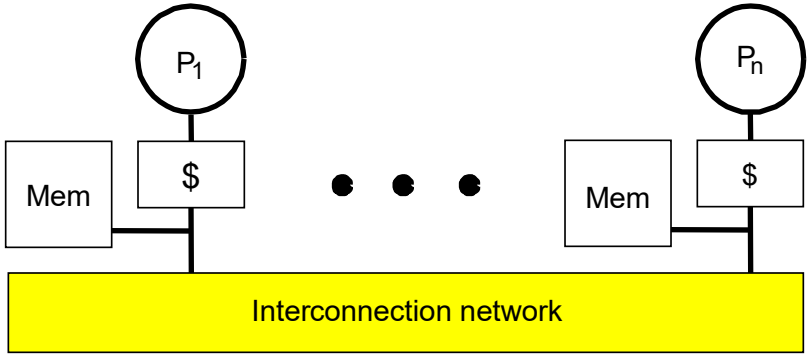
Shared Cache



Crossbar, Interleaved

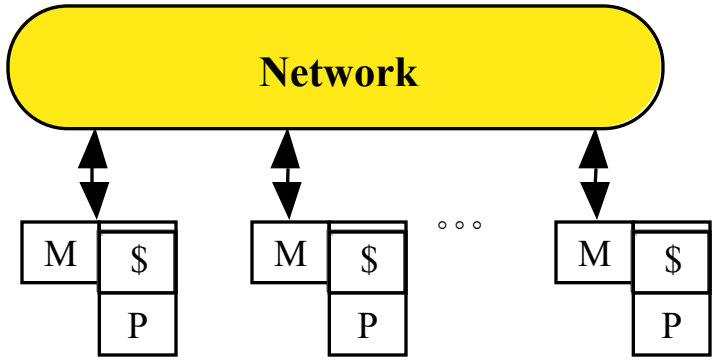


Centralized Memory
UMA



Distributed Memory (NUMA)

Non-Uniform Memory Access (NUMA) SMPs



- Distributed memory
- ***Memory is physically resident close to each processor***
- ***Memory is still shared!***
- *Non-Uniform Memory Access* (NUMA)
  - Local memory and remote memory
  - Access to local memory is faster, remote memory slower
  - Access is non-uniform
  - Performance will depend on data locality
- Cache coherency is still an issue (more difficult)
- Interconnection network architecture is more scalable!!!

Cache Coherency and SMPs

- Caches play key role in SMP performance
  - Reduce average data access time
  - Reduce bandwidth demands placed on shared interconnect
- Private processor caches create a problem
  - Copies of a variable can be present in multiple caches
  - A write by one processor may not become visible to others
    - They will keep accessing stale value in their caches

  $\Rightarrow$ *Cache coherence* problem
- What can be done about it?
  - *Organize the memory hierarchy*
  - Detect and take actions to eliminate the problem

Advantages of Shared Memory Architectures

- Compatibility with SMP hardware
- Ease of programming when communication patterns are complex or vary dynamically during execution
- Ability to develop applications using familiar SMP model, attention only on performance critical accesses
- Lower communication overhead, due to implicit communication and memory mapping to implement protection in hardware, rather than through I/O system
- Need: HW-controlled caching to reduce remote communication by caching of all data, both shared and private

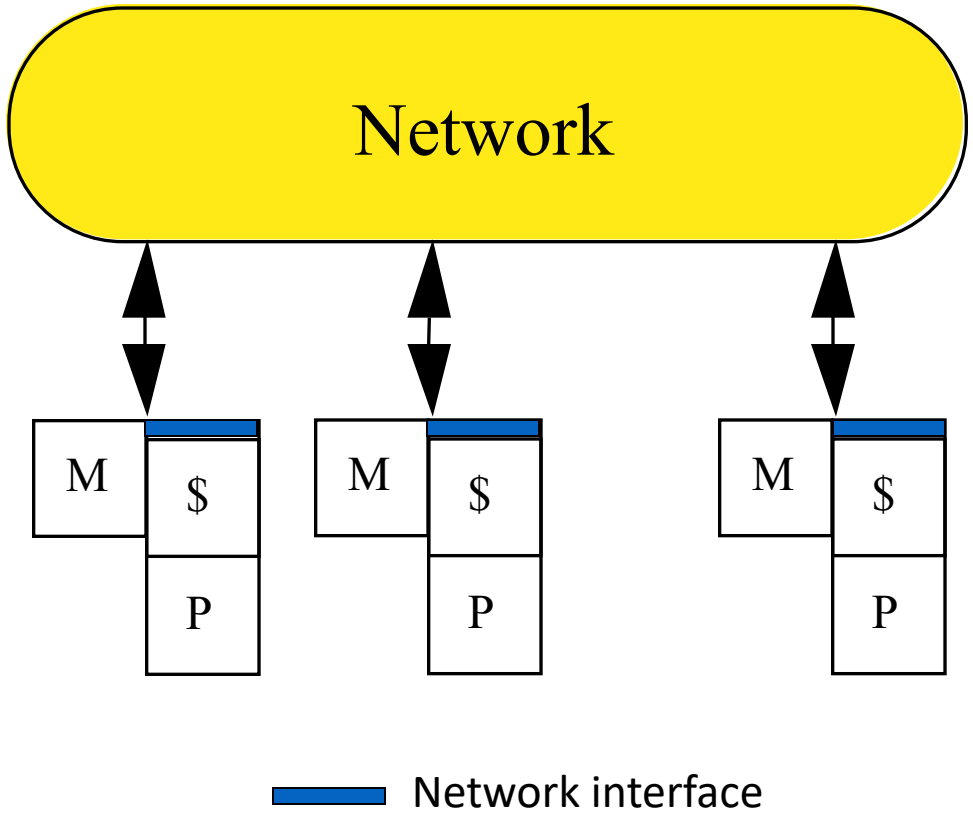
Distributed Memory Multiprocessors

Distributed Memory Multiprocessors

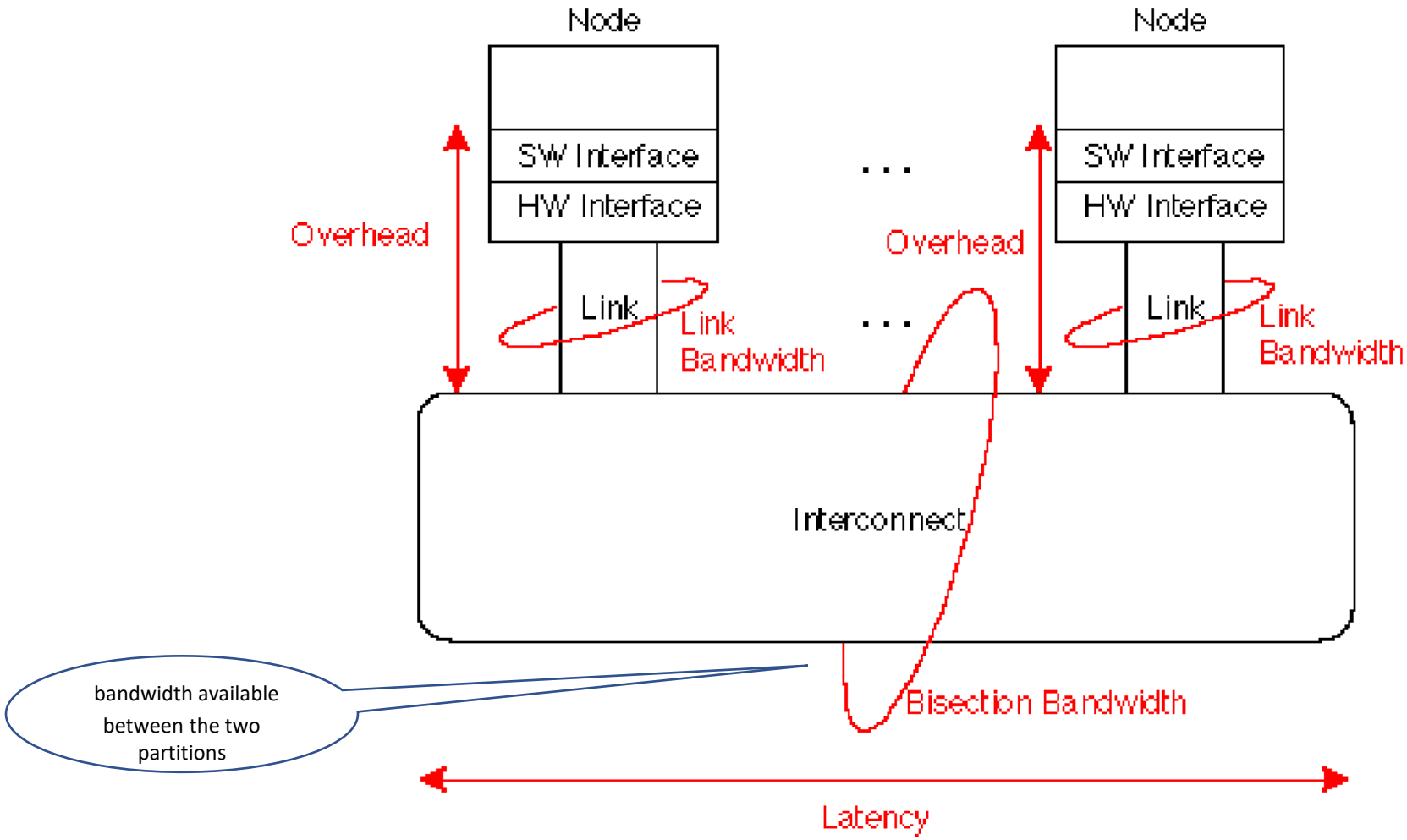
- Each processor has a local memory
  - ***Physically separated memory address space***
- Processors must communicate to access non-local data
  - Message communication (message passing)
    - *Message passing architecture*
  - Processor interconnection network
- ***Parallel applications must be partitioned across***
  - ***Processors: execution units***
  - ***Memory: data partitioning***
- Scalable architecture
  - Small incremental cost to add hardware (cost of node)

Distributed Memory (MP) Architecture

- Nodes are complete computer systems
  - Including I/O
- Nodes communicate via interconnection network
  - Standard networks
  - Specialized networks
- Network interfaces
  - Communication integration
- Easier to build



Network Performance Measures



Overhead: latency of interface vs. Latency: network

Network Performance Metrics: Latency and Bandwidth

- Bandwidth= *maximum capacity of a communications link to transmit data over a network connection in a given amount of time*
  - Need high bandwidth in communication
  - Match limits in network, memory, and processor
  - Network interface speed vs. network bisection bandwidth
- Latency=*measures the time it takes for some data to get to its destination across the network*.
  - Performance affected since processor may have to wait
  - Harder to overlap communication and computation
  - Overhead due to communication is a problem in many machines
- Latency hiding
  - Increases programming system burden
  - Examples: communication/computation overlaps, prefetch

Scalable, High-Performance Interconnect

- Interconnection network is the core of parallel architecture (... next lecture...)
- Requirements and tradeoffs at many levels
  - Elegant mathematical structure
  - Deep relationship to algorithm structure
  - Hardware design sophistication
- Little consensus
  - Performance metrics?
  - Cost metrics?
  - Workload?
  - …

Message Passing Model

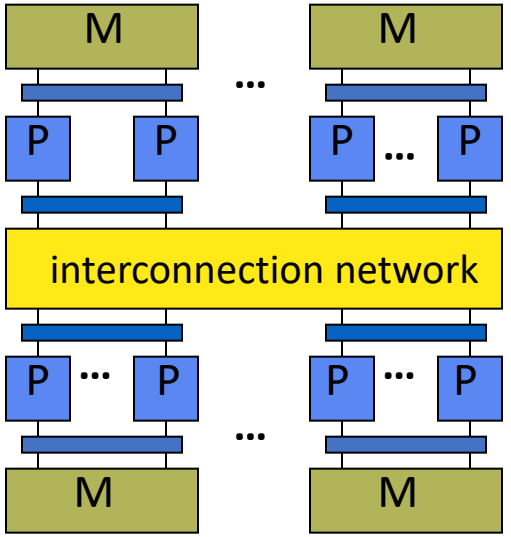
- Hardware maintains send and receive message buffers
- Send message (synchronous)
  - Build message in local message send buffer
  - Specify receive location (processor id)
  - Initiate send and wait for receive acknowledge
- Receive message (synchronous)
  - Allocate local message receive buffer
  - Receive message byte stream into buffer
  - Verify message (e.g., checksum) and send acknowledge
- Memory to memory copy with acknowledgement and pairwise synchronization

Advantages of Distributed Memory Architectures

- The hardware can be simpler (especially versus NUMA) and is more scalable
- Communication is explicit and simpler to understand
- Explicit communication focuses attention on costly aspect of parallel computation
- Synchronization is naturally associated with sending messages, reducing the possibility for errors introduced by incorrect synchronization
- Easier to use sender-initiated communication, which may have some advantages in performance

Clusters of SMPs

- Clustering
  - Integrated packaging of nodes
- Motivation
  - Amortize node costs by sharing packaging and resources
  - Reduce network costs
  - Reduce communications bandwidth requirements
  - Reduce overall latency
  - More parallelism in a smaller space
  - Increase node performance
- Many scalable parallel systems today are built as SMP clusters !!!



Heterogeneous Clusters of SMPs

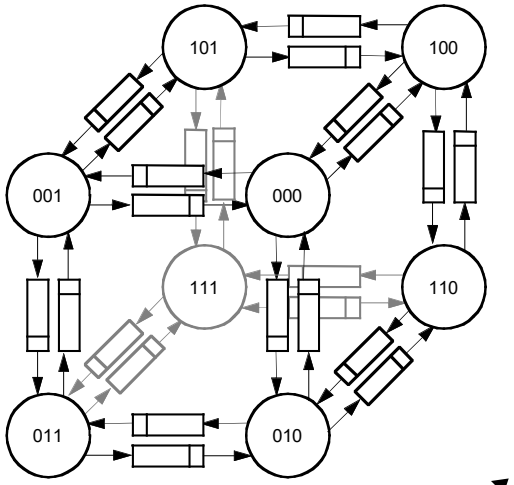
- Diverse Nodes: Not all nodes in the cluster are identical.
  - Some may feature Intel processors, others AMD, or they may have a different number of CPU cores.
  - Accelerators (GPU/FPGA): A heterogeneous cluster often includes nodes equipped with graphics cards (NVIDIA/AMD) to accelerate intensive mathematical computations.

- The Challenge: In a heterogeneous environment, **load balancing** is critical.
  - Balanced partitioning is not good enough
    - it should be dependent on the nodes' capacity and performance

Examples...

Example:CalTech Cosmic Cube

- First distributed memory message passing system
- Hypercube-based communications network



- Chuck Seitz, Geoffrey Fox

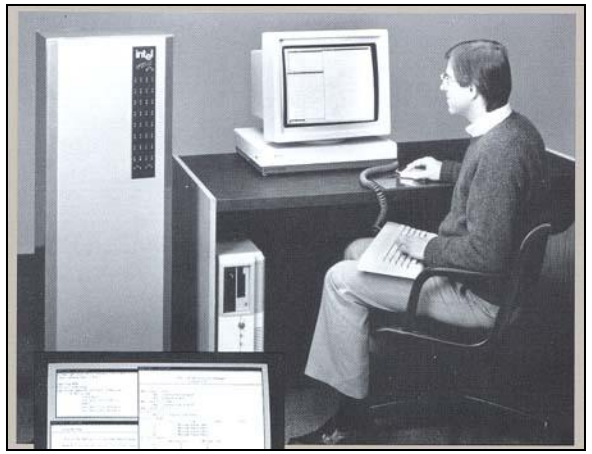


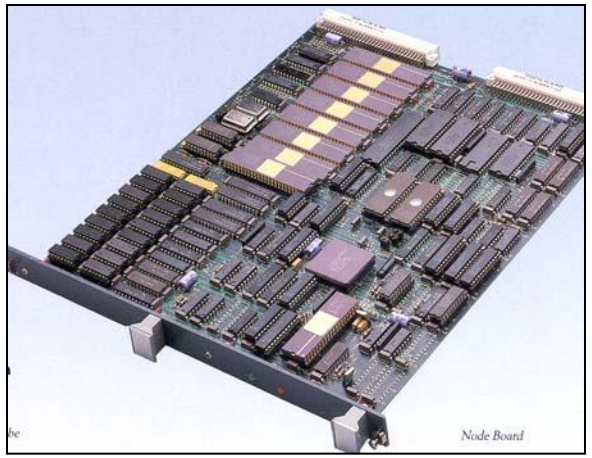
FIFO on each link
- store and forward

Compute node

Example:Intel iPSC/1, iPSC/2, iPSC/860

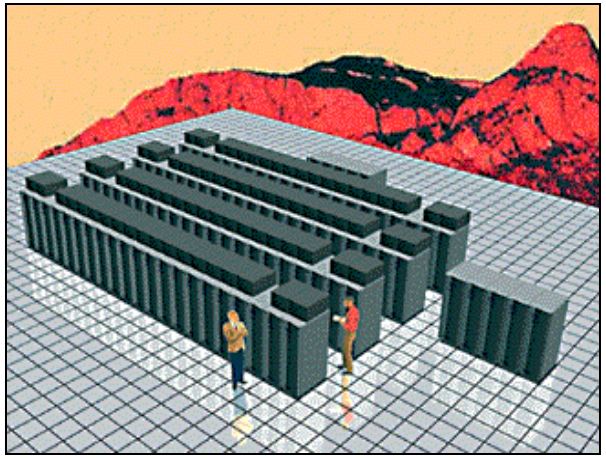
- Shift to general links
  - DMA, enabling non-blocking ops
    - Buffered by system at destination until recv
  - Store&forward routing
- Diminishing role of topology
  - Any-to-any pipelined routing
  - node-network interface dominates communication time
  - Simplifies programming
  - Allows richer design space



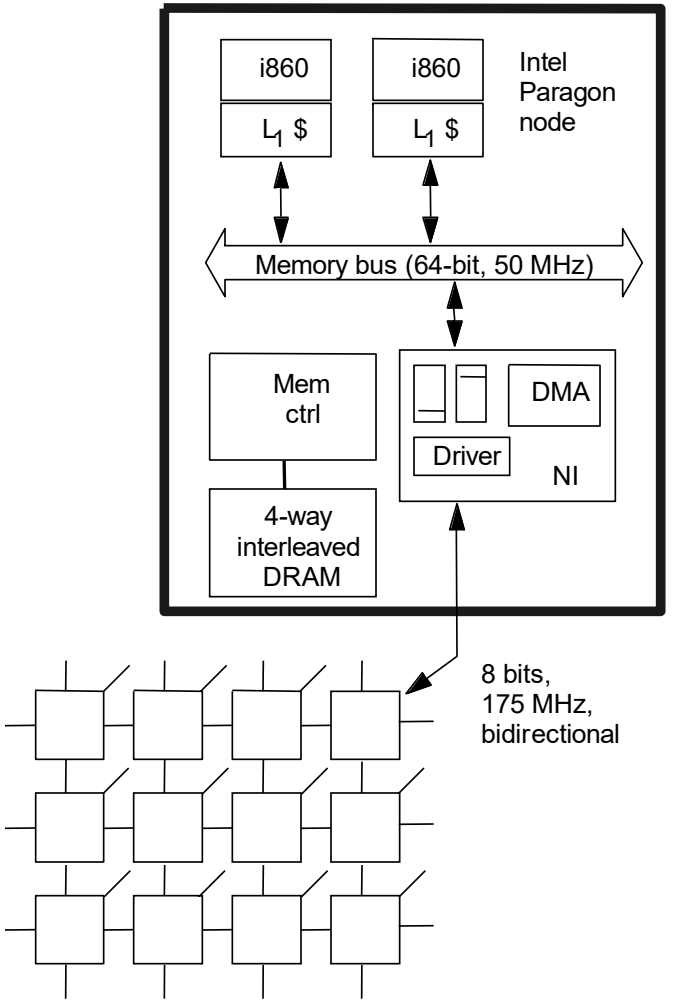


Example: Intel Paragon and ASCI Red

- DARPA project machine
  - Intel i860 processor
  - 2D grid network with processor node attached to every switch
  - 8bit, 175 MHz bidirectional links
- Forerunner design for ASCI Red
  - First Teraflop computer

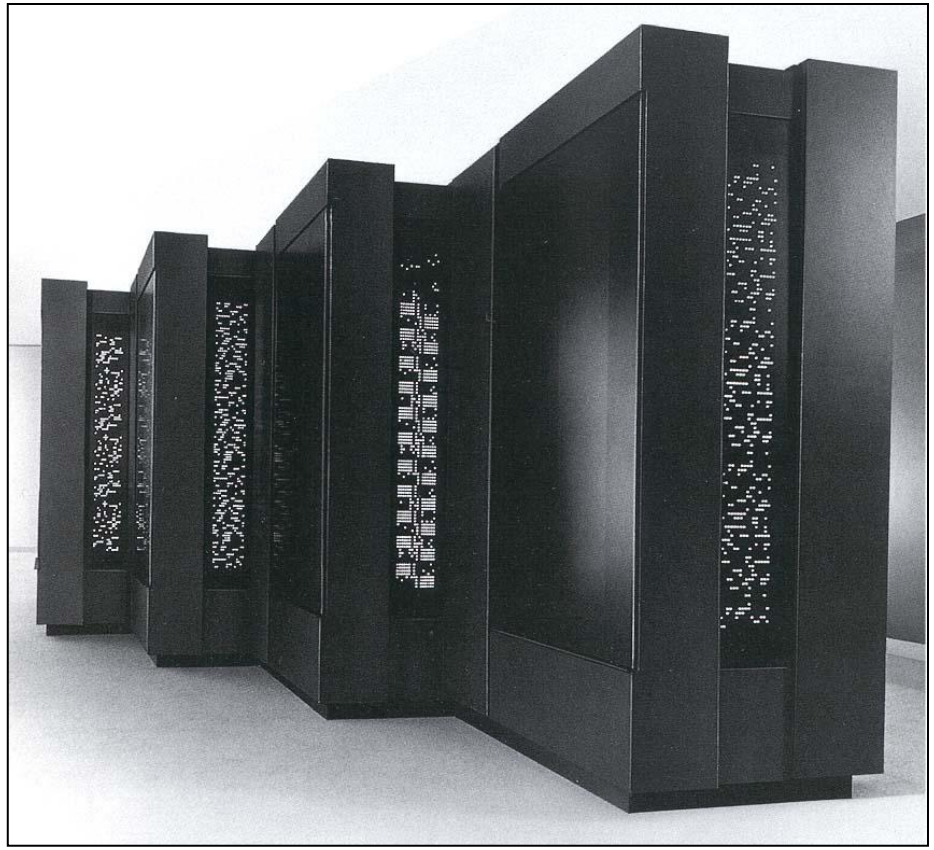






Example: Thinking Machine CM-5

- Repackaged SparcStation
  - 4 per board
- Fat-Tree network
- Control network for global synchronization
- Suffered from hardware design and installation problems



Example: Berkeley Network Of Workstations (NOW)

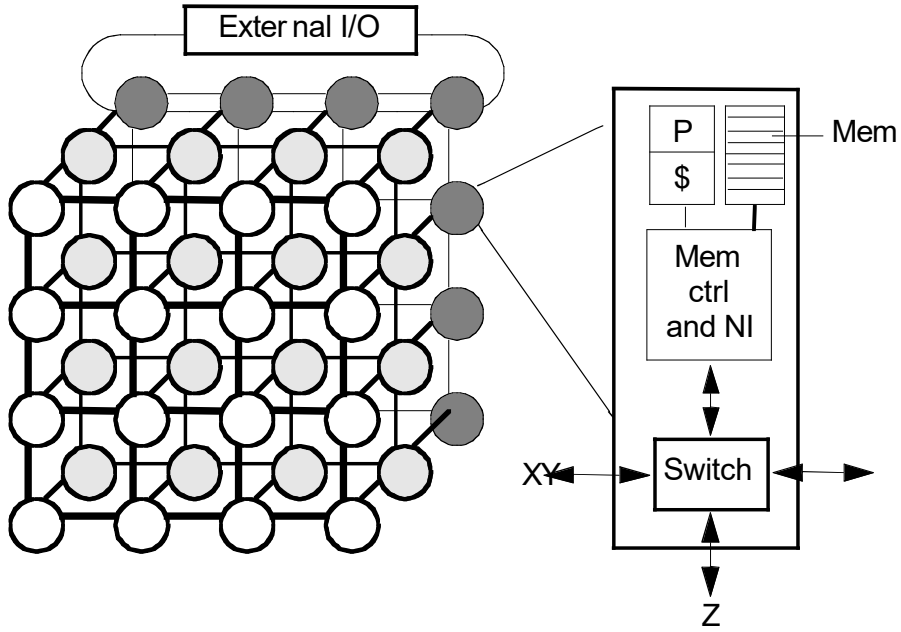
- 100 Sun Ultra2 workstations
- Inteligent network interface
  - proc + mem
- Myrinet network
  - 160 MB/s per link
  - 300 ns per hop



Example: Cray T3E

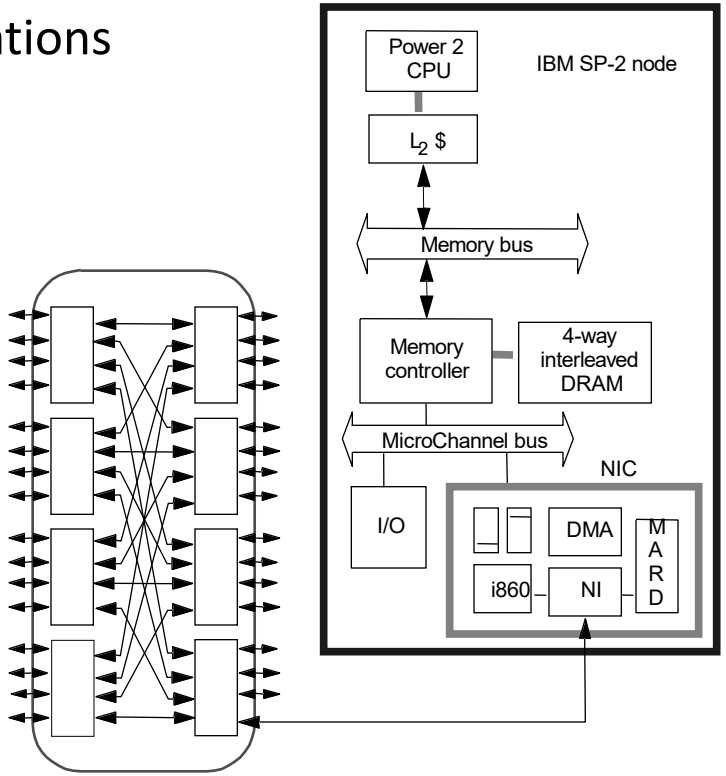
- Up to 1024 nodes
- 3D torus network
  - 480 MB/s links
- No memory coherence
- Access remote memory
  - Converted to messages
  - SHared MEMory communication
    - *put* / *get* operations
- Very successful machine





Example: IBM SP-2

- Made out of essentially complete RS6000 workstations
- Network interface integrated in I/O bus
- SP network very advanced
  - Formed from 8-port switches
- Predecessor design to
  - ASCI Blue Pacific (5856 CPUs)
  - ASCI White (8192 CPUs)







Example: NASA Columbia

- System hardware
  - 20 SGI Altix 3700 superclusters
    - 512 Itanium2 processors (1.5 GHz)
    - 1 TB memory
  - 10,240 processors (now 13,312)
  - NUMAflex architecture
  - NUMAlink “fat tree” network
  - Fully shared memory!!!
- Software
  - Linux with PBS Pro job scheduling
  - Intel Fortran/C/C++ compilers



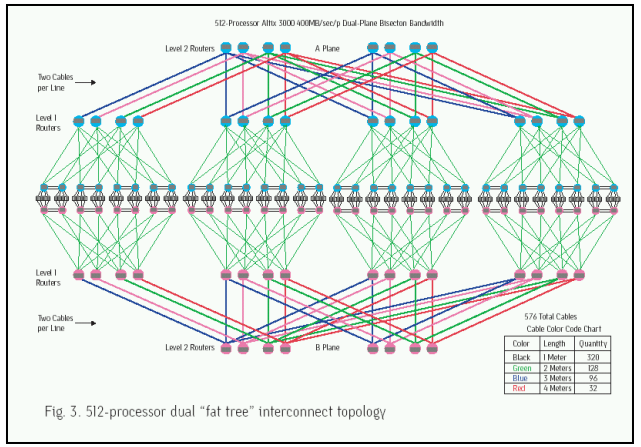
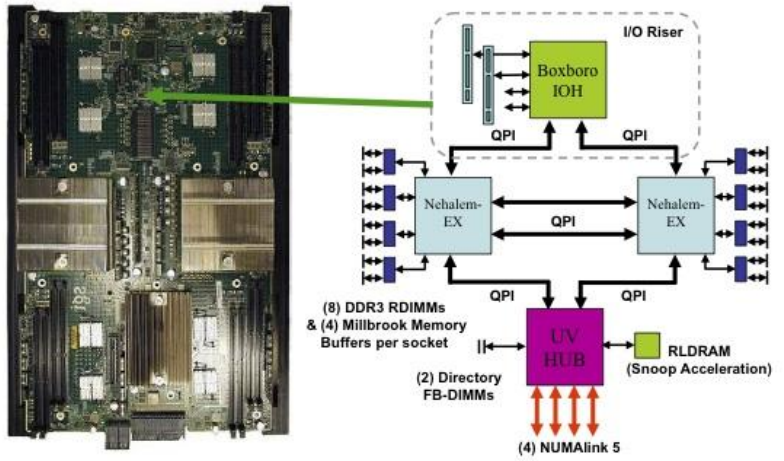
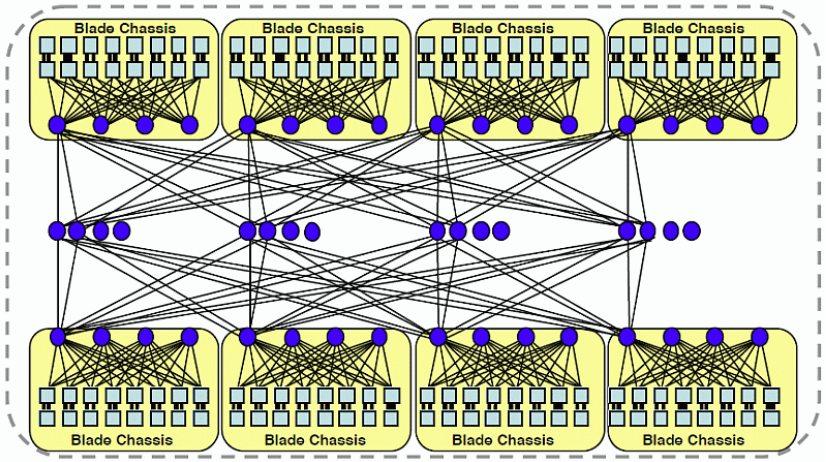


Fig. 3. 512-processor dual “fat tree” interconnect topology

Example: SGI Altix UV

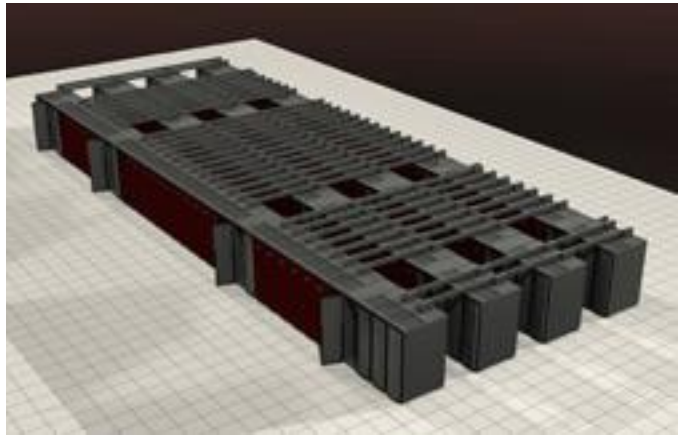
- Latest generation scalable shared memory architecture
- Scaling from 32 to 2,048 cores
  - Intel Nehalem EX
- Architectural provisioning for up to 262,144 cores
- Up to 16 terabytes of global shared memory in a single system image (SSI)
- High-speed 15GB per second interconnect NUMAlink 5

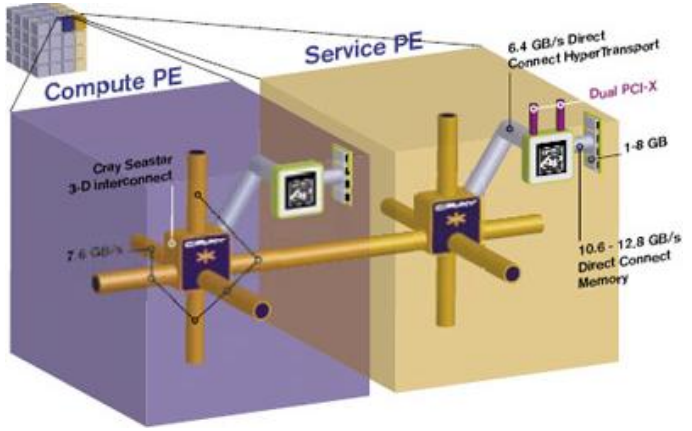




Example: Sandia Red Storm

- System hardware
  - Cray XT3
  - 135 compute node cabinets
  - 12,960 processors
    - AMD Opteron dual-core
  - 320 / 320 service / I/O node processors
  - 40 TB memory
  - 340 TB disk
  - 3D mesh interconnect
- Software
  - Catamount compute node kernel
  - Linux I/O node kernel
  - MPI

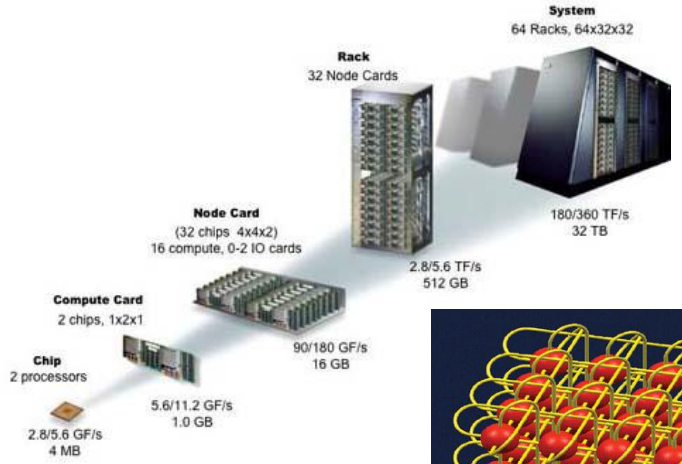


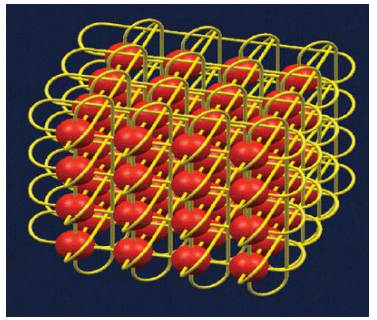


Example: LLNL BG/L

- System hardware
  - IBM BG/L (BlueGene)
  - 65,536 dual-processor compute nodes
    - PowerPC processors
    - “double hummer” floating point
  - I/O node per 32 compute nodes
  - 32x32x64 3D torus network
  - Global reduction tree
  - Global barrier and interrupt networks
  - Scalable tree network for I/O
- Software
  - Compute node kernel (CNK)
  - Linux I/O node kernel (ION)
  - MPI
  - Different operating modes







Example: Tokyo Institute of Technology TSUBAME

- System hardware
  - 655 Sun Fire X4600 servers
  - 11,088 processors
    - AMD Opteron dual-core
  - ClearSpeed accelerator
  - InfiniBand network
  - 21 TB memory
  - 42 Sun Fire X4500 servers
  - 1 PB of storage space
- Software
  - SuSE Linux Enterprise Server 9 SP3
  - Sun N1 Grid Engine 6.0
  - Lustre Client Software

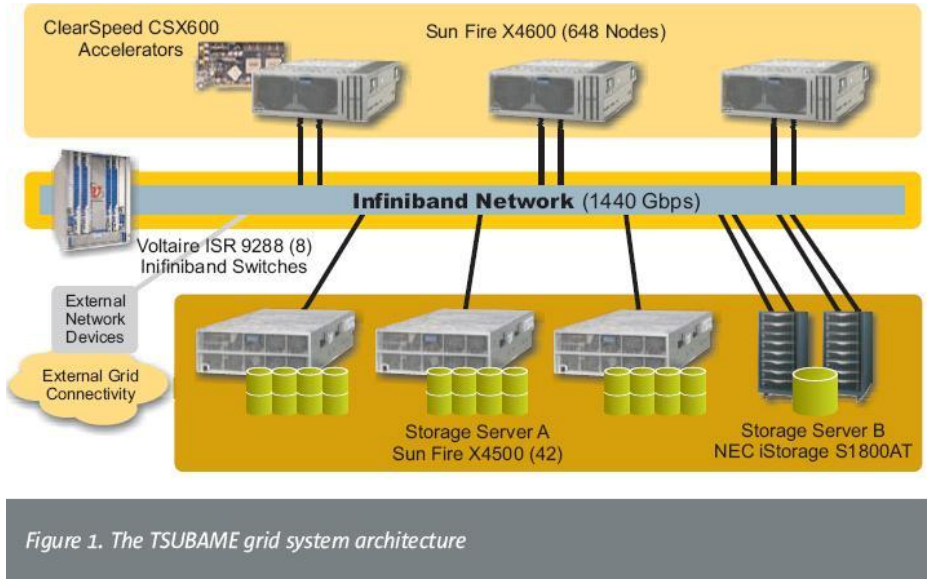
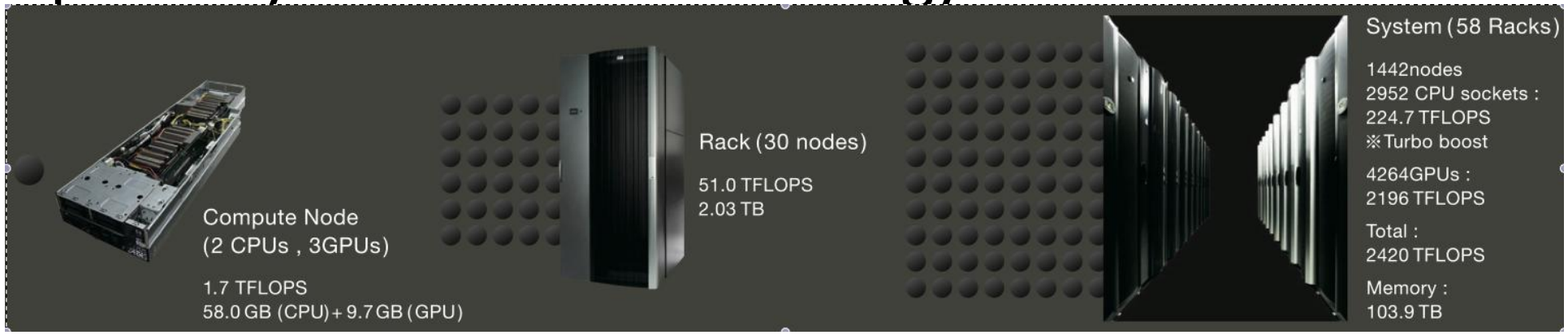


Figure 1. The TSUBAME grid system architecture

Example: Tokyo Institute of Technology TSUBAME2



Thin Node 1408 nodes



HP ProLiant SL390s

GPU : NVIDIA Tesla M2050 (Fermi Core)×3 515GFLOPS VRAM 3GB/GPU

CPU : Intel Xeon X5670 2.93GHz ×2
6 core/socket 76.7 GFLOPS (12cores/node) ※Turbo boost : 3.196GHz

Memory : 58GB DDR3 1333MHz (partly 103GB)

SSD : 60GB ×2 (120GB/node) (partly 120GB ×2 (240GB/node))

Medium Node 24 nodes



HP ProLiant DL580 G7

CPU: Intel Xeon X7550
(Nehalem-EX)
2.0 GHz ×4 sockets
(32cores/node)

GPU: NVIDIA Tesla S1070

Memory: 137 GB (DDR3 1066MHz)

SSD: 120GB ×4 (480GB/node)

Infiniband: QDR

Fat Node 10 nodes



HP ProLiant DL580 G7

CPU: Intel Xeon X7550
(Nehalem-EX)
2.0 GHz ×4 sockets
(32cores/node)

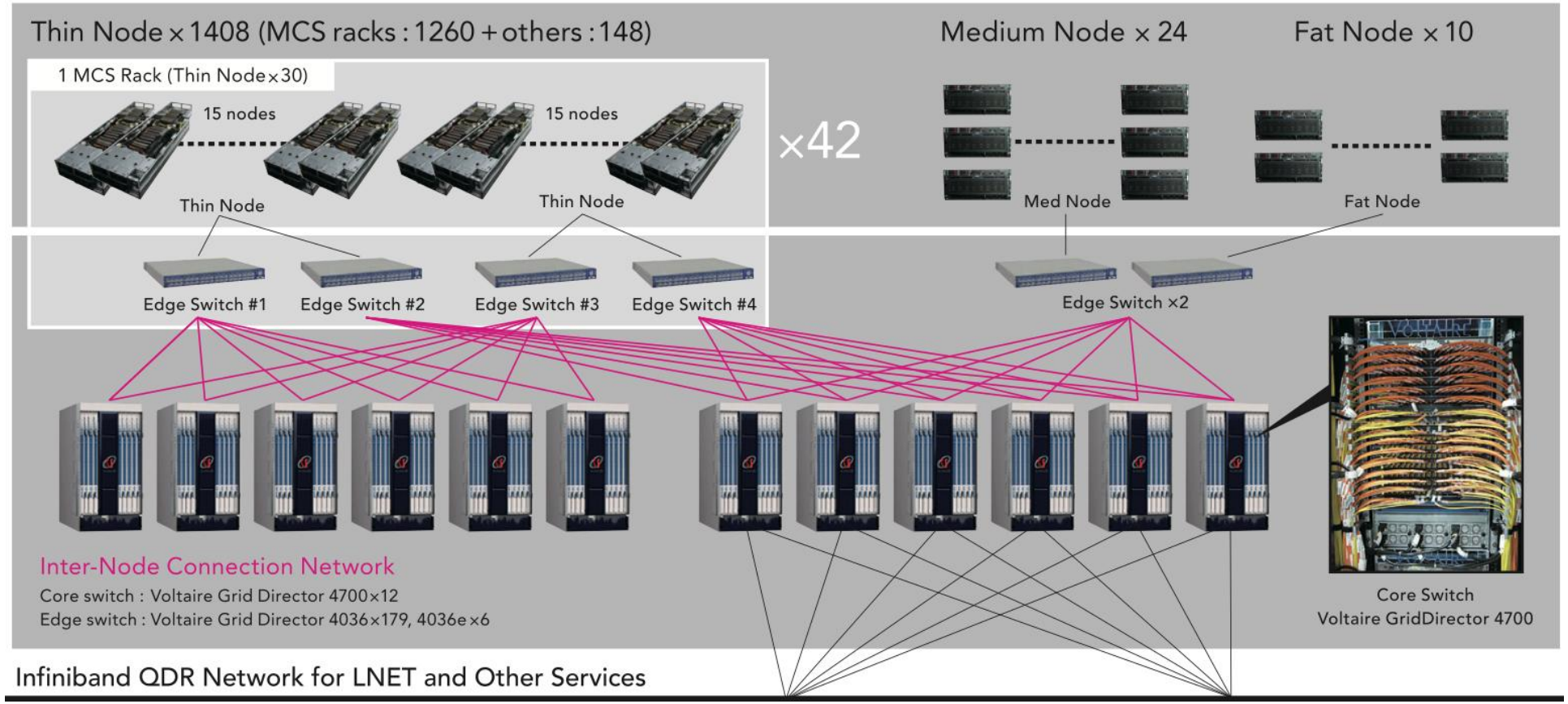
GPU: NVIDIA Tesla S1070

Memory: 274 GB (8 nodes),
548 GB (2 nodes)
DDR3 1066MHz

SSD: 120GB ×5 (600GB/node)

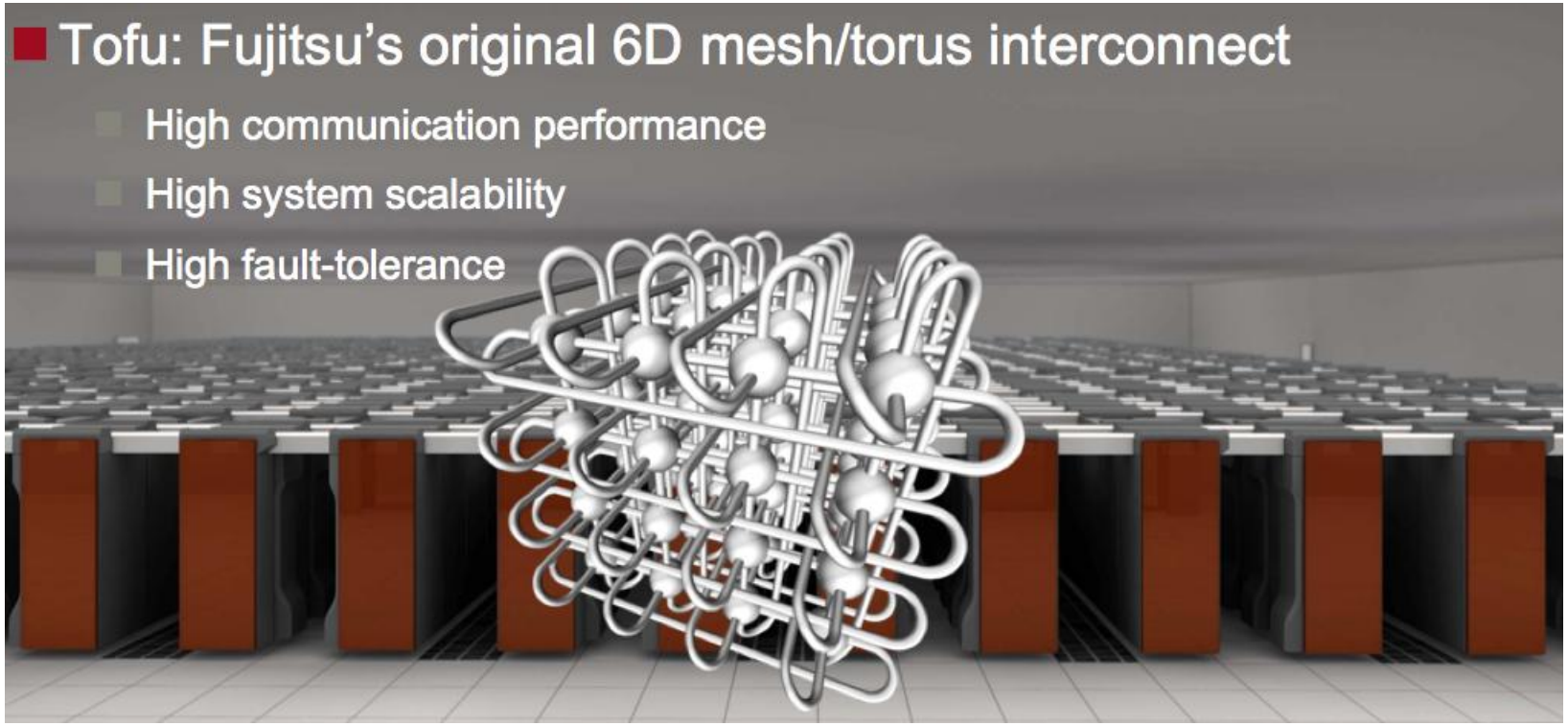
Infiniband: QDR

TSUBAME2 – Interconnect



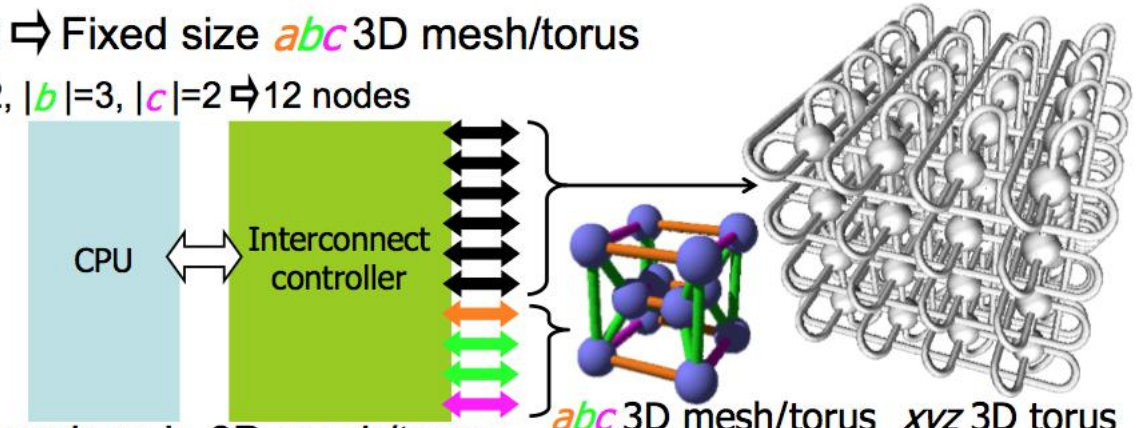
Example: Japanese K Computer – Interconnect

- 80,000 CPUs (SPARC64 VIIIfx), 640,000 cores
- 800 racks
- 8.6 Petaflops (Linpack)

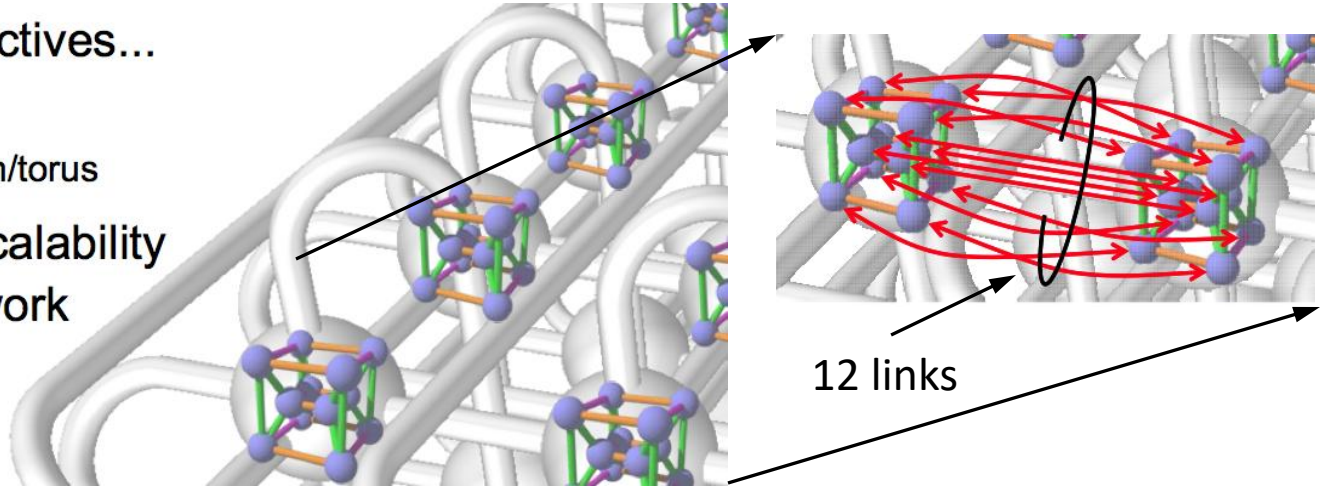


Japanese K Computer – Interconnect

- 6 links ⇨ Scalable *xyz* 3D torus
- 4 links ⇨ Fixed size *abc* 3D mesh/torus
  - |*a*|=2, |*b*|=3, |*c*|=2 ⇨ 12 nodes

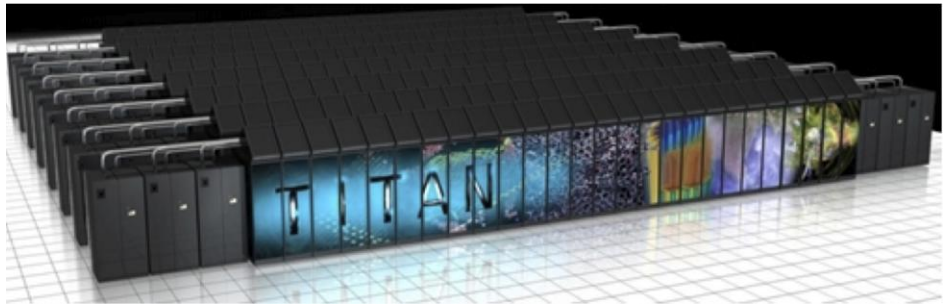


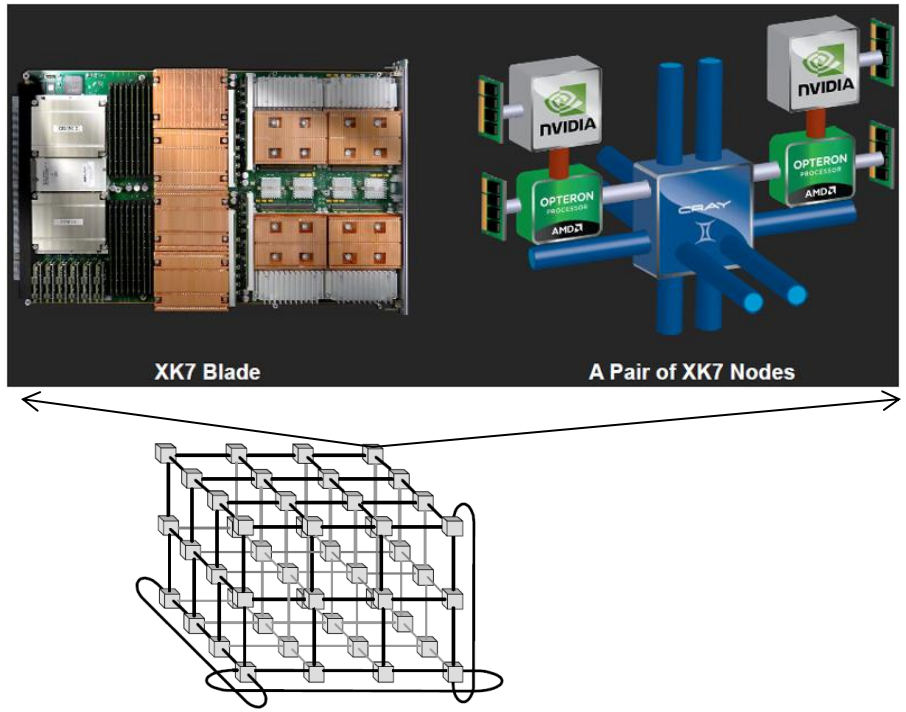
- Total topology is 6D mesh/torus
  - Cartesian product of *xyz* and *abc* mesh/torus
- From the other perspectives...
  - Overlaid twelve *xyz* torus
  - X x Y x Z array of *abc* mesh/torus
- Twelve times higher scalability than the 3D torus network



Example: ORNL Titan (http://www.olcf.ornl.gov/titan)

- Cray XK7
  - 18,688 nodes
  - AMD Opteron
    - 16-core Interlagos
    - 299,008 Opteron cores
  - NVIDIA K20x
    - 18,688 GPUs
    - 50,233,344 GPU cores
- Gemini interconnect
  - 3D torus
- 20+ petaflops





Lecture 3

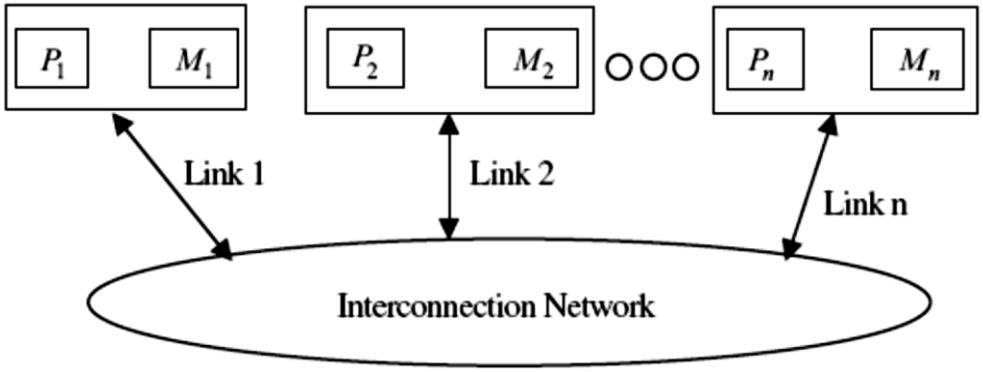
Interconnection Networks for Parallel Computers

References:

- Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar

``**Introduction to Parallel Computing''**, **Addison Wesley, 2003**

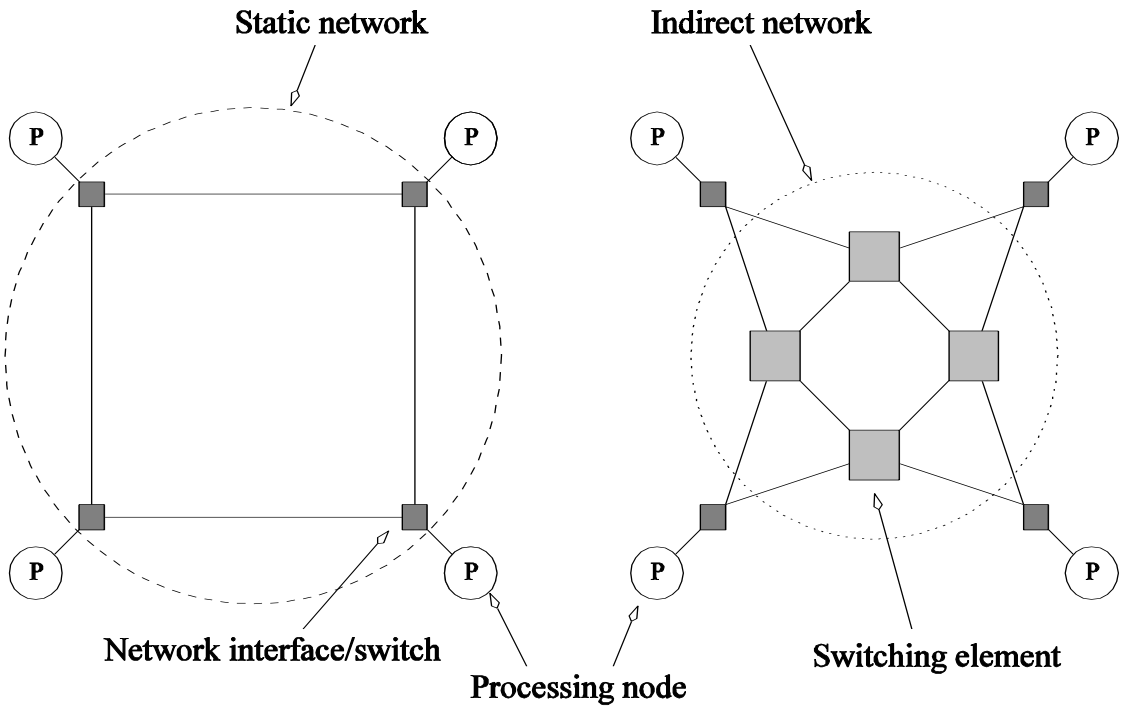
Message passing systems



Interconnection Networks for Parallel Computers

- Interconnection networks carry data between processors and to memory.
- Interconnects are made of
  - *switches* and
  - *links* (wires, fiber).
- Interconnects are classified as static or dynamic.
  - ***Static networks*** consist of point-to-point communication *links* among processing nodes and are also referred to as *direct* networks.
  - ***Dynamic networks*** are built using *switches* and communication *links*. Dynamic networks are also referred to as *indirect* networks.

Static and Dynamic Interconnection Networks



Classification of interconnection networks:
(a) a static network; and (b) a dynamic network.

What Characterizes an Interconnection Network?

- Topology (what)
  - Interconnection structure of the network graph
- Routing Algorithm (which)
  - Restricts the set of paths that messages may follow
  - Many algorithms with different properties
- Switching Strategy (how)
  - How data in a message traverses a route
  - *circuit switching* vs. *packet switching*
- Flow Control Mechanism (when)
  - When a message or portions of it traverse a route
  - What happens when traffic is encountered?

Topological Properties

- Routing distance
  - Number of links on route from source to destination
- Diameter
  - Maximum routing distance
- Average distance
- Partitioned network – bisection width
  - Removal of links resulting in 2 disconnected graphs
  - Minimal cut
- Scaling increment
  - What is needed to grow the network to next valid degree

Interconnection Networks

- Switches map a fixed number of inputs to outputs.
- The total number of ports on a switch is the *degree* of the switch.
- The cost of a switch grows as
  - the square of the degree of the switch,
  - the peripheral hardware linearly as the degree, and
  - the packaging costs linearly as the number of pins (many pins on a port… how many? 2/5/9/25…).

Interconnection Networks: Network Interfaces

- Processors interact to the network via a network interface.
- The network interface may hang off the I/O bus or the memory bus.
- In a physical sense, this distinguishes a cluster from a tightly coupled multicomputer.
- The relative speeds of the I/O and memory buses impact the performance of the network.

Network Topologies

- A variety of network topologies have been proposed and implemented.
- These topologies tradeoff performance for cost.
- Commercial machines often implement hybrids of multiple topologies for reasons of packaging, cost, and available components.

Scalability

- **scalability** is the ability of a system, network, or process,

to handle growing amounts of work

  - informal definition=> in a “graceful” manner
- For example, it can refer to the capability of a system to increase total throughput under an increased load when resources (typically hardware) are added.

  - **Ability to be enlarged to accommodate** growing amounts of work.

    [André B. Bondi, 'Characteristics of scalability and their impact on performance', *Proceedings of the 2nd international workshop on Software and performance,* Ottawa, Ontario, Canada, 2000, pages 195 - 203]

Parallel Architectures Scalability

- **How much can the system be scaled up?**
  - **Adding:**
    - processing units
    - memory units
- **How many processors can be added**

-without degrading the overall system characteristics

  - communication speed,
  - memory access speed,
  - bandwidth, latency,…

-without cost explosion

Bisection bandwidth

- The sum of the capacities of links between two partitions of a network is called the bandwidth of the partition.
- Bisection is the division of a network(graph) into two equal partitions.
- The bisection width is minimum number of links one have to be cut to partition the network in two equal parts.

- The bisection bandwidth of a network topology is the bandwidth available between the two partitions.

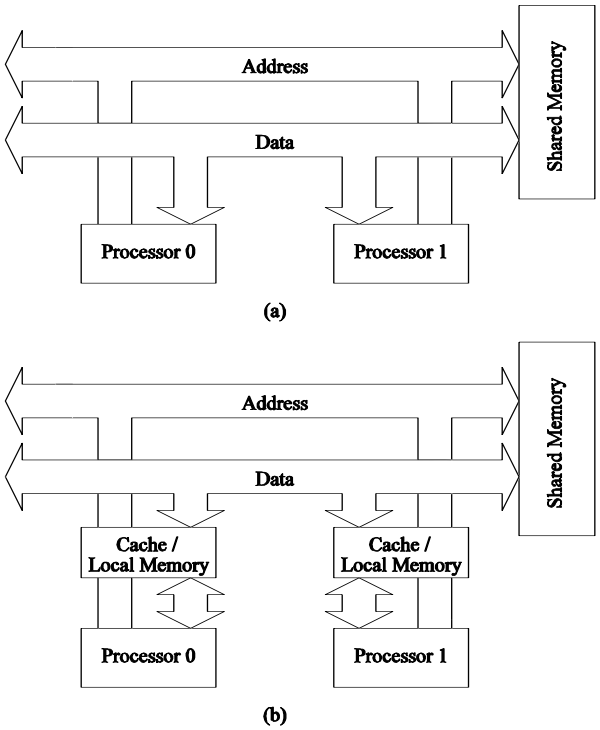
- Bisection bandwidth accounts for the ***bottleneck bandwidth*** of the entire network.

  ->bisection bandwidth represents a very good metric for bandwidth characteristics of the network

Network Topologies: Buses

- Some of the simplest and earliest parallel machines used buses.
- All processors access a common bus for exchanging data.
- The distance between any two nodes is *O(1)* in a bus. The bus also provides a convenient broadcast media.
- However, the bandwidth of the shared bus is a major bottleneck.
- Typical bus-based machines are limited to dozens of nodes.
  Sun Enterprise servers and Intel Pentium based shared-bus multiprocessors are examples of such architectures.

Network Topologies: Buses

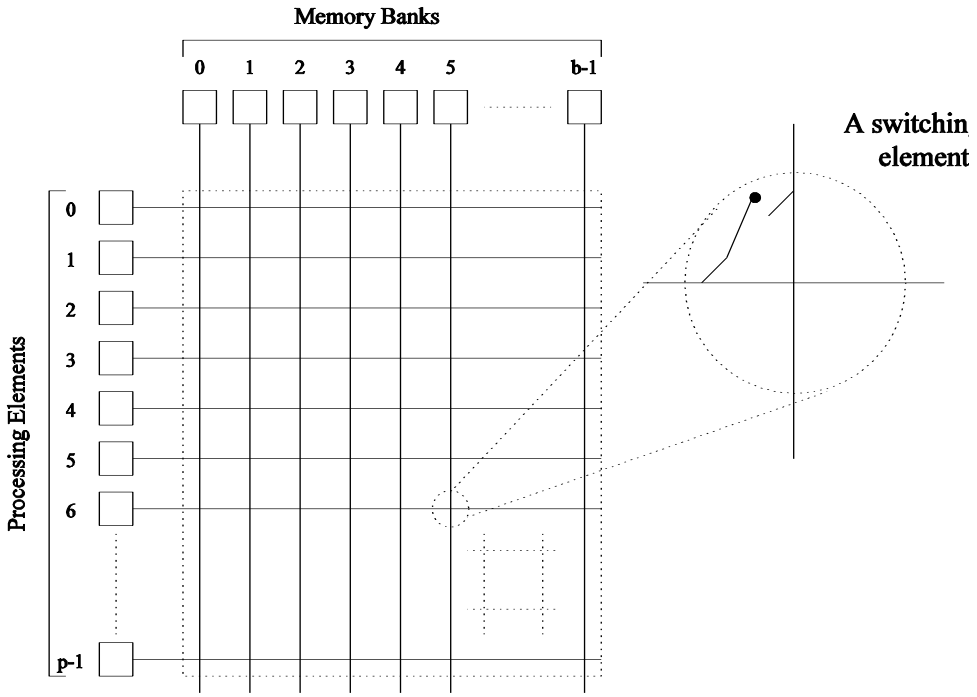


Bus-based interconnects (a) with no local caches; (b) with local memory/caches.

Since much of the data accessed by processors is local to the processor, a local memory can improve the performance of bus-based machines.

Network Topologies: Crossbars

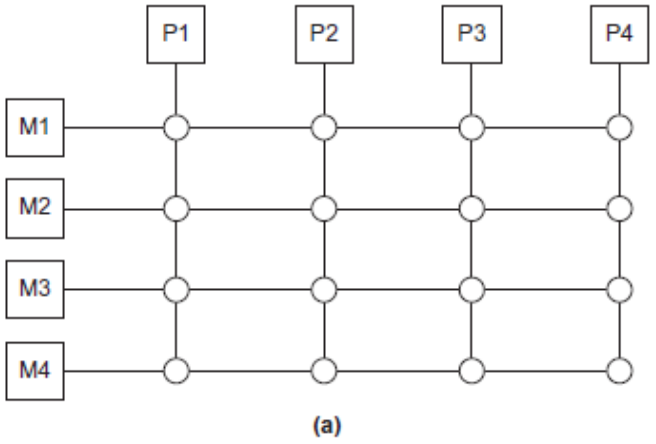
A crossbar network uses an $p \times m$ grid of switches to connect p inputs to m outputs in a non-blocking manner.



Example: a completely non-blocking crossbar network connecting p processors to b memory banks.

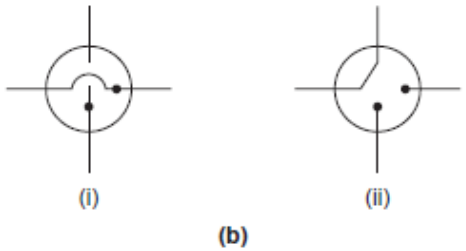
(a)

A crossbar switch connecting 4 processors (P_i) and 4 memory modules (M_j)

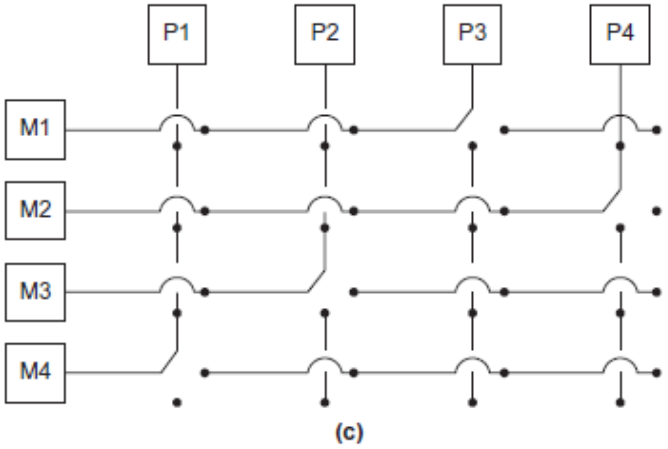


(b)

Configuration of internal switches in a crossbar



(c) Simultaneous memory accesses by the processors



Network Topologies: Crossbars

- The cost of a crossbar of p processors grows as $O(p^2)$.
- This is generally difficult to scale for large values of p.
- Examples of machines that employ crossbars include the Sun Ultra HPC 10000 and the Fujitsu VPP500.



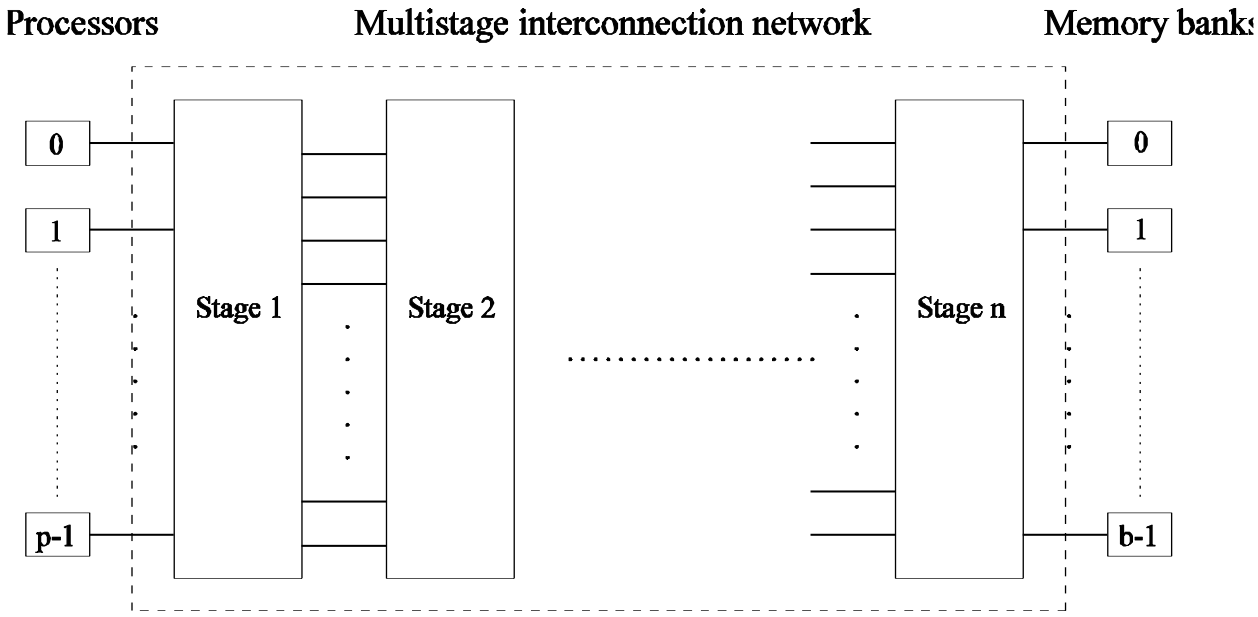
Fujitsu VPP500 Supercomputer Cabinets.

http://www.computerhistory.org/collections/catalog/102685357

Network Topologies: Multistage Networks

- Crossbars have excellent performance scalability but poor cost scalability.
- Buses have excellent cost scalability, but poor performance scalability.
- Multistage interconnects strike a compromise between these extremes.

Network Topologies: Multistage Networks



The schematic of a typical multistage interconnection network.

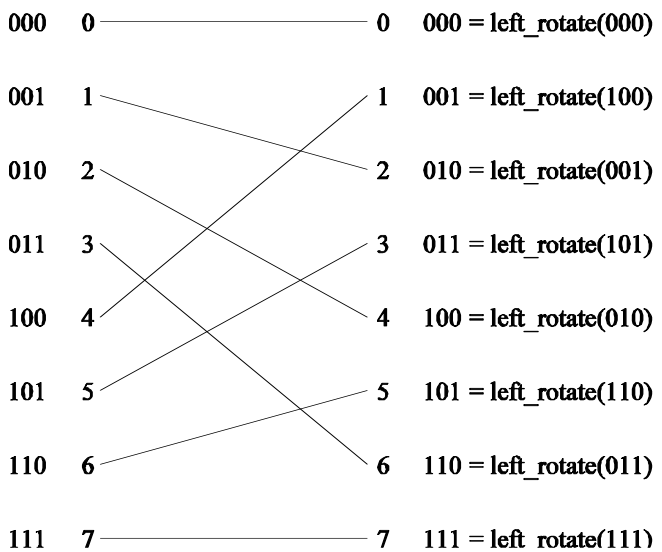
Network Topologies: Multistage Omega Network

- One of the most commonly used multistage interconnects is the Omega network.
- This network consists of *log p* stages, where *p* is the number of inputs/outputs.
- At each stage, input *i* is connected to output *j* if:

$$j = \begin{cases} 2i, & 0 \leq i \leq p/2 - 1 \\ 2i + 1 - p, & p/2 \leq i \leq p - 1 \end{cases}$$

Network Topologies: Multistage Omega Network

Each stage of the Omega network implements a perfect shuffle as follows:



A perfect shuffle interconnection for eight inputs and outputs.

Network Topologies: Multistage Omega Network

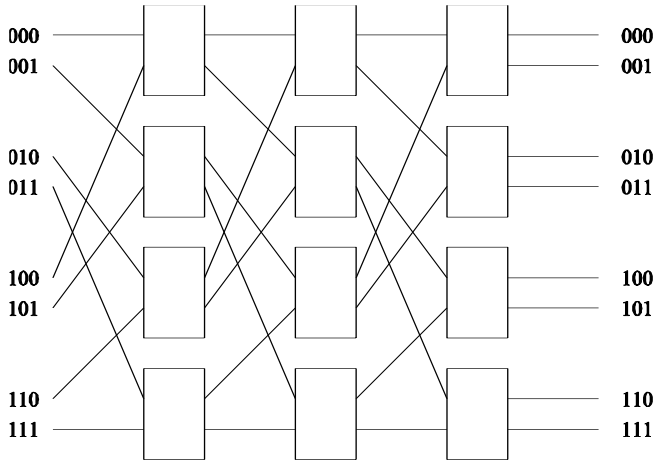
- The perfect shuffle patterns are connected using 2×2 switches.
- The switches operate in two modes – crossover or passthrough.



Two switching configurations of the 2 × 2 switch:
(a) Pass-through; (b) Cross-over.

Network Topologies: Multistage Omega Network

A complete Omega network with the perfect shuffle interconnects and switches can now be illustrated:



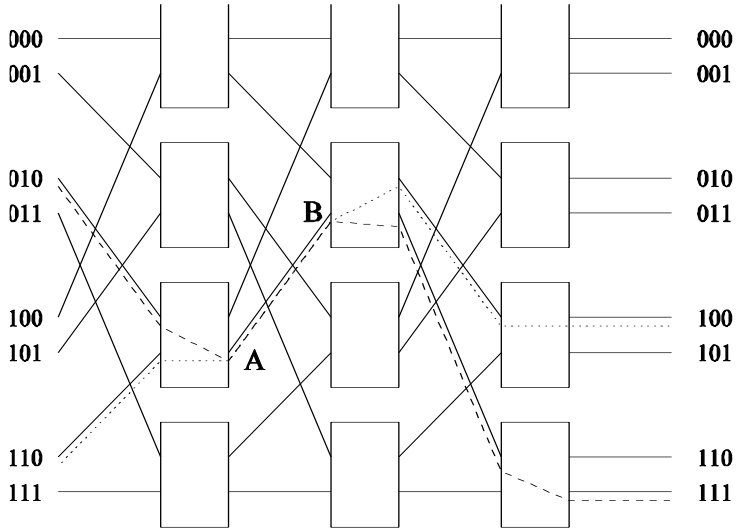
A complete omega network connecting eight inputs and eight outputs.

An omega network has $p/2 \times \log p$ switching nodes, and the cost of such a network grows as $(p \log p)$.

Network Topologies: Multistage Omega Network – Routing

- Let *s* be the binary representation of the source and *d* be that of the destination processor.
- The data traverses the link to the first switching node.
  - ***If the most significant bits of s and d are the same, then the data is routed in pass-through mode by the switch; else, it switches to crossover.***
- This process is repeated for each of the *log p* switching stages.
- Note that this is not a non-blocking switch.

Network Topologies: Multistage Omega Network – Routing



An example of blocking in omega network: one of the messages (010 to 111 or 110 to 100) is blocked at link AB.

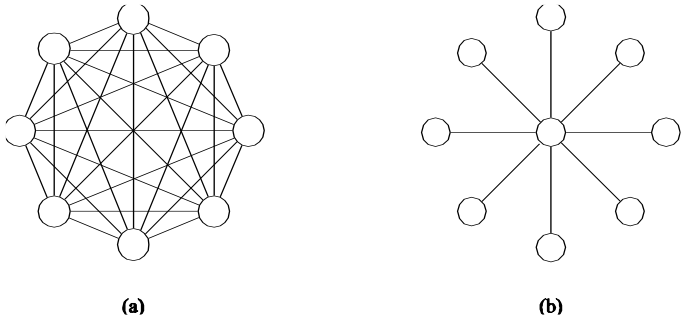
Static Interconnection Networks

Network Topologies: Completely Connected Network

- Each processor is connected to every other processor.
- The number of links in the network scales as $O(p^2)$.
- While the performance scales very well, the hardware complexity is not realizable for large values of p.
- In this sense, ***these networks are static counterparts of crossbars.***

Network Topologies: Completely Connected and Star Connected Networks

Example of an 8-node completely connected network.



(a) A completely-connected network of eight nodes;
(b) a star connected network of nine nodes.

Network Topologies: Star Connected Network

- Every node is connected only to a common node at the center.
- Distance between any pair of nodes is *O(1).*
  - However, ***the central node becomes a bottleneck***.
- In this sense,
  - ***star connected networks are static counterparts of buses.***

Network Topologies: Linear Arrays, Meshes, and *k-d* Meshes

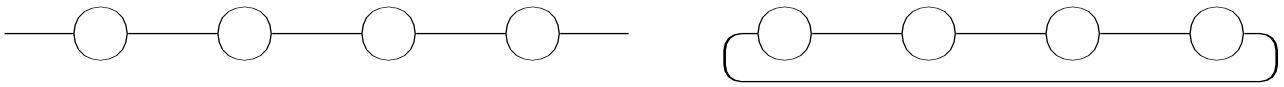
- In a linear array, each node has two neighbors, one to its left and one to its right.
  - If the nodes at either end are connected, we refer to it as a 1-D torus or a ring.

- A generalization to 2 dimensions has nodes with 4 neighbors, to the north, south, east, and west.

- A further generalization to *d* dimensions has nodes with *2d* neighbors.

- A special case of a *d*-dimensional mesh is a hypercube.
  - Here, $d = \log p$, where p is the total number of nodes.

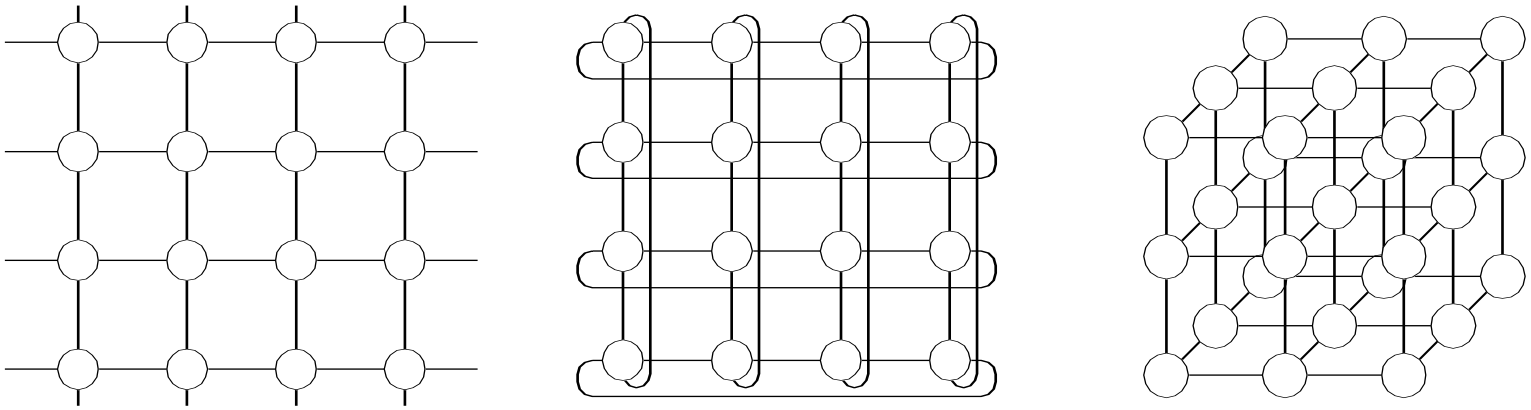
Network Topologies: Linear Arrays



(a) (b)

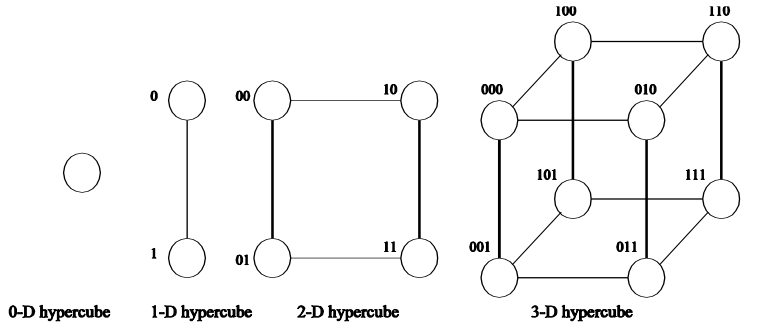
Linear arrays: (a) with no wraparound links; (b) with wraparound link.

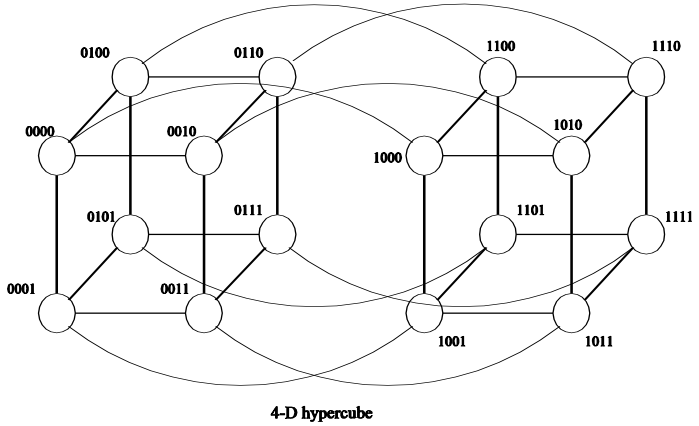
Network Topologies: Two- and Three Dimensional Meshes



Two and three dimensional meshes: (a) 2-D mesh with no wraparound; (b) 2-D mesh with wraparound link (2-D torus); and (c) a 3-D mesh with no wraparound.

Network Topologies: Hypercubes and their Construction



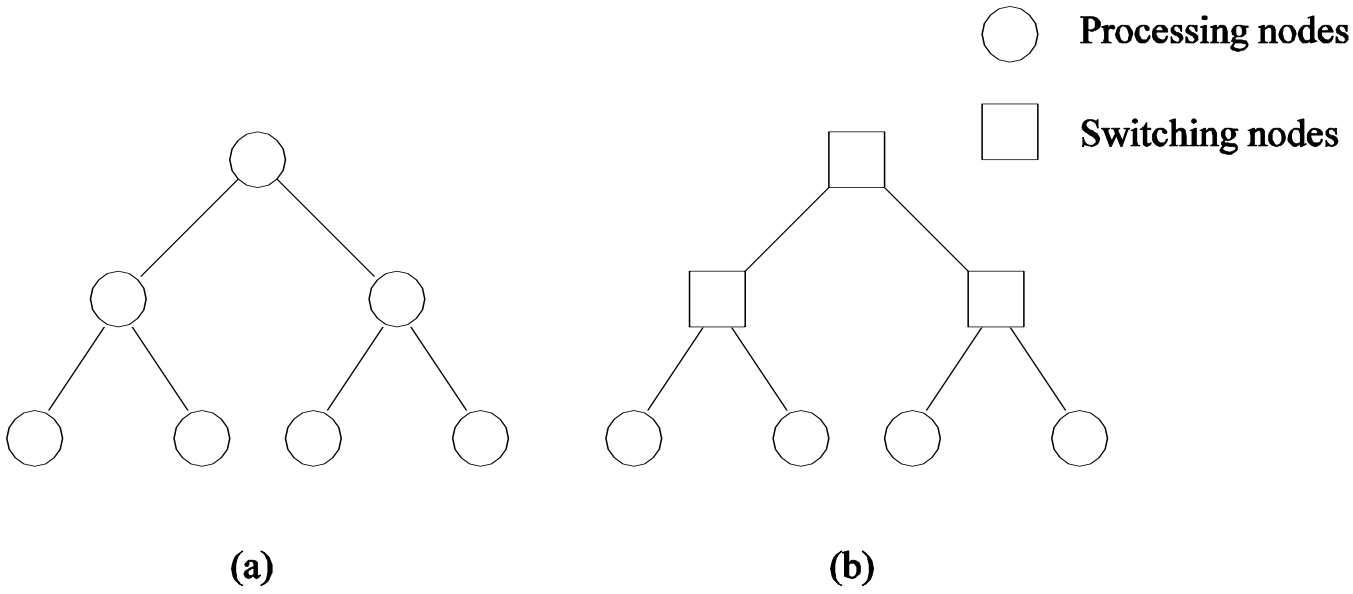


Construction of hypercubes from hypercubes of lower dimension.

Network Topologies: Properties of Hypercubes

- The distance between any two nodes is at most *log p*.
- Each node has *log p* neighbors.
- ***The distance between two nodes is given by the number of bit positions at which the two nodes differ.***

Network Topologies: Tree-Based Networks

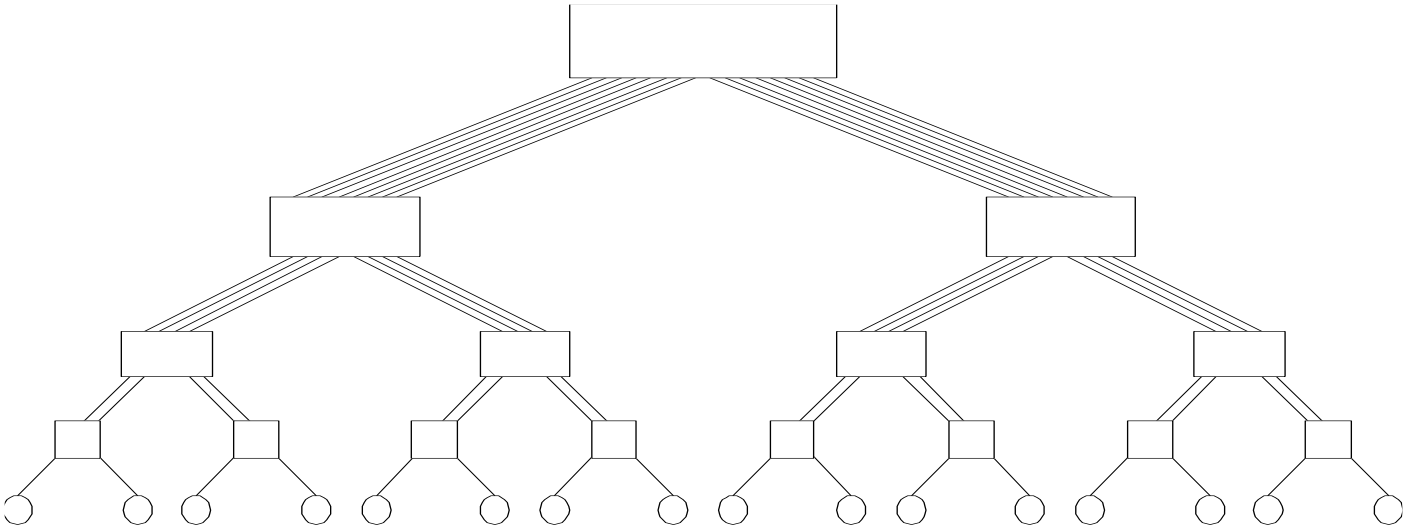


Complete binary tree networks: (a) a static tree network; and (b) a dynamic tree network.

Network Topologies: Tree Properties

- The distance between any two nodes is no more than *2logp*.
- Links higher up the tree potentially carry more traffic than those at the lower levels.
- For this reason, a variant called a fat-tree, fattens the links as we go up the tree.
- Trees can be laid out in 2D with no wire crossings. This is an attractive property of trees.

Network Topologies: Fat Trees



A fat tree network of 16 processing nodes.

Evaluating Static Interconnection Networks

- ***Diameter:*** The distance between the farthest two nodes in the network. The diameter of a linear array is $p - 1$, that of a mesh is $2(\sqrt{p} - 1)$, that of a tree and hypercube is $\log p$, and that of a completely connected network is $O(1)$.
- ***Bisection Width:*** The minimum number of wires you must cut to divide the network into two equal parts. The bisection width of a linear array and tree is 1, that of a mesh is $\sqrt{p}$, that of a hypercube is $p/2$ and that of a completely connected network is $p^2/4$.
  - It determines the maximum possible bandwidth between those two halves, serving as a fundamental bottleneck for data throughput, system latency, and overall parallel processing performance.
- ***Cost:*** The number of links or switches (whichever is asymptotically higher) is a meaningful measure of the cost.
  - However, a number of other factors, such as the ability to layout the network, the length of wires, etc., also factor into the cost.

Evaluating Static Interconnection Networks

Network	Diameter	BisectionWidth	Arc Connectivity	Cost (No. of links)
Completely-connected	1	$p^2/4$	$p-1$	$p(p-1)/2$
Star	2	1	1	$p-1$
Complete binary tree	$2\log((p+1)/2)$	1	1	$p-1$
Linear array	$p-1$	1	1	$p-1$
2-D mesh, no wraparound	$2(\sqrt{p}-1)$	$\sqrt{p}$	2	$2(p-\sqrt{p})$
2-D wraparound mesh	$2\lfloor\sqrt{p}/2\rfloor$	$2\sqrt{p}$	4	$2p$
Hypercube	$\log p$	$p/2$	$\log p$	$(p\log p)/2$
Wraparound *k*-ary *d*-cube	$d\lfloor k/2\rfloor$	$2k^{d-1}$	$2d$	dp

Evaluating Dynamic Interconnection Networks

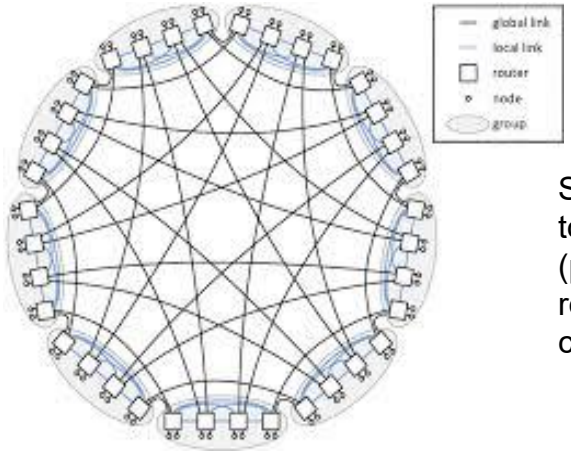
Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Crossbar	1	p	1	p^2
Omega Network	$\log p$	$p/2$	2	$p/2$
Dynamic Tree	$2 \log p$	1	2	$p - 1$

A graph is k arc connected if
k directed disjoint paths between any two nodes u and v exist.

Examples:

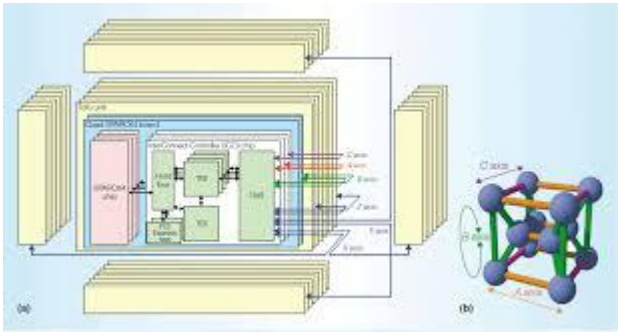
El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE
DOE/NNSA/LLNL
United States

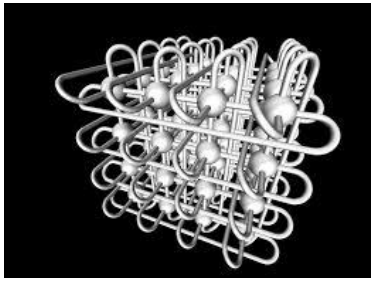
HPE Slingshot is a high-performance, low-latency interconnect technology designed specifically for exascale supercomputing, AI, and data-intensive workloads.
It utilizes a **Dragonfly topology** to provide high-bandwidth, scalable communication across large clusters while maintaining interoperability with standard Ethernet.



Sample Dragonfly topology with h=2 (p=2, a=4), 36 routers and 72 compute nodes.

- **Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu
  RIKEN Center for Computational Science
  Japan
  - The **Tofu interconnect** uses a **6D mesh/torus topology** in which each cubic fragment of the network has the embeddability of a **3D torus graph**, allowing users to run **multiple topology-aware applications.**





Connection to design and programming

Mapping Techniques for Graphs

- Often, we need to embed a known communication pattern into a given interconnection topology.
- We may have an algorithm designed for one network, which we are porting to another topology.

For these reasons, it is useful to understand mapping between graphs.

Mapping Techniques for Graphs: Metrics

- When mapping a graph *G(V,E)* into *G'(V',E'),* the following metrics are important:
  - The maximum number of edges mapped onto any edge in *E'* is called the *congestion* of the mapping.
  - The maximum number of links in *E'* that any edge in *E* is mapped onto is called the *dilation* of the mapping.
  - The ratio of the number of nodes in the set *V'* to that in set *V* is called the *expansion* of the mapping.

Embedding a Linear Array into a Hypercube

- A linear array (or a ring) composed of 2^d nodes (labeled 0 through $2^d - 1$) can be embedded into a *d*-dimensional hypercube by mapping node *i* of the linear array onto node *G(i, d)* of the hypercube. The function *G(i, x)* is defined as follows:

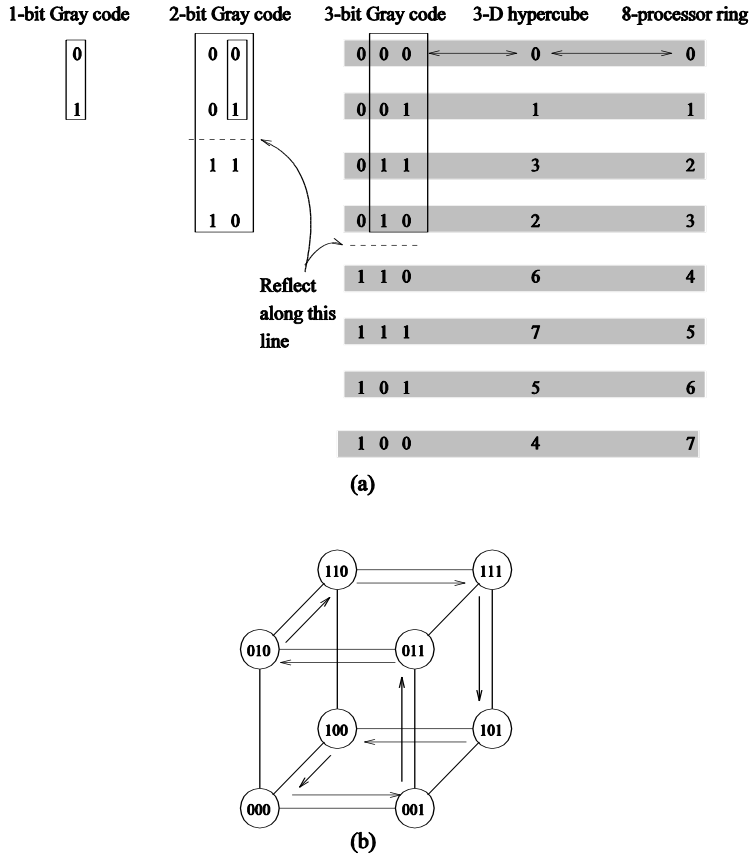
$$
\begin{aligned}
G(0,1) &= 0 \\
G(1,1) &= 1 \\
G(i,x+1) &= \begin{cases} G(i,x), & i < 2^x \\ 2^x + G(2^{x+1} - 1 - i, x), & i \geq 2^x \end{cases}
\end{aligned}
$$

Embedding a Linear Array into a Hypercube

The function *G* is called the *binary reflected Gray code* (RGC).

Since adjoining entries (*G(i, d)* and *G(i + 1, d)*) differ from each other at only one bit position, corresponding processors are mapped to neighbors in a hypercube. Therefore, the congestion, dilation, and expansion of the mapping are all 1.

Embedding a Linear Array into a Hypercube: Example



(a) A three-bit reflected Gray code ring; and (b) its embedding into a three-dimensional hypercube.

Embedding a Mesh into a Hypercube

- A $2^r \times 2^s$ wraparound mesh can be mapped to a 2^{r+s}-node hypercube by :

mapping node

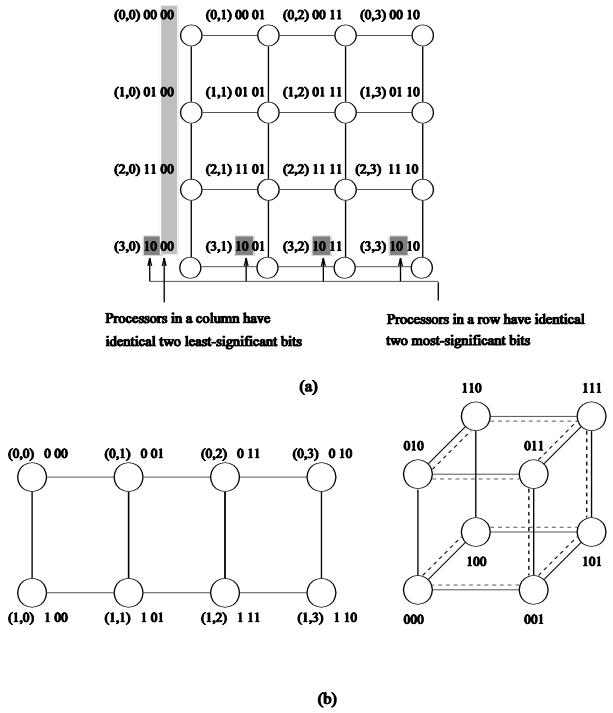
(i, j) of the mesh

onto node

G(i, r− 1*)* || *G(j, s − 1)* of the hypercube

(where || denotes concatenation of the two Gray codes).

Embedding a Mesh into a Hypercube



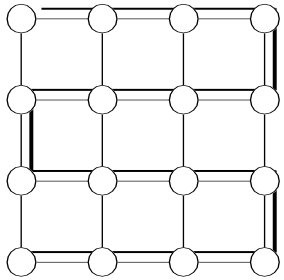
(a) A 4 × 4 mesh illustrating the mapping of mesh nodes to the nodes in a four-dimensional hypercube; and (b) a 2 × 4 mesh embedded into a three-dimensional hypercube.

Once again, the congestion, dilation, and expansion of the mapping is 1.

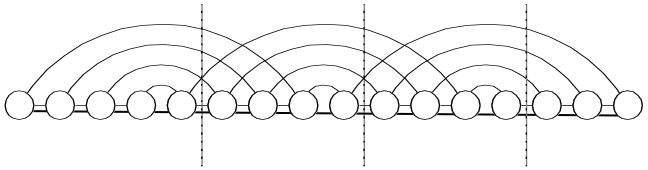
Embedding a Mesh into a Linear Array

- Since a mesh has more edges than a linear array, we will not have an optimal congestion/dilation mapping.
- We first examine the mapping of a linear array into a mesh and then invert this mapping.
- This gives us an optimal mapping (in terms of congestion).

Embedding a Mesh into a Linear Array: Example



(a) Mapping a linear array into a 2D mesh (congestion 1).



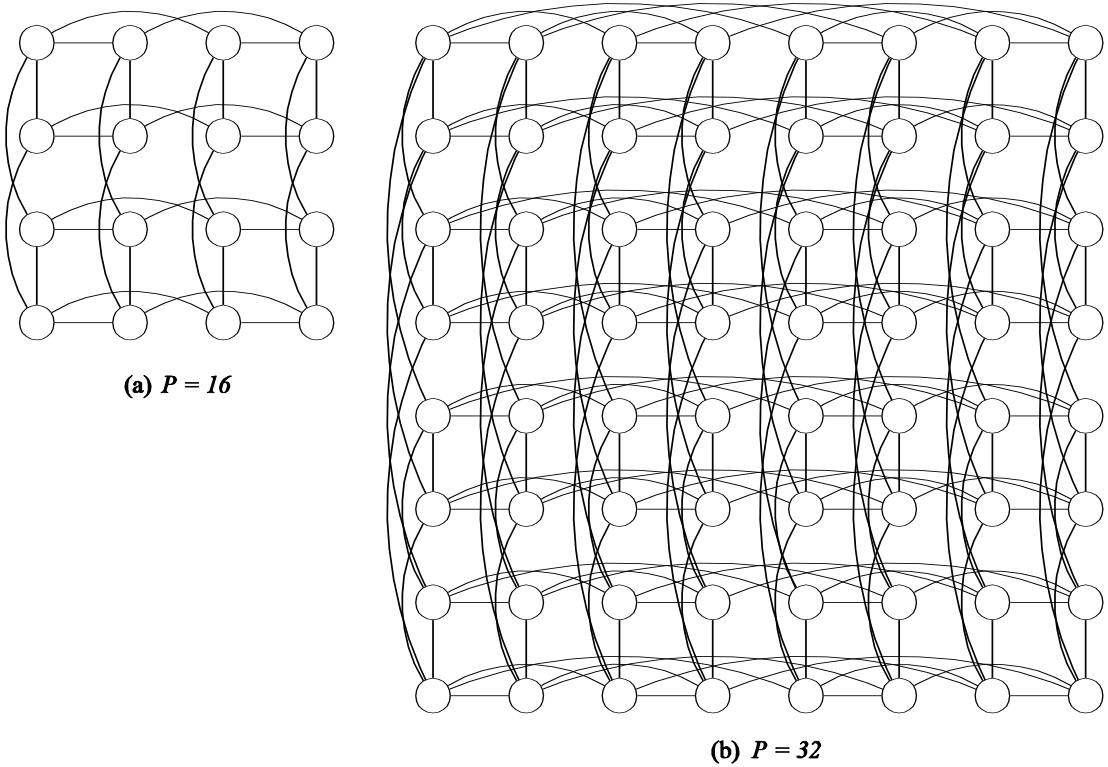
(b) Inverting the mapping - mapping a 2D mesh into a linear array (congestion 5)

(a) Embedding a 16 node linear array into a 2-D mesh; and (b) the inverse of the mapping. Solid lines correspond to links in the linear array and normal lines to links in the mesh.

Embedding a Hypercube into a 2-D Mesh

- Each $\sqrt{p}$ node sub-cube of the hypercube is mapped to a $\sqrt{p}$ node row of the mesh.
- This is done by inverting the linear-array to hypercube mapping.
- This can be shown to be an optimal mapping.

Embedding a Hypercube into a 2-D Mesh: Example

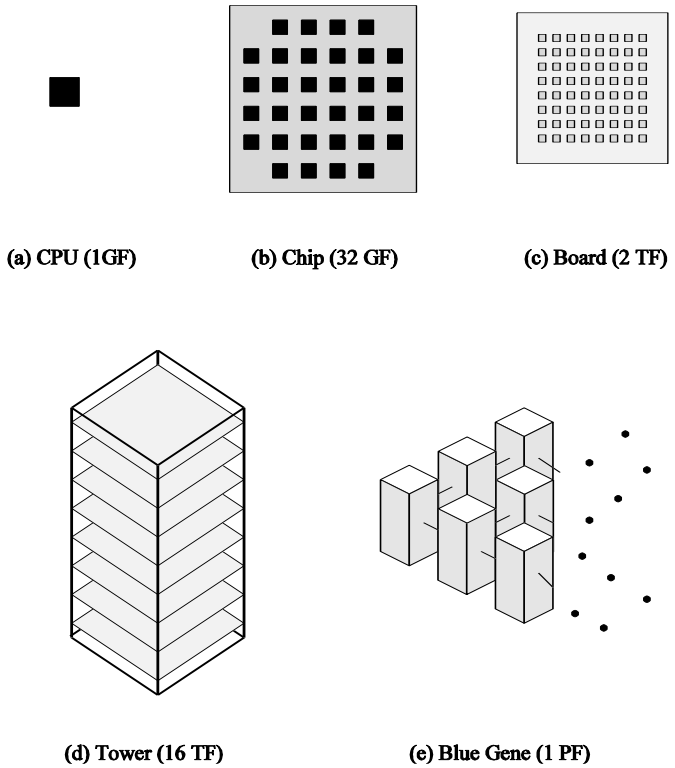


(a) $P = 16$

(b) $P = 32$

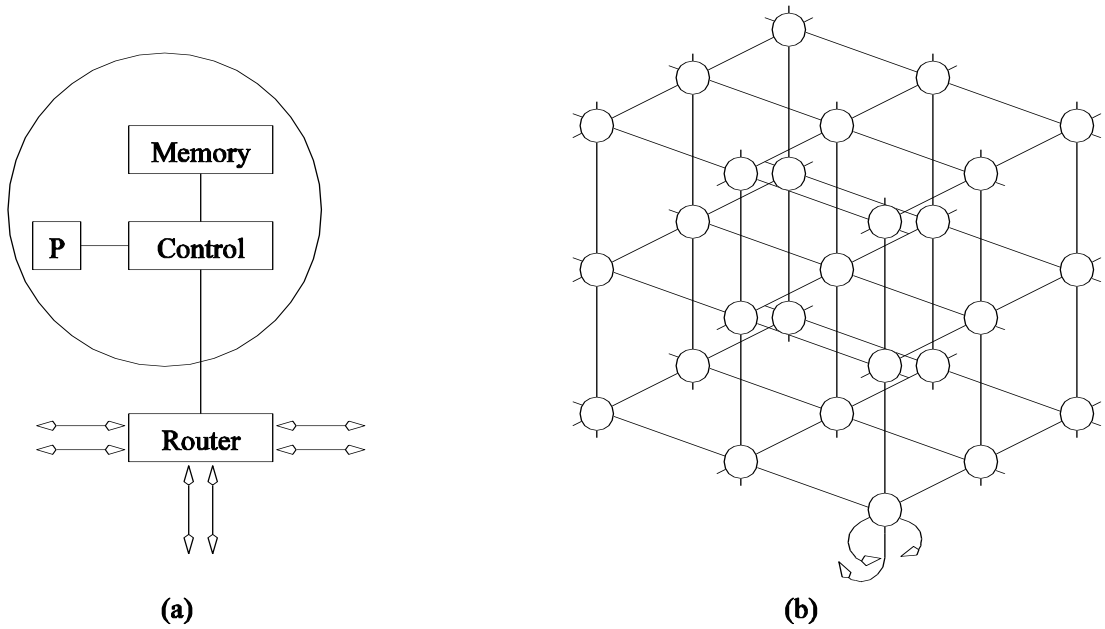
Embedding a hypercube into a 2-D mesh.

Case Studies:
The IBM Blue-Gene Architecture



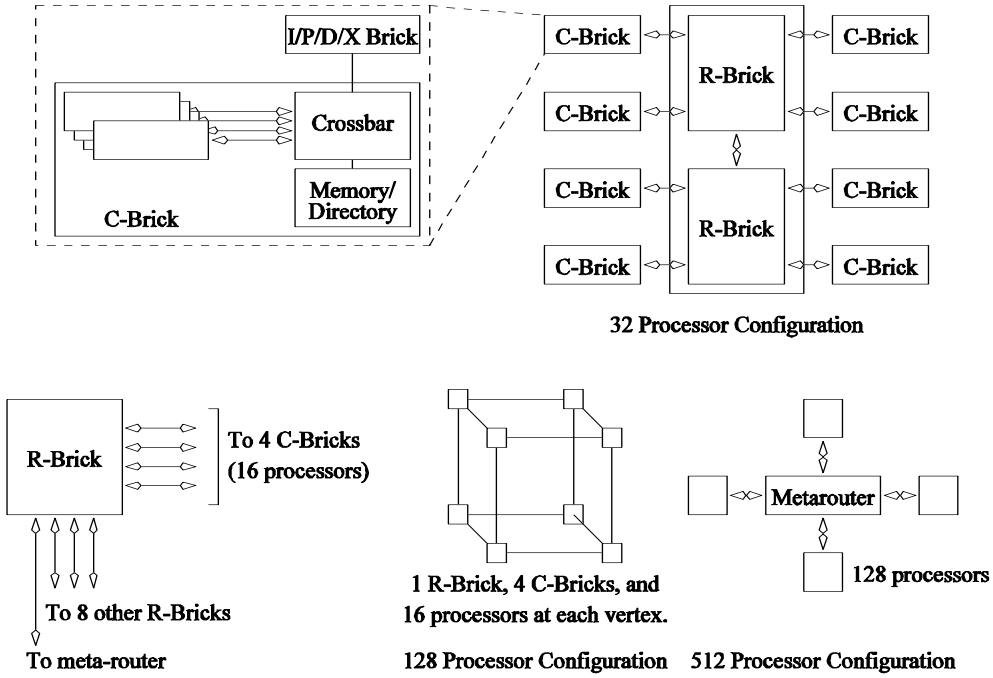
The hierarchical architecture of Blue Gene.

Case Studies:
The Cray T3E Architecture



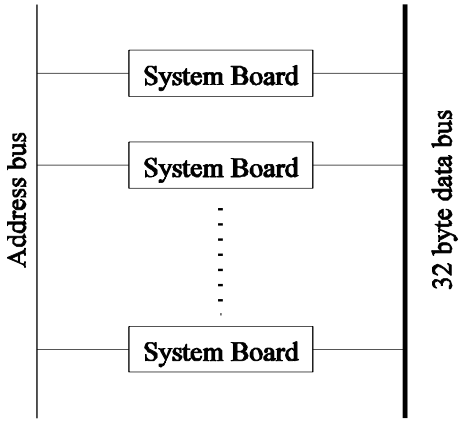
Interconnection network of the Cray T3E:
(a) node architecture; (b) network topology.

Case Studies:
The SGI Origin 3000 Architecture

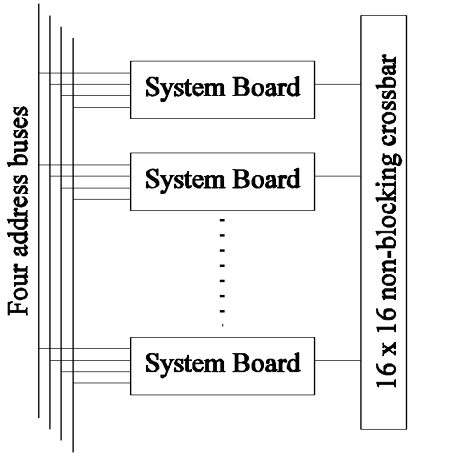


Architecture of the SGI Origin 3000 family of servers.

Case Studies:
The Sun HPC Server Architecture



Sun Ultra 6000 (6 - 30 processors)



Starfire Ultra 1000 (up to 64 processors)

Architecture of the Sun Enterprise family of servers.

Lecture 4

Routing

Communication costs

Taxonomy of communication operations

- **One-to-one communication:**
  - *single-pair communication*: there is just one isolated communicating pair (no problems with deadlocks, congestions,…)
    - shortest path routing in usual interconnection networks.
  - *many one-to-one communications*: several pairs exchange information simultaneously.
  - *permutation routing:* every node is a source of one message and a destination for another message at the same time. This communication pattern implies a permutation among nodes.
- **Collective communication operations**
  - **One-to-many communication:**
    - One node is the sender and several or all nodes are receivers. If the same information is disseminated in multiple copies among the other nodes, these patterns are known as *multicast* and *one-to-all broadcast* communication operations.
    - If the information sent to each destination node is different, the pattern is called *one-to-all scatter*.
  - **All-to-all communication:**
    - *all-to-all broadcast*
    - *scatter*

Routing in Message Passing Networks

- **Routing** is defined as the techniques to select a path over the network channels for sending a message.
  - routing involves the identification of a set of permissible paths that may be used by a message to reach its destination, and a function, **h,** that selects one path from the set of permissible paths.
- Variants:
  - *adaptive* if, for a given source and destination pair, the path taken by the message depends on network conditions, such as network congestion.
  - *oblivious* <-> *deterministic* routing technique=>determines the path using only the source and destination, regardless of the network conditions.

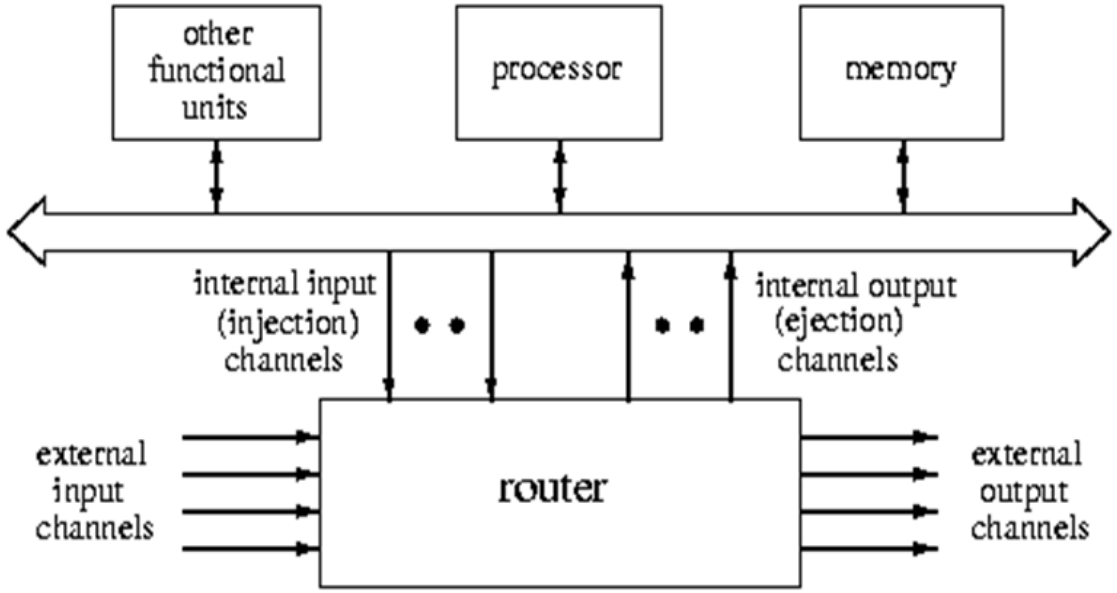
Routing

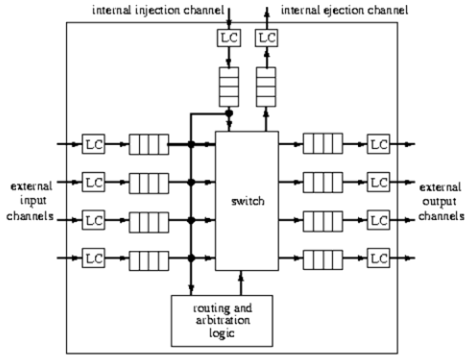
- Given a current node and a destination node, the routing algorithm chooses the next port and channel on which to send out the message.

- A routing algorithm is a function R: N x N -> C.
  - A switch usually uses one of the first three mechanisms to determine the output channel from info in the packet header.
  - A switch needs to route a packet every few cycles, so it needs to be fast.
  - In regular topologies, simple arithmetic suffices.

Router Architecture

Example: standard bus-based computer augmented with a router





router:
a HW coprocessor implementing lower levels of communication protocol. It comprises of a switch, buffers, and routing and arbitration unit

external channels:
interconnect routers and define the topology of a connected direct interconnection network.

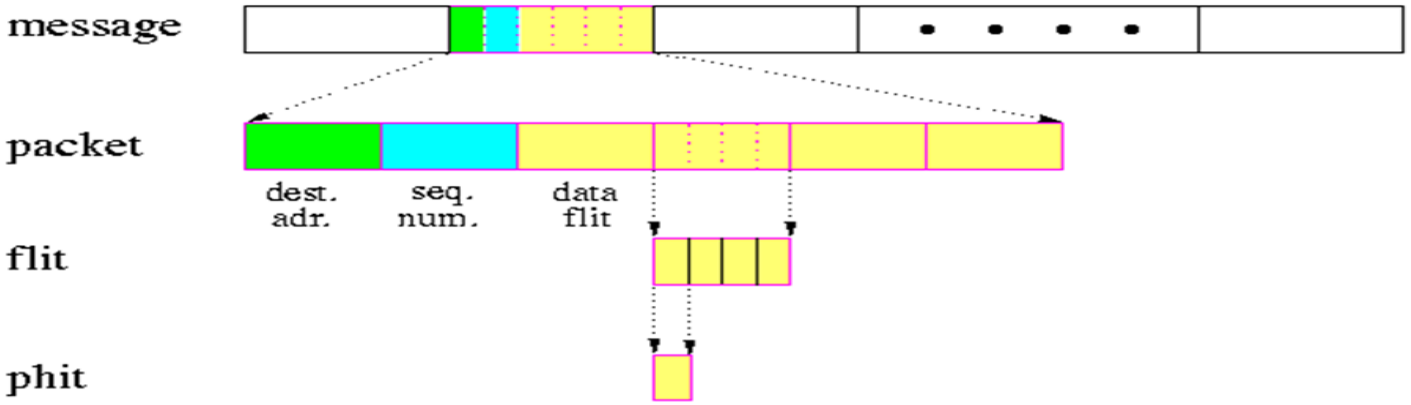
adjacent nodes:
nodes with directly connected routers.

internal channels:
connect the router to the local processing node and implement the physical HW interface between them.

1-port architecture: one injection and one ejection channel

k*-port** architecture: ***k injection and ***k*** ejection channels,

all-port architecture:
of injection channels = **#** of external output channels and
of ejection channels = **#** of external input channels.



- **channel:** consists *of buffers, link controllers, and communication medium* (e.g., coax cable).
- **buffer:** FIFO memory for storing *one* or *several* **units of communication** in transit. They are required for storing transferred data until the next channel is reserved and can overtake them. A router can implement
- **input buffering:** buffers associated with input external channels only
- **output buffering:** buffers associated with output external channels only
- **link controller:** (LC) implements the physical channel flow control communication protocol between neighboring routers (such as handshaking).
- **message:** the unit of communication from the programmer's perspective. Its size is limited only by the user memory space
- **packet:** fixed-size small unit of communication containing *routing information* (e.g., a destination address) and **sequencing** information in its **header**. Its size is of order hundreds or thousands of bytes or words. It consists of *header flits* and *data flits*.
- **flit:** small unit of information at link layer, of size of one or several words. Flits can be of several types and flit exchange protocol typically requires several cycles.
- **phit:** the smallest physical unit of information at the physical layer, which is transferred across one physical link in one cycle.

Routing algorithms

Variants:

A) *centralized* –the routing function is determined by a *centralized controller*. Typically, in SIMD machines.

B) *source* - the routing decisions regarding the entire path are made before sending the message by the source node.

C) *distributed* - each node decides by itself which channel should be used to forward the incoming message. Headers contain only *destination* address. Advantageous for symmetric or regular topologies, since all routers use the same routing algorithm

D) *hybrid* (multiphase) routing: the source node precalculates only addresses of some intermediate nodes and the precise paths between them is determined in a distributed manner by routers.

Implementation of routing algorithm

There are two basic approaches.

- **finite-state machine:** HW or SW algorithm implementing some finite-state automaton.
- **table lookup:** source nodes and/or routers keep routing tables.
  - The number of entries is *O(N)*, where *N* is # of nodes in the network.
  - Source node routing tables contain the whole path specifications, whereas in case of distributed routing the lookup tables just say which output channel(s) correspond(s) to every destination.
  - Routing tables can be either **statically** or **dynamically maintained**.
    - automatic recomputation of routing tables whenever the interconnection network changes, for example, due to deleting a node.

Adaptivity: Routing decisions can be based not only on the addresses, but also on other information.

Non-adaptive algorithms

Deterministic routing algorithms

- always generate the same single routing path for given pair of source and destination addresses, typically a **shortest** one.
  - For source routing, the source node implements pure routing function returning a *unique path* without considering any information about the traffic.
  - For distributed routing, routers make *unique* decisions in every intermediate node.
- Deterministic distributed routing is used in most commercially available parallel machines. It is simple, fast, and performs well under uniform traffic assumption.
- Examples: In mesh-based machines, it is the *dimension-order routing* (XY, XYZ, *e*-cube). It is known to be deadlock-free in meshes and is easy to implement in HW. Nonuniform traffic requires some degree of adaptivity.

Oblivious routing algorithms

- The routing decision are *oblivious to the status* of network traffic.
- Any deterministic routing is oblivious, but oblivious routing is not necessarily deterministic.
- For example, there may exists several shortest paths between the source and destination and the routing algorithm selects **randomly** or **cyclically** one of them.

Oblivious nondeterministic algorithms can distribute uniformly the communication load in situations where adaptive solutions are too expensive or slow.

Adaptive routing algorithms

Use information about network traffic and/or channel status to avoid congested or faulty regions of the network.

- *Source-node adaptive routing* is useful only when the traffic status does not change too fast, otherwise the source node may have obsolete information, and a global status is costly to monitor.
- *Adaptive* routing can easily be combined with *distributed* routing, since routers can react on local congestions using some heuristics, history tables, or probing the neighborhood.
- Sometimes a simple randomized decisions of ``hot-potato'' type suffice to avoid blocking due to busy output channels (Chaos router).
- While being able to avoid deadlocks, adaptive routing must take care of *livelocks*.
- Adaptive routing can be decomposed into two functions:
  - **routing function:** delivers a **set** of possible output channels,
  - **output selection function:** selects one of free output channels among them (if such exists) using local status information.

Minimality and Progressiveness

- **greedy algorithms:** (also called **minimal**, **direct**, **shortest-path**, or **profitable**.)
  - Every routing decision brings the packet *closer* to the destination. Deterministic and oblivious routing algorithms are usually greedy.
- **nongreedy algorithms:** (also called **nonminimal**, **indirect**, **nonprofitable**, or **misrouting**.)
  - They can supply channels which send packets *away* from its destination. This is used exclusively in adaptive routing. It is an *optimistic* approach: instead of waiting in case of blocking, we believe that a longer detour will bring us into a free region where we can resume greedy routing. However, it consumes additional resources and is prone to livelocks.

- **progressive adaptive routing:**
  - Each routing operation allocates a new channel and the length of the route path increases, but not necessarily bringing the packet closer to the destination. Greedy routing is always progressive.
- **backtrack adaptive routing:**
  - The header is allowed to backtrack, releasing previously reserved channels.
  - This is used mainly in fault-tolerant routing when no alternative progressive continuation is possible.
  - It assumes history information to ensure that the same paths are not tried repeatedly.
  - Backtrack is hard and costly implement in wormhole switching, in contrast to circuit switching. Backtrack routing cannot be deterministic.

interconnection networks ⇔ routing algorithms

Given an interconnection network with given topology, its ability to serve as an efficient and reliable communication medium for transferring messages between nodes depends on the level to which the routing algorithm can exploit its structural properties.

This concerns namely:

- **adaptivity**: ability to use the path redundancy of the topology for rerouting packets in case of contention or congestion at links, nodes, or regions.
- **fault tolerance**: similar to the previous property, even though fault tolerant routing need not be necessarily adaptive. It can be achieved for example by storing packets in intermediate nodes and by routing them in more phases.
- **deadlock freedom**: some topologies are known to have deadlock-free routing algorithms.

Routing algorithms examples

- dimension order routing
  - In general, in a mesh or cube, routing is done by moving from lowest (x) dimension to the highest.
  - In a hypercube, it is called *e-cube* routing – XOR based routing

Example- routing in a grid(mesh)

Δx =d.x-s.x, Δ y=d.y-s.y

- West (–x)
- East (+x)
- South (–y)
- North (+y)
- Processor (destination) when Δ x, Δ y = 0
- To accomplish this routing, the switch needs to test the address in the header and increment or decrement one routing field.
- Routing is done in dimension order—first across the x dimension, then the y-dimension, then the z-dimension (if any), etc.

XOR routing in the hypercube - example

- Consider the case whereby
- S = 10(001010) and D = 39(100111).

$$R = S \oplus D$$

- R = (101101); => the message has to be sent along dimensions 0, 2, 3, and 5 in order to reach the destination.
- The order in which these dimensions are traversed is not important.

If we assume that the message will follow the route by traversing the following dimensions 5, 3, 2, and 0.

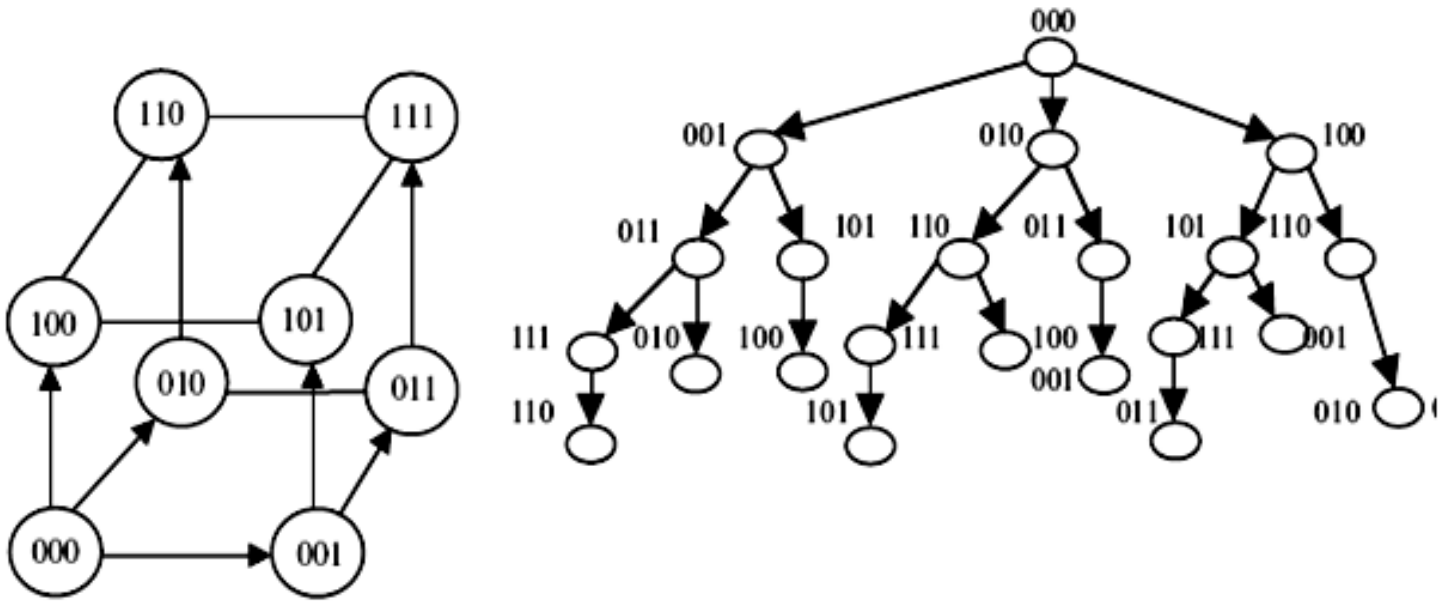
Then the route is totally determined as:

10(001010) →42(101010) →34(100010) →38(100110) →39(100111)s

Routing for Broadcasting and Multicasting

- Practical broadcast and multicast routing algorithms must be deadlock-free and should transmit the message to each destination node in as little time and using as short a path as possible.
- One technique to achieve this is to deliver the message along a common path to as many destinations as possible and then replicate the message and forward each copy on a different channel band for a unique set of destination nodes.
  - The path followed by each copy may further branch in this manner until the message is delivered to every destination node.
  - In such a *tree-based* communication model, the destination set is partitioned at the source and separate copies are sent on one or more outgoing links. A message may be replicated at intermediate nodes and forwarded along multiple outgoing links towards disjoint subsets of destinations.
- Another method to implement a multicast operation uses *separate addressing*. In this case, a separate copy of the message is sent directly from the source to every destination. Clearly, this is an inefficient technique.

Hypercube broadcast tree-based communication



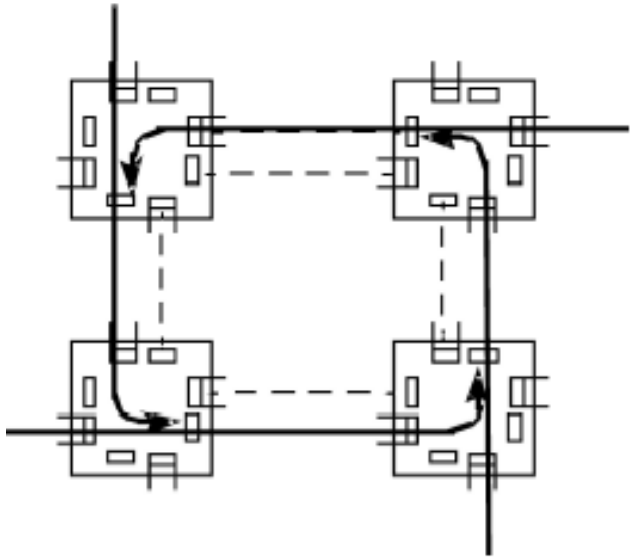
Requirements

Routing should:

- prevent deadlocks, livelocks and starvation
- avoid hot-spots - for this reason, ***two-step routing*** is often used.
  - In this case, a message from source *s* to destination *d* is first sent to a randomly chosen intermediate processor *i* and then forwarded to destination *d*.

Deadlock-free routing

- Deadlock occurs when two or more packets are “circularly” waiting for resources that are held by other packets in the group.



- The diagram illustrates how this can occur with 2-hop messages.
- Each packet is waiting for a link occupied by another packet.

Avoiding deadlocks

- Allow ***rerouting*** (maybe discarding) of the messages participating in a deadlock situation.
  - Rerouting of messages gives rise to nonminimal routing, while discarding messages requires that messages be recovered at the source and retransmitted.
  - This preemptive technique leads to long latency and, therefore, is not used by most message passing networks.
  - straightforward, but inefficient
- ***Ordering*** network resources and requiring that messages request use of these resources in a strict monotonic order.
  - It prevents the occurrence of circular wait,
  - E.g. technique - *channel dependency graph* (CDG)

Livelock

- *Livelock* describes a situation in which a message keeps going around the network and never reaches its destination.
- It is a phenomenon that results from using adaptive routing algorithms where messages are rerouted in the hope to find another path to their destinations.

Solution: using ***priorities***

1. Messages are routed according to their priorities.

2. Once a message has been injected, only a finite number of messages will be injected with higher or equal priority.

Starvation

- A node is said to suffer from *starvation* if it has a message to inject into the network but is never allowed to do so.

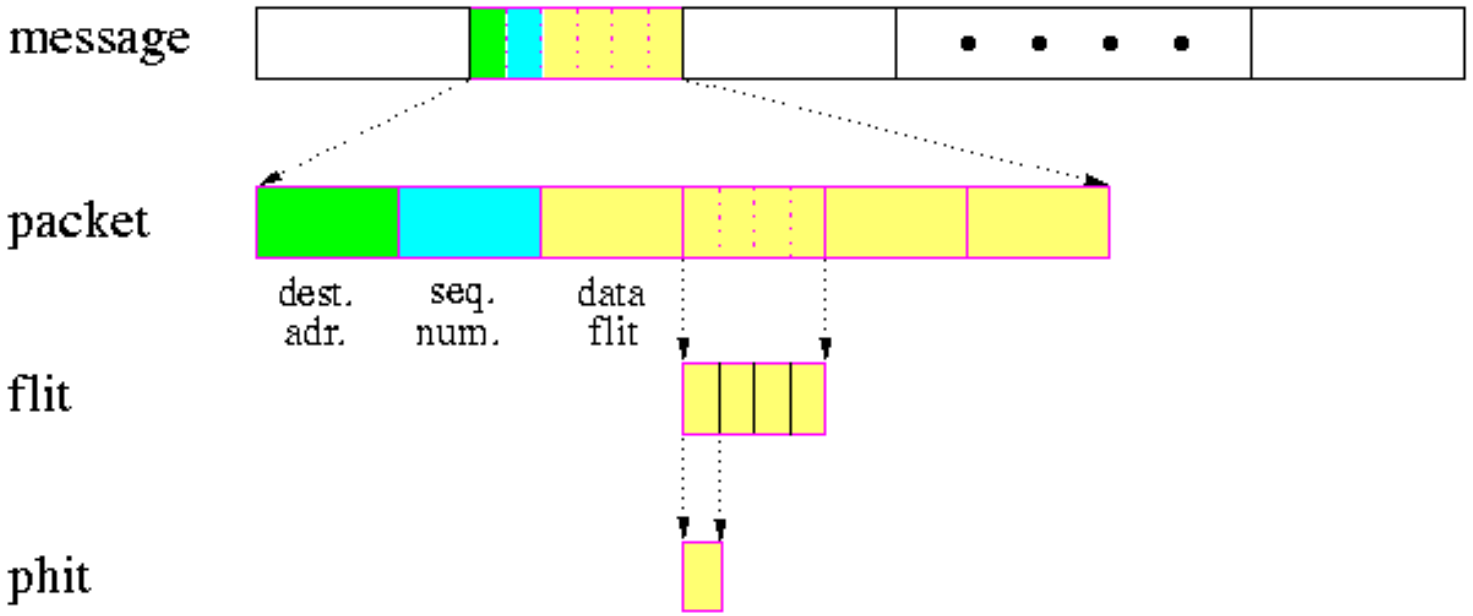
- One solution: ***injection queue***
  - The simplest is to allow ***each node*** to have its ***injection queue***, where it stores the messages, it wants to inject into the network.
  - This queue is considered in the same way as the queues of the incoming links to that node and it competes with them.

Communication Costs in Parallel Machines

- Along with idling and contention, communication is a major overhead in parallel programs.

- The cost of communication is dependent on a variety of features including:
  - programming model semantics,
  - the network topology,
  - data handling and routing, and
  - associated software protocols.

Communication units



Network performance characteristics

- **channel width *w*:** # of bits that a physical channel can transmit simultaneously between two adjacent routers.
  - Equal to the size of a *phit*.
- **channel rate μ :** the peak rate in bits/second at which bits can be transferred over each individual wire of a physical channel.
- **channel bandwidth *B*: $B=w\times\mu$** in bits/second, even though it can be defined also as **$B=\mu$** in phits/second,
- **channel latency t_m:** the inter-router latency of one phit, hence **$t_m=1/\mu = 1/B$** in seconds/phit.
- **startup latency t_s:** the time required for packet framing/unframing, copying data between memory and router buffers, validation and so on, at both source and destination side.
  - It depends mainly on the design of system SW, on its interface to user programs, and on the physical interface between processing nodes and routers.
  - It is a **static** value, *independent of network traffic or distance between nodes*.
- **network latency:** the elapsed time after the head flit of a packet has entered the network at the source router until the tail flit is ejected. The network latency consists of two components:
  - a conflict-free packet **transmission time**, which strongly depends on the switching technique, and
  - the **blocking time**, which includes all possible delays encountered during the lifetime of a packet.

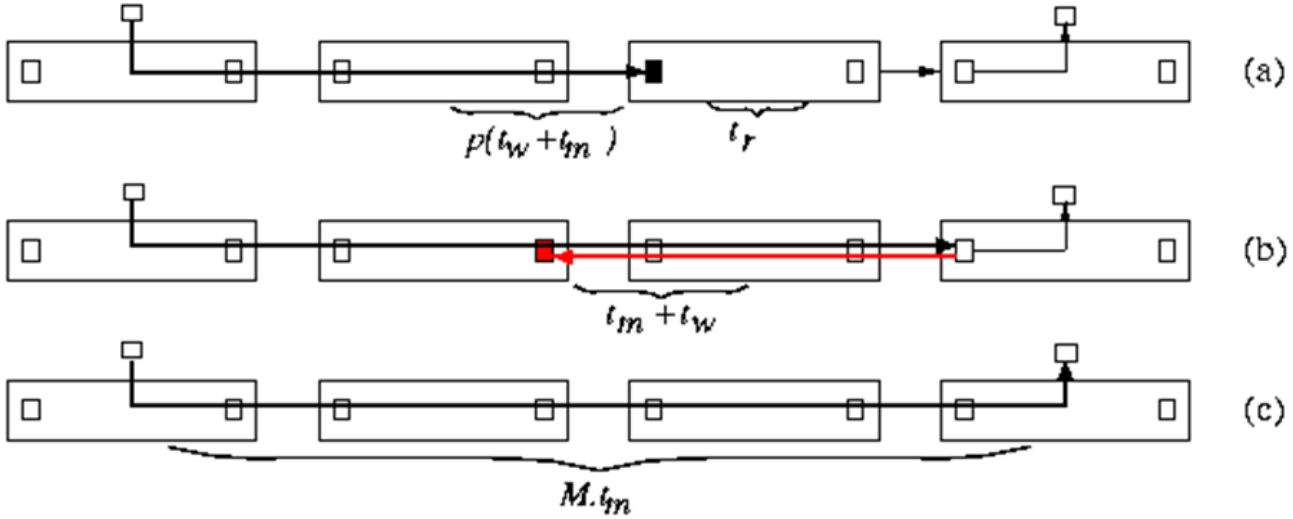
Notations and assumtions

- ***w*** = flit size = phit size = channel width (in bits),
- ***M*** = packet size (in flits),
- $\boldsymbol{t_r}$ = time of routing decision within one router during building a routing path (in seconds),
- $\boldsymbol{t_w}$ = ***intra-router*** switching latency once the router has made the routing decision and a path has been set up through the router (in seconds/phit),
- $\boldsymbol{t_m}$ = *1/B* = ***inter-router*** channel latency (in seconds/flit). The transmission latency of a packet of *M* flits between two adjacent routers is $\boldsymbol{Mt_m}$ seconds.
- ***d*** = the length of a routing path.

Circuit switching (CS)

- The communication between a source and destination has two phases:
  - *circuit establishment* phase and
  - *message transmission* phase.
- A physical path from the source to the destination is reserved *prior* to the transmission of data by injecting a **routing probe**, which contains destination address and some control information. It usually needs more that one phit. Let ***p*** be the size of the probe in phits.
- The probe progresses towards the destination node, reserving physical links as it is transmitted through intermediate routers
- The path is set up after the probe has reached the destination.
- Then an **acknowledgment flit** is sent back
- Upon reception of the acknowledgment, the sender transmits the whole message at the full bandwidth of the path. The path from a HW circuit that is reserved for the whole time of the message transmission
- The circuit is released either by destination node or by the last bits of the message.

Example - CS



- Circuit switching on a path of length 3.

(a) The probe progresses towards the destination router.

(b) The acknowledgment is sent back to the source.

(c) The whole message is transmitted along established circuit using its full bandwidth

CS- latency analysis

- A message of length *M* transmitted to distance *d* takes time

$$t_{CS}=d(t_r+(p+1)(t_w+t_m))+Mt_m$$

Proof:

The probe needs time $d(t_r+p(t_w+t_d))$
to get to the destination,
the acknowledgment takes $d(t_w+t_d)$
to get back, and
the data transmission takes Mt_m

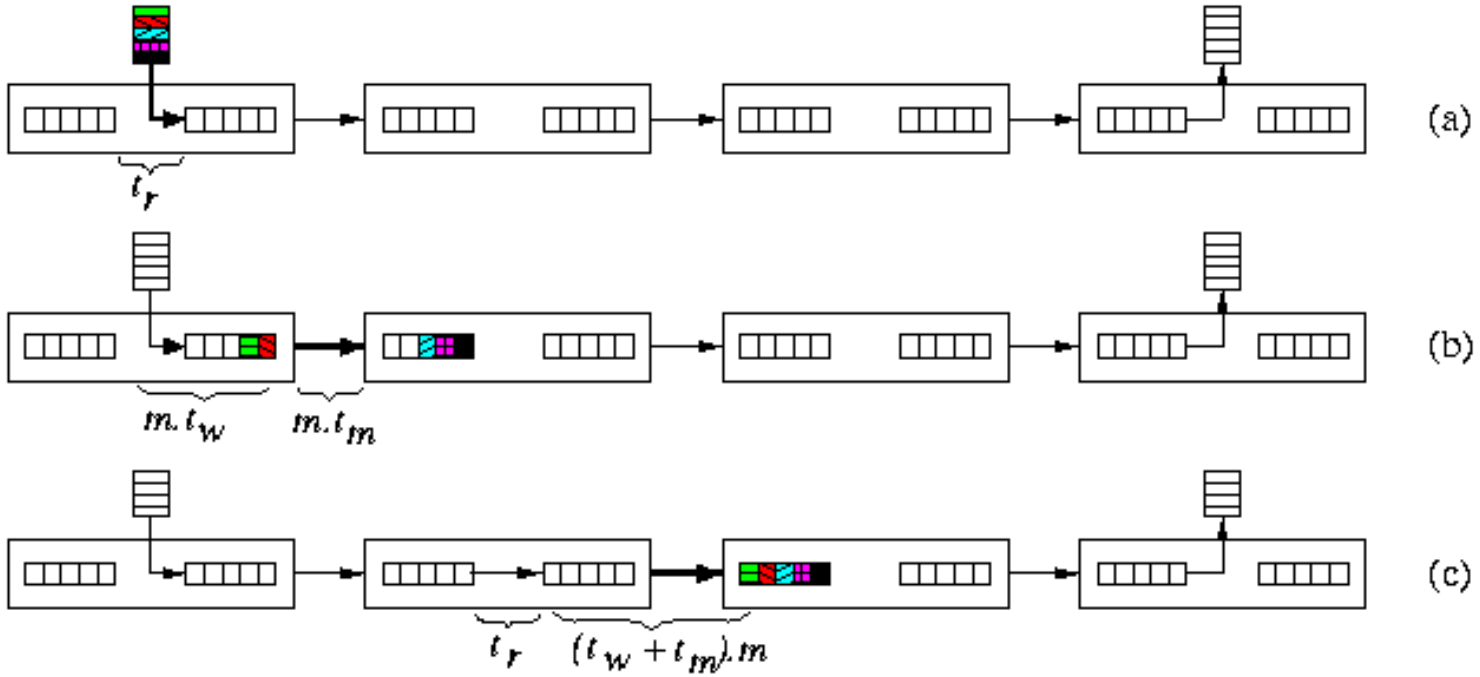
Remarks

- It is advantageous if messages **are infrequent and long**, i.e. the **transmission time is longer than the setup time**. In case of short messages, the whole physical circuit is reserved during the whole setup and transmission part.
- While the probe is buffered at each router, the data bits are not. The circuit operates as a single wire form the source to the destination.
- Messages need not be broken into fixed-length packets, but can be transmitted as continuous flow of bits along the setup circuit. Hence, there are no limits on the length of transmitted data.

Store-and-forward (SF) switching

- This technique is alternatively called **packet switching**.
- The message is split into *fixed-length packets*, every packet consists of *flits*, starting with the *header* flit.
- Every channel has input and output buffers *for one entire packet*.
- Every packet is ***individually*** routed from the source to the destination.
- One step of the SF switching is called **hop**. It consists of copying the whole packet from one output buffer to the next input buffer.
- Routing decisions are made by each intermediate router only **after** the whole packet was completely buffered in its input buffer.

SF -Example



Store-and-forward switching of a packet.

(a) Routing decision is being made in the first router.

(b) The packet is performing the first hop to the second router after has been copied to the output buffer of the first router.

(c) The whole packet after the second hop.

SF-Latency Analysis

- A packet of length *M* transmitted to distance ***d*** takes time

$$\mathbf{t_{SF}=d(t_r+(t_w+t_m)M)}$$

Proof:

- At every router, routing decisions must be made and then the packet hops to the next router, which takes

$t_r + (t_w+t_m)M$,

and this repeats *d* times.

Remarks

- Since ***packets may take different paths***, each packet must carry routing information, error checking, sequencing, and other related header information

- This technique is advantageous, when messages are **short and frequent,** since one transmission makes busy *at most one channel* from the whole path.

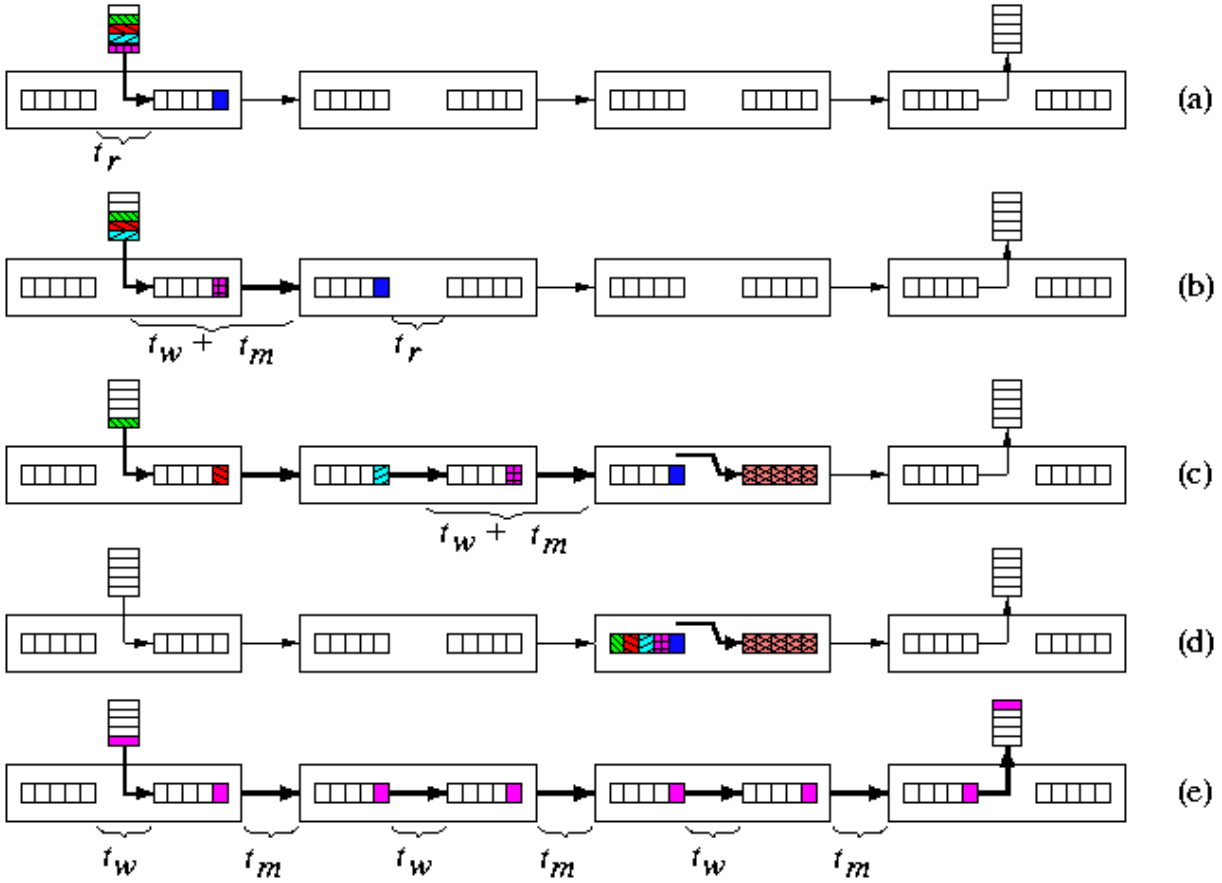
- The need to buffer the whole packet makes either the router design expensive and/or slower, or the size of packets is limited.

- The communication latency is proportional to the **product** of packet size and distance. This forces designers to seek **shortest path routing a**nd use **low-diameter networks.**

Virtual cut-through (VCT) switching

- Messages are split into packets and router have buffers for the whole packet as in SF switching.
- Instead of waiting for the whole packet buffered, the incoming header flit is ***cut through*** into the next router as soon as the routing decision was made and the output channel is free.
- Every further flit is buffered whenever it reaches the router, but it is also immediately cut-through to the next router if the output channel is free.
- In case the header cannot proceed, it waits in the current router, and all the following flits subsequently draw in, possibly releasing the channels occupied so far.
- In case of no resource conflicts along the route, the packet is effectively pipelined through successive routers as a loose chain of flits.
    - All the buffers along the routing path are blocked for other communication requirements.

VCT- Example



Virtual cut-through switching of a packet.

(a) The header flit has moved into the output buffer of the first router.

(b) The header has cut through into the second router while subsequent flits are following its path.

(c) Pulling a chain of flits behind, the header has cut through into a router where its output buffer is reserved for another communication.

(d) The whole chain of packet flits has contracted and the whole packet gets buffered in the first router releasing all previously allocated links.

(e) Flit pipeline moving towards the destination.

VCT Latency Analysis

- A packet of length *M* transmitted to distance *d* takes time

$$t_{VCT}=d(t_r+t_w+t_m)+max(t_w,t_m)M$$

Proof:

- We assume that there is no time penalty for cutting through. Only the header experiences routing delay as well as switching and inter-router latency.
- Once the header flit reaches the destination, the cycle time of the pipeline of packet flits is determined by the maximum of the switching time and inter-router latency.
- This is because we assume that channels have both input and output buffers.
- In case of input buffering only, for example, we would have t_w+t_m instead of $max(t_w,t_m)$.

Remarks

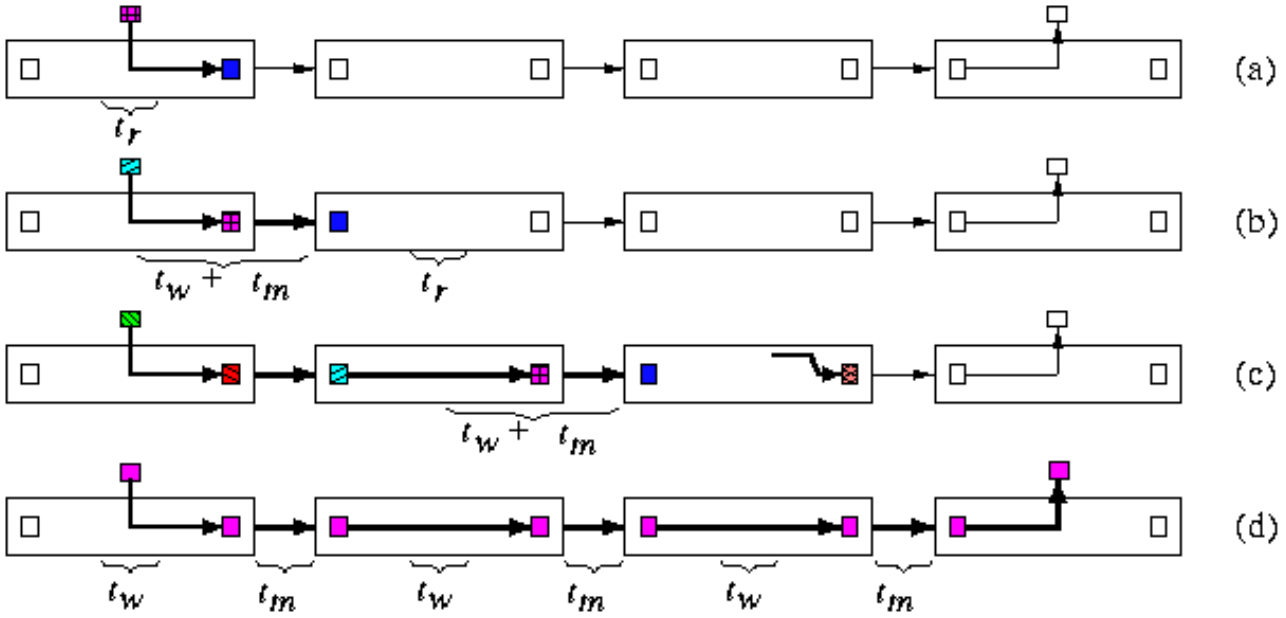
- Only the header flit contains routing information and therefore each incoming data flit is simply forwarded along the same output channel as its predecessor.
- No sequence numbers are needed
- Therefore, transmission of different packets *cannot be interleaved or multiplexed* over one physical channel.
- *Error checks are performed on the entire message.*

.

Wormhole (WH) switching

- The packets are split into flits which are snaked along the route exactly in the pipeline way as in conflict-free VCT switching.
  - Hence, also here transmission of different packets cannot be interleaved or multiplexed freely over one physical channel without additional architectural support.
- The main difference is that routers do not have buffers for the whole packets.
  - Instead, every router has *small* buffers for one or a few flits.
- The header flit again builds a path in the network, which the other flits follow in pipeline.
  - The sequence of buffers and links occupied by flits of a given packet forms the **wormhole**.
  - However, the length of the path here is proportional to the number of flits in the packet. Typically, it spans the whole path between the source and destination.
- If the header cannot proceed due to busy output channels, the whole chain of flits gets *stalled*, occupying flit buffers in routers on the path constructed so far and blocking other possible communications.

Example



- Wormhole switching of a packet.

(a) The header is copied in the output buffer after having done routing decision.

(b) The header flit is transferred to the second router and other flits are following it.

(c) The header flit arrived into a router with busy output channel and the whole chain of flits along the path got stalled, blocking all its channels.

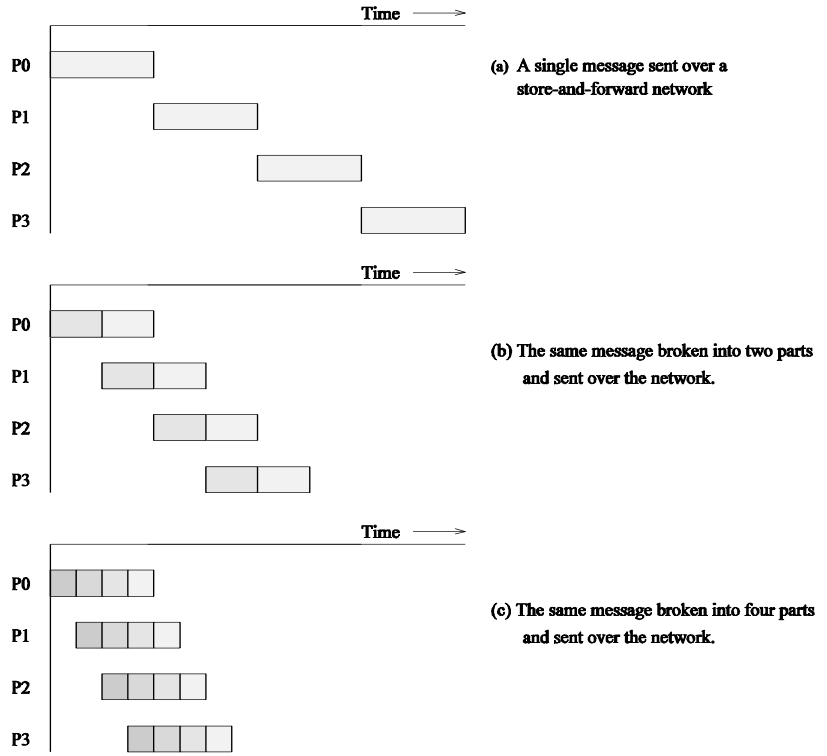
(d) Pipeline of flits in case of conflict-free routing, establishing the wormhole across routers.

Remarks related to WH technique

- Conflict-free communication latency t_{WH} is exactly the same as in VCT switching.
  - Hence, it is proportional to the **sum** of the path length and packet size.

- Wormhole routing allows simple, small, cheap, and fast routers. Therefore, it is the most common switching technique used nowadays in commercial machines.
- Wormhole routers use often only input buffering.
- *Blocking resources in case of stalled pipelines is the main drawback of this technique*.
- Since blocking chains of buffers can easily cause snow-ball effect, WH switching is very **deadlock-prone**.
- This issue is related to the concept of *virtual channels*.
  - We cannot mix flits of different packets into one buffer (= one physical channel), since flits carry no identity flags.
  - However, with adding some control logic, we can split *one physical channel into several virtual channels*. They will have their own buffers but will share (time-multiplex) one single physical channel medium (similarly to processes with own contexts sharing one physical processor).

Packet Routing Techniques



Passing a message from node P_0 to P_3 (a) through a ***store-and-forward*** communication network; (b) and (c) extending the concept to ***cut-through routing***. The shaded regions represent the time that the message is in transit. The startup time associated with this message transfer is assumed to be zero.

Simplified communication performance evaluation

- Consider
  - *Startup time* (t_s): Time spent at sending and receiving nodes (executing the routing algorithm, programming routers, etc.).
  - *Per-hop time* (t_h): This time is a function of number of hops and includes factors such as switch latencies, network delays, etc.
  - *Per-word transfer time* (t_w): This time includes all overheads that are determined by the length of the message.
    - This includes bandwidth of links, error checking and correction, etc.

Simplified Cost Model for Communication

- The cost of communicating a message between two nodes ***d*** hops

$$T_{comm} = t_s + d\,t_h + t_w M$$

  - In this expression, t_h is typically smaller than t_s and t_w. For this reason, the second term is not considered, particularly, when M is large.
  - it is often not possible to control routing and placement of tasks.

- For these reasons, if we ignore the network topology(or if consider a complete network), we can approximate the cost of message transfer by

$$T_{comm} = t_s + t_w M$$

Simplified Cost Model for Communication=>? congestion problem

- It is important to note that the original expression for communication time is ***valid for only uncongested networks***.
- If a link takes multiple messages, the corresponding t_w term must be scaled up by the number of messages.
- Different communication patterns congest different networks to varying extents.
- It is important to understand and account for this in the communication time accordingly.

Networks of Real Machines (~ 2000)

Machine	Topology	Speed	Width	Delay	Flit
nCUBE/2	hypercube	25 ns	1	40 cycles	32
CM-5	fat-tree	25 ns	4	10 cycles	4
SP-2	banyan	25 ns	8	5 cycles	16
Paragon	2D mesh	11.5 ns	16	2 cycles	16
T3D	3D torus	6.67 ns	16	2 cycles	16
DASH	torus	30 ns	16	2 cycles	16
Origin	hypercube	2.5 ns	20	16 cycles	160
Myrinet	arbitrary	6.25 ns	16	50 cycles	16

A message is broken up into *flits* for transfer.

Analytical Modeling of Parallel Systems performance evaluation

Lecture 5 - MPP

References :

“Introduction to Parallel Computing” Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar

“Distributed Systems: Paradigms and Models”, M.Danellutto

Why to evaluate the performance a-priori?

Evaluate the performance before starting writing application code!

- Performance orientation => In case several different implementations are possible, we are interested in knowing which one will provide the better performance.

- Productivity orientation => Which of the possible implementations will eventually provide performance that matching the performance requirements, and it is the less expensive in terms of coding effort.

Additionally:

- we may be interested in knowing how much *overhead* is added by a possible implementation.
- we may be interested in knowing what kind of performance we can achieve by using a given parallelism **exploitation pattern or skeleton** composition
  - in case expected performance of the given pattern doesn't match the requirements we can simply avoid to go on implementing a useless application.

Performance model

- The performance model measures the performance of
  an application on a given target machine
  in function of a full range of parameters depending either on:
  - the application features
    - parallelism exploitation
    - patterns used,
    - time spent in the execution of the sequential portions of code,
    - dimensions of the data types involved

  or

  – on the target machine features
    - communication times,
    - processing power of the different processing elements,
    - topology of the interconnection network,
    - features of the (shared) memory subsystem, etc.

Performance functions

=>a function P mapping values of some performance indicator.

More precisely we are interested in a set of functions $P_1; : : : ;P_h$

- each modeling a different performance indicator

- These performance model functions are used to:
  - predict performance of an application,
  - compare alternative implementations of an application, or
  - evaluate the overheads of an implementation of an application.

Level of accuracy

- Depending on the number of the parameters taken into account, as well as on the final “form" of the performance model, different results maybe achieved.

- different approximation levels =>
  - different accuracies may be achieved in the predicted results.

Topics Overview

- Sources of Overhead in Parallel Programs
- Performance Measures/Metrics for Parallel Systems
- Effect of Granularity on Performance
- Asymptotic Analysis of Parallel Programs

- Scalability of Parallel Systems
- Minimum Execution Time and Minimum Cost-Optimal Execution Time
- Other Scalability Metrics

Analytical Modeling - Basics

- A sequential algorithm is evaluated by its runtime
  - in general, asymptotic runtime as a function of input size
- The asymptotic runtime of a sequential program is identical on any serial platform.
- The parallel runtime of a program depends on
  - the input size & problem complexity
  - the number of processors, and
  - the communication parameters of the machine.
  - ... memory latency...
- An algorithm must therefore be analyzed in the context of the underlying platform.
- A ***parallel system*** *inside an analytical model* is a combination of
  - a parallel algorithm and an
  - an underlying platform.

Sources of Overhead in Parallel Programs

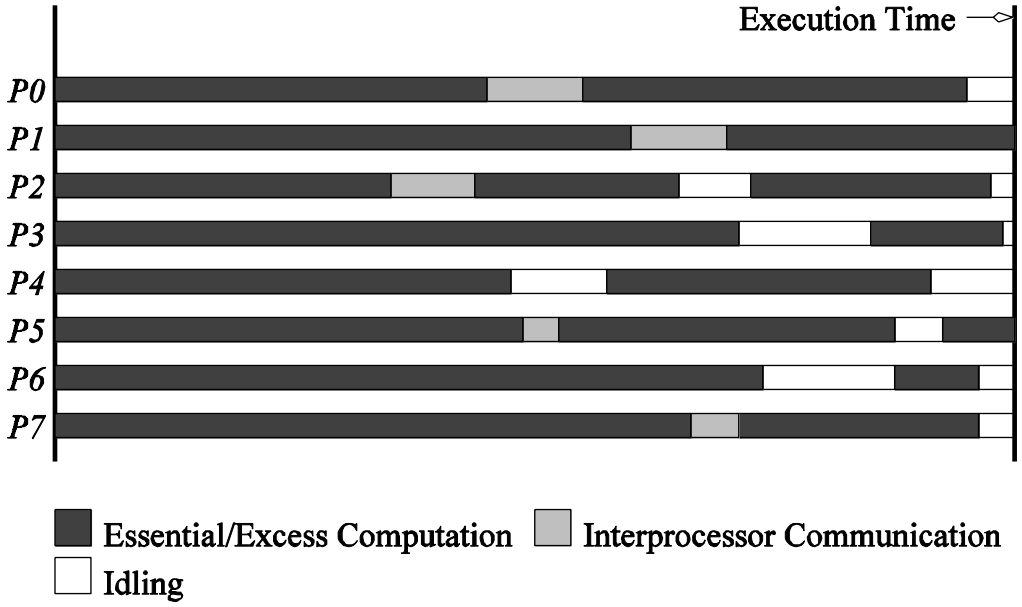
- If we use two processors, shouldn't our program run twice as fast?
  ☺
- No!!!
  ☹

A number of overheads, including:

- wasted/supplementary computation,
- communication,
- idling,
- contention

cause degradation in performance.

Example



The execution profile of a hypothetical parallel program executing on eight processing elements. Profile indicates times spent performing computation (both essential and excess), communication, and idling.

Sources of Overheads in Parallel Programs

- *Interprocess interactions*:
  Processes working on any non-trivial parallel problem will need to 'talk' to each other.

- *Idling*:
  Processes may idle because of load imbalance, synchronization, or serial components.

- *Excess Computation*:
  - This is the computation which is not performed by the serial version.
  - This might be because:
    - the serial algorithm is difficult to parallelize, or
    - that some computations are repeated across processors to minimize communication/synchronization.

Performance Metrics for Parallel Systems: Execution Time

- *Serial runtime* of a program is the time elapsed between the beginning and the end of its execution on a sequential computer.
- *The parallel runtime* is the time that elapses from the moment the first processor starts to the moment the last processor finishes execution.
- We denote the serial runtime by $\boldsymbol{T_s}$ and the parallel runtime by $\boldsymbol{T_P}$.
- The same notation for time-complexity!

Analytical Modeling - Basics

A number of performance measures are intuitive:

- Wall clock time - the time from the start of the first processor to the stopping time of the last processor in a parallel ensemble.

  – But how does this scale when
    - the number of processors is changed

    or
    - the program is ported to another machine altogether?

  – How much faster is the parallel version? This implies the obvious follow-up question:
    – what's the baseline serial version with which we compare?

- Raw FLOPS count

  - What good are FLOPS counts? when they are not enough?

FLOPS = sockets x (cores/ sockets) x clock x (Flops/cycle)

Communication /Synchronization costs (see Lecture 4)

- Distributed memory
  - The cost of communication is dependent on a variety of features including
    - the programming model semantics,
    - the network topology,
    - data handling and routing, and
    - associated software protocols.
- Shared memory
  - Synchronization

Performance Metrics for Parallel Systems: Total Parallel Overhead

- Let T_{all} be the total time collectively spent by all the processing elements.
- T_S is the serial time.
- $T_{all} - T_S$ is the total time spend by all processors in non-goal computation work. This is called the *overhead*.
- The total time collectively spent by all the processing elements could be approximate as:

  $T_{all} = p\ T_P$ (p is the number of processors).
- The overhead function (T_o) is therefore given by

$$T_o = p\ T_P - T_S$$

Performance Metrics for Parallel Systems: ***Speedup***

- What is the benefit from parallelism?

(general definition)

- Speedup (***S***) is the ratio of
  - the time taken to solve a problem on a single processor (the method used for solving could be different)

  to
  - the time required to solve the same problem on a parallel computer with ***p*** identical processing elements.

Alternative Definitions…

Speedup Factor

$$S(n) = \frac{\text{Execution time using one processor (single processor system)}}{\text{Execution time using a multiprocessor with n processors}} = \frac{t_s}{t_p}$$

where t_s is execution time on a single processor and t_p is execution time on a multiprocessor. $S(n)$ gives increase in speed by using multiprocessor. Underlying algorithm for parallel implementation might be (and is usually) different.

Speedup factor can also be cast in terms of computational steps:

$$S(n) = \frac{\text{Number of computational steps using one processor}}{\text{Number of parallel computational steps with n processors}}$$

Maximum speedup is (usually) n with n processors (*linear speedup*).

Speed-up types

- **relative**
  - T_s = parallel program executed on one processor (of the same machine where parallel program is executed)

- **absolute**
  - consider the best sequential algorithm that solve the problem!

- Theoretical and empirical speedup

Amdahl's Law - pessimistic

- Amdahl's Law states the speedup of processing based on the percentages of the serial part and parallel part:

Considering that:
Total sequential execution time complexity is = 1
seq = sequential computation part(fraction);
par = parallel computation part(fraction)
seq+par=1
n= # processors

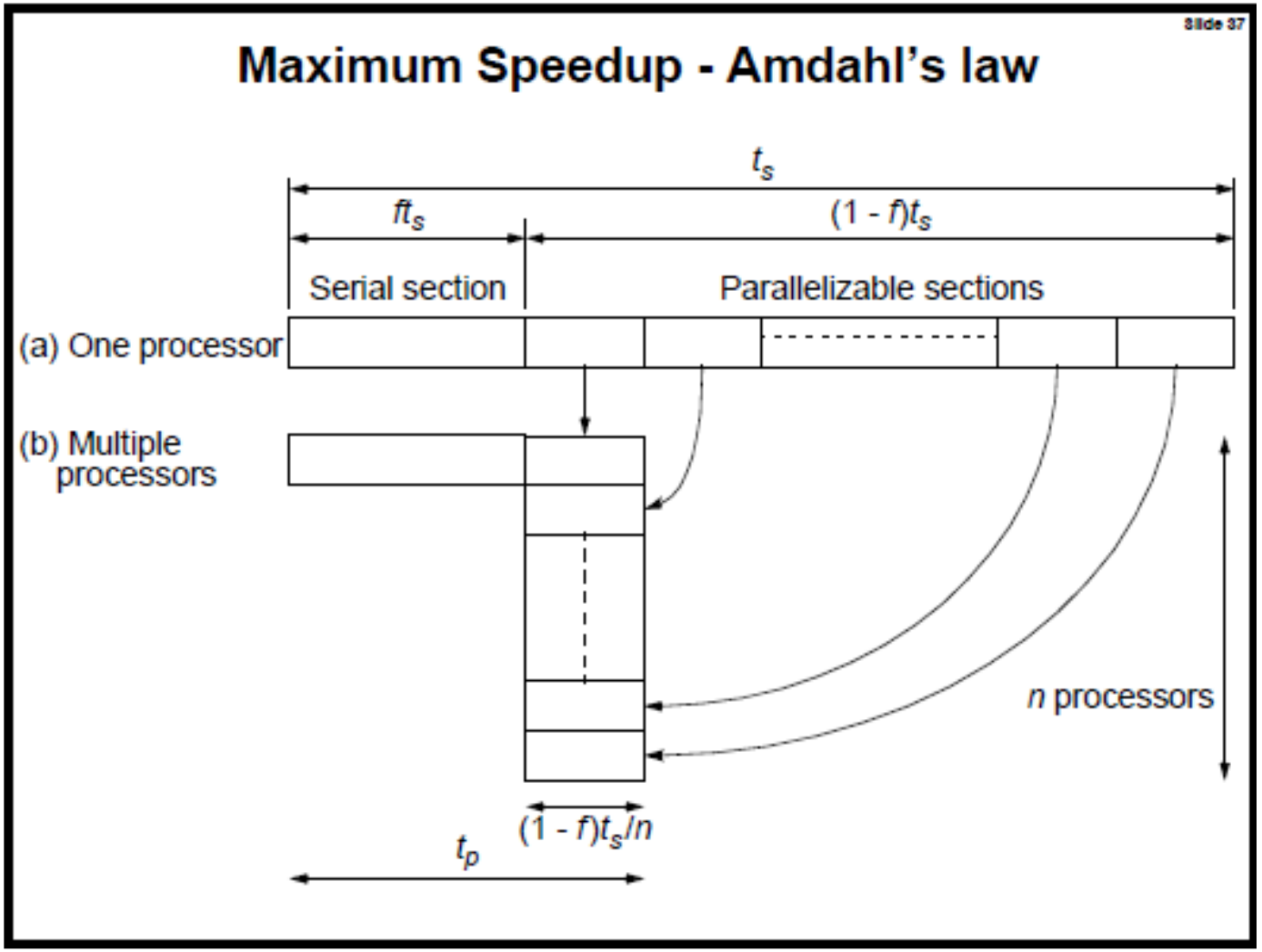
speedup = 1/(seq + par/n)

- When *n* -> *infinity*, then the speedup approaches to 1/seq.
- That is to say the upper limit of speedup is bounded by the cost of the serial part.
- But Amdahl's Law does not take into account the size of the problem being solved!

Alternative presentation

$$S(n) = \frac{t_s}{ft_s + (1-f)t_s/n} = \frac{n}{1+(n-1)f}$$

seq= f ;
par = 1-f



Amdahl's law

f – sequential fraction

$S(n) = (f + (1-f)/n)^{-1}$

$$S(n) \leq f^{(-1)}$$

- If f=5% then S(n) ≤ 100/5=20 even if n=10000…, n=1000…
- The speed-up of a program from parallelization is limited by how much of the program can be parallelized.
- This is true iff **f** does not depend on the size of the problem!

Gustafson's law - optimistic

- Gustafson's law is another law in computing, closely related to Amdahl's law. It states that the speedup with *n* processors is

Remarks:

- Amdahl's law assumes:
  - a fixed problem size and that
  - the running time of the sequential part (fraction) of the program is independent of the number of processors,
  - the fraction of the sequential part remains the same independent on the problem size
- Gustafson's law takes the opposite view
- Gustafson's Law rationalizes that
  - as the size of the problem grows, the serial part will become a smaller and smaller percentage of the entire process

Gustafson's law – more detailed analysis

- Let m = size of the problem, n = # of processors,

Gustafson's Law states that:

considering that:

T_p = seq(m) + par(m) = 1 par(m) = 1 – seq(m),

when m -> infinity, seq(m) -> 0,

then

speedup =seq(m) + n(1 – seq(m))

when n processor are applied for the parallel part

- As m -> infinity, and as seq(m) becomes a smaller and smaller percentage, the speedup approaches n.
- In other words, as programs get larger, having multiple processors will become more advantageous, and it will get close to n times the performance with n processors as the percentage of the serial part diminishes.
- **It assumes the absolute cost of the serial part is constant and does not grow with the size of the problem.**

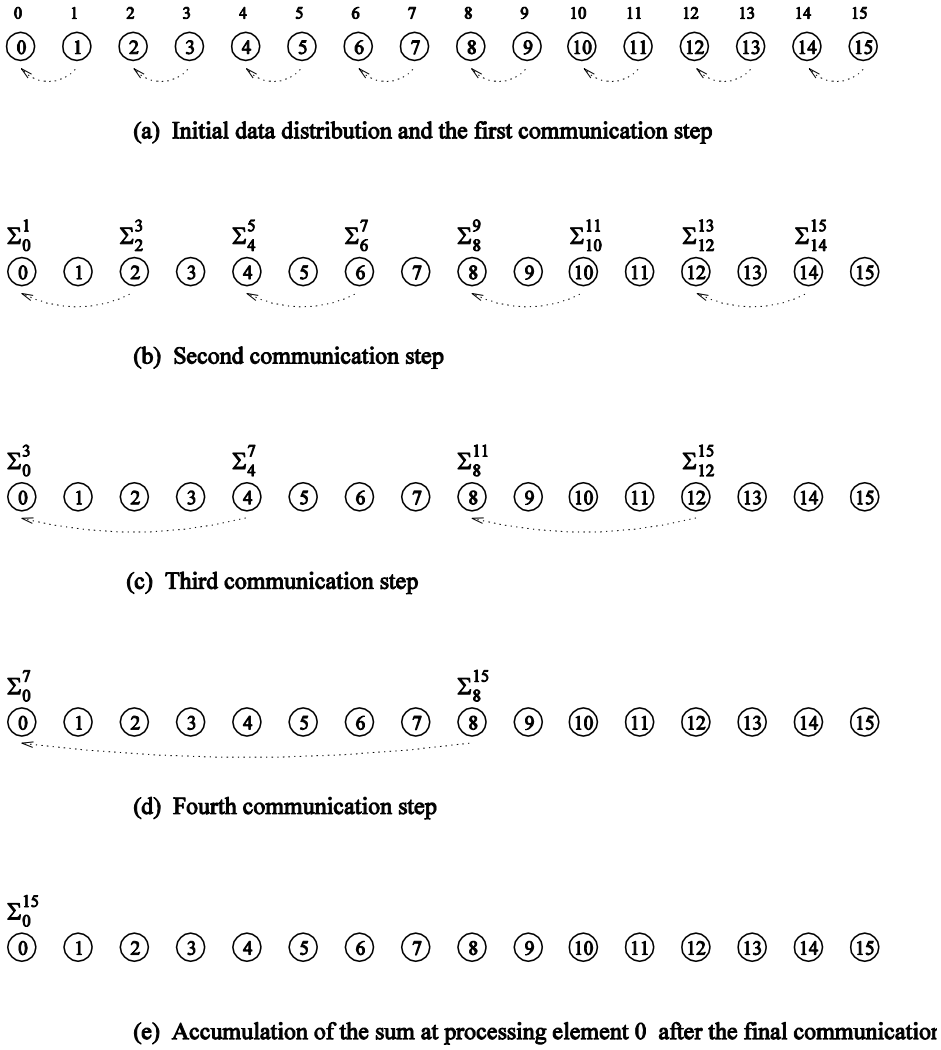
Order of magnitude - Example

- Assume that a task has two independent parts, A and B.
- B takes roughly 25% of the time of the whole computation.
- With effort, a programmer may be able to make this part (B) five times faster, but this only reduces the time for the whole computation by a little.
- In contrast, one may need to perform less work to make part A twice as fast.
- This will make the computation much faster than by optimizing part B, even though B got a greater speed-up (5 × versus 2 ×).

Performance Metrics: Example

- Consider the problem of adding ***n*** numbers by using ***n*** processing elements.
- If ***n*** is a power of two, we can perform this operation in **log *n*** steps by propagating partial sums up a logical binary tree of processors.

Performance Metrics: Example



Computing the global sum of 16 partial sums using 16 processing elements .

Σ^{j}_{i} denotes the sum of numbers with consecutive labels from ***i*** to ***j***.

Performance Metrics evaluation:
Adding on a distributed memory systems

=>If an addition takes constant time - t_c and

- communication of a single word takes time $t_s + t_w$,
- $p=n$

$$T_P = \log n \, [(t_s + t_w) + t_c]$$

$$T_S = n \, t_c$$

$$S(n) = T_S / T_P = (n / \log n) \, (t_c / (t_s + t_w + t_c))$$

$$T_P = \Theta (\log n)$$

- We know that $T_S = \Theta (n)$
- Asymptotic Speedup S is given by $S = \Theta (n / \log n)$

Performance Metrics: Absolute Speedup

- For a given problem, there might be many serial algorithms available. These algorithms may have different asymptotic runtimes and may be parallelizable to different degrees.

- For the purpose of computing speedup, usually the best sequential program is considered as the baseline.

Performance Metrics: Speedup Example – relative versus absolute -

- Consider the problem of parallel bubble sort.
- The serial time for bubblesort is 150 seconds.
- The parallel time for odd-even sort (efficient parallelization of bubble sort) is 40 seconds.
- The speedup would appear to be 150/40 = 3.75. (relative speed-up!)
- But is this really a fair assessment of the system?
- What if serial quicksort only took 30 seconds? In this case, the speedup is 30/40 = 0.75. This is a more realistic assessment of the parallel system.

Performance Metrics: Speedup Bounds

- Speedup can be as low as 0 (the parallel program never terminates).
- Speedup, in theory, should be upper bounded by n *(n = #processors)* – we can only expect a n-fold speedup if we use as many resources we need.
  - A speedup greater than n is possible only if each processing element spends less than time T_S / n solving the problem.
  - In this case, a single processor could be time slided to achieve a faster serial program, which contradicts our assumption of fastest serial program as basis for speedup.
  - Exceptions exist: search algorithms, others

Asymptotic Analysis of Parallel Programs

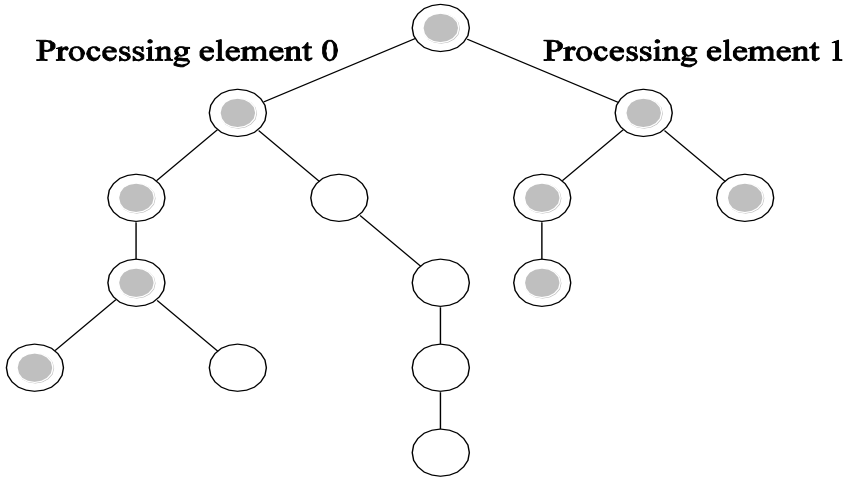
Consider the problem of **sorting** a list of ***n*** numbers.

- The fastest serial programs for this problem run in time Θ(***n*** **log** ***n***).
- Consider four parallel algorithms, A1, A2, A3, and A4.
- The table shows number of processing elements, parallel runtime, speedup, efficiency and cost.

Algorithm	A1	A2	A3	A4
p	n^2	$\log n$	n	$\sqrt{n}$
T_P	1	n	$\sqrt{n}$	$\sqrt{n}\log n$
S	$n\log n$	$\log n$	$\sqrt{n}\log n$	$\sqrt{n}$
E	$\frac{\log n}{n}$	1	$\frac{\log n}{\sqrt{n}}$	1
pT_P	n^2	$n\log n$	$n^{1.5}$	$n\log n$

Performance Metrics: Superlinear Speedups

One reason for superlinearity is that the parallel version does less work than corresponding serial algorithm.



Searching an unstructured tree for a node with a given label, `S', on two processing elements using depth-first traversal. The two-processor version with processor 0 searching the left subtree and processor 1 searching the right subtree expands only the shaded nodes before the solution is found. The corresponding serial formulation expands the entire tree. It is clear that the serial algorithm does more work than the parallel algorithm.

Performance Metrics: Superlinear Speedups

Resource-based superlinearity: The higher aggregate cache/memory bandwidth can result in better cache-hit ratios, and therefore superlinearity.

Example: A processor with 64KB of cache yields an 80% hit ratio. If ***two*** processors are used, since the problem size/processor is smaller (less data), the hit ratio goes up to 90%. Of the remaining 10% access, 8% come from local memory and 2% from remote memory.

If DRAM access time is 100 ns, cache access time is 2 ns, and remote memory access time is 400ns, this corresponds to a speedup of 2.43!

Performance Metrics: Efficiency

- Efficiency is a measure of the fraction of time for which a processing element is usefully employed.

- Mathematically, it is given by

$$E = \frac{S}{p}. \quad (2)$$

- Following the bounds on speedup, efficiency can be as low as 0 and as high as 1.

Cost of a Parallel System

- Cost is the product of parallel runtime and the number of processing elements used $C = p \times T_P$
- Cost reflects the sum of the time that each processing element spends solving the problem.
- A parallel system is said to be ***cost-optimal*** if the cost of solving a problem on a parallel computer is asymptotically identical to serial cost - $C = \Theta(T_s)$
- Since $E = T_S / p\,T_P$, for cost optimal systems, $E = O(1)$.
- Cost is sometimes referred to as ***Work*** or *processor-time product*.

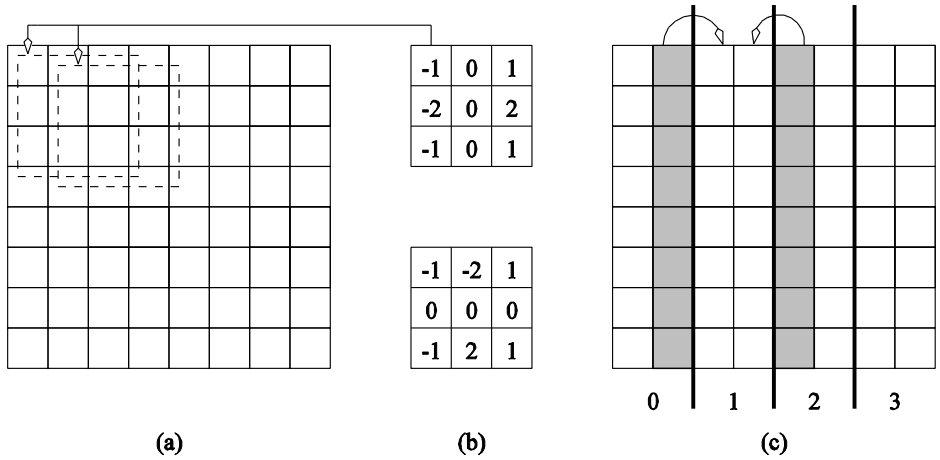
Cost of a Parallel System: Adding numbers Example

Consider the problem of adding numbers on processors.

- We have, $\boldsymbol{T_P} = \mathbf{log}\ \boldsymbol{n}$ (for $\boldsymbol{p} = \boldsymbol{n}$).
- The cost of this system is given by $\boldsymbol{p}\ \boldsymbol{T_P} = \boldsymbol{n}\ \mathbf{log}\ \boldsymbol{n}$.
- Since the serial runtime of this operation is $\Theta(\boldsymbol{n})$, the algorithm is **not cost optimal**.

Parallel Time, Speedup, and Efficiency Example

Consider the problem of edge-detection in images. The problem requires us to apply a ***3*** x ***3*** template to each pixel. If each multiply-add operation takes time t_c, the serial time for an ***n*** x ***n*** image is given by $T_S = t_c n^2$.



Example of edge detection: (a) an ***8*** x ***8*** image; (b) typical templates for detecting edges; and (c) partitioning of the image across four processors with shaded regions indicating image data that must be communicated from neighboring processors to processor 1.

Parallel Time, Speedup, and Efficiency Example (continued)

- One possible parallelization partitions the image equally into vertical segments, each with n^2 / p pixels.

- The boundary of each segment is $2n$ pixels. This is also the number of pixel values that will have to be communicated.
  - $T^{Communication}_{p}(n) = 2(t_s + t_w n)$.

- Templates may now be applied to all n^2 / p pixels in time
  - $T^{Computation}_{p}(n) = 9\, t_c n^2 / p$.

Parallel Time, Speedup, and Efficiency Example (continued)

- The total time for the algorithm is therefore given by:

$$T_P = 9t_c\frac{n^2}{p} + 2(t_s + t_w n)$$

- The corresponding values of speedup and efficiency are given by:

$$S = \frac{9t_c n^2}{9t_c\frac{n^2}{p} + 2(t_s + t_w n)}$$

and

$$E = \frac{1}{1 + \frac{2p(t_s + t_w n)}{9t_c n^2}}.$$

$C = (9\ t_c n^2/p + 2(t_s + t_w n))p = 9\ t_c n^2 + 2p(t_s + t_w n)$

Impact of Non-Cost-Optimality

Consider a sorting algorithm that uses n processing elements to sort the list in time $(\log n)^2$.

- Since the serial runtime of a (comparison-based) sort is $n \log n$, the speedup and efficiency of this algorithm are given by $n / \log n$ and $1 / \log n$, respectively.
- The $p\ T_P$ product of this algorithm is $n\ (\log n)^2$.
- This algorithm is not cost optimal but only by a factor of $\log n$.
- If $p < n$, assigning the n tasks to p processors gives $T_P = (\log n)^2 (n / p)$.
- The corresponding speedup of this formulation is $p / \log n$.
- For a given **p** this speedup goes down as the problem size n is increased!

Granularitaty

- *Granularitatea(“grain size”) calitative parameter that characterizes:*
  1. *Parallel architectures (number of processors, memory units)*
  2. *Parallel applications*

- *Application Granularity = minimum size of a sequential unit of a program (expressed in no of instructions) = minimum grain size*
  - *sequential unit = part in a program with no synchronization and communications operations*
- *Medium Application Granularity = average of all grain sizes*
  - *Can be approximated as the ratio between the total time over communication/synchronization time*

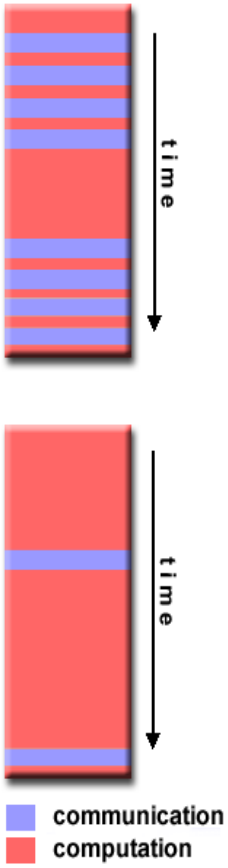
- Each instruction stream has a certain granularity.

Granularity(in parallel applications)

Granularity measures the amount of computation in relation to communication.

- could be approximated by the ratio of computation to the amount of communication
- it is determined by the minimum size (no of instructions) of a unity of computation work (in which no communication and synchronization take place)

- **Fine-grain Granularity:**
  - Relatively small amounts of computational work are done between communication/synchronization events
  - Low computation to communication /synchronization ratio
  - Facilitates load balancing
  - Implies high communication overhead and less opportunity for performance enhancement
  - If granularity is too fine it is possible that the overhead required for communications and synchronization between tasks takes longer than the computation.
- **Coarse-grain Granularity:**
  - Relatively large amounts of computational work are done between communication/synchronization events
  - High computation to communication ratio
  - Implies more opportunity for performance increase
  - Harder to load balance efficiently



System granularity

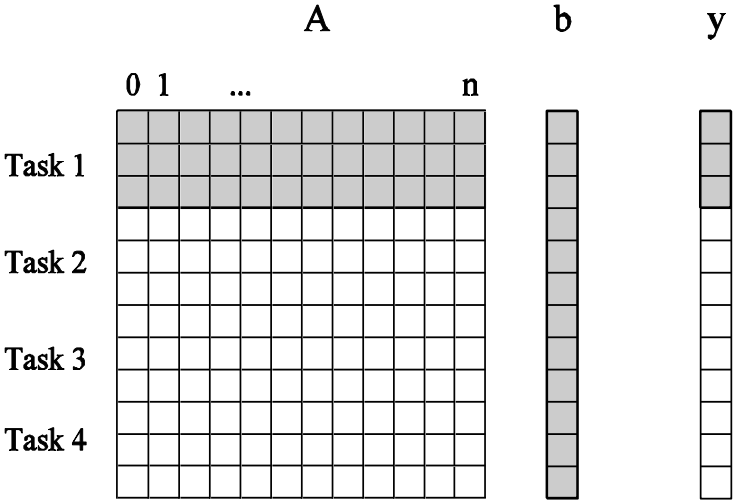
- *For a given parallel architecture there is a a minimum value o the application granularity that marks the limit under which the system performance drop significantly= system **granularity***

- *Why?*
  - The overhead arrive to be comparable with the computation time
- What we whish?
  - $\Rightarrow$ For parallel system low granularity
  - $\Rightarrow$ For parallel applications high granularity.

- Exceptions: classes of application with low application granularity but which are executed on specific architectures with low system granularity
  - Systolic/pipeline applications

Granularity of Task Decompositions

- The number of tasks into which a problem is decomposed **determines(not equal !!!)** its granularity.
- Decomposition into a large number of tasks results in fine-grained decomposition and the decomposition into a small number of tasks results in a coarse grained decomposition.



A coarse grained counterpart to the dense matrix-vector product example. Each task in this example corresponds to the computation of three elements of the result vector.

Granularity…cont.

- Periods of computation are typically separated from periods of communication or synchronization events.
- In most cases the overhead associated with communications and synchronization is high relative to computation speed.

Which is the Best?

- The finer the granularity is, the greater the potential for parallelism and hence speed-up, but the greater the overheads of synchronization and communication.

- In order to attain the best parallel performance, the best balance between load and communication overhead needs to be found.
  - If the granularity is too fine, the performance can suffer from the increased communication overhead.
  - If the granularity is too coarse, the performance can suffer from load imbalance.

- ***The most efficient granularity is dependent on the algorithm and the hardware environment in which it runs.***

Effect of Granularity on Performance

- Often, using fewer processors improves efficiency of parallel systems.
- Using fewer than the maximum possible number of processing elements to execute a parallel algorithm is called *scaling down* a parallel system.
- A naive way of scaling down is to think of each processor in the original case as a virtual processor and to assign virtual processors equally to scaled down processors.
- Since the number of processing elements decreases by a factor of n / p, the computation at each processing element increases by a factor of n / p.
- ***The communication cost should not increase by this factor*** since some of the virtual processors assigned to a physical processors might talk to each other.
  - This is the main reason of the improvement from building granularity!!!

Building Granularity: Example

- Consider the problem of adding n numbers on p processing elements such that $p < n$ and both n and p are powers of 2.
- Use the parallel algorithm for n processors, except, in this case, we think of them as virtual processors.
- Each of the p processors is now assigned n / p virtual processors.
- The the $\log n$ steps of the original algorithm are simulated in $(n / p) \log p$ steps on p processing elements.
- Subsequent $\log n - \log p$ steps do not require any communication.

Building Granularity: Example (continued)

- The overall parallel execution time of this parallel system is $\Theta\left(\left(\boldsymbol{n} / \boldsymbol{p}\right) \log \boldsymbol{p}\right)$.

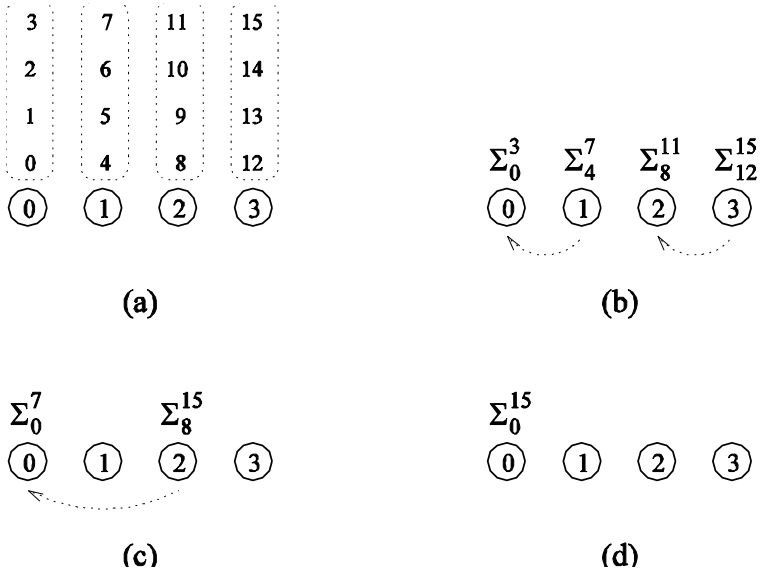
- The cost is $\Theta\left(\boldsymbol{n} \log \boldsymbol{p}\right)$, which is asymptotically higher than the $\Theta\left(\boldsymbol{n}\right)$ cost of adding $\boldsymbol{n}$ numbers sequentially.

- Therefore, **the parallel system is not cost-optimal.**

Building Granularity: Example (continued)

Can we build granularity in the example in a cost-optimal fashion?

- Each processing element locally adds its n / p numbers in time $\Theta(n / p)$.
- The p partial sums on p processing elements can be added in time $\Theta(n / p)$.



A cost-optimal way of computing the sum of 16 numbers using four processing elements.

Building Granularity: Example (continued)

- The parallel runtime of this algorithm is

$$T_P = \Theta(n/p + \log p),$$

- so long as

$$n = \Omega(p \log p)$$

- The cost is cost-optimal $C(n) = pT_p(n) = \Theta(n + p \log p)$

- $T_s(n) = \Theta(n) = C(n) = pT_p(n) = \Theta(n)$
- $C(n, p) = \Theta(T_s(n))$ the condition for cost optimality
- => this variant is **cost-optimal**

Asymptotic Analysis of sorting algorithms

- If the metric is speed, algorithm A1 is the best, followed by A3, A4, and A2 (in order of increasing T_P).
- In terms of efficiency, A2 and A4 are the best, followed by A3 and A1.
- In terms of cost, algorithms A2 and A4 are cost optimal, A1 and A3 are not.

It is important to identify the objectives of analysis and to use appropriate metrics!

Asymptote analysis

- The maximum speedup asymptote is given by the linear function- $f(n) = n$.

(so, a **line** that a curve approaches, as it heads towards infinity)

Proof:

- If we assume that $s(n) > n$ then the portions of the algorithm executed in parallel on the n processing elements will take a time
  $Tp < Tseq/n$ and therefore we could have performed sequentially the n computations taking Tp on a single processing element, taking an overall time

$T_{all} = n*Tp < n*Tseq / n < Tseq$

which is in contradiction with the assumption $Tseq$ was the “best" sequential time.

- But there are exceptions…

Lecture 6
Analytical performance analysis

Scalability

Performance expectations

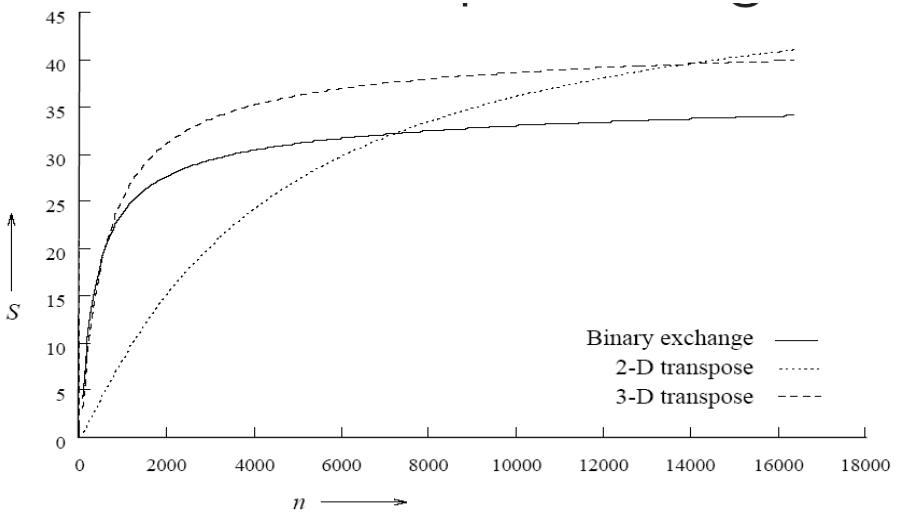
When using 1 processor, the sequential program runs for 100 seconds.

When we use 10 processors, should the program run for 10 times faster?

- This could work theoretically only for embarrassingly parallel computations
  - parallel computations that can be divided into completely independent computations that can be executed simultaneously.
  - There may have no interaction between separate processes; sometime the results need to be collected.
  - Embarrassingly parallel applications are the kind that can scale up to a very large number of processors. Examples: Monte Carlo analysis, numerical integration, 3D graphics rendering, and many more.
- In other types of applications, the computation components interact and have dependencies, which make difficult the usage of a large number of processors.

Scalability of Parallel Systems

How do we extrapolate performance from small problems and small systems to larger problems on larger configurations?



*Example: T*hree parallel algorithms for computing an n-point Fast Fourier Transform (FFT) on 64 processing elements. A comparison of the speedups obtained using 64 processing elements with $t_c = 2$, $t_w = 4$, $t_s = 25$, and $t_h = 2$.

It is difficult to infer scaling characteristics from observations on small datasets on small machines.

- The ***scalability*** (strong) of a parallel system is a measure of its capacity to increase speedup in proportion to the number of processing elements.
- It reflects a parallel system's ability to utilize increasing processing resources effectively.

Speedup and Strong scaling

- Let T_1 be the execution time for a computation to run on 1 processor and T_p be the execution time for the computation (**with the same input – same problem**) to run on p processors.

$speedup(p) = T_1 / T_p$

- Factor by which the use of p processors speeds up execution time relative to 1 processor for the same problem.
- When the analysis if for a fixed problem size, this is referred to as "strong scaling".

- Typically, $1 \leq speedup\ (p) \leq p$
- The speedup is ideal if $speedup = p$
- Linear speedup: $speedup(p) = k \times p$ for some constant $0 < k < 1$

Obs. In strong scaling analysis, we are interested in determining how much we can increase the number of processes for a fixed problem size, such that to preserve the efficiency.

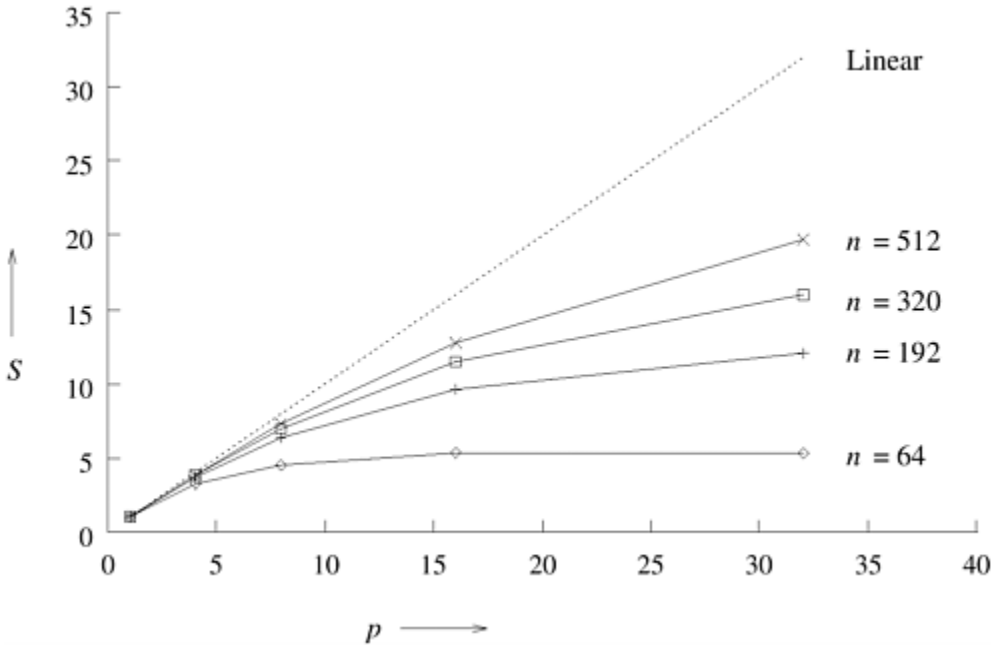
Example: Adding numbers(1)

Speedup and efficiency as functions of the number of processing elements

$$T_P = \frac{n}{p} + 2\log p$$

$$S = \frac{n}{\frac{n}{p} + 2\log p}$$

$$E = \frac{1}{1 + \frac{2p\log p}{n}}$$



Obs.

- A larger instance of the same problem yields higher speedup and efficiency for the same number of processing elements.
- Speedup tends to saturate, and efficiency drops as a consequence of Amdahl's law.

Example: Adding numbers(2)

Efficiency: The effect of increasing the problem size keeping the number of processing elements constant (column analysis).

n	*p* = 1	*p* = 4	*p* = 8	*p* = 16	*p* = 32
64	1	***0.8***	0.57	0.33	0.17
192	1	0.92	***0.8***	0.6	0.38
320	1	0.95	0.87	0.71	0.5
512	1	0.97	0.91	***0.8***	0.62

Analysis:

The efficiency of adding 64 numbers using 4 processing elements is 0.80.

If the number of processing elements is increased to 8 and the size of the problem is scaled up to add 192 numbers, the efficiency remains 0.80.

Increasing *p* to 16 and *n* to 512 results in the same efficiency.

- The parallel systems that have this ability to ***maintain efficiency at a fixed value*** by simultaneously increasing the number of processing elements and the size of the problem they are considered ***scalable*** parallel systems.

Weak scaling

- Weak scaling analysis evaluates the speedup, efficiency or the running time

of a parallel algorithm for different problem sizes, while ensuring that the problem size per processor remains constant.

- A common practice in weak scaling analysis consists in doubling both the size of the problem and the number of processors. Still, this do not always assure that the problem size per processor remains constant.

If the running time or the efficiency remains constant, then the algorithm is scalable.

Example: Adding numbers

n'=2n; p'=2p ; n/p=k

T_p= n/p+2log p=k+2log p; S=p/(1+2log p/k)

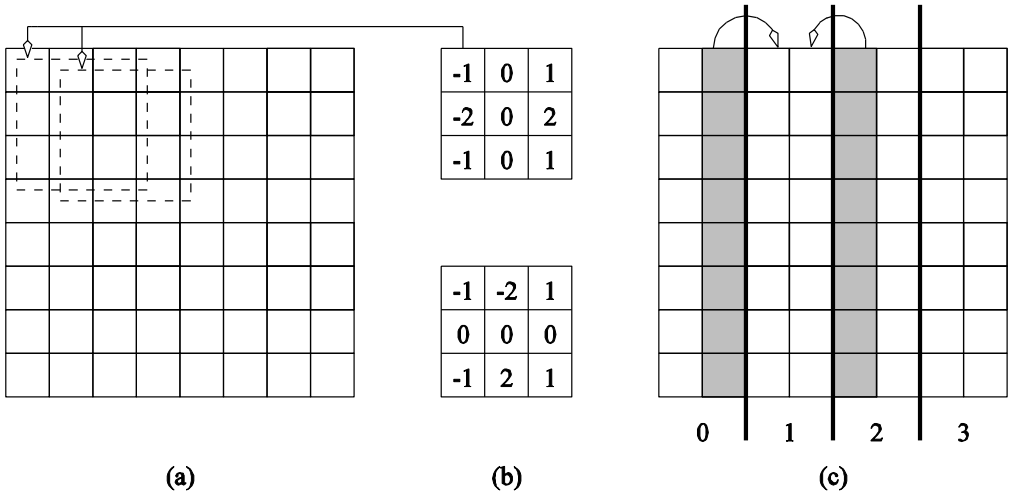
T'_p= n'/p'+2log p' = k+2log p+2; S'=p'/(1+2log p'/k)

E=1/(1+2log p/k); E'=1/(1+2log p/k+2/k); => E>E' but 2/k=2p/n<<1

For example, if k=200 and p=2^10 => E=1/(1+10/100)=100/110=0.(90)

E'= 1/(1+10/100+1/100)=100/111=0.(900)

Example: Matrix Convolution distributed memory



Problem size per processor k= n^2/p
n'=2n => p'=4p

$Ts= 9n^2$

$Tp=9n^2/p+2(t_s+nt_w)$

$S=p/(1+2(t_s+nt_w)p/ 9n^2)$

$E=1/(1+2(t_s+nt_w)/9k)$

$E'=1/(1+2(t_s+2nt_w)/9k)$

Scaling Characteristics of Parallel Programs

- The efficiency of a parallel program can be written as:

$$E = \frac{S}{p} = \frac{T_S}{pT_P}$$

or

$$E = \frac{1}{1 + \frac{T_o}{T_S}}.$$

- The total overhead function T_o is an increasing function of p *and depends on* T_S .
  - $T_o = pT_p - T_s$
  - $pT_p = T_s + T_o$

Overhead impact

- Total overhead function T_o is a function of both problem size T_s and the number of processing elements p.
- For a given problem size (i.e., the value of T_S remains constant), as we increase the number of processing elements, generally T_o increases (??? how much), and so the overall efficiency of the parallel program goes down.

$$E = \frac{S}{p} = \frac{T_S}{pT_P}$$

- In many cases, T_o grows sublinearly with respect to T_s.

$$E = \frac{1}{1 + \frac{T_o}{T_S}}.$$

- In such cases, the **efficiency increases if the problem size is increased** keeping the number of processing elements constant.

- For such systems, we can
simultaneously increase the problem size and number of processors
to keep **efficiency constant**.

- We call such systems ***scalable* parallel application systems**.

Scaling Characteristics of Parallel Programs

- Cost-optimal parallel systems have an efficiency of Θ(1).
- Scalability and cost-optimality are therefore related.
- A scalable parallel system can always be made cost-optimal if the number of processing elements and the size of the computation are chosen appropriately.

Example: adding numbers

- The parallel system for adding *n* numbers on *p* processing elements is cost-optimal when $n = \Omega(p \log p)$.
- The same parallel system is scalable if n is increased in proportion to $O(p \log p)$ as *p* is increased, which means: n'/n= (p'log p') /(p log p)
- Example on slide 6:
  - n=64 p=4 => E= 0.80.
  - n=192 p=8 => E= 0.80. … 192/64= 3 ; 8*3/4*2=3
  - n=512 p=16 => E= 0.80. … 512/64=8 ; 16*4/4*2=8

Metrics of Scalability

Basic:

- Scalability (scalab(n)) the ratio between the parallel execution time with parallelism degree equal to 1 and the parallel execution time with parallelism degree equal to n.

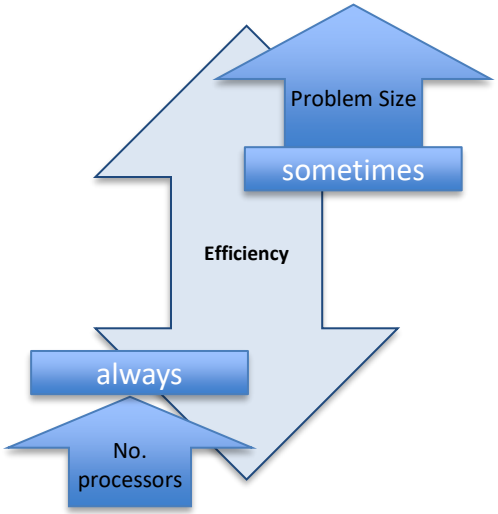
$$scalab(n) = \frac{T_{par}(1)}{T_{par}(n)}$$

- It measures how efficient is the parallel implementation in achieving better performances on larger parallelism degrees.
- As for the speedup, the asymptote is linear (f(n) = n) proof – similar to that for speed-up.
- However, scalability does not provide any comparison with the “best" sequential computation.

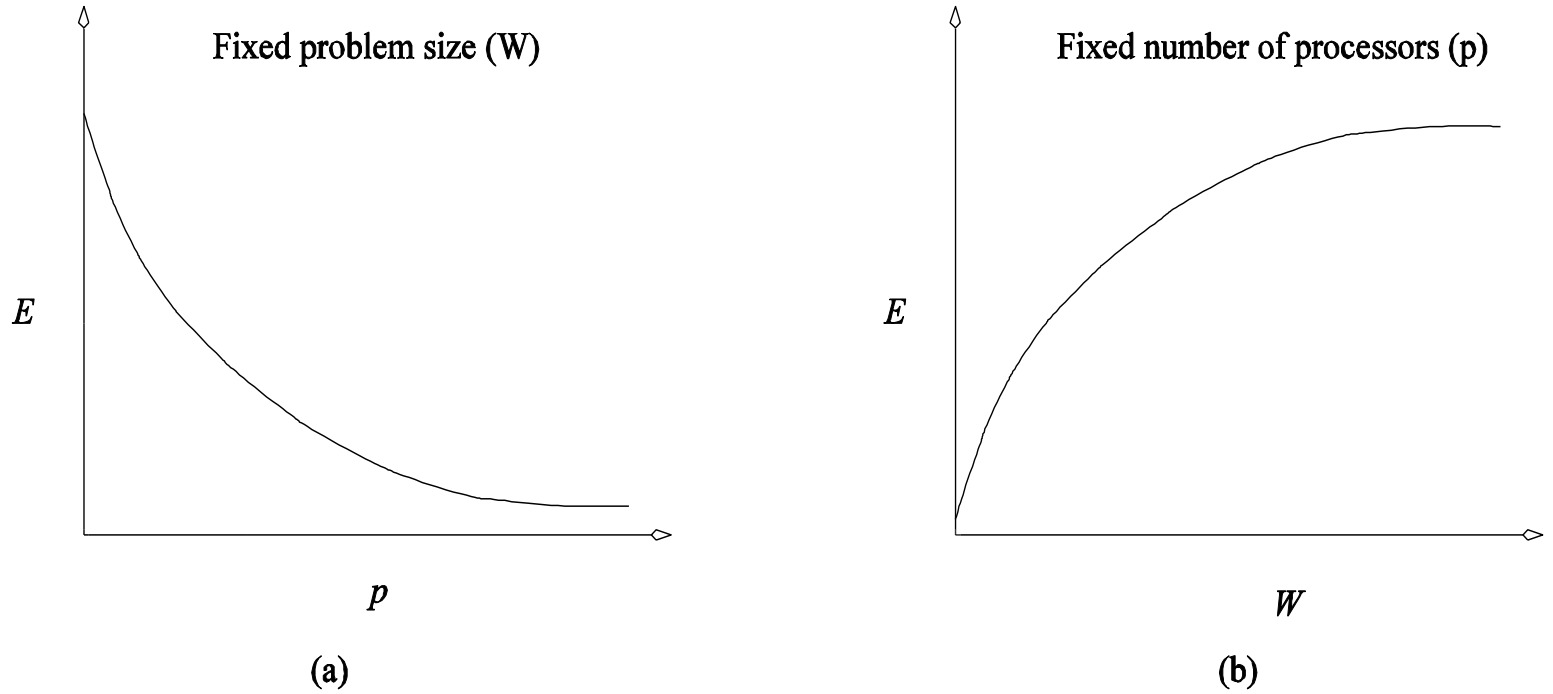
Iso-efficiency Metric of Scalability

Recall that:

- For all systems, as we increase the number of processing elements, the overall efficiency of the parallel system goes down, if we maintain a given problem size constant.
- For many systems, the efficiency of a parallel system increases if *the problem size(work) is increased* while keeping the number of processing elements constant.



Isoefficiency Metric of Scalability



Variation of efficiency:

(a) as the number of processing elements is increased for a given problem size; and

(b) as the problem size (W) is increased for a given number of processing elements. The phenomenon illustrated in graph (b) is not common to all parallel systems.

Isoefficiency Metric of Scalability

- **What is the rate at which the problem size must be increased with respect to the number of processing elements to keep the efficiency fixed?**

- This rate determines the scalability of the system.
  The slower this rate, the better.

- Before we formalize this rate, we define
  - the problem size ***W*** *(work)* as the asymptotic number of operations associated with the best serial algorithm to solve the problem.

Isoefficiency Metric of Scalability

- We can write parallel runtime as:

$$T_P = \frac{W + T_o(W,p)}{p}$$

- The resulting expression for speedup is

$$\begin{aligned} S &= \frac{W}{T_P} \\ &= \frac{Wp}{W + T_o(W,p)}. \end{aligned}$$

- Finally, we write the expression for efficiency as

$$\begin{aligned} E &= \frac{S}{p} \\ &= \frac{W}{W + T_o(W,p)} \\ &= \frac{1}{1 + T_o(W,p)/W}. \end{aligned}$$

Isoefficiency Metric of Scalability

- For scalable parallel systems, efficiency can be maintained at a fixed value (between 0 and 1) if the ratio T_o / W is maintained at **a constant value**.
- For a desired value E of efficiency,

$$E = \frac{1}{1 + T_o(W,p)/W},$$

$$\frac{T_o(W,p)}{W} = \frac{1-E}{E},$$

$$W = \frac{E}{1-E} T_o(W,p).$$

- If $K = E / (1 - E)$ is a constant depending on the efficiency to be maintained, since T_o is a function of W and p, we have

$$W = K T_o(W,p).$$

Isoefficiency Metric of Scalability

- The problem size ***W*** can usually be obtained as a function of ***p*** by algebraic manipulations to keep efficiency constant.
- This function is called the *isoefficiency function*.
- This function determines the ease with which a parallel system can maintain a constant efficiency and hence achieve speedups increasing in proportion to the number of processing elements

Isoefficiency Metric: Example

- The overhead function for the problem of adding ***n*** numbers on ***p*** processing elements is approximately ***2p* log *p*** .

$$W = KT_o(W, p).$$

- Substituting T_o by ***2p* log *p*** , we get

W = K*2*p* log p

Thus, **the asymptotic isoefficiency** function for this parallel system is

$$\Theta(p \log p)$$

- If the number of processing elements is increased from ***p*** to ***p'***, the problem size (in this case, n) must be increased by a factor of

(***p'* log *p'***) / (***p* log *p***) to get the same efficiency as on ***p*** processing elements.

Example:

- if ***p=4, p'=8 => W' = (8/4)(3/2)W => W'= 3W***

Isoefficiency Metric: Example

- Consider a more complex example where $T_o = p^{3/2} + p^{3/4}W^{3/4}$
- Using only the first term of T_o in $W = KT_o(W, p)$.

$W = K p^{3/2}$

- Using only the second term, yields the following relation between W and p:

$$W = Kp^{3/4}W^{3/4}$$

$$W^{1/4} = Kp^{3/4}$$

$$W = K^4p^3$$

The larger of these two asymptotic rates determines the asymptotic isoefficiency.

This is given by $\Theta(p^3)$

Cost-Optimality and the Isoefficiency Function

- A parallel system is cost-optimal if and only if

$$pT_P = \Theta(W).$$

- From this, we have:

$$\begin{aligned} W + T_o(W,p) &= \Theta(W) \\ T_o(W,p) &= O(W) \\ W &= \Omega(T_o(W,p)) \end{aligned}$$

- If we have an isoefficiency function ***f***(***p***), then it follows that the relation ***W*** = Ω(***f***(***p***)) must be satisfied to ensure the cost-optimality of a parallel system as it is scaled up.

Example: adding numbers **W = K*2*p* log p** => ***W*** = Ω(***plog p***) => n= Ω(***plog p***)

Lower Bound on the Isoefficiency Function

- For a problem consisting of ***W*** units of work, no more than ***W*** processing elements can be used cost-optimally.

- The problem size must increase at least as fast as Θ(***p***) to maintain fixed efficiency; hence, Ω(***p***) is the asymptotic lower bound on the isoefficiency function.

Degree of Concurrency and the Isoefficiency Function

- The maximum number of tasks that can be executed simultaneously at any time in a parallel algorithm is called its ***degree of concurrency*.**
- If ***C***(***W***) is the degree of concurrency of a parallel algorithm, then for a problem of size ***W***, no more than ***C***(***W***) processing elements can be employed effectively.

Degree of Concurrency and the Isoefficiency Function: Example

Consider solving a system of equations in variables by using Gaussian elimination ($\boldsymbol{W} = \Theta(\boldsymbol{n}^3)$)

- The $\boldsymbol{n}$ variables must be eliminated one after the other, and eliminating each variable requires $\Theta(\boldsymbol{n}^2)$ computations.
- At most $\Theta(\boldsymbol{n}^2)$ processing elements can be kept busy at any time.
- Since $\boldsymbol{W} = \Theta(\boldsymbol{n}^3)$ for this problem, the degree of concurrency $\boldsymbol{C}(\boldsymbol{W})$ is $\Theta(\boldsymbol{W}^{2/3})$ (n- expressed in function of W) .
- Given $\boldsymbol{p}$ processing elements, the problem size should be at least $\Omega(\boldsymbol{p}^{3/2})$ to use them all.

Minimum Execution Time and Minimum Cost-Optimal Execution Time

Often, we are interested in the minimum time to solution.

(find the optimal number of processors)

- We can determine the minimum parallel runtime T_P^{min} for a given W by differentiating the expression for T_P w.r.t. p and equating it to zero.

$$\frac{\mathrm{d}}{\mathrm{d}p} T_P = 0$$

- If p_0 is the value of p as determined by this equation, $T_P(p_0)$ is the minimum parallel time.

Minimum Execution Time: Example

Consider the minimum execution time for adding ***n*** numbers.

$$T_P = \frac{n}{p} + 2\log p.$$

Setting the derivative w.r.t. ***p*** to zero, we have ***p = n/ 2*** . The corresponding runtime is

$$T_P^{min} = 2\log n.$$

(One may verify that this is indeed a min by verifying that the second derivative is positive).

Note that at this point, the formulation is not cost-optimal.

Minimum Cost-Optimal Parallel Time

- Let $T_P^{cost_opt}$ be the minimum cost-optimal parallel time.
- If the isoefficiency function of a parallel system is $\Theta(f(p))$, then a problem of size W can be solved cost-optimally if and only if
  $W = \Omega(f(p))$.
- In other words, for cost optimality, $p = O(f^{-1}(W))$.
- For cost-optimal systems, $T_P = \Theta(W/p)$, therefore,
- $$T_P^{cost_opt} = \Omega\left(\frac{W}{f^{-1}(W)}\right).$$

Minimum Cost-Optimal Parallel Time: Example

Consider the problem of adding ***n*** numbers.

- $W = n$, $T_p = n/p + \log p$
- The isoefficiency function ***f***(***p***) of this parallel system is Θ(***p*** **log** ***p***).
- W = n = p log p
- From this, we have ***p*** ≈ ***n*** /**log** ***n*** .
- At this processor count, the parallel runtime is:

$$T_P^{cost_opt} = \log n + \log\left(\frac{n}{\log n}\right)$$

$$= 2\log n - \log\log n.$$

- Note that both T_P^{min} and $T_P^{cost_opt}$ for adding ***n*** numbers are Θ(**log** ***n***). This may not always be the case.

Other scalability metrics

Serial Fraction *f – another metric for scalability*

- If the serial runtime of a computation can be divided into a totally parallel and a totally serial component, we have:

$$W = T_{ser} + T_{par}.$$

- From this, we have,

$$T_P = T_{ser} + \frac{T_{par}}{p}.$$

$$T_P = T_{ser} + \frac{W - T_{ser}}{p}$$

Serial Fraction ***f*** *(… Amdahl law…)*

- The serial fraction ***f*** of a parallel program is defined as:

$$f = \frac{T_{ser}}{W}.$$

- Therefore, we have:

$$T_P = f \times W + \frac{W - f \times W}{p}$$

$$\frac{T_P}{W} = f + \frac{1 - f}{p}$$

Serial Fraction

- Since $S = W / T_P$, we have

$$\frac{1}{S} = f + \frac{1 - f}{p}.$$

- From this, we have:

$$f = \frac{1/S - 1/p}{1 - 1/p}.$$

- If f increases with the number of processors, this is an indicator of rising overhead, and thus an indicator of poor scalability.

Serial Fraction: Example

Consider the problem of estimating the serial component of the matrix-vector product (on a hypercube).

We have:

$$f = \frac{\frac{t_c n^2}{p} + t_s \log p + t_w n}{t_c n^2} \Big/ \frac{1 - 1/p}{}$$

or

$$f = \frac{t_s p \log p + t_w n p}{t_c n^2} \times \frac{1}{p - 1}$$

$$f \approx \frac{t_s \log p + t_w n}{t_c n^2}$$

Here, the denominator is the serial runtime and the numerator is the overhead.

Isospeed scalability metric

X.H. Sun and D. Rover, "Scalability of Parallel Algorithm–Machine Combinations", *IEEE Transaction on Parallel Distributed Systems*, Vol. 5, pp.599–613, 1994.

- Isospeed scalability is a performance metric used to evaluate parallel systems by measuring if the average speed per processor remains constant as system size (p) increases, provided the workload (W) is scaled proportionally.
- An algorithm-machine combination is considered scalable under isospeed if the average speed per processor can be maintained by increasing the workload.
- The average unit speed (speed per processor) is defined as the system's achieved speed divided by the number of processors, in homogeneous environment.

- The isospeed scalability function is $$\Psi(p, p') = \frac{p' W}{p W'}$$

where p and p' are the initial and scaled number of processors,
and W and W' are the initial and scaled work (problem size) respectively.

isospeed-e scalability metric

X.H. Sun, Y. Chen and M. Wu, "Scalability of Heterogeneous Computing", *Proceedings of 34th International Conference on Parallel Processing*, pp.557-564, 2005.

- In *isospeed-e scalability* metric a new concept of *marked speed* is introduced to describe the combined computing power of a general parallel computing system.

- The marked speed of a computing node is the (benchmarked) sustained speed of that node. It represents the computational capability of that node. It can be calculated based on hardware peak performance, which in general is higher than actually delivered performance. In practice, computation intensive benchmarks can be used to measure the marked speed of each node.

- Once the marked speed of a computing node is measured, it is used as a constant parameter, like CPU frequency or memory access latency, in all experimental analyses.

- The marked speed of a computing system is defined as the sum of the marked speed of each node that composes the computing system. Let *Ci* denote the marked speed of node *i*. In a heterogeneous environment, *Ci* may be different from each other due to the heterogeneity. In homogeneous environment, all *Ci* are the same. Let *C* stand for the marked speed of a computing system.

… short summary

- An increase of the number of processors leads to a decrease of efficiency.
- The increase of problem size causes an increase in efficiency.
- A scalable parallel problem can be solved efficiently by increasing the number of processors and the problem size simultaneously.

Lecture 7

PRAM models

Computational networks

Brent Theorem

PRAM – Reference model

- Theoretical Model of Parallel Computation

used well for performance evaluation

- The abstract model used for serial computing is known as RAM (for random-access machine, or model).

- PRAM is the counterpart for parallel computing.

RAM model

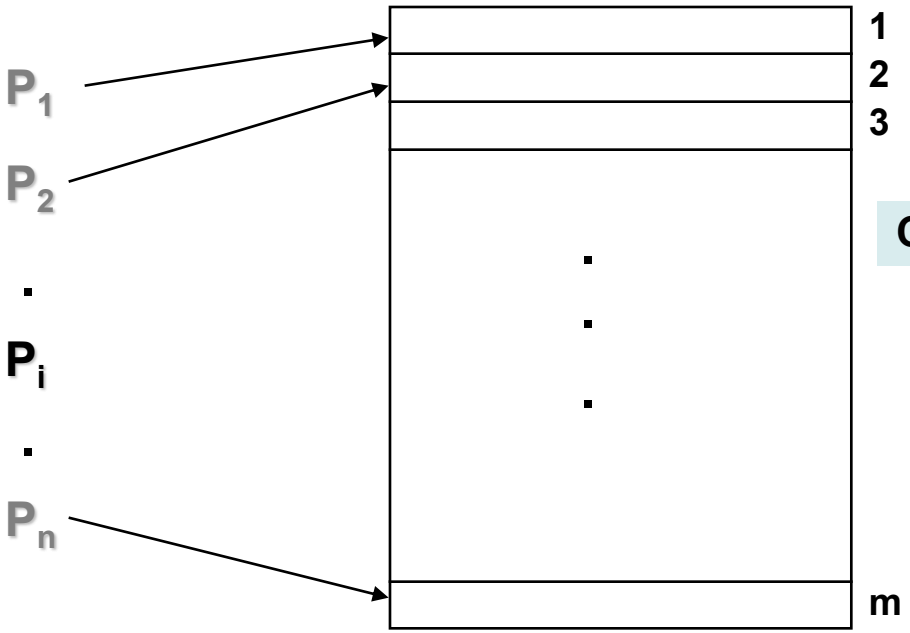
- **Random Access Machine** is a favorite model of a sequential computer. Its main features are:
  - Computation unit with a user defined program.
  - Read-only input tape and write-only output tape.
  - **Unbounded** number of local **memory** cells.
  - Each memory cell is capable of holding an integer of unbounded size.
  - Instruction set includes operations for :
    - moving data between memory cells,
    - comparisons
    - conditional branches, and
    - simple arithmetic operations.
  - Execution starts with the first instruction and ends when a HALT/STOP instruction is executed.
  - All operations take unit time regardless of the lengths of operands.
  - **Time complexity** = the number of instructions executed.
  - **Space complexity** = the number of memory cells accessed.

Parallel RAM-> PRAM model

- Parallel Random Access Machine
- Shared-memory multiprocessor
- **unlimited number of processors,** each
  - has unlimited local memory
  - knows its ID
  - able to access the shared memory
- **unlimited shared memory**

PRAM MODEL

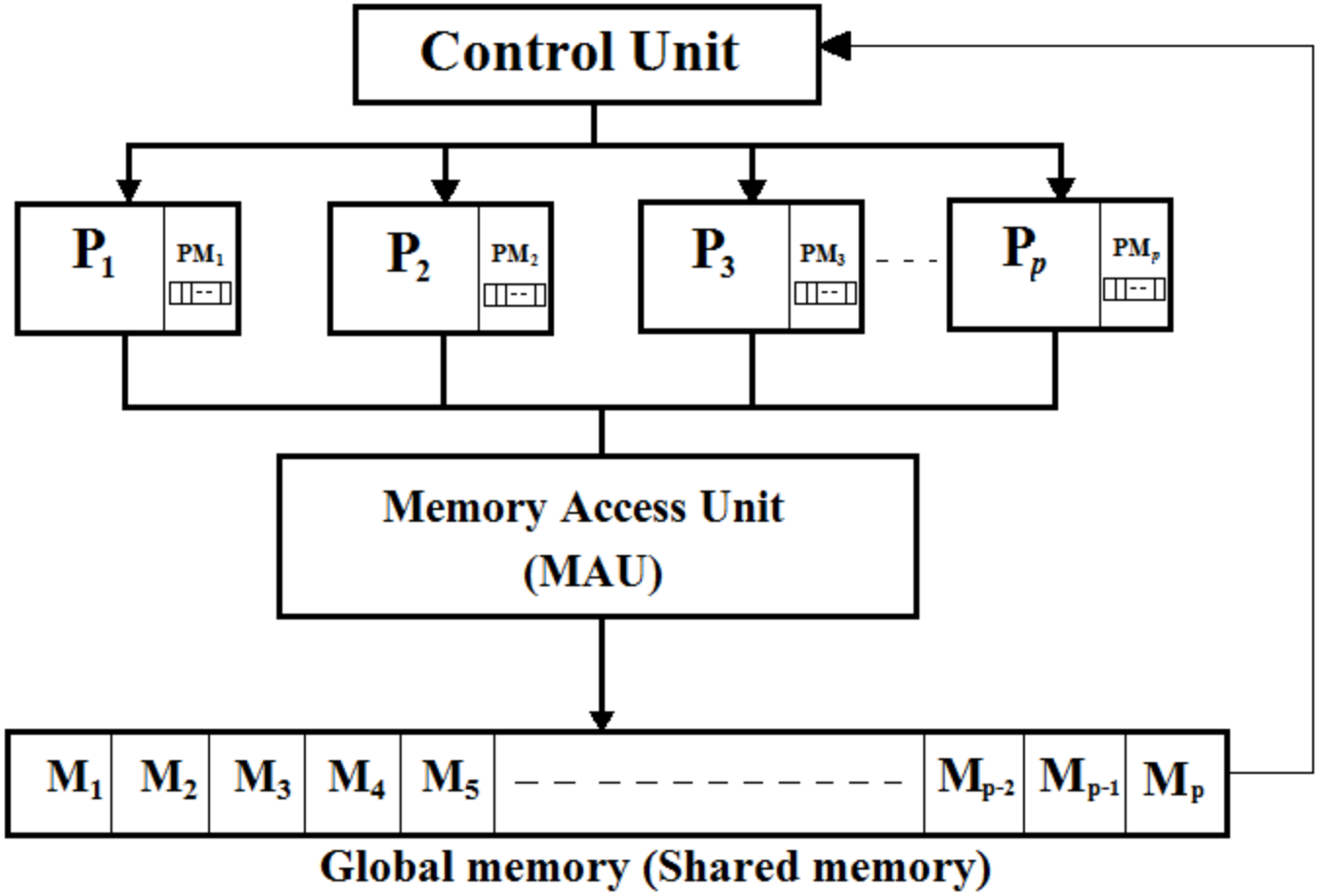
***n* RAM processors connected to a common memory of *m* cells**



ASSUMPTION: at each time unit each P_i can
- reading from memory,
- make an internal computation and
- writing into memory.

CONSEQUENCE: any pair of processors P_i P_j can communicate in constant time!
P_i writes the message in cell x at time t
P_j reads the message in cell x at time t+1

PRAM architecture model



PRAM

- Inputs/Outputs are placed in the shared memory. (designated address)
- Memory cell stores an arbitrarily large integer.
- Each instruction takes a unit time.
- **Instructions are synchronized across the processors**
- A PRAM instruction executes in **3-phase cycles**.
  - **Read** (if any) from a shared memory cell.
  - **Local computation** (if any).
  - **Write** (if any) to a shared memory cell.

PRAM Complexity Measures

- for each individual processor
  - ***time***: number of instructions executed
  - ***space***: number of memory cells accessed

- PRAM machine
  - ***time***: time taken by the longest running processor
  - ***space***: total number of memory cells accessed
  - ***hardware***: maximum number of active processors

Technical Issues for PRAM

- How processors are activated
- How shared memory is accessed

- Too many processors gives problems with synchronization

PRAM execution

- During computation steps processors have a special activation register that specifying the maximum index of an active processor.
- Initially, only the first processor is in active mode, it computes the number of required active processors and loads this register, and then the other corresponding processors start executing their programs.
- The computation will continue until the first processor halts, at which time all other active processors are halted.

Processor Activation – 2 variants

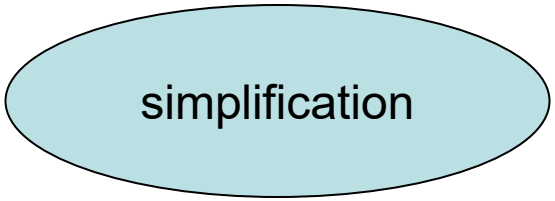
- P_0 places the number of processors (p) in the designated shared-memory cell
  - each active P_i (where $i < p$), starts executing
  - $O(1)$ time to activate
  - all processors halt when P_0 halts

- Active processors explicitly activate additional processors via FORK instructions
  - tree-like activation
  - $O(\log p)$ time to activate

Parallel time complexity = *the time elapsed for P_0's computation.*

PRAM

- To describe a program in PRAM model we need to specify **n** programs (n=no of processors)
  - 1 program defined based on the ID => n programs
- Pros: It is a very good **conceptual model** for designing efficient parallel algorithms
  - due to simplicity and possibility of simulating efficiently PRAM algorithms on more realistic parallel architectures



WT paradigm

basic parallel statement

for all x in X do in parallel
instruction(x)

Vectors sum – PRAM

Algorithm 1 (Vector sum on a PRAM)
Input: Vectors $v[1..n]$ and $w[1..n]$ in shared memory.
Output: Vector $z[1..n]$ in shared memory.

PRAM with ***n*** processors

Processors: $P_0 \ldots P_{n-1}$

Algorithm of P_i

Read v[i]
Read w[i]
z[i] ← v[i]+w[i]
Write z[i]

One step

Vectors sum – bounded parallelism

PRAM with ***p*** processors

Processors: $P_0 \ldots P_{p-1}$

P_i

Algorithm 1 (Vector sum on a PRAM)
Input: Vectors $v[1..n]$ and $w[1..n]$ in shared memory.
Output: Vector $z[1..n]$ in shared memory.

```
1  local integer h
2  for h = 1 to ⌈n/p⌉ do
3     if (h − 1)p + i ⩽ n then
4        z[(h − 1)p + i] ← v[(h − 1)p + i] + w[(h − 1)p + i]
5     endif
6  enddo
```

Algorithm (WT paradigm)

```
forall i in [1..n] do in parallel
        z[i] = v[i]+ w[i]
forall i in [1..m] do in parallel
        z[i] = v[i]+ w[i]
```

The Work-Time paradigm

- **Work-Time** = A higher-level abstraction than PRAM.

- In the PRAM model, ***algorithms are presented as a program to be executed by all the processors****;* in each step an operation is performed simultaneously by all active processors.

- In the WT model, each step may contain an arbitrary number of operations to be performed simultaneously, and the *scheduling of these operations over processors is left implicit.*
  - we will use the **forall** construct to denote such concurrent operations, and we drop explicit mention of the processor *id* and *p*, the number of processors.

- In fact the **forall** construct is the only construct that distinguishes a WT algorithm from a sequential algorithm.

Example

Algorithm 2 (Sequence reduction, WT description)
Input: Sequence a with $n = 2^k$ elements of type T, binary associative operator $\oplus : T \times T \to T$.
Output: $S = \oplus_{i=1}^{n} a_i$.

T REDUCE(**sequence**⟨T⟩ a, $\oplus : T \times T \to T$)

1 $T\ B[1..n]$
2 **forall** $i \in 1 : n$ **do**
3 $\quad B[i] \leftarrow a_i$
4 **enddo**
5 **for** $h = 1$ **to** k **do**
6 $\quad$ **forall** $i \in 1 : n/2^h$ **do**
7 $\quad\quad B[i] \leftarrow B[2i-1] \oplus B[2i]$
8 $\quad$ **enddo**
9 **enddo**
10 $S \leftarrow B[1]$
11 **return** S

The *step complexity* of the algorithm, denoted by $S(n)$, is the number of steps that the algorithm executes. (**Depth**)
If $W_i(n)$ is the number of simultaneous operations at parallel step i, then

$$W(n) = \sum_{i=1}^{S(n)} W_i(n).$$

$$S_{2-4}(n) = \Theta(1)$$
$$S_{6-8}(n) = \Theta(1)$$
$$S_{5-9}(n) = kS_{6-8}(n) = \Theta(\lg n)$$
$$S_{10}(n) = \Theta(1)$$
$$S(n) = S_{2-4}(n) + S_{5-9}(n) + S_{10}(n) = \Theta(\lg n)$$

$$W_{2-4}(n) = \Theta(n)$$
$$W_{6-8}(n,h) = \Theta(\tfrac{n}{2^h})$$
$$W_{5-9}(n) = \textstyle\sum_{h=1}^{k} W_{6-8}(n,h) = \Theta(n)$$
$$W_{10}(n) = \Theta(1)$$
$$W(n) = W_{2-4}(n) + W_{5-9}(n) + W_{10}(n) = \Theta(n)$$

Work-Depth analysis…

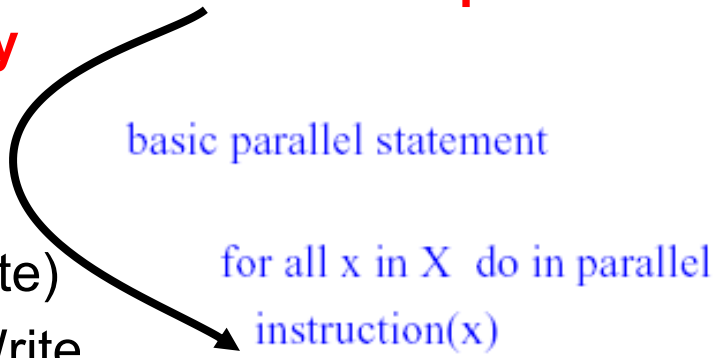
Variant…

```
for h =1 to k do
    forall  i ∈ 1,n do
        if ( i % 2^h =0 and i+2^(h-1)<=n ) then
                B[i] ← B[i]+ B[i+2^(h-1)]
        endif
    enddo
enddo
```

Shared-Memory Access

Concurrent (C) means, many processors can do the operation simultaneously in the same memory

Exclusive (E) not concurent



- EREW (Exclusive Read Exclusive Write)
- CREW (Concurrent Read Exclusive Write
  - Many processors can read simultaneously the same location, but only one can attempt to write to a given location
- ERCW
- CRCW
  - **Many processors can write**/read **at**/from **the same memory location**

example

As a simple example of a CREW PRAM computation, consider the problem of computing $ab+ac+bd+cd$ from inputs a, b, c, d. Let $p_i t_j$ denote the computation performed by processor i at time step j. Then we have

$$
\begin{aligned}
p_1 t_1 &: b + c \Rightarrow x \\
p_1 t_2 &: a * x \Rightarrow y \\
p_2 t_2 &: x * d \Rightarrow z \\
p_1 t_3 &: y + z \Rightarrow \text{result}
\end{aligned}
$$

Basic submodels for CRCW PRAM

- **PRIORITY CRCW:** the processors are assigned fixed distinct priorities and the processor with the highest priority is allowed to complete WRITE.

- **ARBITRARY CRCW:** one randomly chosen processor is allowed to complete WRITE. The algorithm may make no assumptions about which processor was chosen.

- **COMMON CRCW:** all processors are allowed to complete WRITE **iff** all the values to be written are equal. Any algorithm for this model has to make sure that this condition is satisfied. If not, the algorithm is illegal and the machine state will be undefined.

- **OPERATOR CRCW**: for concurrent WRITE operation each processor will update the location by applying a given associative operator e.g. summation.

Example - searching

- Assume *p*-processor PRAM, ***p<n*.** Assume that shared memory contains ***n*** distinct items and *P0* owns value ***x***.
  The task is to let *P0* know whether ***x*** occurs within the input array.

- EREW PRAM algorithm:
  - *P0* broadcasts *x* to *P1,...,Pp* in *log p* steps using **binary broadcast tree**.
  - All processors perform local searches, each on *[n/p]* items in *[n/p]* steps.
  - Every processor defines a flag *Found* and all processors perform a **parallel reduction**.

    T(n,p) = O(log p + n/p)

- CREW PRAM algorithm: A similar approach, but *P1,...,Pp* can read *x* simultaneously in *O(1)* time. But the final reduction takes *O(log p)* time anyway, so
  T(n,p) = O(log p + n/p)

- COMMON CRCW PRAM algorithm: The final step takes now also *O(1)* time, those processors with the flag *Found* set can write simultaneously into *P0*'s cell
  T(n,p) = O(n/p).

Example CRCW-PRAM

- Initially
  - table **A** contains values 0 and 1
  - **output** contains value 0

$$\text{for each } 1 \leq i \leq 5 \text{ do in parallel}$$
$$\text{if } A[i] = 1 \text{ then output=1;}$$

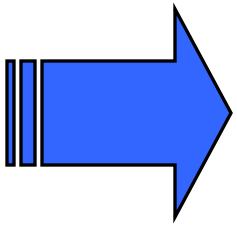
- The program computes the "Boolean OR" of A[1], A[2], A[3], A[4], A[5]

Example CREW-PRAM

- Assume initially table **A** contains [0,0,0,0,0,1] and we have the following parallel program

for each $1 \leq i \leq 5$ do in parallel
$A[i] := A[i] + A[i+1]$

p_i : $1 \leq i < 5$



then the consecutive values of the tables A (in parallel step 0, 1, 2, 3, 4, 5) correspond to the Pascal triangle, the nonzero elements in the n-th row are

$$\binom{n}{0}, \binom{n}{1}, \binom{n}{2}, \cdots \binom{n}{n}$$

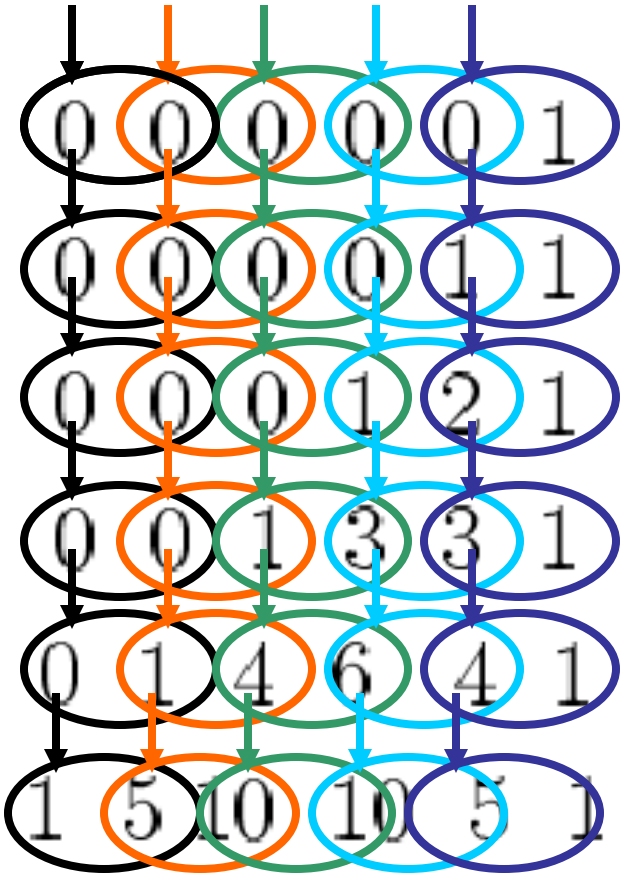
for $n = 0, 1, 2, 3, 4, 5, 6.$

Pascal triangle

$$\binom{n}{0}, \binom{n}{1}, \binom{n}{2}, \ldots \binom{n}{n}$$

for $n = 0, 1, 2, 3, 4, 5, 6.$

$p_i : 1 \leq i < 5$



PRAM CREW (WT)

for each $1 \leq i \leq 5$ do in parallel
$A[i] := A[i] + A[i+1]$

Operator CRCW Example- Rank Sort

A 2 steps algorithm - ***rank sort***

• Compute the rank (position in sorted order) for each element in parallel

– Compare all possible pairings of input values in parallel,

n^2-fold parallelism

– CRCW model with update on write using summation (+)

• Move each value to its correctly sorted position according to the rank: n-fold parallelism

• O(1) running time

Rank sort on a PRAM

- Compute the rank for all possible pairings of inputs in parallel, n2-fold parallelism
- Move each value in position according to the rank: n-fold parallelism

```
forall i=0:n-1, j=0:n-1
        if ( x[i] > x[j] ) then rank[i] = 1 endif
forall i=0:n-1
        y[rank[i]] = x[i]
```

PRAM Variants =>Synchronization

- The standard PRAM posits a rigid execution pattern in which all processors are synchronized by a global clock.

- Several variants ease this restriction:
  - APRAM
  - Asynchronous PRAM (irregular synchronization points)
  - XPRAM (Periodic synchronization between intervals of asynchronous execution)
- **While these models incorporate synchronization, they do not charge an explicit cost for it !!!**

PRAM Variants => Latency

- The cost of non local memory accesses has a severe effect on the performance in massively parallel computers.

=>Several PRAMs were designed to remedy the unit memory cost idealization.

- LPRAM has been augmented by charging a cost of ***l*** units to access global memory.
- An elaboration of this model = BPRAM, augmented this by charging ***l*** units for the first message from global memory and a variable cost ***b*** for each additional memory access in a contiguous block.
- Thus, the BPRAM provides incentives for one level of reference locality and for block transfer a form of data parallelism.

PRAM Variants => Bandwidth

- DRAM - A PRAM variant which assumes two classes of memory and includes a mechanism for assigning a non unit cost to a remote access
  - The DRAM is important because it eliminates the paradigm of global shared memory and replaces it with only private distributed memory.
  - While the **topology** of the communication network is **ignored** the DRAM incorporates the notion of limited bandwidth.
  - This model proposes a cost function for a nonlocal memory access which is based on the maximum possible congestion for a given data partition and execution sequence.
  - While the function is somewhat complicated it attempts to provide scheduling incentives to respect limited access to nonlocal data .
- Another PRAM variant proposed in the PRAMm incorporates bandwidth limitations by restricting the size of global shared memory to m memory locations The model is a CRCW PRAM except that in any given step only *m* accesses can be serviced.

Constrained PRAM models

- **Bounded** number of shared **memory cells**. This may be called a **small memory PRAM**.
  - If the input data set exceeds the capacity of the shared memory, the input and/or output values can be distributed evenly among the processors.

- **Bounded** size of a **machine word** and/or **memory cell**. This parameter is usually called **word size of PRAM**.

- **Bounded** number of **processors**.
  This may be called a **small PRAM**. If the number of threads of execution is higher, processors may interleave several threads.

- Constraints on simultaneous access to shared memory cells: handling **access conflicts(EREW, CRCW…)**.

Communication constraint

- Limiting the amount of PRAM shared memory corresponds to restricting the *amount of information that can be communicated between processors in one step*.

- For example, a *distributed memory* machine with processors interconnected by a *shared bus* can be modeled as a PRAM with *one shared memory cell*.

Computational power

- Having this range of submodels, we must ask how they compare as to the ability to execute parallel algorithms. Various submodels may have different **computational power**.

Definition

PRAM submodel A is **computationally stronger** than submodel B, written A>=B, if any algorithm written for B will run **unchanged** on A in the same parallel time, assuming the same basic properties.

Lemma

PRIORITY >= ARBITRARY >= COMMON >= CREW >= EREW

Simulation of large PRAMs on small PRAMs

- Small PRAMs can simulate larger PRAMs.
- Even though relatively simple, the following two simulations are very useful and notoriously used.
  - The first result is related to decreasing the number of processors.
  - The second result is related to decreasing the number of memory cells.

Lemma 1

Assume $p' < p$.

Any problem that can be solved on a p-processor PRAM in t steps can be solved on a p'-processor PRAM in

$t' = O(t * p/p')$

steps assuming the same size of shared memory.

(see Lecture 6 – Building granularity)

Proof:

- Partition p simulated processors into p' groups of size p/p' each.
- Associate each of the p' simulating processors with one of these groups.
- Each of the simulating processors simulates one step of its group of processors by:
  - executing all their READ and local computation substeps first,
  - executing their WRITE substeps then.

Vectors sum

PRAM with p processors

Processors: $P_0 \ldots P_{p-1}$

Algorithm 1 (Vector sum on a PRAM)

Input: Vectors $v[1..n]$ and $w[1..n]$ in shared memory.

Output: Vector $z[1..n]$ in shared memory.

```
1  local integer h
2  for h = 1 to ⌈n/p⌉ do
3     if (h − 1)p + i ⩽ n then
4        z[(h − 1)p + i] ← v[(h − 1)p + i] + w[(h − 1)p + i]
5     endif
6  enddo
```

•We assume that *h, l, n* and *p* are in the local memory.
•Under this assumption, all references to shared memory in Algorithm 1 are exclusive, and the algorithm requires only an EREW PRAM (Exclusive Read Exclusive Write).
•Algorithm 1 requires on the order of *n/p* steps to execute, so the parallel (concurrent) running time $T_c(n, p) = O(n/p)$.

Lemma 2

Assume *m'*<*m*.

Any problem that can be solved on a *p*-processor and *m*-cell PRAM in *t* steps can be solved on a *max(p, m')*-processor *m'*-cell PRAM in *O(t * m/m')* steps.

Proof:

- Partition *m* simulated shared memory cells into *m'* continuous segments S_i of size *m/m'*.
- Each simulating processor P'_i, *1*<= *i*<= *p*, will simulate processor P_i of the original PRAM.
- Each simulating processor P'_i, *1*<= *i* <= *m'*, stores the initial contents of S_i into **its local memory** and will use *M'[i]* as an auxiliary memory cell for simulation of accesses to cells of S_i.
- Simulation of one original READ operation:
  each P'_i, *i=1,...,max(p, m')* repeats for *k=1,...,m/m'*:
  - write the value of the *k*-th cell of S_i into *M'[i]*, *i=1,...,m'*,
  - read the value which the simulated processor P_i, *i=1,...,p*, would read in this simulated substep, if it appeared in the shared memory.
- The local computation substep of P_i, *i=1,...,p*, is simulated in one step by P'_i.
- Simulation of one original WRITE operation is analogous to that of READ.

PRAM models equivalance

- Two models M_1 and M_2 are **PRAM-equivalent** if:

Any algorithm running in time $T(n)$ on M_1 with $P(n)$ processors can be simulated on M_2 with at most **polylogarithmic overhead** in time and a comparable number of processors, and vice versa.

Formally:

If Model A solves a problem in time $T(n)$ using $P(n)$ processors

Then Model B is equivalent if it can simulate A in:

$$T'(n) = O\big(T(n) \cdot \mathit{polylog}(n)\big)$$

- And the reverse simulation also holds.

Simulation of stronger PRAM models on weaker ones

- It is very useful to know efficient simulations of stronger PRAM models on weaker ones, since a stronger model is more convenient for the design of algorithms, whereas weaker models, such as EREW, are closer to real parallel computers.
- Since it is technologically difficult to build full massively parallel CREW or CRCW PRAM computers, it is important to understand the costs of simulating the CREW or CRCW machines on EREW.
- Any multiple access has to be converted into a series of exclusive accesses.
- The most important are simulations of the strongest PRIORITY CRCW on the weakest EREW.

Lemma

Assume PRIORITY CRCW with the priority scheme based trivially on indexing: lower indexed processors have higher priority.

One step of *p*-processor *m*-cell PRIORITY CRCW can be simulated by a *p*-processor *mp*-cell EREW PRAM in *O(log p)* steps.

Proof:(Constructive.)

- Each PRIORITY processor *Pk* is simulated by EREW processor *P'k*.
- Each shared memory cell *M[i]*, *i=1,...,m*, of PRIORITY is simulated by an **array** of *p* shared memory cells *M'[i,k]*, *k=1,...,p*, in the EREW PRAM.
  - *M'[i,1]* plays the role of *M[i]*.
  - *M'[i,2],...,M'[i,p]* are **auxiliary cells** used for resolving conflicts, initially **empty**, and organized as **internal nodes(p-1) of a complete binary tree** *Ti* with *p* leaves. The height of every *Ti* is *[log p]*.

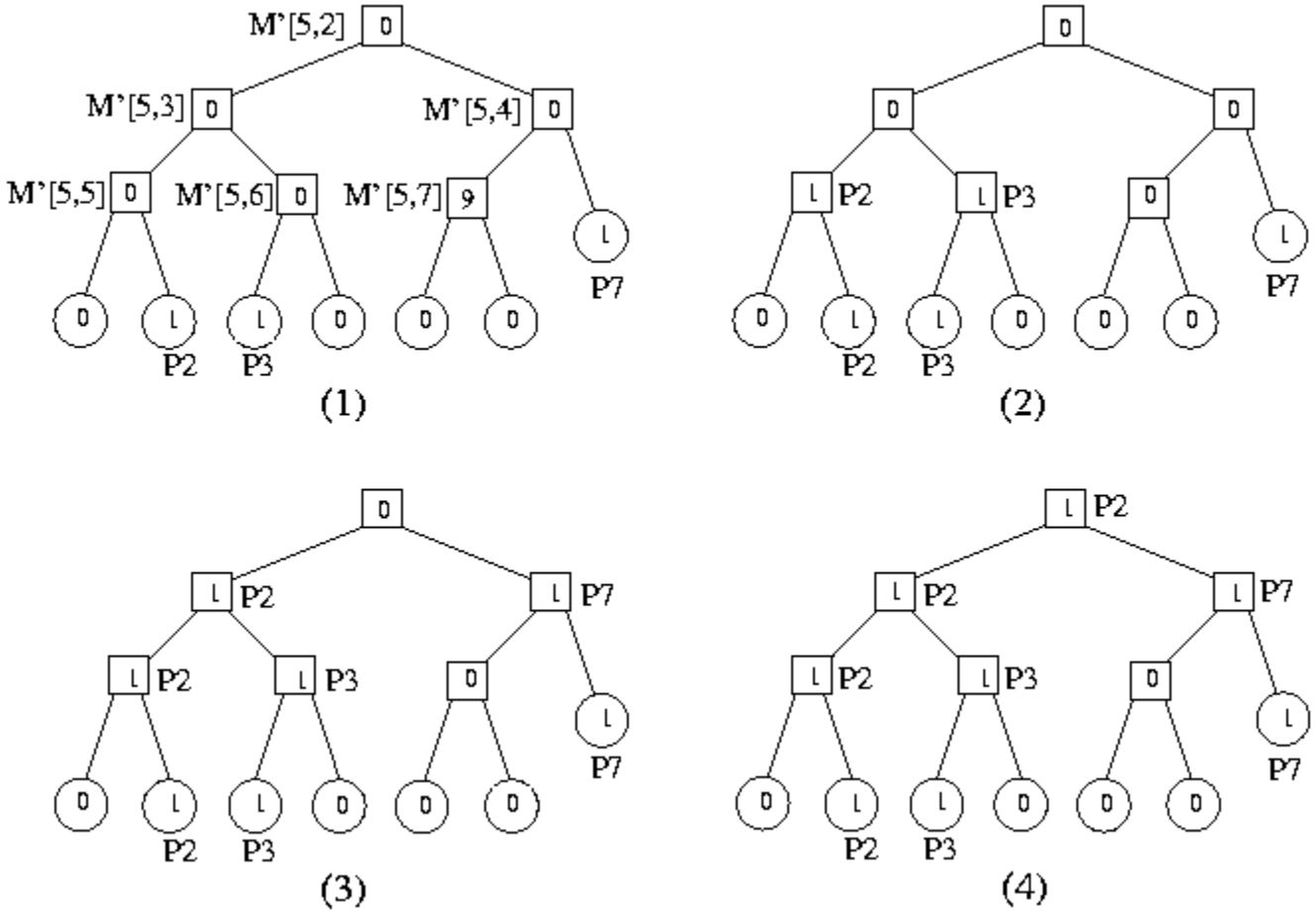
- **Simulation of a PRIORITY WRITE substep:** Each EREW processor must find out whether it is the processor with the lowest index within the group of processors asking to write to the same cell, and if so, it must become the group winner and perform the WRITE operation. The other processors in the group will fail. This is done as follows:

  – If *Pk* wants to write into *M[i]*, processor *P'k* turns **active** and becomes *k*-th leaf of *Ti*. It knows whether it is the right or left child of its parent.
  – Each active **left** processor stores its ID into the parent cell in its tree, marks it as **occupied**, and remains active.
  – Each active **right** processor checks its parent cell. If this is empty, it stores its ID into it, and remains active. If it is occupied, it becomes idle (he lost).
  – This is repeated *log p* times at further levels of the trees.
  – The processor who managed to proceed to the root of *Ti*, becomes the winner who can write into *M[i]*. Processors which used *Ti* must then sweep down *Ti* in the reverse order to reset the cells *M'[i,2],...,M'[i,p]* to empty.

- **Simulation of a PRIORITY READ substep:** is similar.
  - The same sweep-ups of the auxiliary trees are performed in parallel to determine the winners in the groups.
  - The winners will read the values from the cells *M'[*,1]*
  - During the cleaning sweep-down, the read value is distributed to the losers.

Example:

p=7, processors *P2, P3*, and *P7* wish to write into *M[5]*.



Lemma

One step of PRIORITY CRCW with *p* processors and *m* shared memory cells can be simulated by an EREW PRAM in *O(log p)* steps with *p* processors and *m+p* shared memory cells.

Proof:(Constructive.)

- Each PRIORITY processor *Pk* is simulated by EREW processor *P'k*.
- Each PRIORITY cell *M[i]* is simulated by EREW cell *M[i]*.
- EREW uses an **auxiliary** array *A* of *p* cells.

- If *Pk* wants to access *M[i]*, processor *P'k* writes pair *(i,k)* into *A[k]*. If *Pk* does not want to access any PRIORITY cell, processor *P'k* writes *(0,k)* into *A[k]*.
- All *p* processors sort the array *A* into lexicographic order using *(log p)*-time parallel sort.
  - Each *P'k* appends to cell *A[k]* a flag *f*:

  f=0, if the first component of A[k] is either 0 or it is the same as the first component of A[k-1]

  f=1 otherwise.

  Further steps differ for simulation of WRITE or READ.

- **PRIORITY WRITE:**
  - Each *P'k* reads the triple *(i,j,f)* from cell *A[k]* and writes it into *A[j]*.
  - Each *P'k* reads the triple *(i,k,f)* from cell *A[k]* and writes into *M[i]* iff *f=1*.
- **PRIORITY READ:**
  - Each *P'k* reads the triple *(i,j,f)* from cell *A[k]*.
  - If *f=1*, it reads value *vi* from *M[i]* and overwrites the third component in *A[k]* the flag *f*, with *vi*.
  - In at most *log p* steps, this third component is then distributed in subsequent cells of *A* until it reaches either the end or an element with a different first component.
  - Each *P'k* reads the triple *(i,j,vi)* from cell *A[k]* and writes it into *A[j]*.
  - Each *P'k* who asked for a READ reads the value *vi* from the triple *(i,k,v)* in cell *A[k]*.

Example

Assume *p=7*, *m=4*, and

- *P1* wants to access *M[2]*,
- *P2* wants to access *M[4]*,
- *P3* wants to access *M[2]*,
- *P4* wants to access *M[1]*,
- *P5* wants to access *M[4]*,
- *P6* wants to access *M[2]*,
- *P7* wants to access no cell at all.

- Array *A* in the first three steps of simulation:

(2,1,)	*(4,2,)*	*(2,3,)*	*(1,4,)*	*(4,5,)*	*(2,6,)*	*(0,7,)*
(0,7,)	*(1,4,)*	*(2,1,)*	*(2,3,)*	*(2,6,)*	*(4,2,)*	*(4,5,)*
(0,7,0)	*(1,4,1)*	*(2,1,1)*	*(2,3,0)*	*(2,6,0)*	*(4,2,1)*	*(4,5,0)*

- Array *A* in simulation of WRITE:

(2,1,1)	*(4,2,1)*	*(2,3,0)*	*(1,4,1)*	*(4,5,0)*	*(2,6,0)*	*(0,7,0)*

- Array *A* in simulation of READ:

(0,7,0)	*(1,4,v_1)*	*(2,1,v_2)*	*(2,3,0)*	*(2,6,0)*	*(4,2,v_4)*	*(4,5,0)*
(0,7,0)	*(1,4,v_1)*	*(2,1,v_2)*	*(2,3,v_2)*	*(2,6,0)*	*(4,2,v_4)*	*(4,5,v_4)*
(0,7,0)	*(1,4,v_1)*	*(2,1,v_2)*	*(2,3,v_2)*	*(2,6,v_2)*	*(4,2,v_4)*	*(4,5,v_4)*
(2,1,v_2)	*(4,2,v_4)*	*(2,3,v_2)*	*(1,4,v_1)*	*(4,5,v_4)*	*(2,6,v_2)*	*(0,7,0)*

Conclusion

- Any polylog-time PRAM algorithm is robust with respect to PRAM models.

PRAM is an attractive and important model for designers of parallel algorithms. Why?

It inspires real world system and algorithm designs.

- It is **natural**: the number of operations executed per one cycle on *p* processors is at most *p*.
- It is **strong**: any processor can read or write **any** shared memory cell in unit time.
- It is **simple**: it abstracts from any communication or synchronization overhead, which makes the complexity and correctness analysis of PRAM algorithms easier. Therefore,
- It is **useful**: it is an idealization of existing (and nowaday more and more abundant) shared memory parallel machines.
- It can be used as a **benchmark**: If a problem has no feasible/efficient solution on PRAM, it has no feasible/efficient solution on any parallel machine.

PRAM articulates fundamental limitations

– If a PRAM algorithm is inefficient, then so is any parallel algorithm!

- But the opposite is not true!

– We can use the PRAM to identify some inefficient algorithms

THE PRAM IS A THEORETICAL (UNFEASIBLE) MODEL

- The interconnection network between processors and memory would require a very large amount of area .

- **The message-routing on the interconnection network would require time proportional to network size** (i. e. the assumption of a constant access time to the memory is not realistic).

WHY THE PRAM IS A REFERENCE MODEL?

- Algorithm's designers can forget the communication problems and focus their attention on the parallel computation only.

- **There exist algorithms simulating any PRAM algorithm on bounded degree networks**.

E. G. A PRAM algorithm requiring time T(n), can be simulated in a mesh of tree in time $T(n)\log_2 n/\log\log n$, that is each step can be simulated with a slow-down of $\log_2 n/\log\log n$.

- Instead of design **ad hoc** algorithms for bounded degree networks, design more general algorithms for the PRAM model and simulate them on a feasible network.

- **For the PRAM model there exists a well developed body of techniques and methods to handle different classes of computational problems.**
- **The discussion on parallel model of computation is still HOT**

The actual trend:

COARSE-GRAINED MODELS

- **The degree of parallelism allowed is independent from the number of processors.**
- **The computation is divided in supersteps, each one includes**
  - **local computation**
  - **communication phase**
  - **syncronization phase**

Lecture 8:Sorting Networks

Reference: Thomas Sauerwald, lecture on Sorting Networks

Comparison Networks

Sorting Network

Sorting Networks Batcher’s

Overview: Sorting Networks

(Serial) Sorting Algorithms

- Well-known several (comparison-based) sorting algorithms: Insertion sort, Bubble sort, Merge sort, Quick sort, Heap sort
- execute one operation at a time
- can handle arbitrarily large inputs
- sequence of comparisons is not set in advance

Sorting Networks

- only perform comparisons
- can only handle inputs of a fixed size
- sequence of comparisons is set in advance
- Comparisons can be performed in parallel

Allows to sort *n* numbers in sublinear time!

Simple concept, but surprisingly deep and complex theory!

Comparison Networks

Comparison Network

- A comparison network consists solely of wires and comparators:
  - comparator is a device with, on given two inputs, x and y, returns two outputs x' and y' $(operates\ in\ O(1))$
  - wire connect output of one comparator to the input of another
  - special wires: n input wires $a_1, a_2, \ldots, a_n$ and n output wires $b_1, b_2, \ldots, b_n$

Convention: use the same name for both a wire and its value.

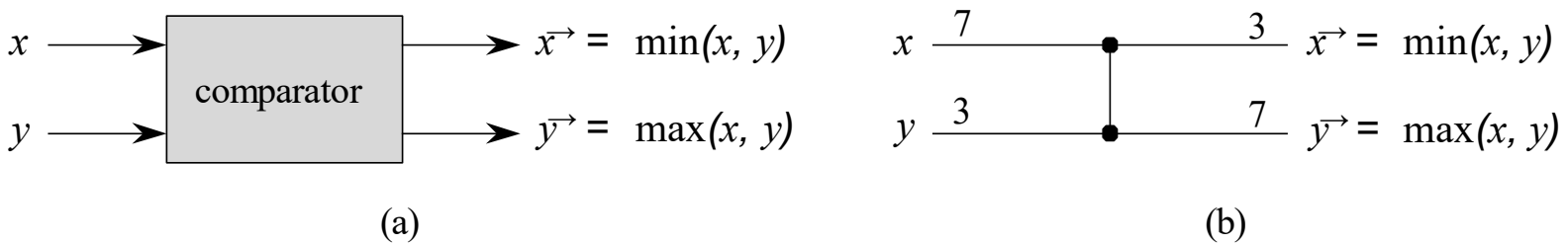


Figure 1

(a) A comparator with inputs x and y and outputs $\vec{x}$ and $\vec{y}$.

(b) The same comparator, drawn as a single vertical line.

Inputs $x = 7$, $y = 3$ and outputs $\vec{x} = 3$, $\vec{y} = 7$ are shown.

Sorting Networks

A sorting network is a comparison network which works correctly (that is, it sorts every input

Comparison Network

- A comparison network consists solely of wires and comparators:
  - comparator is a device with, on given two inputs, x and y, returns two outputs x' and y'
  - wire connect output of one comparator to the input of another
  - special wires: n input wires $a_1, a_2, \ldots, a_n$ and n output wires $b_1, b_2, \ldots, b_n$

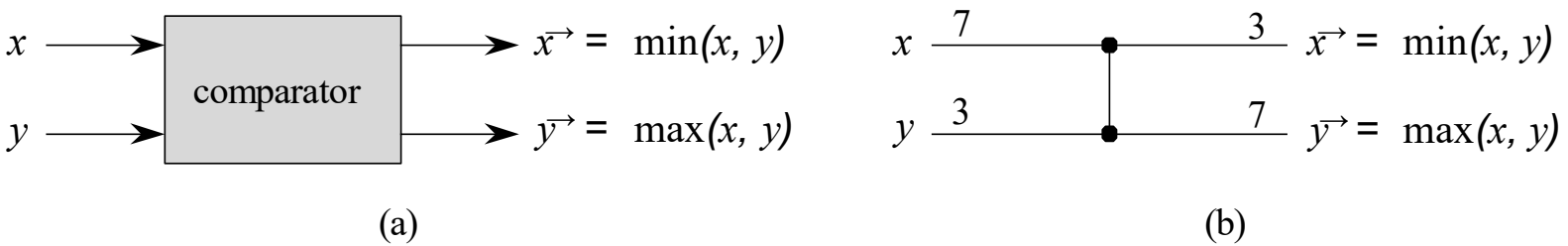
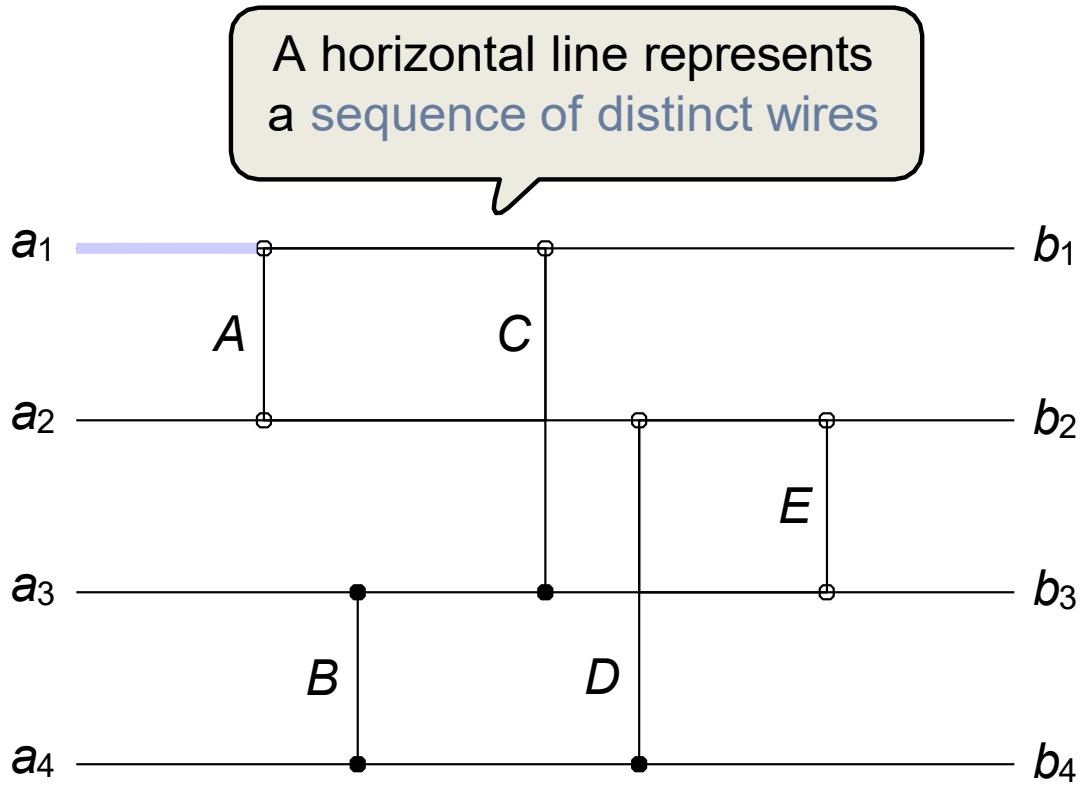
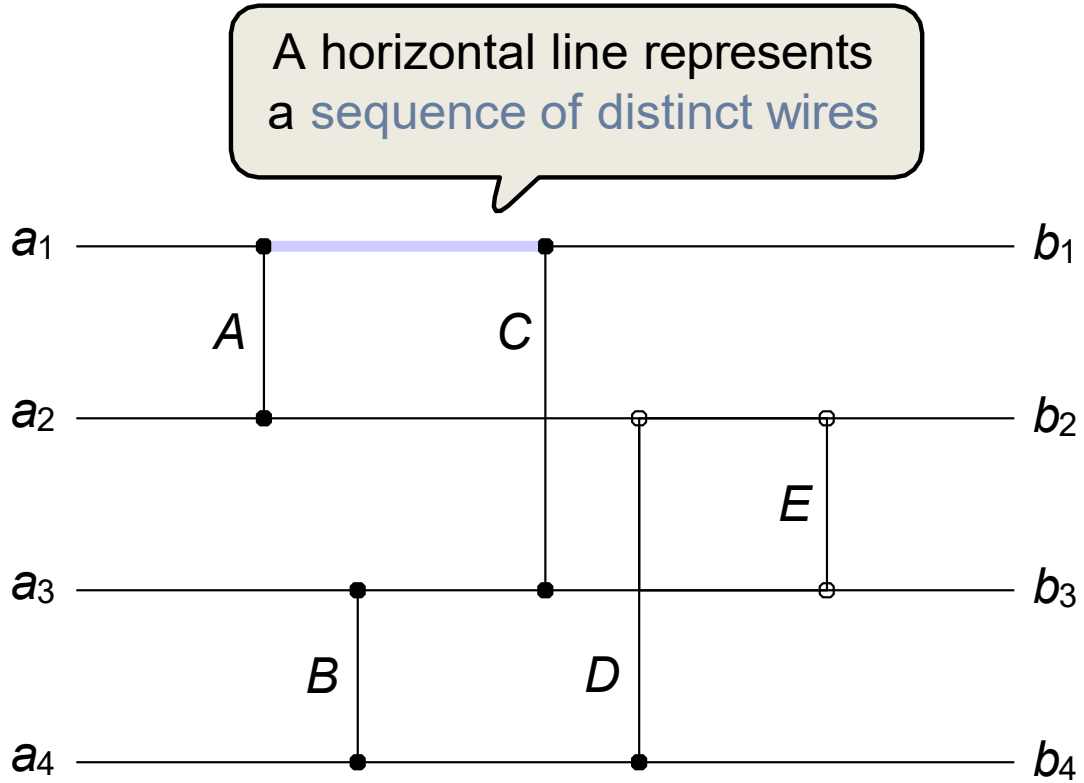


Figure 1 **(a)** A comparator with inputs x and y and outputs $\vec{x}$ and $\vec{y}$. **(b)** The same comparator, drawn as a single vertical line. Inputs $x = 7$, $y = 3$ and outputs $\vec{x} = 3$, $\vec{y} = 7$ are shown.

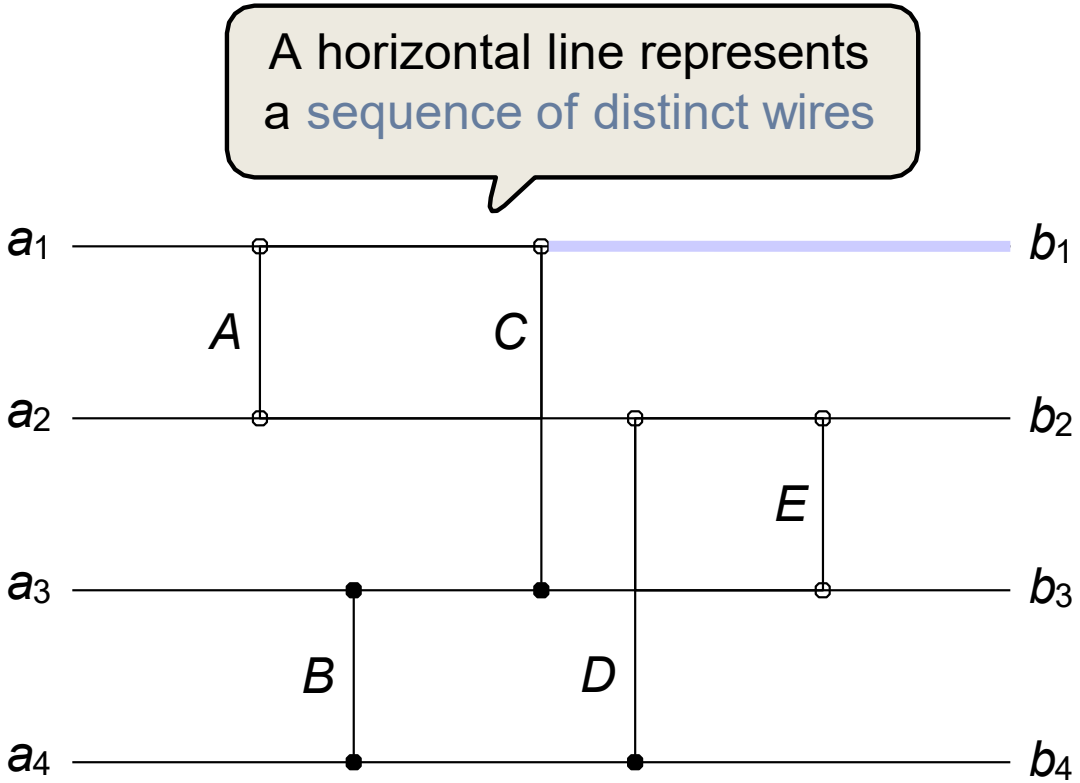
Example of a Comparison Network



Example of a Comparison Network)

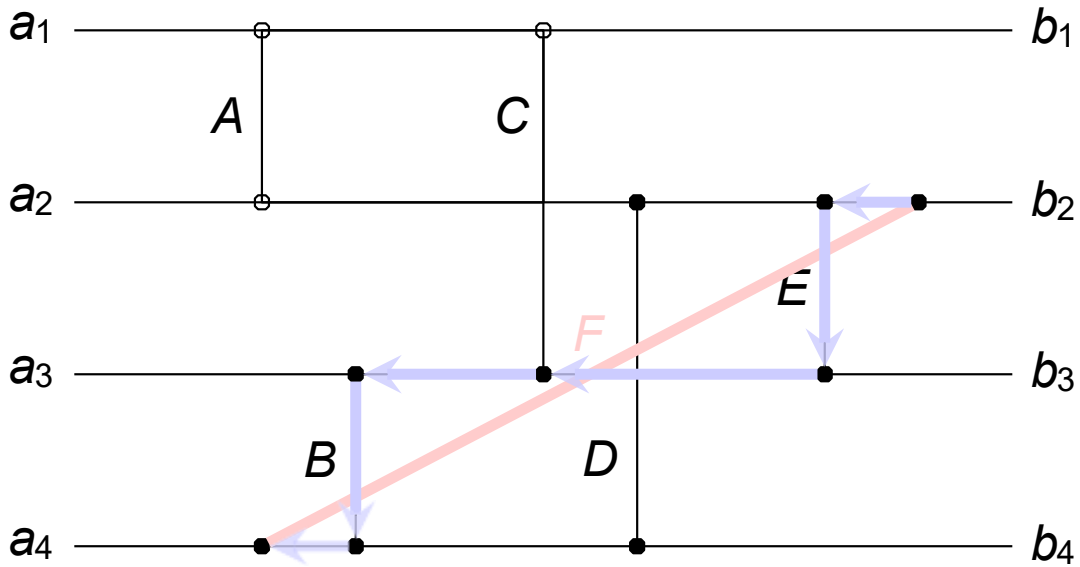


Example of a Comparison Network



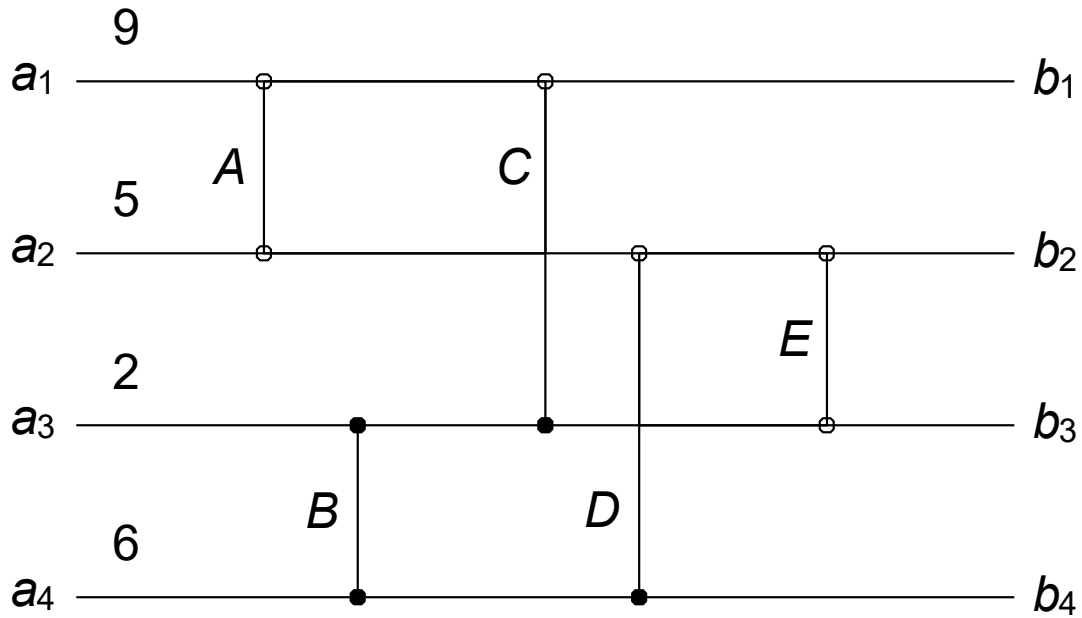
Example of a Comparison Network

Interconnections between comparators must be acyclic

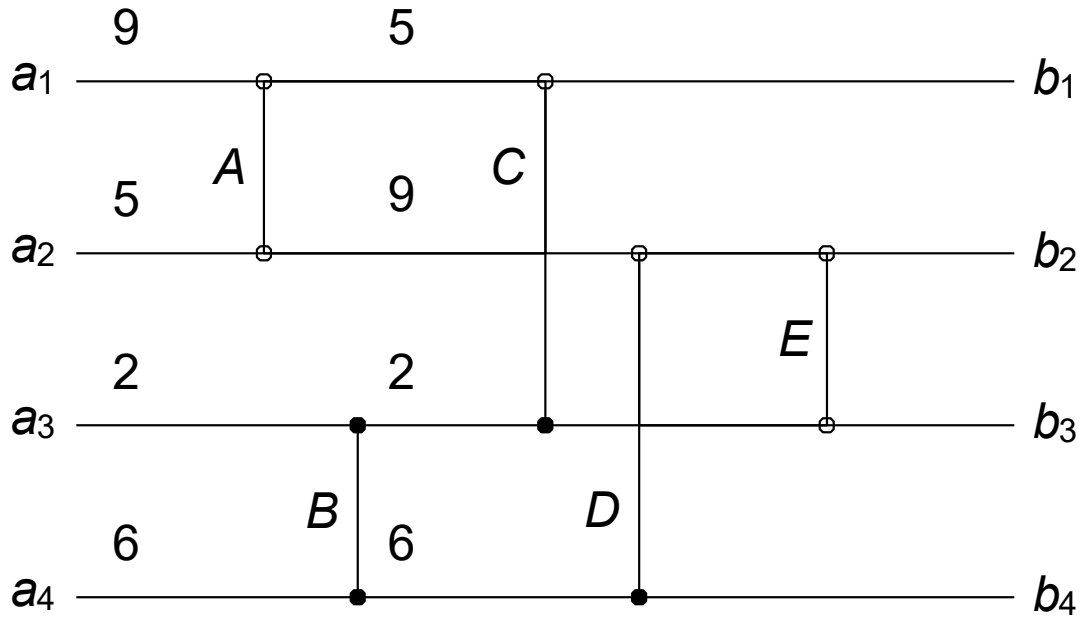


Tracing back a path must never cycle back on itself and go through the same comparator twice.

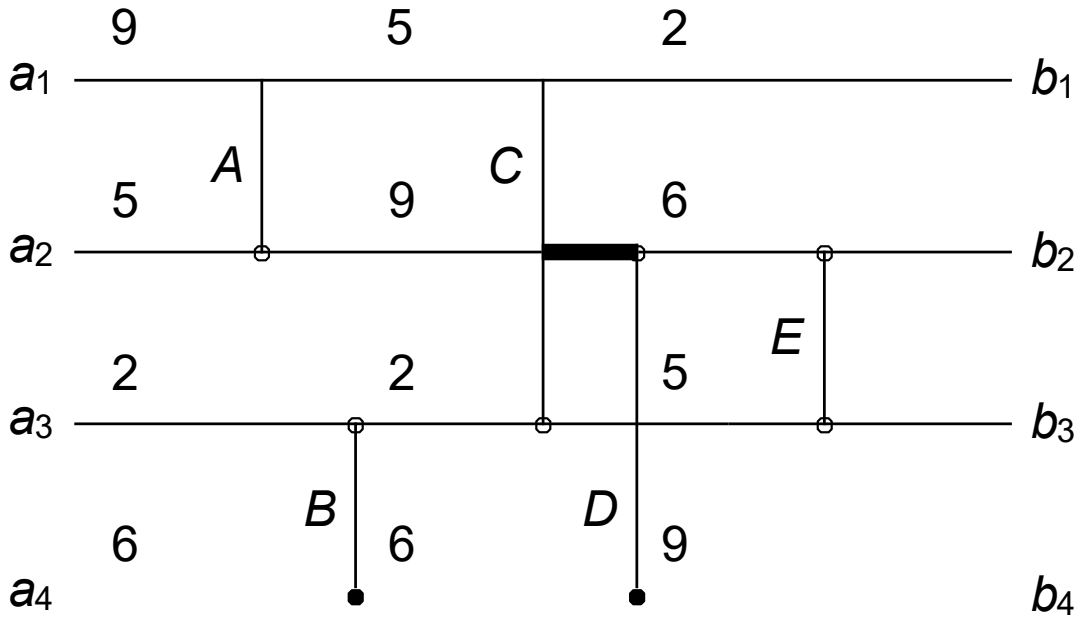
Example of a Comparison Network



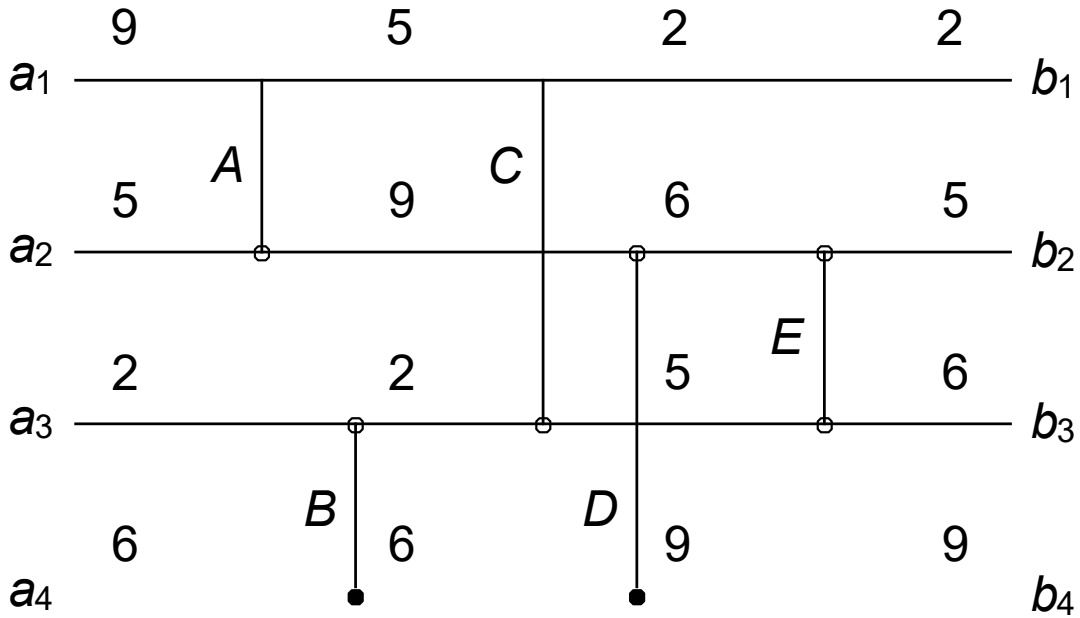
Example of a Comparison Network



Example of a Comparison Network (Figure 27.2)

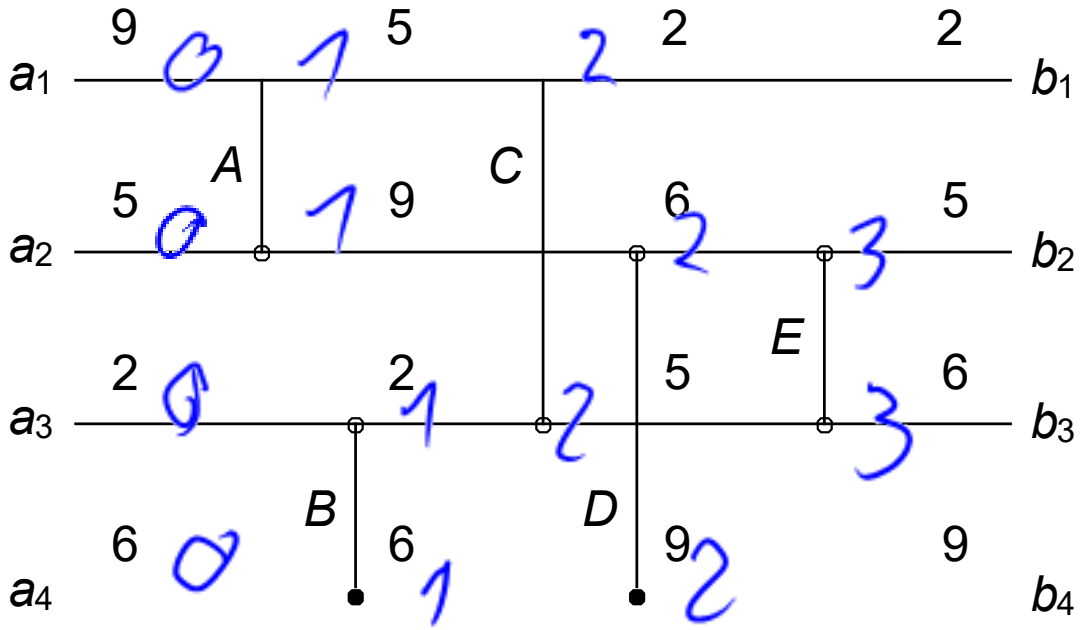


Example of a Comparison Network



This network is in fact a sorting network!

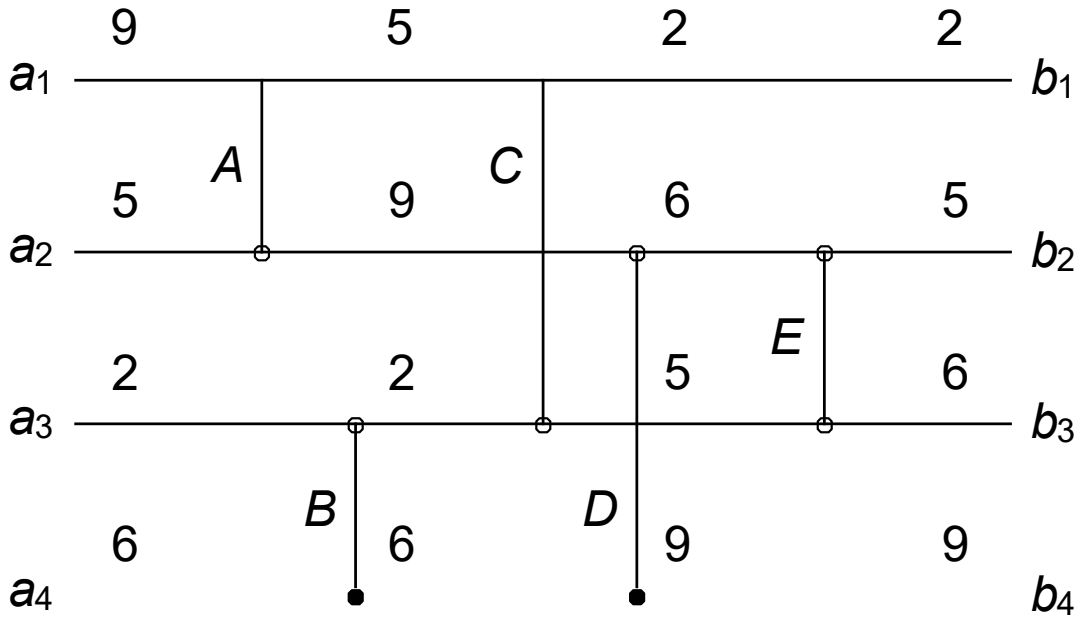
Example of a Comparison Network



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

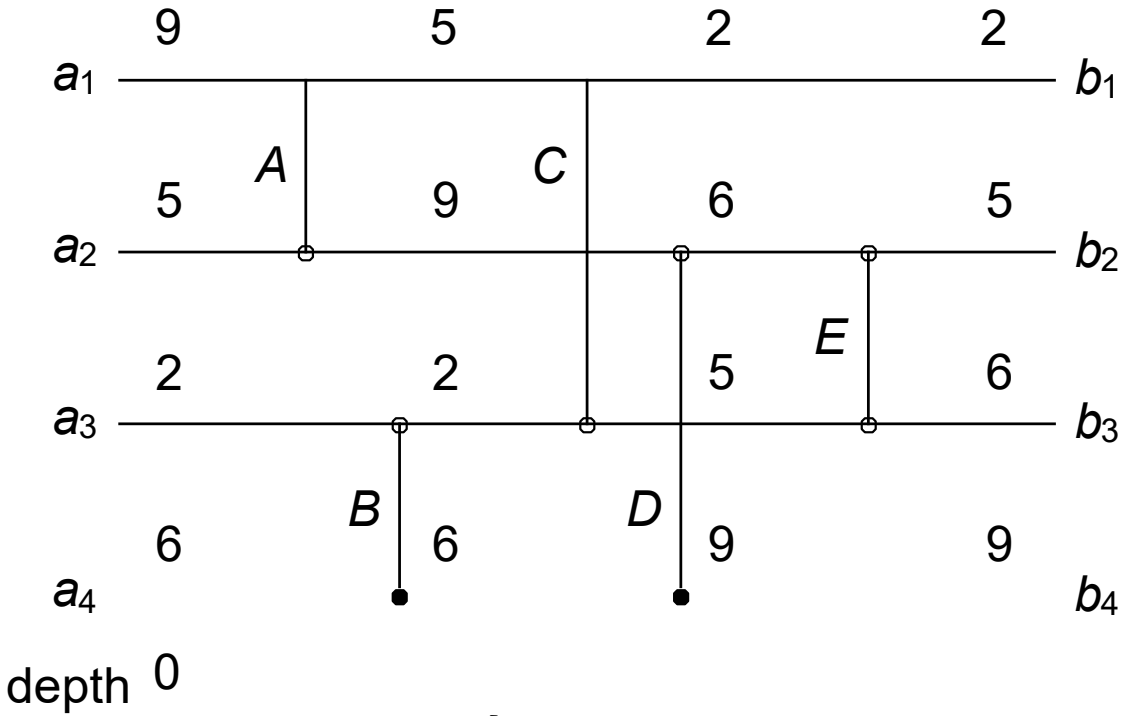
Example of a Comparison Network



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

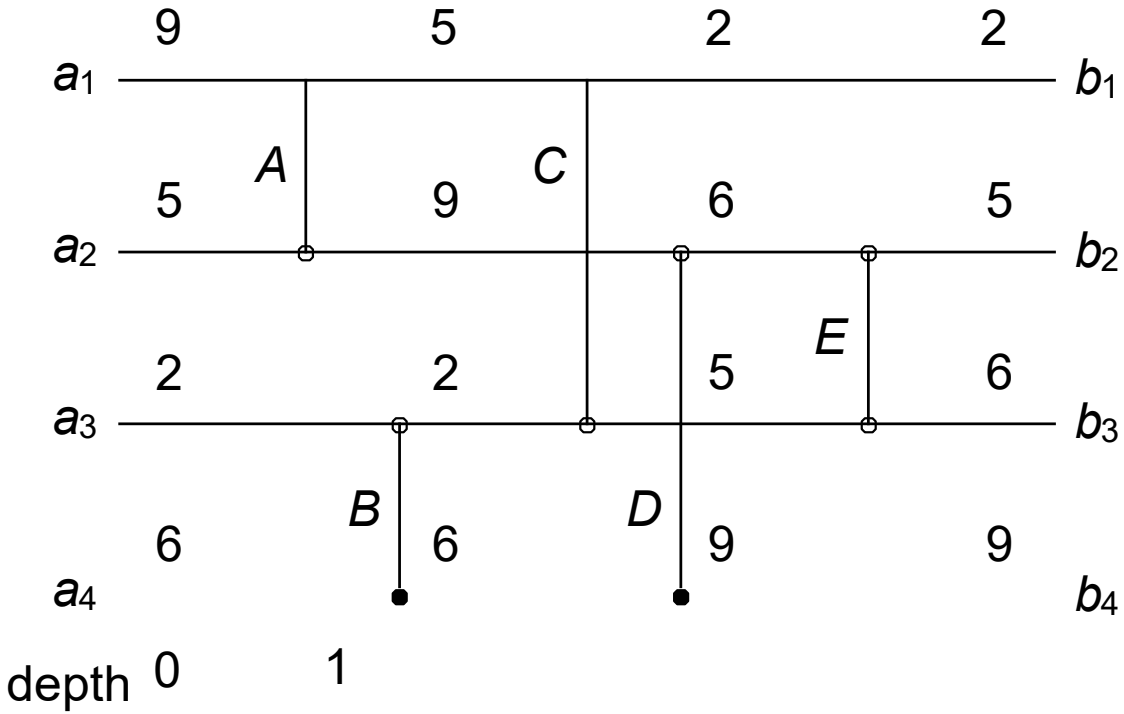
Example of a Comparison Network



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

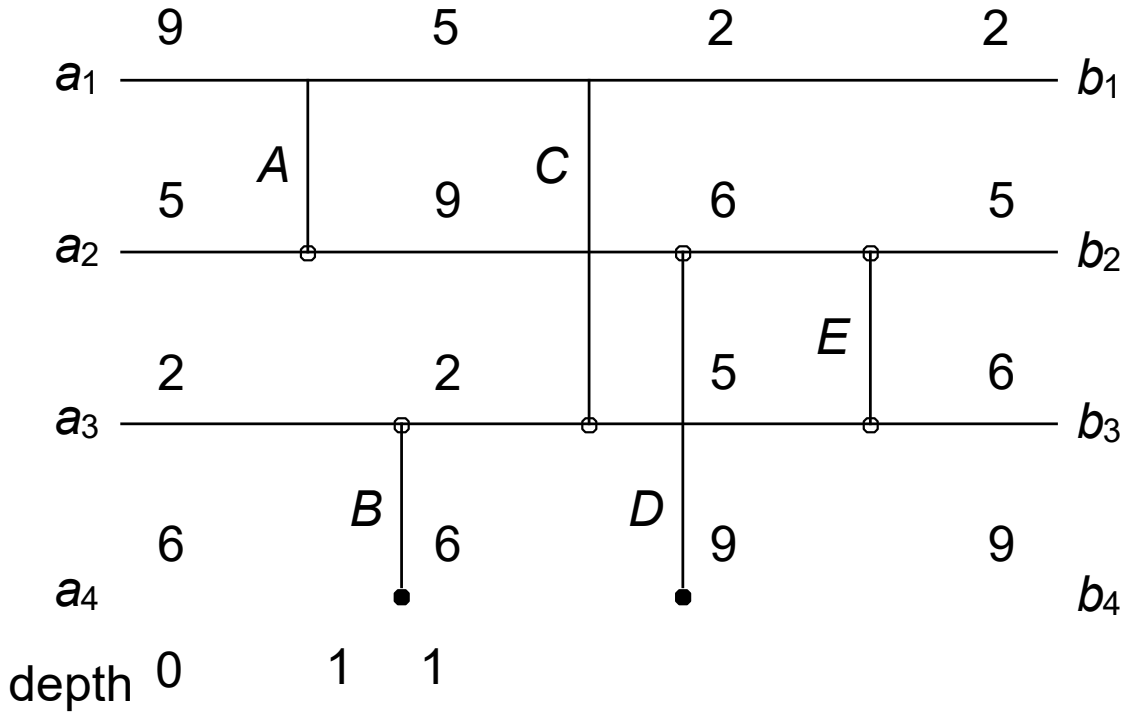
Example of a Comparison Network (Figure 27.2)



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

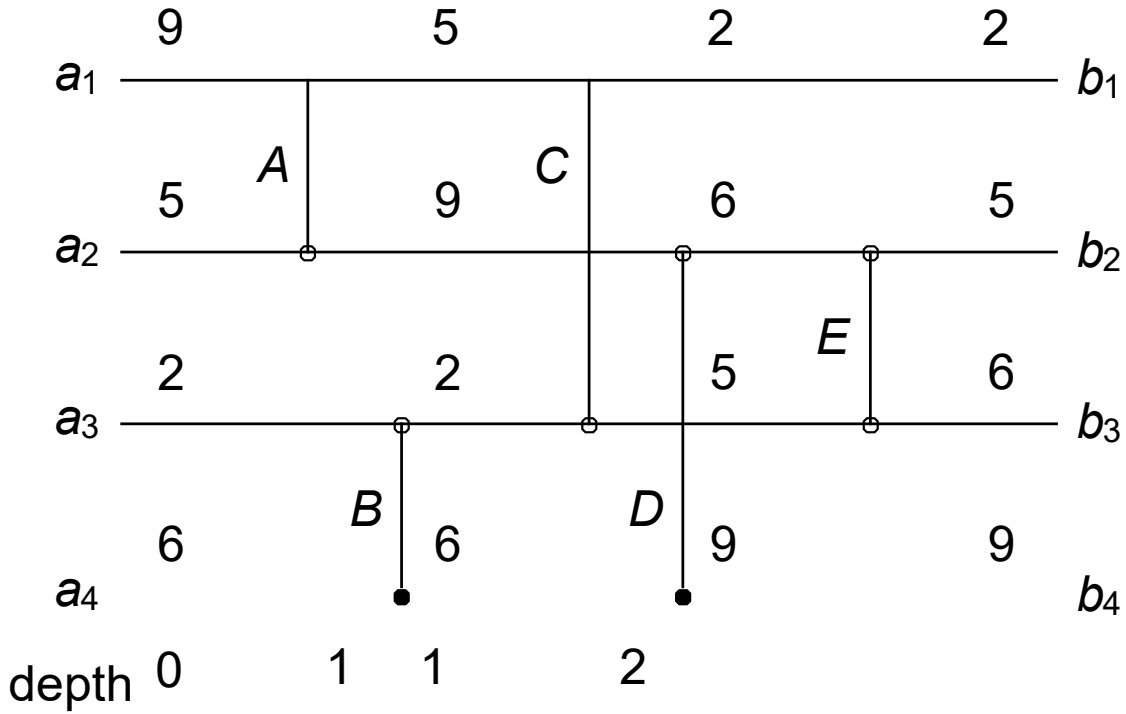
Example of a Comparison Network (Figure 27.2)



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

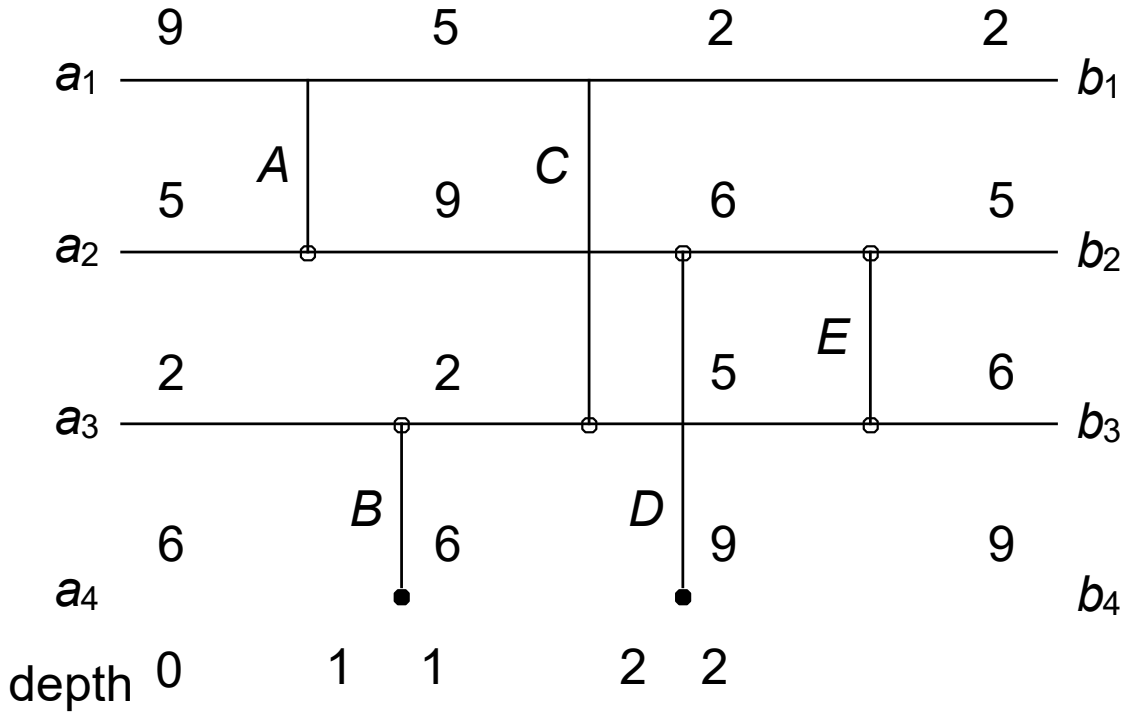
Example of a Comparison Network (Figure 27.2)



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

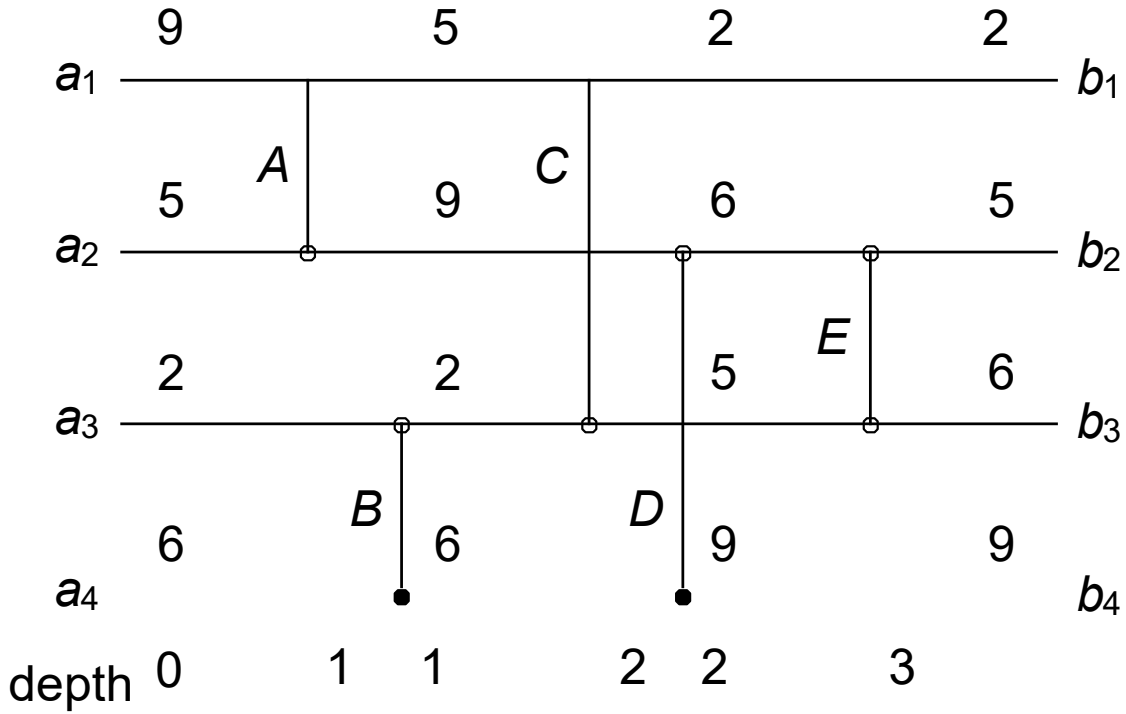
Example of a Comparison Network (Figure 27.2)



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

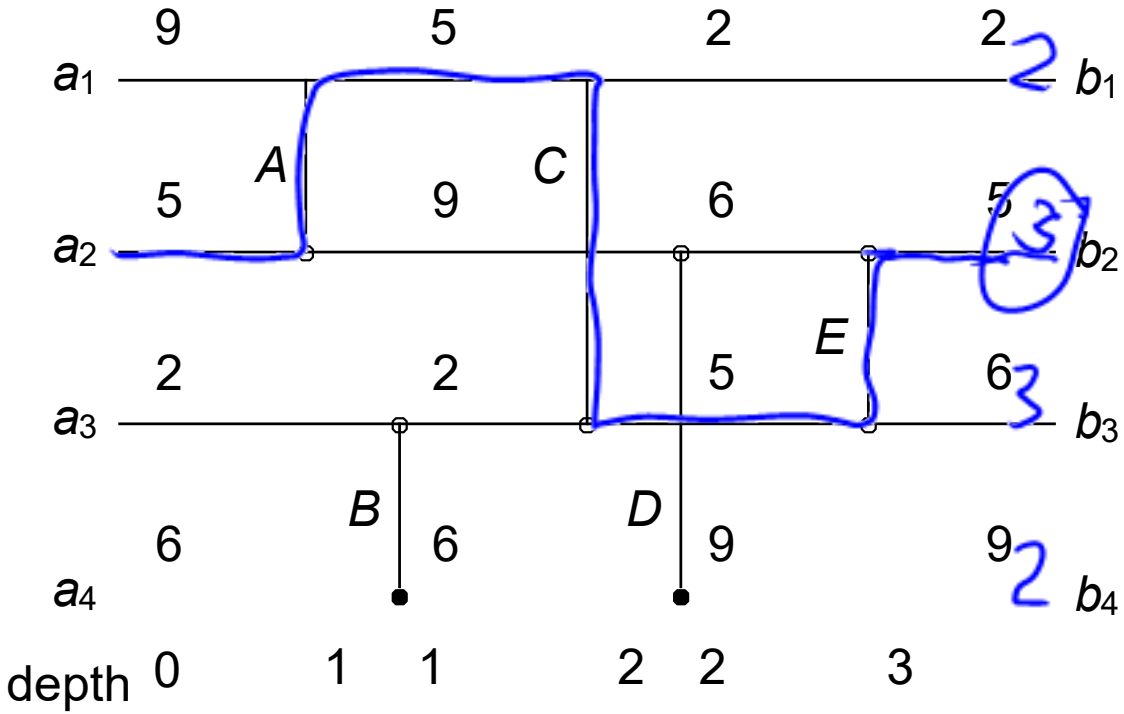
Example of a Comparison Network (Figure 27.2)



Depth of a wire:

- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

Example of a Comparison Network (Figure 27.2)



Maximum depth of an output wire equals total running time

Depth of a wire:

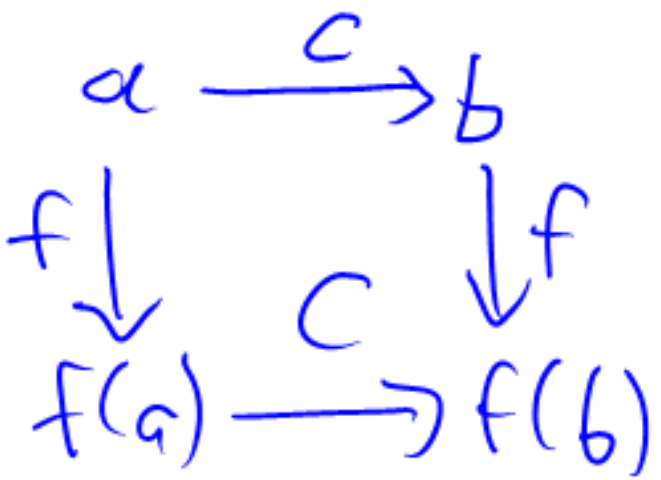
- Input wire has depth 0
- If a comparator has two inputs of depths d_x and d_y, then outputs have depth $\max\{d_x, d_y\} + 1$

Zero-One Principle

Zero-One Principle: A sorting networks works correctly on arbitrary inputs if it works correctly on binary inputs.

Lemma 1

If a comparison network transforms the input $a = \langle a_1, a_2, \ldots, a_n \rangle$ into the output $b = \langle b_1, b_2, \ldots, b_n \rangle$, then for any monotonically increasing function f, the network transforms $f(a) = \langle f(a_1), f(a_2), \ldots, f(a_n) \rangle$ into $f(b) = \langle f(b_1), f(b_2), \ldots, f(b_n) \rangle$.



Zero-One Principle

Zero-One Principle: A sorting networks works correctly on arbitrary inputs if it works correctly on binary inputs.

Lemma 1

If a comparison network transforms the input $a = \langle a_1, a_2, \ldots, a_n \rangle$ into the output $b = \langle b_1, b_2, \ldots, b_n \rangle$, then for any monotonically increasing function f, the network transforms $f(a) = \langle f(a_1), f(a_2), \ldots, f(a_n) \rangle$ into $f(b) = \langle f(b_1), f(b_2), \ldots, f(b_n) \rangle$.

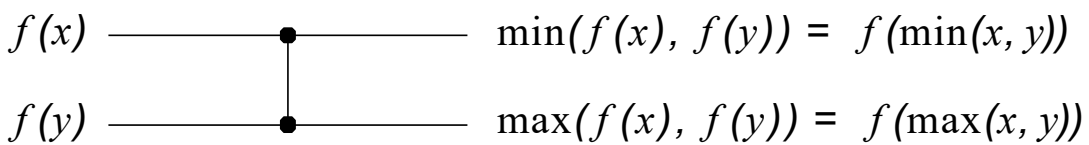


Figure 4 The operation of the comparator in the proof of Lemma.1. The function f is monotonically increasing.

Zero-One Principle

Zero-One Principle: A sorting networks works correctly on arbitrary inputs if it works correctly on binary inputs.

Lemma 1

If a comparison network transforms the input $a = \langle a_1, a_2, \ldots, a_n \rangle$ into the output $b = \langle b_1, b_2, \ldots, b_n \rangle$, then for any monotonically increasing function f, the network transforms $f(a) = \langle f(a_1), f(a_2), \ldots, f(a_n) \rangle$ into $f(b) = \langle f(b_1), f(b_2), \ldots, f(b_n) \rangle$.

Theorem 2 (Zero-One Principle)

If a comparison network with n inputs sorts all 2^n possible sequences of 0's and 1's correctly, then it sorts all sequences of arbitrary numbers correctly.

Proof of the Zero-One Principle

Theorem.2 (Zero-One Principle)

If a comparison network with n inputs sorts all 2^n possible sequences of 0's and 1's correctly, then it sorts all sequences of arbitrary numbers correctly.

Proof:

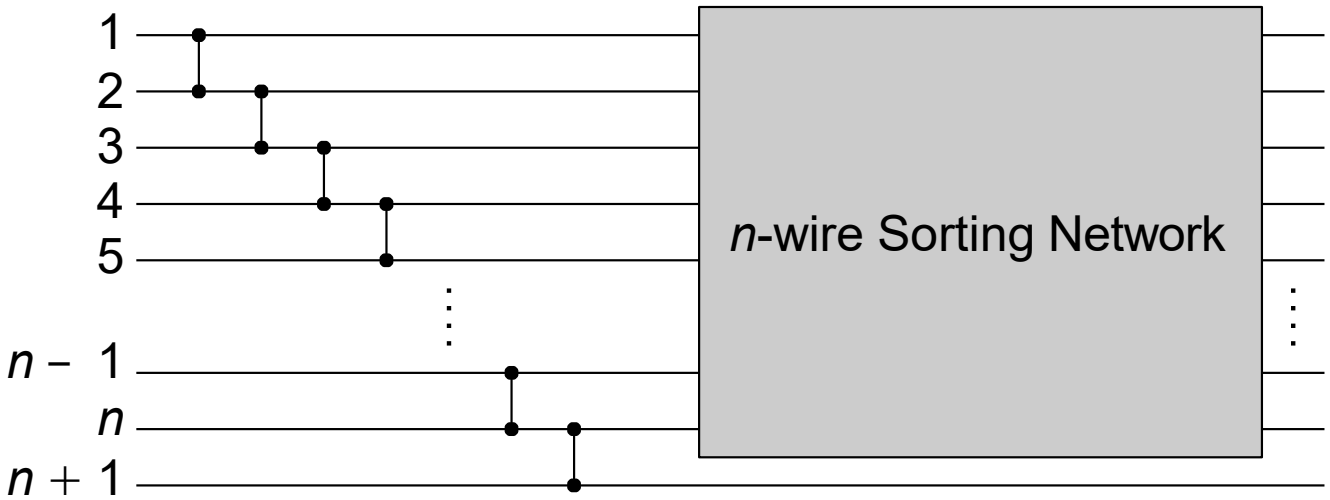
- For the sake of contradiction, suppose the network does not correctly sort.
- Let $a = \langle a_1, a_2, \ldots, a_n \rangle$ be the input with $a_i < a_j$, but the network places a_j before a_i in the output
- Define a monotonically increasing function f as:

$$f(x) = \begin{cases} 0 & \text{if } x \le a_i, \\ 1 & \text{if } x > a_i. \end{cases}$$

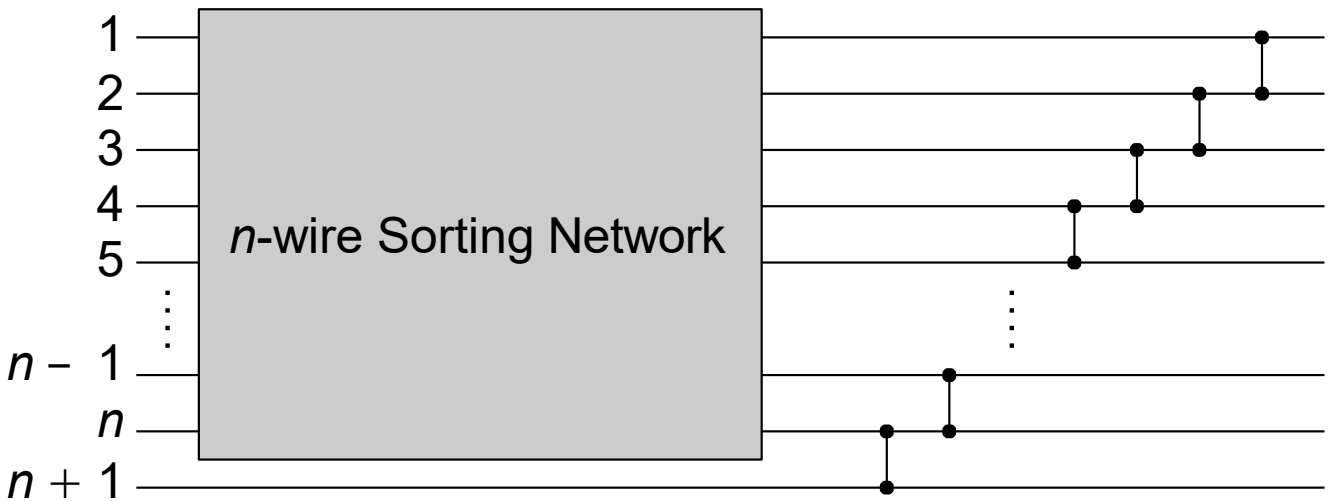
- Since the network places a_j before a_i, by the previous lemma $\Rightarrow f(a_j)$ is placed before $f(a_i)$
- But $f(a_j) = 1$ and $f(a_i) = 0$, which contradicts the assumption that the network sorts all sequences of 0's and 1's correctly

□

Some Basic (Recursive) Sorting Networks

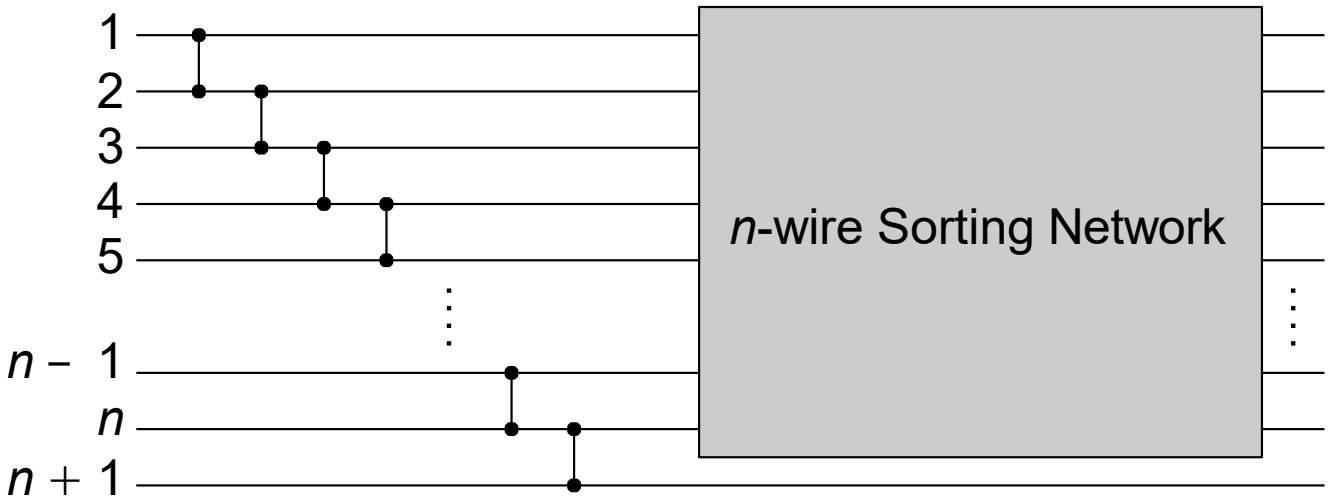


Bubble Sort



Insertion Sort

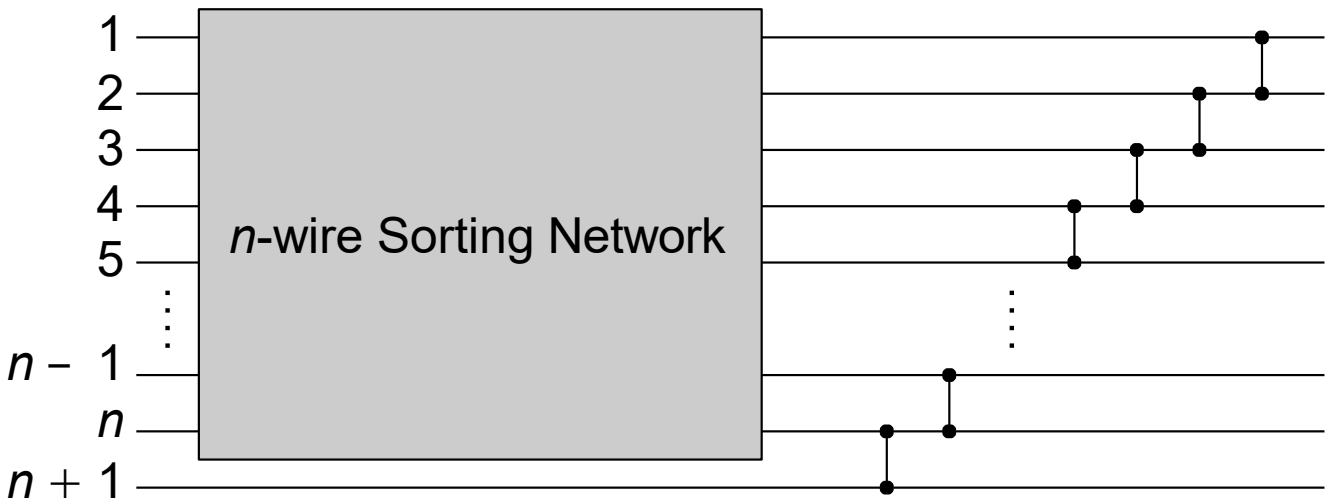
Some Basic (Recursive) Sorting Networks



Bubble Sort

These are Sorting Networks, but with depth Θ(*n*).

T(N)=n+T(n-1)



Insertion Sort

Bitonic Sequences

Bitonic Sequence

A sequence is bitonic if it monotonically increases and then monotonically decreases or can be circularly shifted to become monotonically increasing and then monotonically decreasing.

Sequences of one or two numbers are defined to be bitonic.

Examples:

- $\langle 1, 4, 6, 8, 3, 2 \rangle$ bitonic
- $\langle 6, 9, 4, 2, 3, 5 \rangle$ bitonic
- $\langle 9, 8, 3, 2, 4, 6 \rangle$ bitonic
- $\langle 4, 5, 7, 1, 2, 6 \rangle$ not bitonic
- binary bitonic sequences: $0^i 1^j 0^k$, or, $1^i 0^j 1^k$, for $i, j, k \geq 0$.

Towards Bitonic Sorting Networks

Half-Cleaner

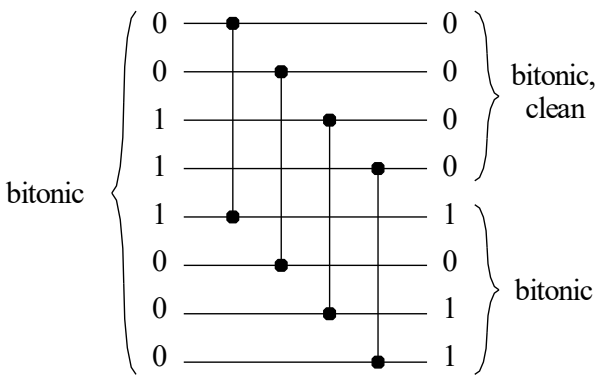
A half-cleaner is a comparison network of depth 1 in which input wire i is compared with wire $i + n/2$ for $i = 1, 2, \ldots, n/2$.

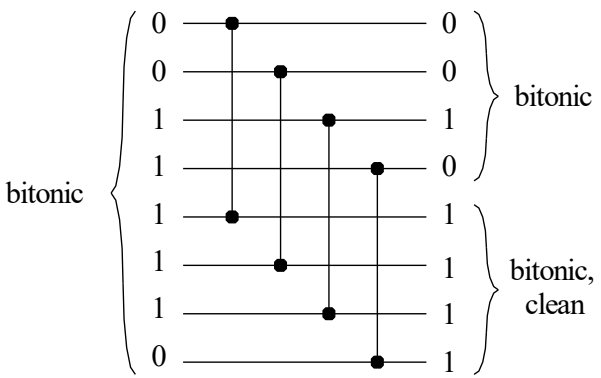
We always assume that n is even

Lemma.3

If the input to a half-cleaner is a bitonic sequence of 0's and 1's, then the output satisfies the following properties:

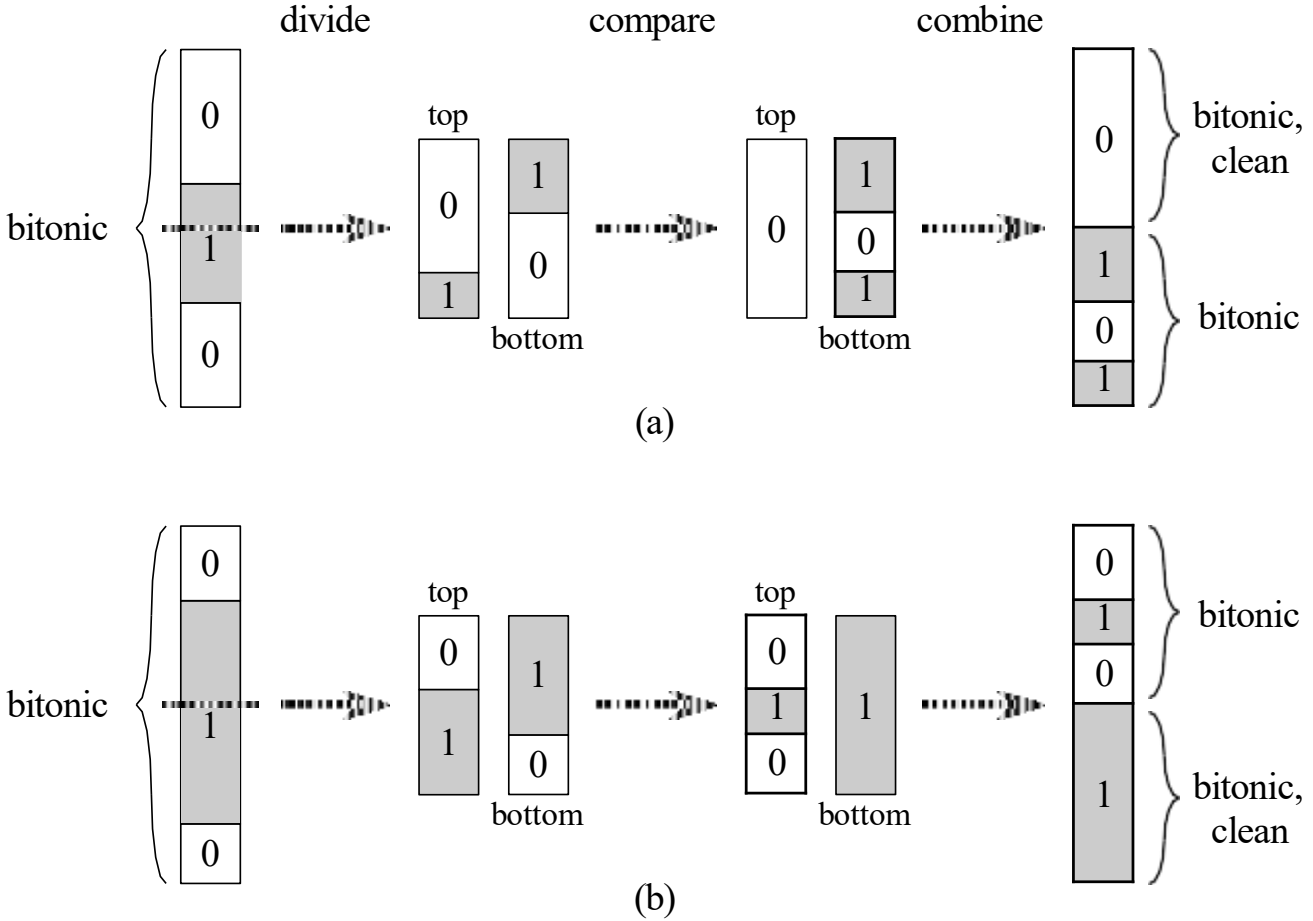
- both the top half and the bottom half are bitonic,
- every element in the top is not larger than any element in the bottom,
- at least one half is clean.





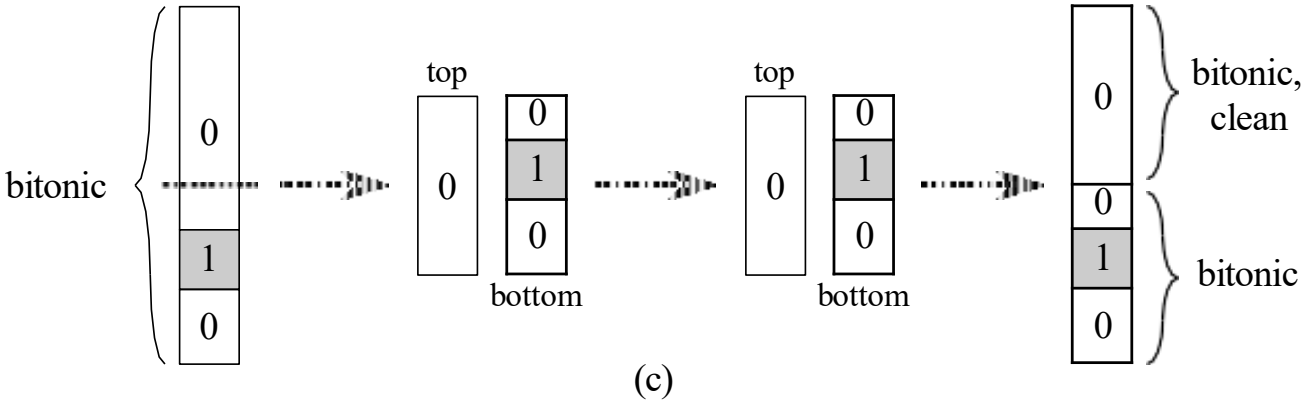
Proof of Lemma 3

W.l.o.g. assume that the input is of the form $0^i 1^j 0^k$, for some $i, j, k \geq 0$.

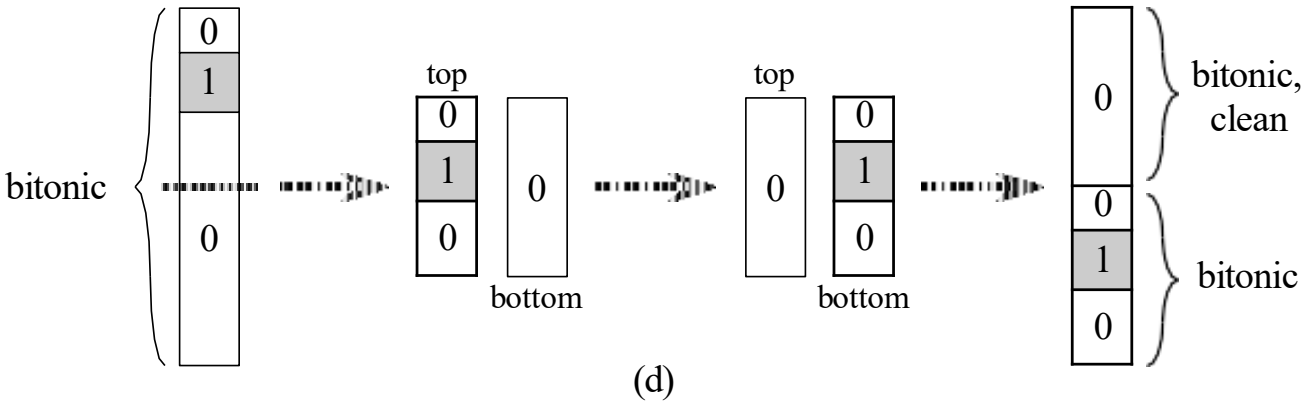


Proof of Lemma 3

W.l.o.g. assume that the input is of the form $0^i 1^j 0^k$, for some $i, j, k \geq 0$.



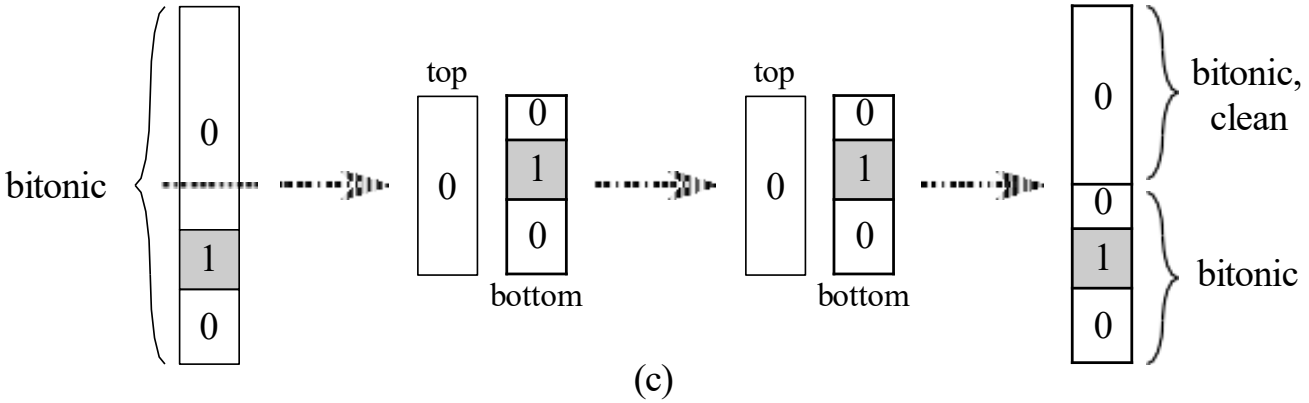
(c)



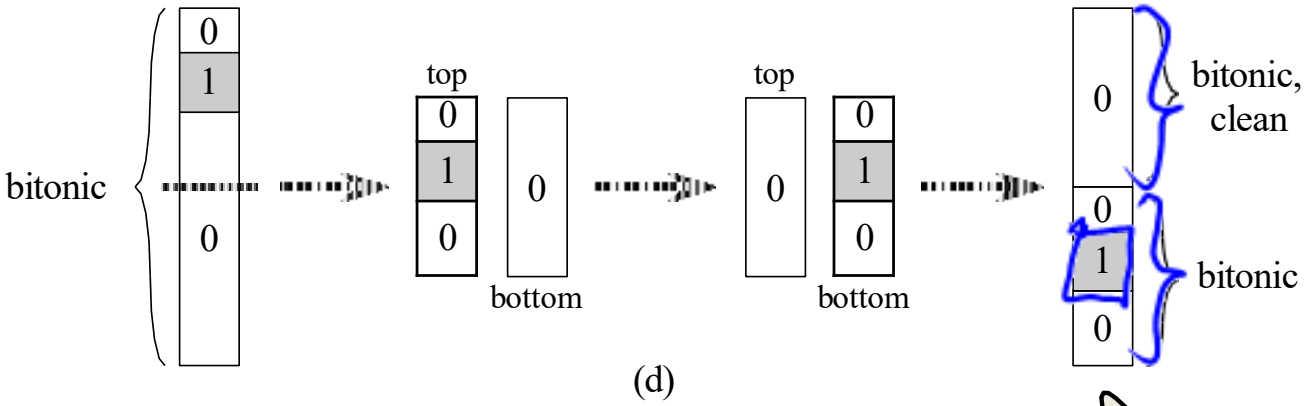
(d)

Proof of Lemma 3

W.l.o.g. assume that the input is of the form $0^i 1^j 0^k$, for some $i, j, k \geq 0$.



(c)



(d)

This suggests a recursive approach, since it now suffices to sort the top and bottom half separately.

The Bitonic Sorter

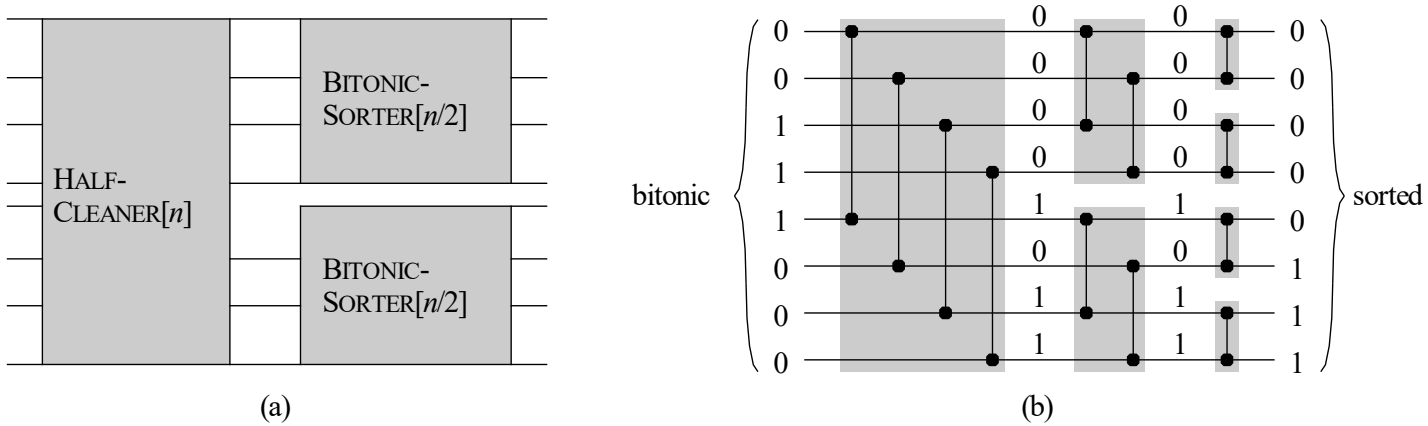


Figure.9 The comparison network BITONIC-SORTER[n], shown here for $n = 8$.

(a) The recursive construction: HALF-CLEANER[n] followed by two copies of BITONIC-SORTER[$n/2$] that operate in parallel.

(b) The network after unrolling the recursion. Each half-cleaner is shaded. Sample zero-one values are shown on the wires.

Recursive Formula for depth $D(n)$:

we will always assume that n is a power of 2.

$$D(n) = \begin{cases} 0 & \text{if } n = 1, \\ D(n/2) + 1 & \text{if } n = 2^k. \end{cases}$$

BITONIC-SORTER[n] has depth $\log n$ and sorts any zero-one bitonic sequence.

Merging Networks

Merging Networks

- can merge two sorted input sequences into one sorted output sequences
- will be based on a modification of BITONIC-SORTER[n]

Basic Idea:

- consider two given sequences $X = 00000111$, $Y = 00001111$
- concatenating X with Y^R (the reversal of Y) $\Rightarrow$ 000001111110000

This sequence is bitonic!

Hence in order to merge the sequences X and Y, it suffices to perform a bitonic sort on X concatenated with Y^R.

Construction of a Merging Network (1/2)

- Given two sorted sequences $\langle a_1, a_2, \ldots, a_{n/2} \rangle$ and $\langle a_{n/2+1}, a_{n/2+2}, \ldots, a_n \rangle$
- We know it suffices to bitonically sort $\langle a_1, a_2, \ldots, a_{n/2}, a_n, a_{n-1}, \ldots, a_{n/2+1} \rangle$
- Recall: first half-cleaner of BITONIC-SORTER[n] compares i and $n/2 + i$

⇒ First part of MERGER[n] compares inputs i and $n - i$ for $i = 1, 2, \ldots, n/2$

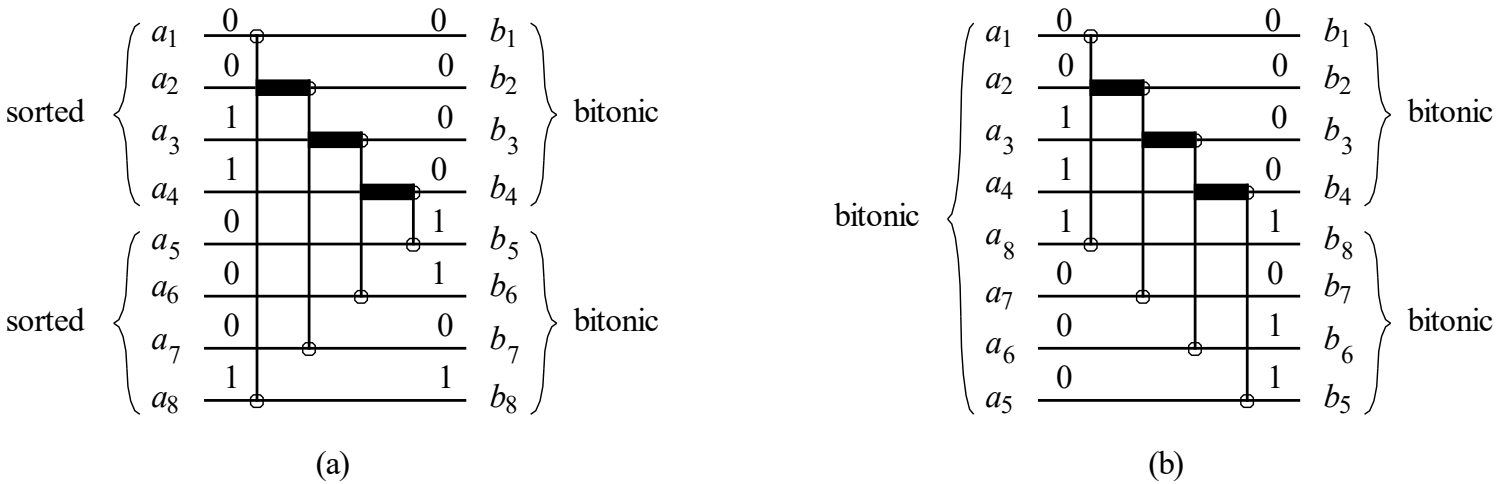


Figure.10 Comparing the first stage of MERGER[n] with HALF-CLEANER[n], for $n = 8$.
(a) The first stage of MERGER[n] transforms the two monotonic input sequences $\rightarrow a_1, a_2, \ldots, a_{n/2}\uparrow$ and $\rightarrow a_{n/2+1}, a_{n/2+2}, \ldots, a_n\uparrow$ into two bitonic sequences $\rightarrow b_1, b_2, \ldots, b_{n/2}\uparrow$ and $\rightarrow b_{n/2+1}, b_{n/2+2}, \ldots, b_n\uparrow$.
(b) The equivalent operation for HALF-CLEANER[n]. The bitonic input sequence $\rightarrow a_1, a_2, \ldots, a_{n/2-1}, a_{n/2}, a_n, a_{n-1}, \ldots, a_{n/2+2}, a_{n/2+1}\uparrow$ is transformed into the two bitonic sequences $\rightarrow b_1, b_2, \ldots, b_{n/2}\uparrow$ and $\rightarrow b_n, b_{n-1}, \ldots, b_{n/2+1}\uparrow$.

Construction of a Merging Network (1/2)

- Given two sorted sequences $\langle a_1, a_2, \ldots, a_{n/2} \rangle$ and $\langle a_{n/2+1}, a_{n/2+2}, \ldots, a_n \rangle$
- We know it suffices to bitonically sort $\langle a_1, a_2, \ldots, a_{n/2}, a_n, a_{n-1}, \ldots, a_{n/2+1} \rangle$
- Recall: first half-cleaner of BITONIC-SORTER[n] compares i and $n/2 + i$

$\Rightarrow$ First part of MERGER[n] compares inputs i and $n - i$ for $i = 1, 2, \ldots, n/2$

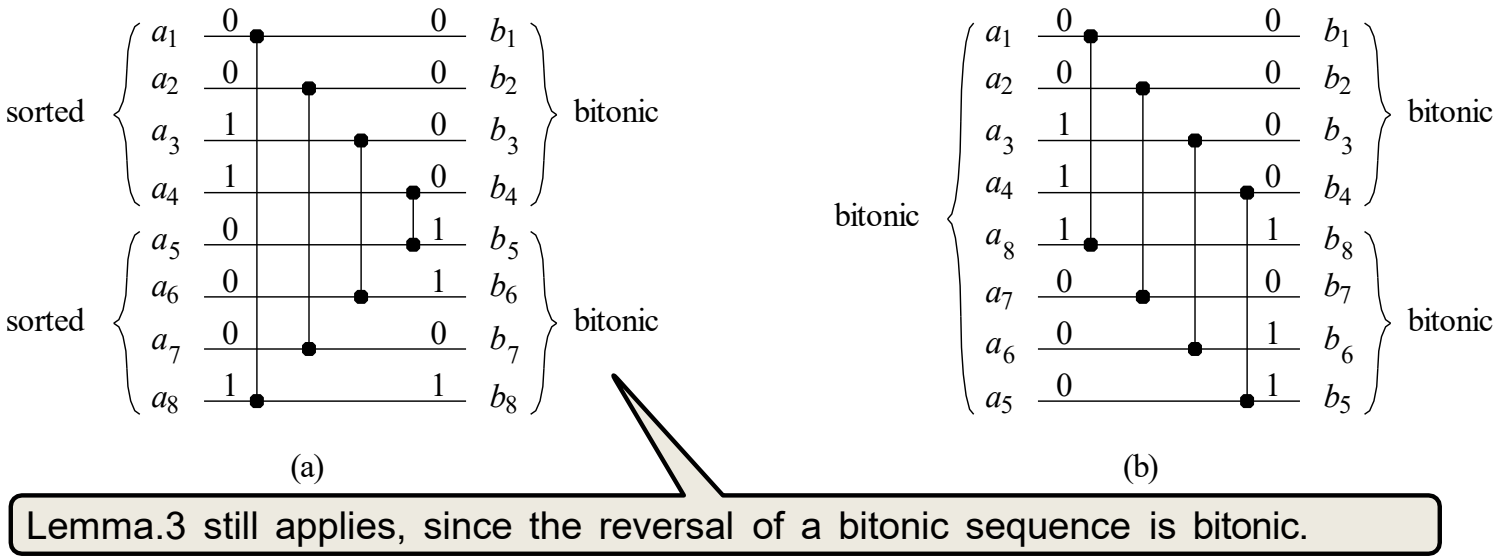


Figure 10 Comparing the first stage of MERGER[n] with HALF-CLEANER[n], for $n = 8$. **(a)** The first stage of MERGER[n] transforms the two monotonic input sequences $\rightarrow a_1, a_2, \ldots, a_{n/2}\uparrow$ and $\rightarrow a_{n/2+1}, a_{n/2+2}, \ldots, a_n\uparrow$ into two bitonic sequences $\rightarrow b_1, b_2, \ldots, b_{n/2}\uparrow$ and $\rightarrow b_{n/2+1}, b_{n/2+2}, \ldots, b_n\uparrow$. **(b)** The equivalent operation for HALF-CLEANER[n]. The bitonic input sequence $\rightarrow a_1, a_2, \ldots, a_{n/2-1}, a_{n/2}, a_n, a_{n-1}, \ldots, a_{n/2+2}, a_{n/2+1}\uparrow$ is transformed into the two bitonic sequences $\rightarrow b_1, b_2, \ldots, b_{n/2}\uparrow$ and $\rightarrow b_n, b_{n-1}, \ldots, b_{n/2+1}\uparrow$.

Construction of a Merging Network (1/2)

- Given two sorted sequences $\langle a_1, a_2, \ldots, a_{n/2} \rangle$ and $\langle a_{n/2+1}, a_{n/2+2}, \ldots, a_n \rangle$
- We know it suffices to bitonically sort $\langle a_1, a_2, \ldots, a_{n/2}, a_n, a_{n-1}, \ldots, a_{n/2+1} \rangle$
- Recall: first half-cleaner of BITONIC-SORTER[n] compares i and $n/2 + i$

$\Rightarrow$ First part of MERGER[n] compares inputs i and $n - i$ for $i = 1, 2, \ldots, n/2$

- Remaining part is identical to BITONIC-SORTER[n]

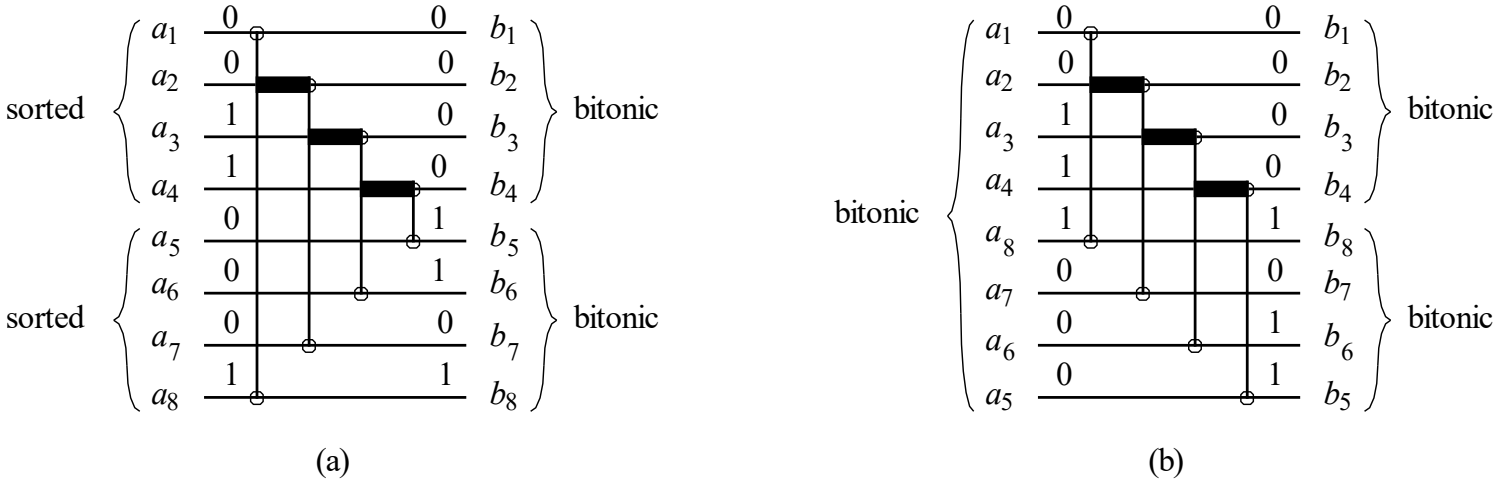


Figure 10 Comparing the first stage of MERGER[n] with HALF-CLEANER[n], for $n = 8$. **(a)** The first stage of MERGER[n] transforms the two monotonic input sequences $\rightarrow a_1, a_2, \ldots, a_{n/2}\uparrow$ and $\rightarrow a_{n/2+1}, a_{n/2+2}, \ldots, a_n\uparrow$ into two bitonic sequences $\rightarrow b_1, b_2, \ldots, b_{n/2}\uparrow$ and $\rightarrow b_{n/2+1}, b_{n/2+2}, \ldots, b_n\uparrow$. **(b)** The equivalent operation for HALF-CLEANER[n]. The bitonic input sequence $\rightarrow a_1, a_2, \ldots, a_{n/2-1}, a_{n/2}, a_n, a_{n-1}, \ldots, a_{n/2+2}, a_{n/2+1}\uparrow$ is transformed into the two bitonic sequences $\rightarrow b_1, b_2, \ldots, b_{n/2}\uparrow$ and $\rightarrow b_n, b_{n-1}, \ldots, b_{n/2+1}\uparrow$.

Construction of a Merging Network (2/2)

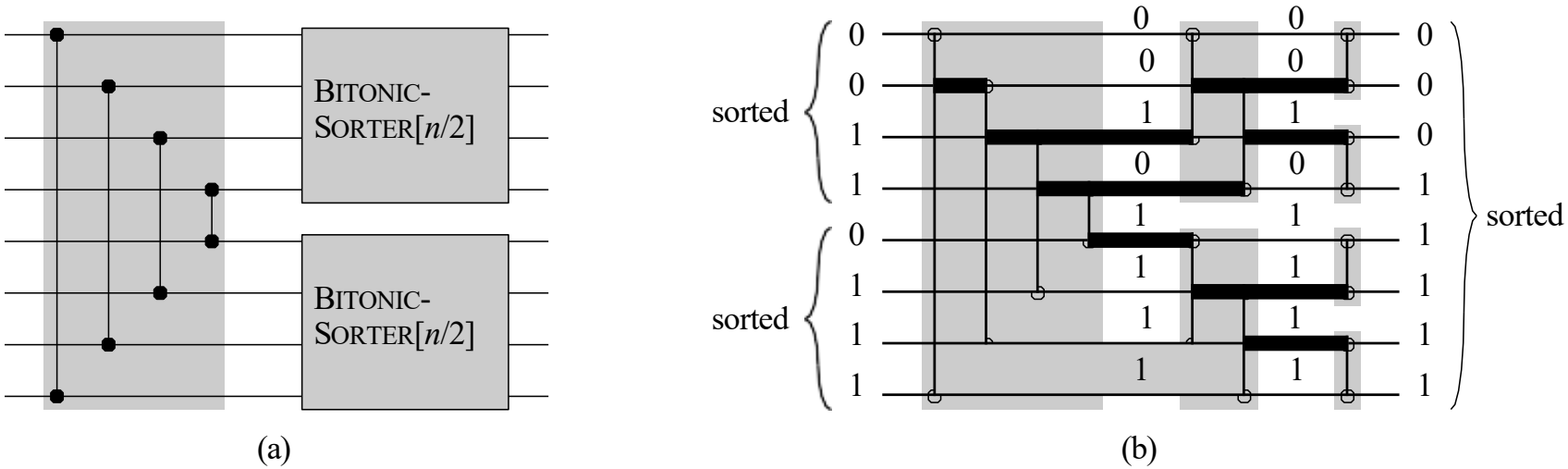


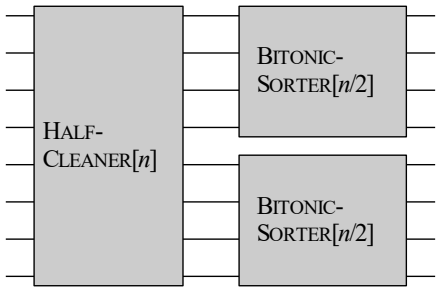
Figure 11 A network that merges two sorted input sequences into one sorted output sequence.
The network MERGER[n] can be viewed as BITONIC-SORTER[n]with the first half-cleaner altered to compare inputs i and $n - i + 1$ for $i = 1, 2, \ldots, n/2$. Here, $n = 8$.
(a) The network decomposed into the first stage followed by two parallel copies of BITONIC-SORTER[$n/2$].
(b) The same network with the recursion unrolled. Sample zero-one values are shown on the wires, and the stages are shaded.

Construction of a Sorting Network

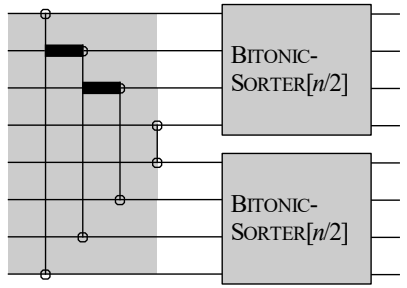
Main Components

1. BITONIC-SORTER[n]
   - sorts any bitonic sequence
   - depth $\log n$
2. MERGER[n]
   - merges two sorted input sequences
   - depth $\log n$

1.

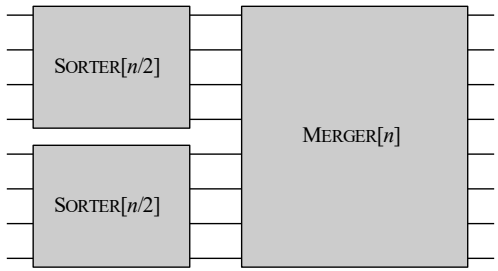


2.



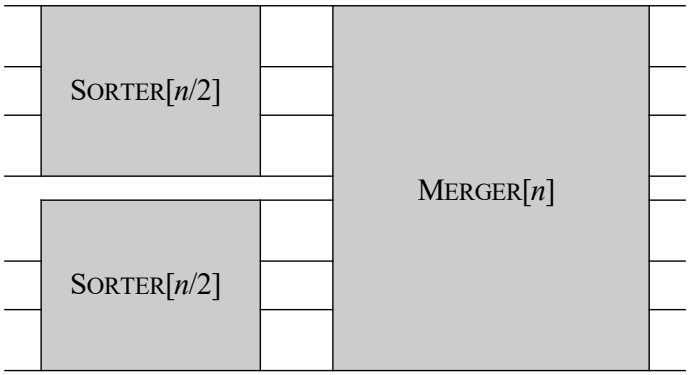
Batcher's Sorting Network

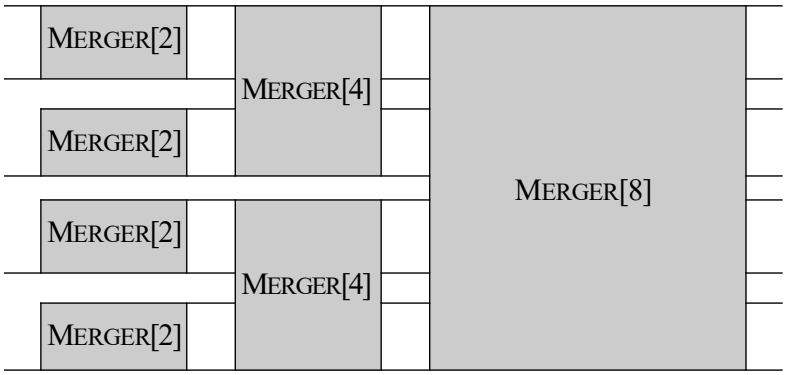
- SORTER[n] is defined recursively:
  - If $n = 2^k$, use two copies of SORTER[$n/2$] to sort two subsequences of length $n/2$ each. Then merge them using MERGER[n].
  - If $n = 1$, network consists of a single wire.

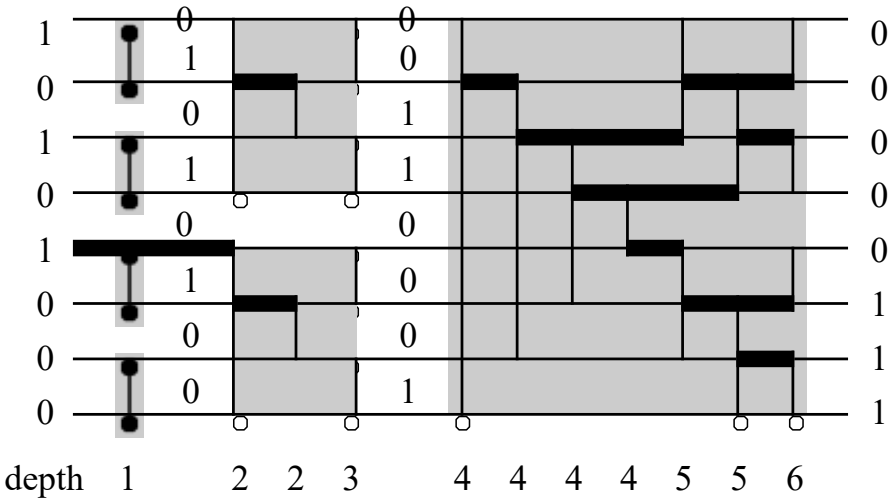


can be seen as a parallel version of merge sort

Unrolling the Recursion





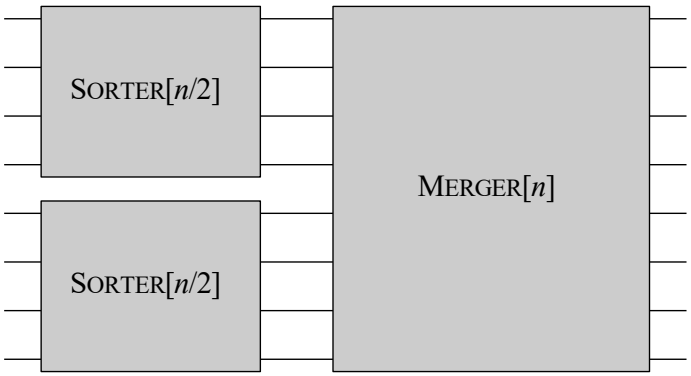


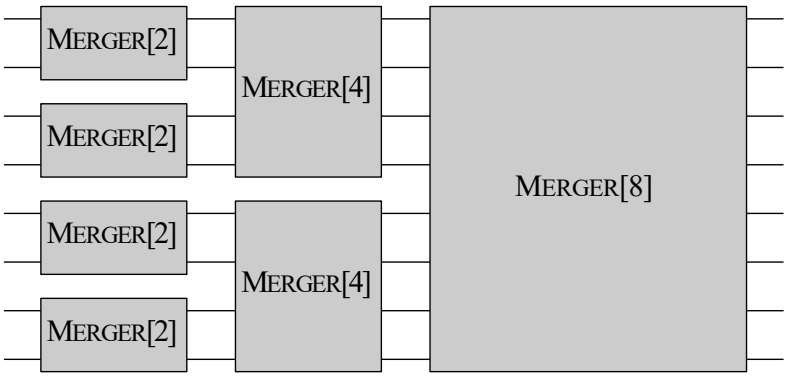
Recursion for $D(n)$:

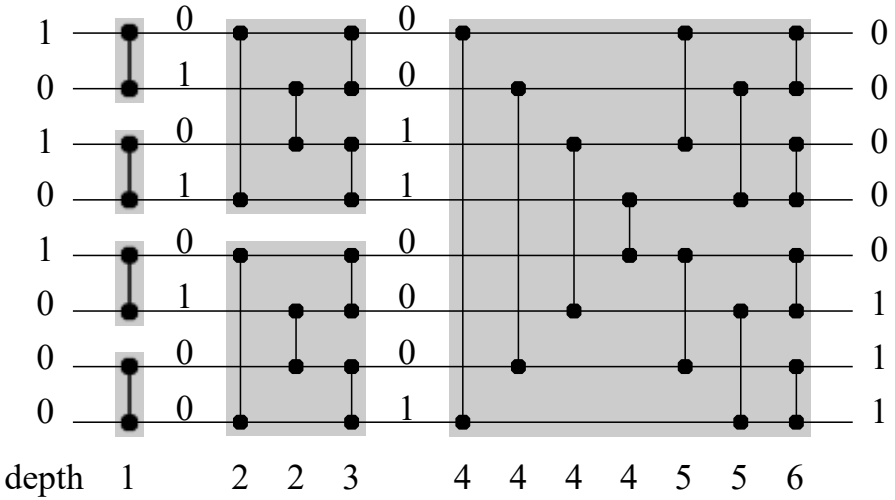
$$D(n) = \begin{cases} 0 & \text{if } n = 1, \\ D(n/2) + \log n & \text{if } n = 2^k. \end{cases}$$

Solution: $D(n) = \Theta(\log^2 n)$.

Unrolling the Recursion







Recursion for $D(n)$:

$$D(n) = \begin{cases} 0 & \text{if } n = 1, \\ D(n/2) + \log n & \text{if } n = 2^k. \end{cases}$$

Solution: $D(n) = \Theta(\log^2 n)$.

SORTER[n] has depth $\Theta(\log^2 n)$ and sorts any input.

A Glimpse at the AKS Network

Ajtai, Komlós, Szemerédi (1983)

There exists a sorting network with depth $O(\log n)$.

Quite elaborate construction and involves huge constants

Perfect Halver

A perfect halver is a comparator network that, given any input, places the $n/2$ smaller keys in $b_1, \ldots, b_{n/2}$ and the $n/2$ larger keys in $b_{n/2+1}, \ldots, b_n$.

Perfect halver of depth $\log_2 n$ exist $\sim$ yields sorting networks of depth $\Theta((\log n)^2)$.

Approximate Halver

An (n, g)-approximate halver, $g < 1$, is a comparator network that for every $k = 1, 2, \ldots, n/2$ places at most gk of its k smallest keys in $b_{n/2+1}, \ldots, b_n$ and at most gk of its k largest keys in $b_1, \ldots, b_{n/2}$.

Lecture 8

Brent Theorem

Sorting Networks

Brent's Theorem

- Relates the work and time complexities of a parallel algorithm described in the WT formalism to its running time on a *p*-processor PRAM.

Theorem 1 (Brent 1974) *A WT algorithm with (depth) step complexity S(n) and work complexity W(n) can be simulated on a p-processor PRAM in no more than* $\lfloor \frac{W(n)}{p} \rfloor + S(n)$ *steps.*

Brent's Theorem allows conversion of a WT algorithm into a bounded PRAM algorithm.

Proof:

For each time step i, for $1 \leq i \leq S(n)$, let $W_i(n)$ be the number of operations in that step. We simulate each step of the WT algorithm on a p-processor PRAM in parallel ste$\lceil \frac{W_i(n)}{p} \rceil$ scheduling the $W_i(n)$ operations on the p processors in groups of p operations at a time.

The last group may not have p operations if p does not divide $W_i(n)$ evenly. In this case, we schedule the remaining operations among the smallest-indexed processors.

Given this simulation strategy, the time to simulate step $W_i(n)$ of the WT algorithm will be $\lceil \frac{W_i(n)}{p} \rceil$

and the total time for a p processor PRAM to simulate the algorithm is

$$\sum_{i=1}^{S(n)} \lceil \frac{W_i(n)}{p} \rceil \leqslant \sum_{i=1}^{S(n)} (\lfloor \frac{W_i(n)}{p} \rfloor + 1) \leqslant \lfloor \frac{W(n)}{p} \rfloor + S(n).$$

Remark

- There are a number of complications that this simple sketch of the simulation strategy does not address.
- For example, to preserve the semantics of the **forall** construct,
  - we should generally not update any element of the left-hand side of a WT assignment until we have evaluated all the values of the right-hand side expression.
  - This can be accomplished by the introduction of a temporary result that is subsequently copied into the left hand side.

Computational Networks

A *Computational Network* is a directed acyclic graph whose vertices are subdivided into three sets:

Input vertices: These vertices have no incoming edges.

Output vertices: These vertices have no outgoing edges.

Interior vertices: These vertices have incoming edges and a single outgoing edge.

- Each interior vertex is labeled with an elementary operation.
- The number of incoming edges to an interior vertex is called ***fan-in***, and
- the number of outgoing edges is called ***fan-out***.
- The maxima of these two quantities over entire graph is called, respectively, **the fan-in and the fan-out of the graph.**

- The length of the longest path from any input vertex to any output vertex is called ***depth*** of the computational network.

A *computation* on a computational network

The *computation* performed by a computation network, on a given set of inputs is defined to be the data that appears on the output vertices as a result of the following procedure:

- Apply the input data to the input vertices.
- Transmit data along directed edges. Whenever an interior vertex is encountered, wait until data arrives along all of its incoming edges, and then perform the indicated elementary operation. Transmit the result of the computation along all of the outgoing edges.
- The procedure terminates when there is no data at the interior vertices.

Brent's Theorem (with CN formulation)

Let N be a computational network with n interior nodes and depth d, and bounded fan-in.

Then the computations performed by N can be carried out by a CREW-PRAM computer with p processors in time O(n/p+d).

(The total time depends upon the fan-in of *N* -- we consider it as a constant of proportionality.)

->This result is normally applied to modifying parallel algorithms to use fewer processors.

Proof:

- In order to simulate the computation of *N* we define the ***depth*** of each vertex - v, *n,* of *N*, to be the maximum length of any path from an input vertex to *v*
  - clearly this is the greatest distance that any amount of input-data has to travel to reach *v*.

- We assume that that we have a data-structure in the memory of PRAM for encoding a vertex of *N*, and that has a field with
  - a pointer to the vertex that receives the output of its computation, if it is an interior vertex.
  - This field is *null* if the vertex is an output vertex.

- The simulation is done inductively – >

  we simulate all of the computations that take place at vertices of depth ≤ k-1 before simulating the computation on vertices of depth k-1.

  Suppose that there are n_i interior vertices of N whose depth is precisely i; then $(\text{sum i: } 1 \leq i \leq d : n_i) = n$.

  After simulating the computations on vertices of depth k-1, the computations on nodes of depth k could be simulated, since the inputs of these nodes are now available.

Proposition

When performing the computations on nodes of depth k, the order of the computations is irrelevant.

This is due to the definition of depth – it implies that the output of any vertex of depth *k* is input to a vertex of strictly higher depth (since depth is the length of the longest path from an input vertex to the vertex in question).

The simulation of the computations at depth *k*

- Processors read the data from the output areas of the data-structures for vertices at depth *k*-1.
- Processors perform required computations.
- Since there are n_i nodes of depth *i*, and the computations can be performed in any order, the execution-time of this phase is $[n_i / p] \leq n_i / p + 1$.
- The total execution-time is thus

$$(\text{sum } i : 1 \leq i \leq d : [n_i / p]) \leq (\text{sum } i : : 1 \leq i \leq d : n_i / \text{p} + 1) = \text{n} / \text{p} + \text{d}$$

Corollary

$$\frac{T_1}{p} \leq T_p \leq \frac{T_1}{p} + T_\infty.$$

T_1 – the sequential implementation
T_∞ - the parallel implementation with unbounded number of processors
T_p –the parallel implementation using p processors

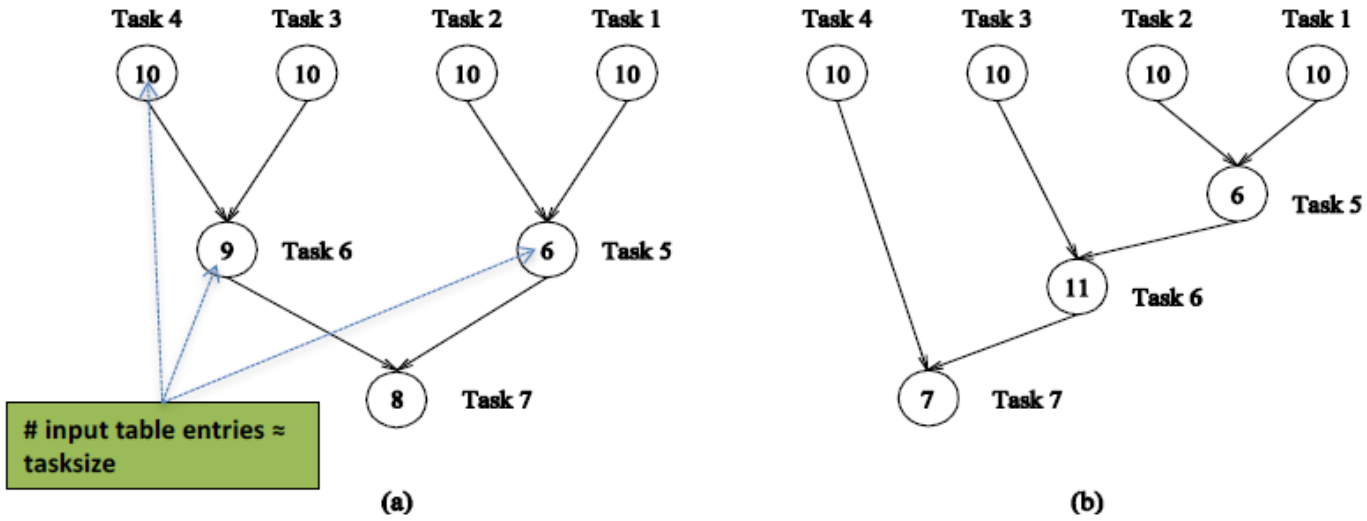
Brent Scheduling Principle

- This principle specifies that it is possible to reduce the number of processors used in parallel algorithms, without increasing the asymptotic execution time. In general, the execution time increases somewhat when the number of processors is reduced.

- Suppose algorithm A has the property that its computations can be expressed in terms of a computational network with *n* vertices, and depth *d* and has a bounded fan-in. Then algorithm A can be executed on a CREW-PRAM computer with *p* processors in time *O*(n/p+d).

- Re-arrange the corresponding computational network by reducing the depth may lead to optimization!

Example - applying database queries two decomposition variants:



Example

- Adding the numbers given in an array of length *n*.
  A possible computational network is a binary tree with *n* leaves.

  - If $n=2^k$, then the binary tree has the depth equal to *k*,
    and the all the interior nodes are equal to $2^k - 1 = 2^{k-1}+\ldots+2+1$, ->W(n)= 2^k
  - If we use 2^{k-1} processors the time complexity is equal to *k* *(S(n)=k)*.

    $$\lfloor \frac{W(n)}{p} \rfloor + S(n)$$

  - If we use p processors, the time complexity is
    T(n)=(2^k)/ p +k
  - If p = 2^{k-q} then T(n)= 2^q+*k* which means *O*(*k*) = O(log n) if $2^q = O(k)$.

  - Generally If p= 2^k /c*k then T(n)=(c+1)k = O(k)

- Brent's theorem has interesting implications related to *work efficiency* of an algorithm.

Remind…

- The amount of work performed by a parallel algorithm is the total number of computations executed by that algorithm

- A parallel algorithm is said to be *cost-optimal* if $(\boldsymbol{pT_P) = \Theta (T_S)}$.

  $\boldsymbol{E} = \boldsymbol{T_S} / (\boldsymbol{pT_P})$ => for cost optimal systems $\boldsymbol{E} = O(1)$.

scaling down…again

- Using fewer than the maximum possible number of processing elements to execute a parallel algorithm is called *scaling down* a parallel system.

- A naive way of scaling down is to think of each processor in the original case as a virtual processor and to assign virtual processors equally to scale down processors.

- Since the number of processing elements decreases by a factor of $\boldsymbol{n} / \boldsymbol{p}$, the computation at each processing element increases by a factor of $\boldsymbol{n} / \boldsymbol{p}$.

- The communication cost should not increase by this factor since some of the virtual processors assigned to a physical processor might talk to each other.
  This is the main reason for the improvement from building granularity.

Example

Algorithm 2 (Sequence reduction, WT description)
Input: Sequence a with $n = 2^k$ elements of type T, binary associative operator $\oplus : T \times T \to T$.
Output: $S = \oplus_{i=1}^{n} a_i$.

T REDUCE(**sequence**⟨T⟩ a, $\oplus : T \times T \to T$)

```
1  T B[1..n]
2  forall i ∈ 1 : n do
3     B[i] ← a_i
4  enddo
5  for h = 1 to k do
6     forall i ∈ 1 : n/2^h do
7        B[i] ← B[2i − 1] ⊕ B[2i]
8     enddo
9  enddo
10 S ← B[1]
11 return S
```

Algorithm 3 (Sequence reduction, PRAM description)

Input: Sequence a with $n = 2^k$ elements of type T, binary associative operator $\oplus : T \times T \to T$, and processor id i.

Output: $S = \oplus_{i=1}^{n} a_i$.

T PRAM-REDUCE(**sequence**⟨T⟩ a, $\oplus : T \times T \to T$)

```
1  T B[1..n]
2  local integer h, j, ℓ
3  for ℓ = 1 to ⌈n/p⌉ do
4     if (ℓ − 1)p + i ⩽ n then
5        B[(ℓ − 1)p + i] ← a_{(ℓ−1)p+i}
6     endif
7  enddo
8  for h = 1 to k do
9     for ℓ = 1 to ⌈(n/2^h)/p⌉ do
10       j ← (ℓ − 1)p + i
11       if j ⩽ n/2^h then
12          B[j] ← B[2j − 1] ⊕ B[2j]
13       endif
14    enddo
15 enddo
16 if i = 1 then
17    S ← B[1]
18 endif
19 return S
```

$$T_C(n,p) = \lceil \frac{n}{p} \rceil \Theta(1) + \sum_{h=1}^{k} \lceil \frac{n/2^h}{p} \rceil \Theta(1) + \Theta(1) = O(\frac{n}{p} + \lg n)$$

Brent Theorem - interpretation

- Brent's theorem specifies for a sequential algorithm with d time steps, and a total of W operations, that a runtime T

  T =d + (W-d)/p

is definitely possible on a shared memory machine (PRAM) with p processors.

Proof:

(sum i : $1 \le i \le d$: $[n_i / p]$) $\le$ (sum i : $1 \le i \le d$: $(n_i+p-1)/ p$)
= (n-d) / p + d

- There may be an algorithm that solves this problem faster, or it may be possible to implement this algorithm faster (by scheduling instruction differently to minimize idle processors, for instance), but it is definitely possible to implement this algorithm in this time given p processors.

Sum – again… (1)

Variant 1: step through the array, adding each value in turn to an accumulator.

```
for (i=0;i < length(a); i++) { sum = sum + a(i); }
```

There are n operations, so W = n.

T = n + 0/p.

(W-S) = 0,

so no matter how many processors are available, this algorithm will take time n.

Sum – again... (2)

sum(a) = ((A0 + A1) + (A2 + A3)) + ((A4 + A5) + (A6 + A7)) etc...

For an array of length n, the longest chain(s) will be of length **log(n).**

S = log(n).

W = n (as before).

Possible time =>

T = log(n) + (n - log(n))/p.

Remarks regarding number summation

- No matter how many processors are used, there can be no implementation of this algorithm that runs faster than O (log(n)).
- If we have n processors, the algorithm can be implemented in log(n) time
- If we have one processor, the algorithm can be implemented in n time.
- If we consider the amount of work done in each case, with one processor, we do n work, with log(n) processors we do n work, but with n processors we do nlog(n) work.
  - The implementations with 1 or log(n) processors, therefore are cost optimal, while the implementation with n processors is not.

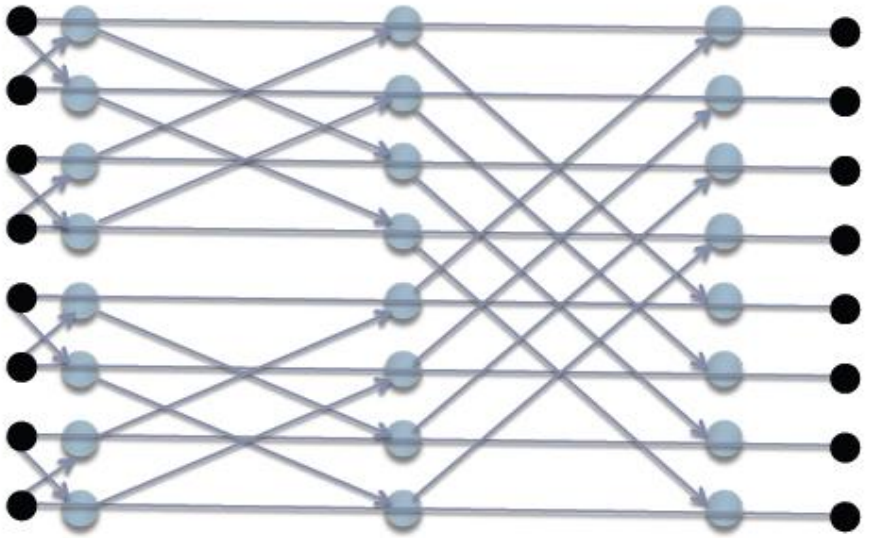
Conclusion

- It is important to remember that Brent's theorem does not tell us how to implement any of these algorithms in parallel;
  - it merely tells us what is possible.

- The Brent's theorem implementation may be hideously ugly compared to the naive implementation.

- Key to understanding Brent's theorem is understanding time steps.
  In a single time step every instruction that has no dependencies is executed, and therefore **t** is equal to the length of the longest chain of instructions that depend on the results of other instructions (as any shorter chains will be finished executing by (or before) the time the longest chain has).

Radix 2 (FFT Graph)



- N inputs, N outputs, Size = NlgN, Depth = lgN
- The reverse of an FFT graph is an FFT graph

FFT Graph Formula

- At stage i, node $a_n...a_1$ has
  - inputs $a_n...a_{i+1}0\ a_{i-1}...a_1$ and $a_n...a_{i+1}1\ a_{i-1}...a_1$
  - outputs $an...a_{i+2}\,0\ a_i...a_1$ and $a_n...a_{i+2}1\ a_i...a_1$

- Reverse FFT: Same structure, with address bits reversed

FFT Computation

- At stage i, node $a_n...a_1$ has
  - inputs $a_n...a_{i+1}0a_{i-1}...a_1$ and $a_n...a_{i+1}1a_{i-1}...a_1$
  - outputs $a_n...a_{i+2}0a_i...a_1$ and $an...a_{i+2}1a_i...a_1$

- Assume $p = 2k$, $n \geq 2k$ ($p \leq \sqrt{N}$)

1. Each processor computes on a block of contiguous

inputs $a_n...a_{n-k+1}x...x$ (p distinct blocks) – n-k stages

2. Transpose: $a_n...a_{n-k+1}a_{n-k}...a_1$? $a_{n-k}...a_1\ a_n...a_{n-k+1}$

3. Each processor computes on a block of contiguous inputs – k stages.

- Communication volume N, communication depth 1

Sorting networks

- See Lecture 8-SN.pdf

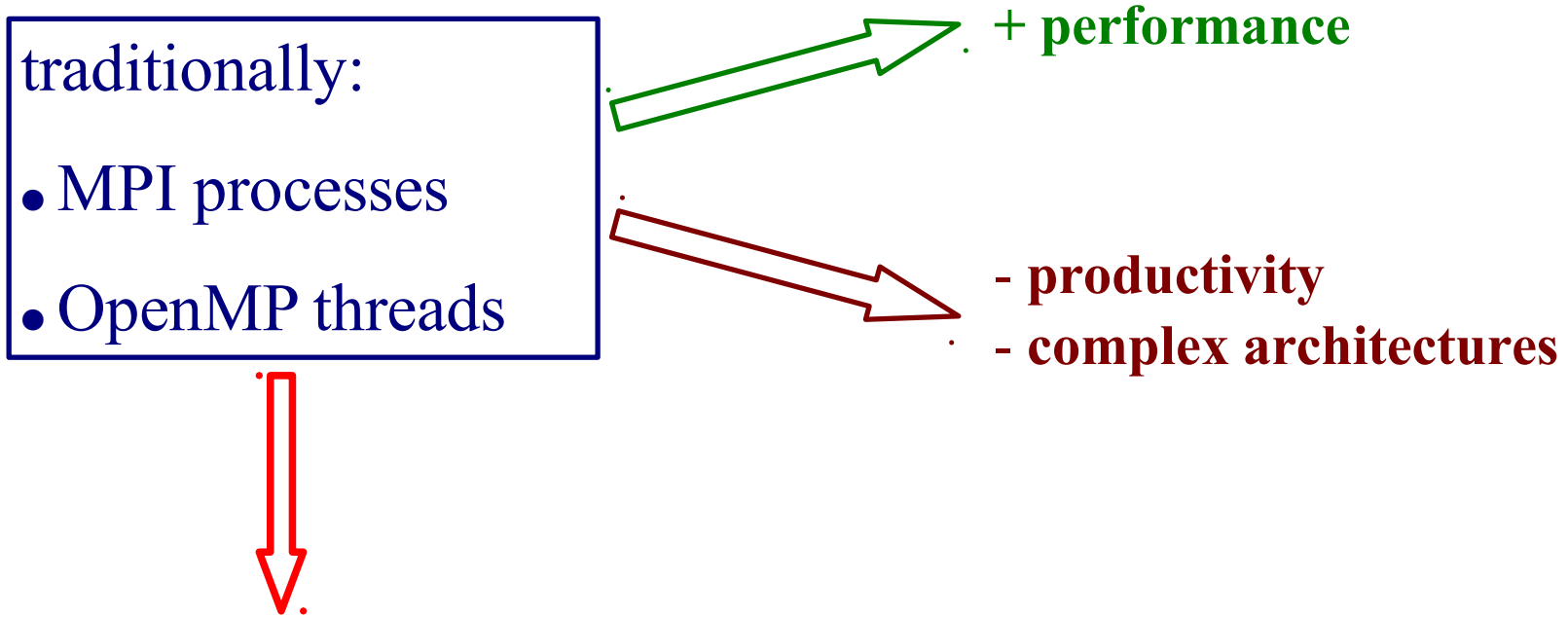
Lecture 9
Task-Based Parallel Programming Models

References:

Claudia Fohry. An Overview of Task-Based Parallel Programming Models. Universität Kassel, Germany

OpenMP: https://www.openmp.org/wp-content/uploads/OpenMP-UMT-Tasking-1.pdf

Parallel Programming in HPC



novel proposals:

UPC, CAF, Chapel, Cilk, TBB, Legion, Charm++, Go, Spark, ...

PGAS TaPP Actors CSP MapReduce ...

„Programming Model“

= a set of abstractions that simplify and structure the way the programmer thinks about and expresses a parallel algorithm



- realized by languages, language extensions and libraries
- concrete system may support multiple programming models
  - e.g. OpenMP: thread-based and task-based programming

Task

A task is defined by :

- The operation that is to be performed (implementation).
- The way in which the operations are related to other operations (dependencies).
- The data that is involved in the operation (input and output).

Task Dependencies: Example 1

```
#include <stdio.h>

void function1(int a, int b) {

    printf("The sum is %d.\n", a + b);

}

void function2(int b) {

    printf("The sum is %d.\n", 10 + b);

}

int main() {

    int a = 10, b = 7;

    function1(a, b);

    function2(b);

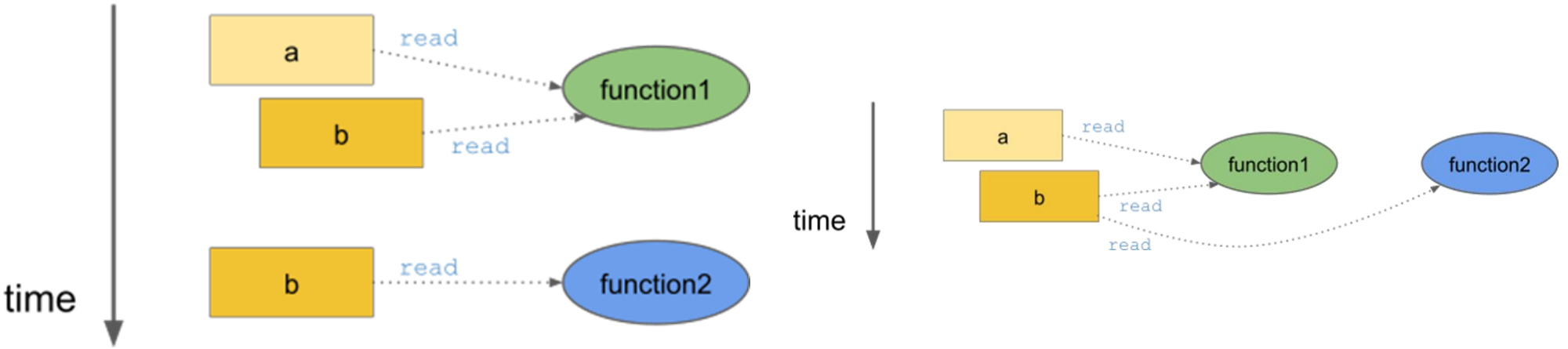
    return 0;

}
```

Task 1
Operation: function1.
Input: variables a and b.
Output: updated variable b.
Depends on: none

Task 2
Operation: function2.
Input: variable b.
Output: none.
Depends on: Task1

Execution


time
a
read
function1
b
read
b
read
function2
time
a
read
function1
b
read
read
function2

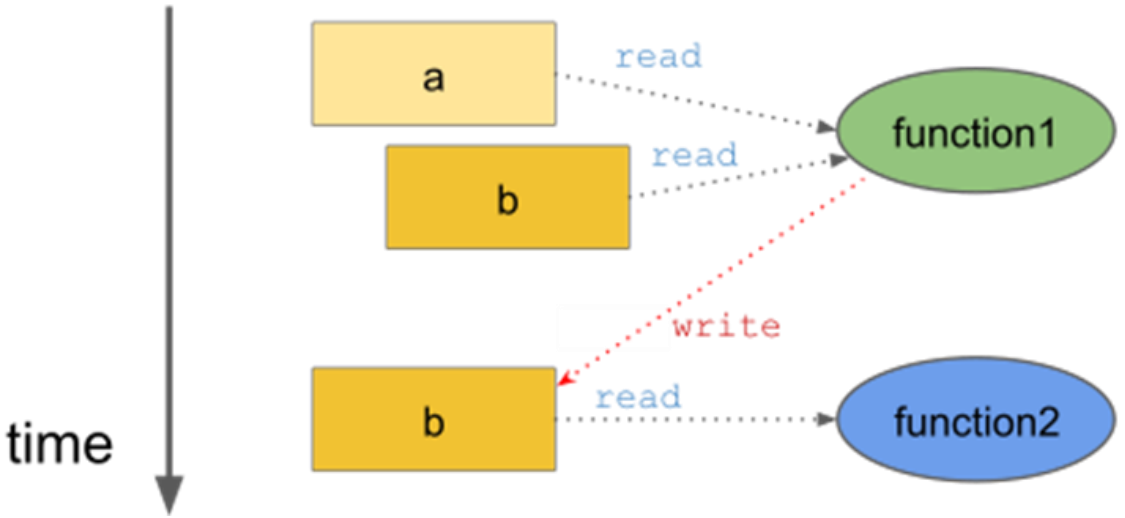
Example 2

```
#include <stdio.h>
void function1(int a, int *b) {
    printf("The sum is %d.\n", a + *b);
    *b += 3;
}

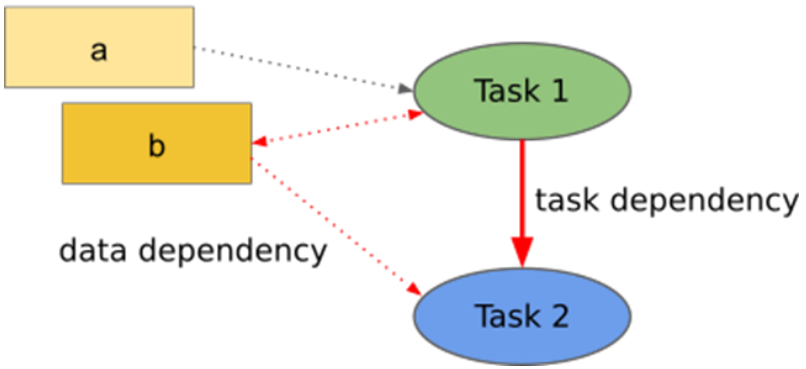
void function2(int b) {
    printf("The sum is %d.\n", 10 + b);
}

int main() {
    int a = 10, b = 7;
    function1(a, &b);
    function2(b);
    return 0;
}
```

Data dependency – variable b



Task dependency representation – Task Graph Directed Acyclic Graph (DAG)



A task-graph can also be constructed **explicitly**, i.e. a programmer defines both

- the task implementations and
- the task dependencies.

=> **Possible parallelization**

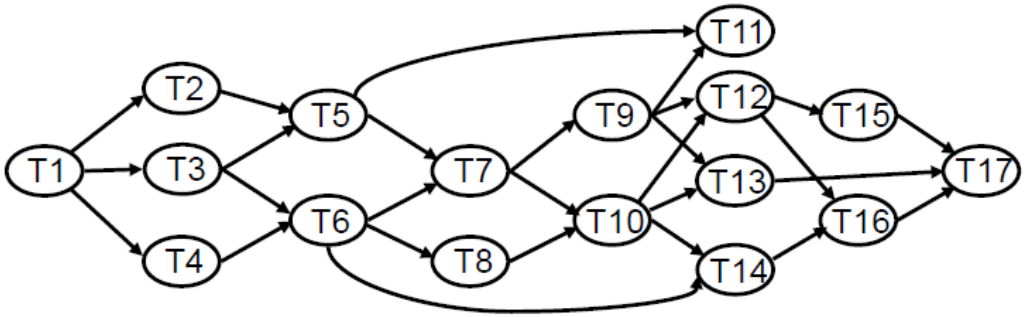
Sequential (Implicit) execution:
The runtime systems deduces the task dependencies from the sequential order and the input and output variables.
⇒the task graph is constructed **implicitly**. The runtime system then executes the task in a **sequentially consistent order**.
That is, the runtime system may execute the task in any order as long as it respects the task dependencies.

Task decomposition -Task graph

- Divide work into tasks that can be executed concurrently.
- Many different decompositions possible for any computation.
- Tasks may have same, different, or even indeterminate sizes.
- Tasks may be independent or have non-trivial order.
- Conceptualize tasks and ordering as task dependency DAG.

DAG - example

—node = task
—edge = control dependency



Different task decompositions may lead to significant differences with respect to their eventual parallel performance.

Data Dependencies

- **True(Flow)-Dependence**: Task 1 computes a value stored at A, and Task 2 retrieves a value stored at A.
- **Anti-Dependence**: Task 1 retrieves a value stored at A, and Task 2 computes a value stored at A
- **Input-Dependence**: Task 1 and Task 2 both retrieve a value stored at A
- **Output-Dependence**: Task 1 and Task 2 both compute a value stored at A

Interaction vs Dependency graphs

- Subtasks generally exchange data with others in a decomposition.

- The graph of tasks (nodes) and their interactions/data exchange (edges) is referred to as a task interaction graph.

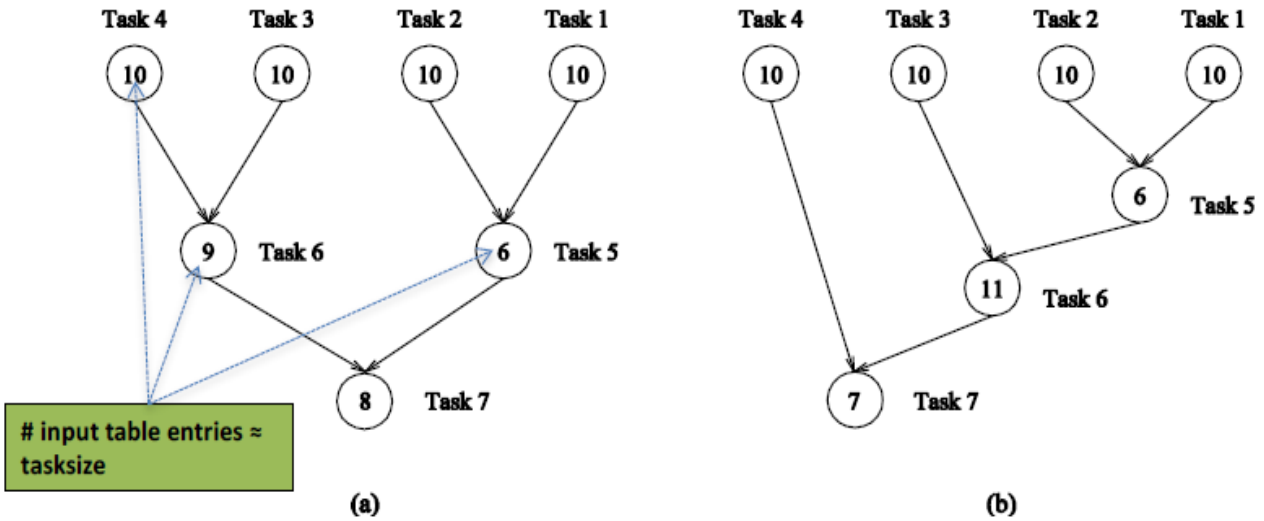
support etc.

- ***Task interaction graphs*** represent input- and output- dependencies, whereas
- ***Task dependence graphs*** represent true-dependencies or anti-dependencies.

Critical Path Length

- A directed path in the task dependency graph represents a sequence of tasks that must be processed one after the other.
- The longest path determines the shortest time in which the program can be executed in parallel.
- The length of the longest path in a task dependency graph is called the critical path length.

Examples- two database query decompositions:

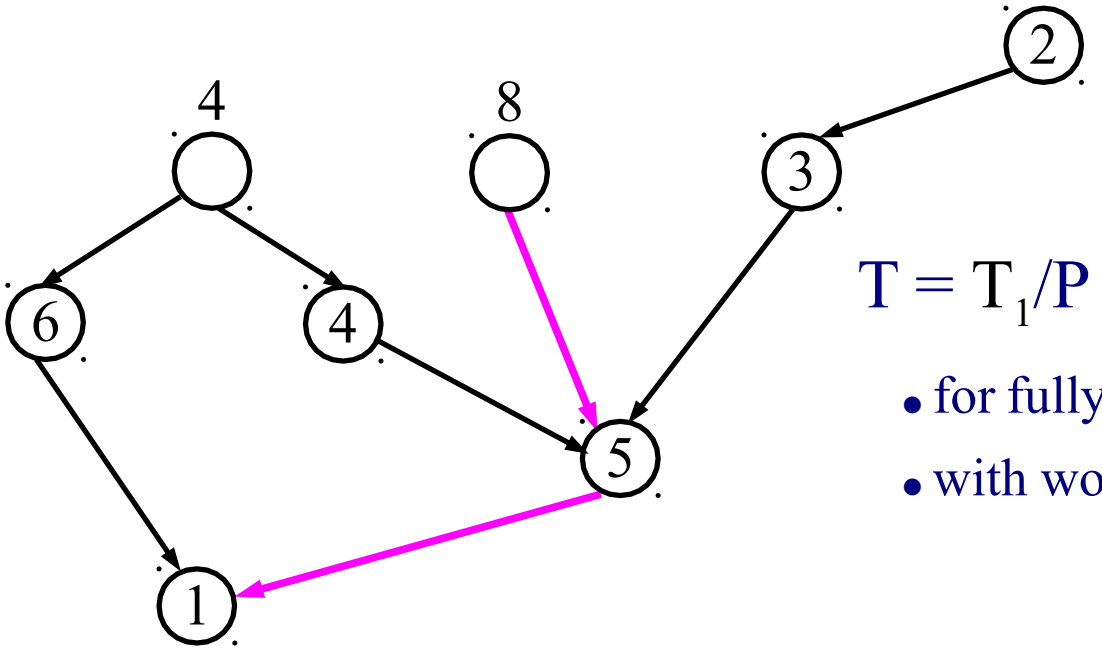


Performance analysis

- task graph = DAG representation of TaPP algorithm / program
- labels = each node labeled with its time-complexity
- critical path = path that maximizes sum of node labels
- T_∞: Span = sum of node labels on critical path

  = time that would be required with infinite number of processors
- T_1: Work= total sum of node labels

  = sequential running time

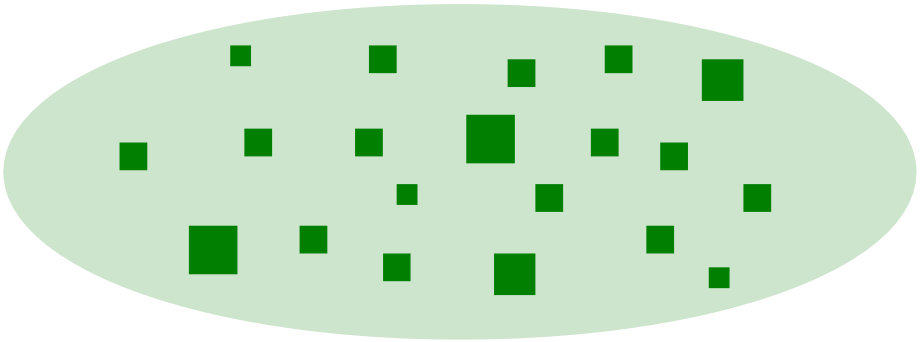


$T = T_1/P + O(T_\infty)$

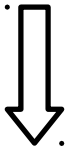
- for fully strict graphs (fork/join)
- with work-stealing scheduler

Tasks

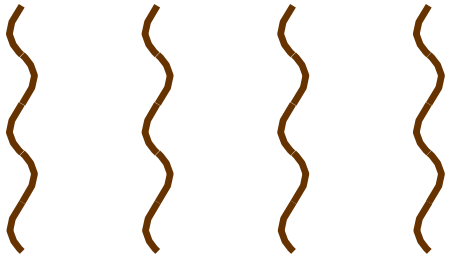
tasks



runtime system



threads / processes



operating system



cores



- part of the computation that can be performed in parallel to other tasks
- number is large, architecture-independent
- generated at runtime
- may generate other tasks
- same or different code
- possibly dependencies, parent-child relation, communication

TaPP:
Task-Based Parallel Programming

Programmers describe computation as a set of tasks and specify their cooperation

⟶tasks are fine-grained

⟶responsibility for task mapping is shifted to runtime system

Details are realization-dependent:

- **cooperation mechanism**
- **task-internal parallelism**
- **memory model**
- **…**

Alternative terms:

- **task-parallel programming**
- **task-oriented parallel programming**
- **Asynchronous Many Task (AMT) programming**

Pros of TaPP - Overview

Key properties:

- higher abstraction level
- automatization of scheduling relieves human programmer
- scheduling is converted from static to dynamic activity

Major potential gains:

- productivity
- portability, heterogeneity, scalability
- load balancing, irregularity support
- fault tolerance, elasticity

Pros of TaPP - Productivity

- higher abstraction level: no need to consider physical resources

  ⟶programmers can concentrate on application
- evolutionary adoption: tasks internally coded in traditional languages ⟶ integrate legacy codes
- interoperability: different tasks in different languages
- large-scale software development: different tasks by different programmers
- determinism: tasks as deterministic, composable units
- clarity: separation of concerns, e.g. task definition vs. task execution, task code vs. and dependencies

Pros of TaPP - Adaptivity

- portability: incl. shared / distributed memory, GPUs, hybrid / heterogeneous
- heterogeneity: versions of same task can run on different resources
- dynamic load balancing supports:
  - irregular applications: varying task number / sizes
  - dynamic environments, e.g.:
    - fault Tolerance: units for failure containment → re-run tasks
    - elasticity: addition and deletion of resources

Pros of TaPP - Performance

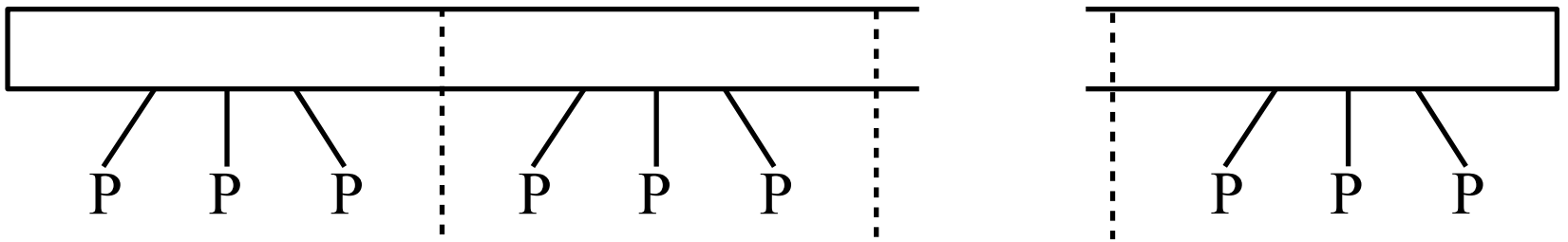
- contra: overhead for task management, loss of control
- predictable performance: tight complexity bounds for subclasses
- performance portability: same program may run efficiently on different architectures
- scalability: typically large number of tasks
- avoidance of global synchronization: start individual task when inputs available ⟶ parallelism among stages / libraries
- latency hiding: switch to another task when blocked
- less communication costs: tasks may be moved to their data ⟶
- performance loss is limited

Cons of TaPP

- overhead for task management: reduced performance
- higher abstraction level: loss of control can reduce performance
- overdecomposition: identify more parallelism than hardware provides → challenge for algorithm design, increases overhead
- focus on code partitioning: no associated concept for partitioning data (unlike in object orientation)
- many proposals: not yet an established / prevailing model
- usage: not yet widely used in HPC
- new language problem: low user base, small library, limited tool support etc.

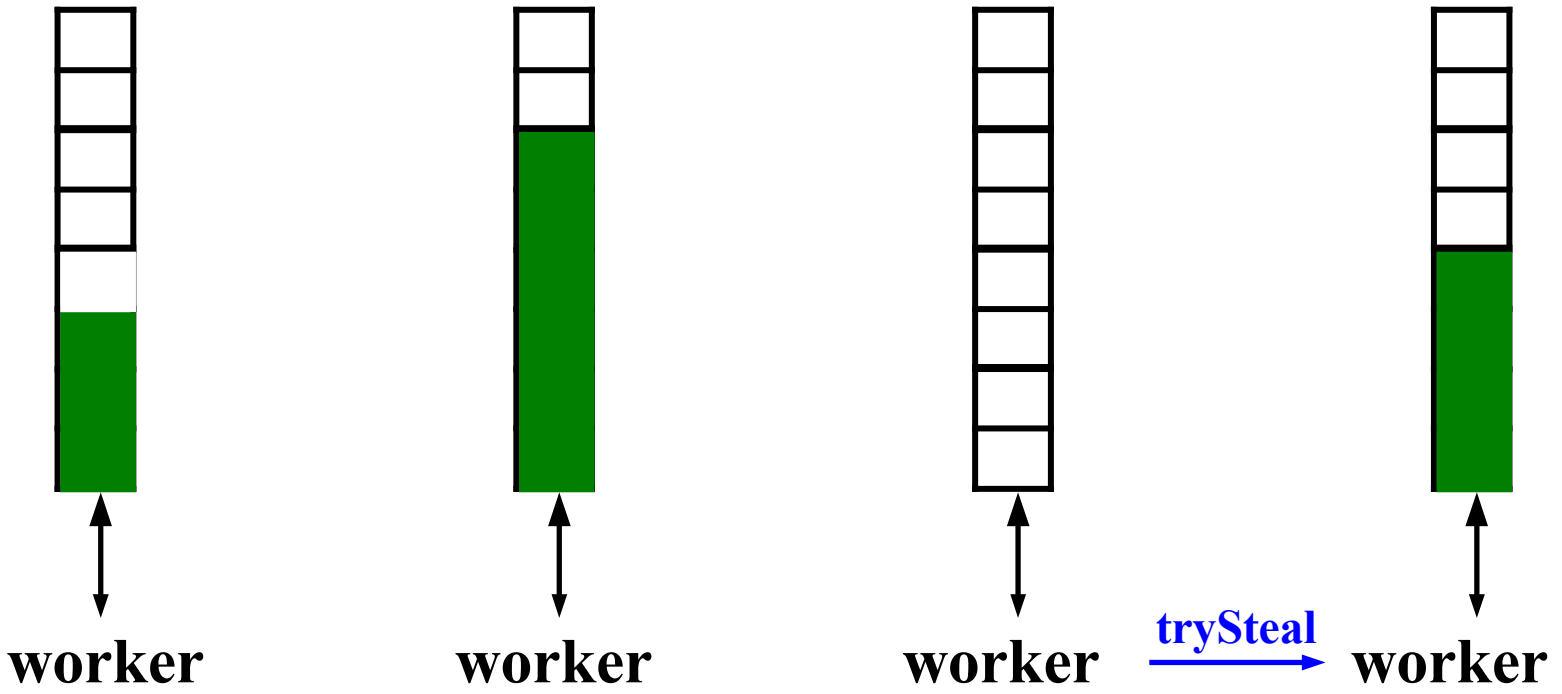
Design Options for TaPP Models

- realization by library, language extension, pragmas, language
- language binding: C++, Java, new language, ...
- target architecture:
  - shared memory: workers are threads
  - distributed memory: workers are processes
  - partitioned global address space (PGAS): intra-place vs. global load balancing



- support for:
  - task cancellation, task priorities, task-internal parallelism, grain-
  - size control, task preemption, fault tolerance, elasticity reduction
  - and synchronization constructs etc.

Work Stealing



- typical implementation approach
- variants:
  - victim selection: random, locality-based etc.
  - work-first vs. help-first
  - ...

Core Design Options

Types of Task Cooperation:

- independent: tasks run independently, contribute task results to final result
- fork-join: each task returns result to parent task, root task computes result
- side effects: communication through read/write accesses to a shared memory
- dataflow: tasks exchange data via inputs/outputs, ready when inputs available
- message-driven: tasks are methods of objects, invoked on message receipt

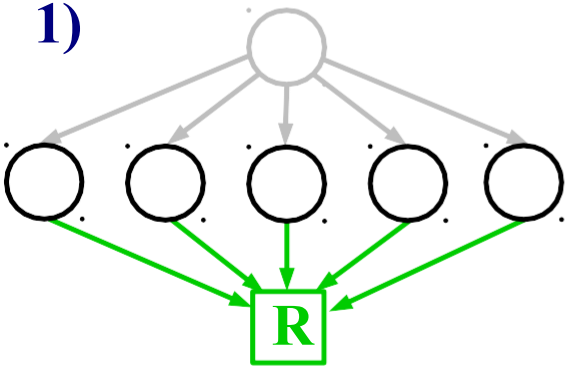
Types of Task Mapping (for distributed memory):

- unbound: system maps tasks globally, independent of data
- place-bound: user maps data and tasks to PGAS places (e.g. Chapel)
- data distribution-controlled: tasks are mapped to data locations, data distribution is chosen separately (e.g. Legion, StarPU, ParSEC)
- object-controlled: tasks (method calls) run on node of their object, objects are placed automatically and may migrate (e.g. Charm++, HPX)

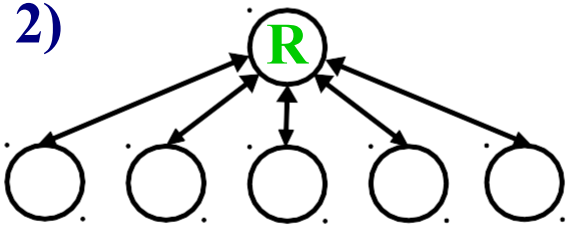
Independent Tasks

- static or dynamic tasks
- task results are combined into final results
  - e.g. by reduction with commutative, associative operator
- typically, all tasks have same code
- task graphs:

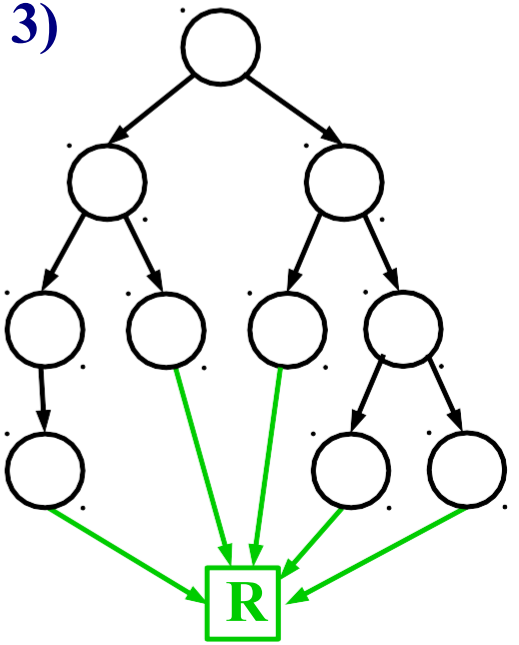
1)



2)

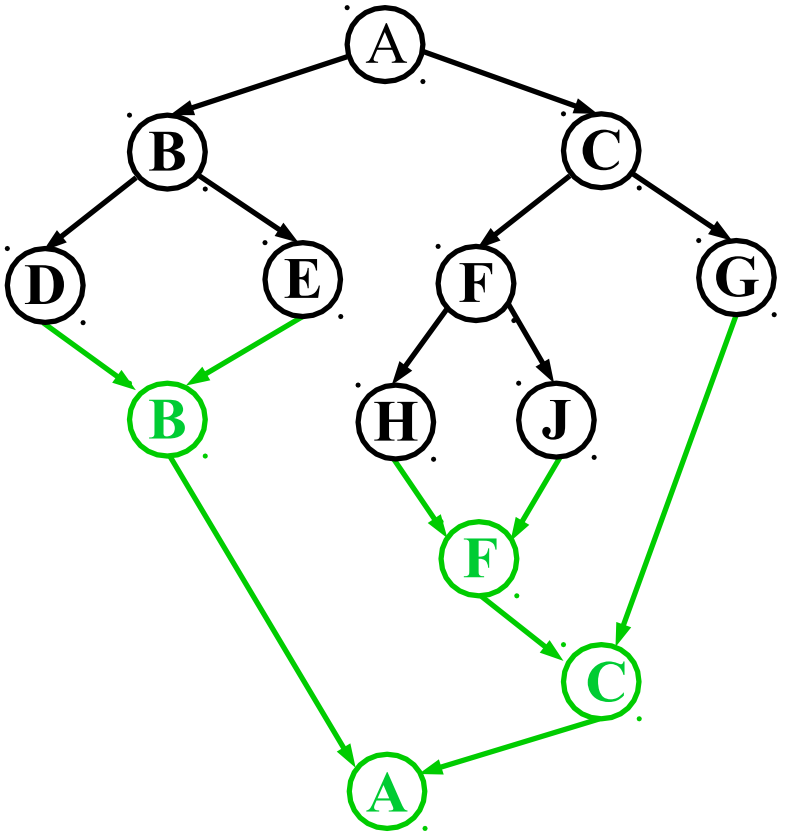


3)



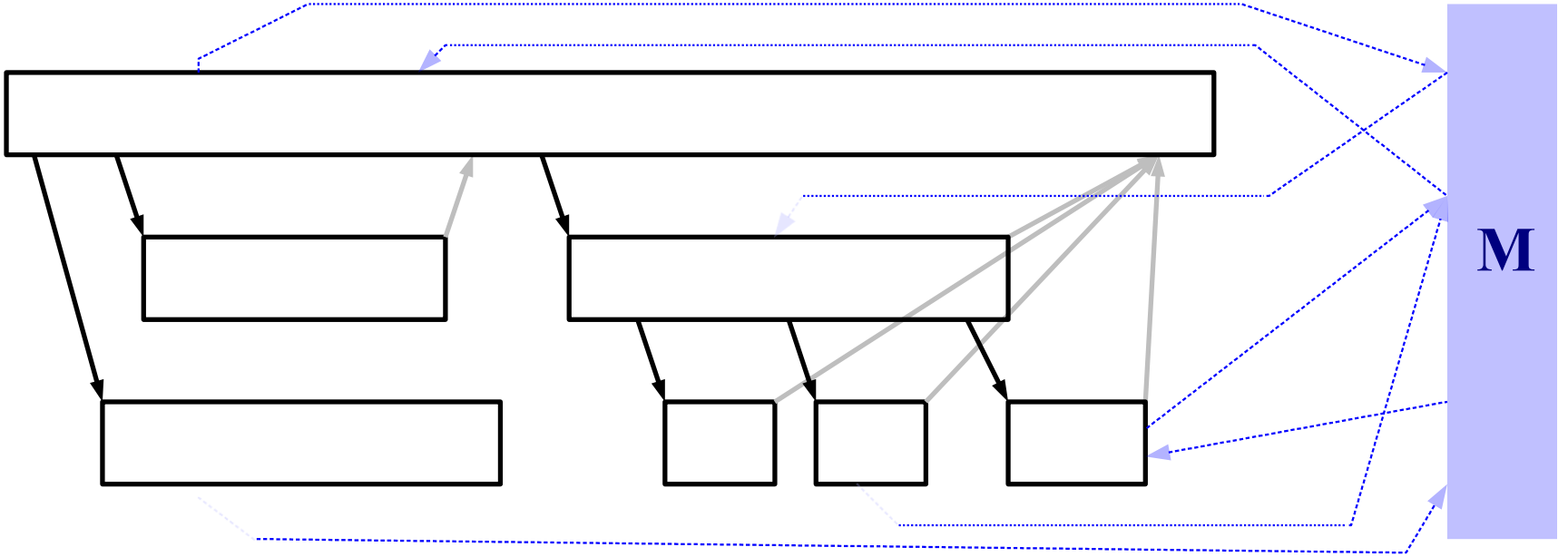
Fork-Join

- computation tree, each task returns result to parent
  - children can not outlive parents
- inside task: computation, spawn, result receipt in any order



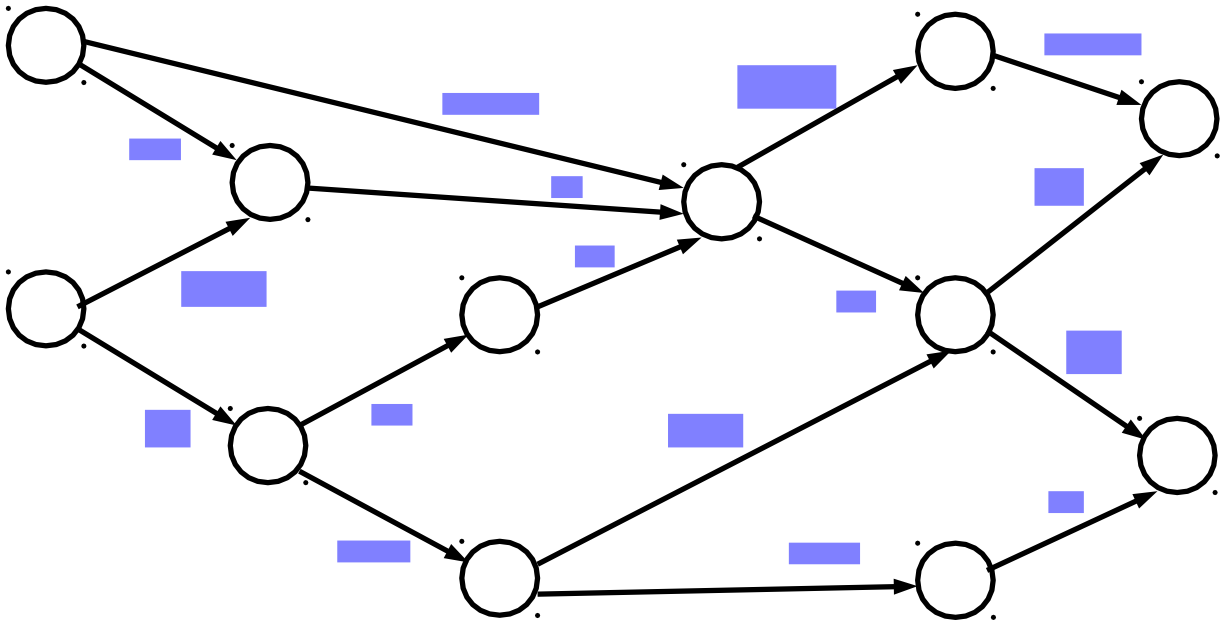
Cooperation through Side Effects

- control-driven: tasks are ready when spawned
- tasks communicate *during* execution
  - possibly bidirectional
- tasks may synchronize with their parent or ancestor
  - often: async-finish (e.g. Chapel, X10, APGAS Library)



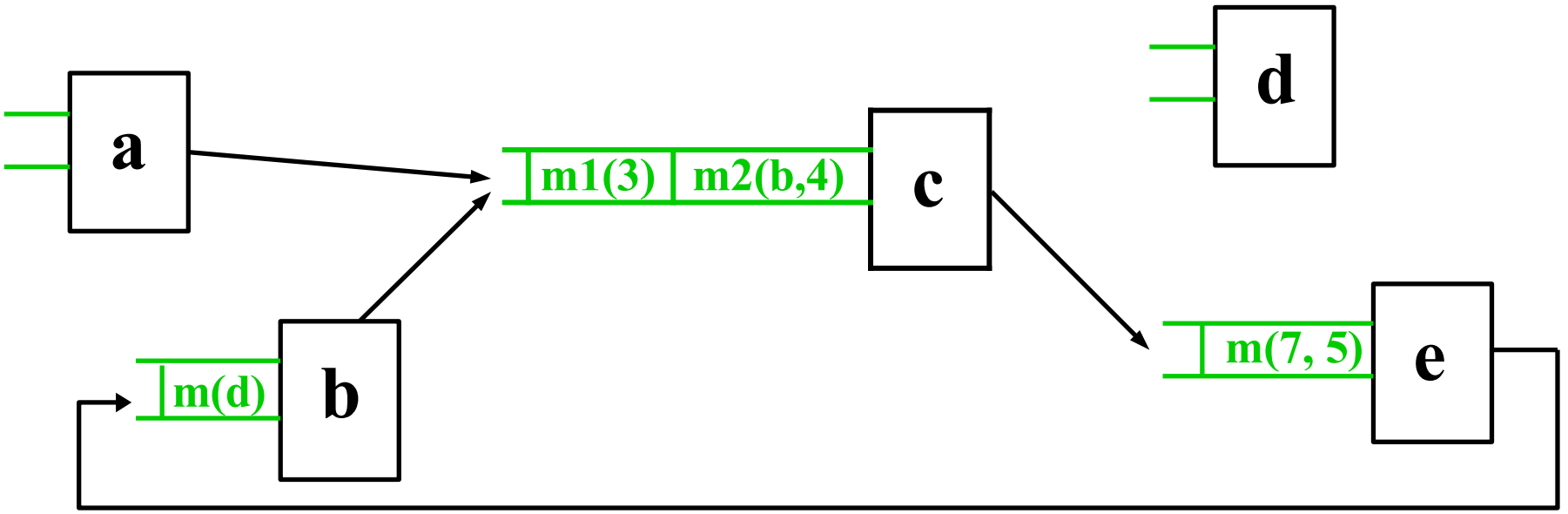
Cooperation through Dataflow

- tasks are functions, any acyclic task graph
- tasks become ready when inputs available
  - uncoupling of task generation and task start
- specification of dependencies:
  - explicit: between tasks (e.g. Cpp-Taskflow, TBB, ParSEC)
  - implicit: indirect via data accesses (e.g. OpenMP tasks, StarPU, Legion)



Message-Driven Cooperation

- related to actor model
- tasks are methods of distributed objects
- invoked on message receipt
  - subject to scheduling from incoming message queue
- tasks may modify object data
  - sequential message receipt ⟶synchronization



Summary features

- concept of task-based parallel programming:
- programmers describe computation as a set of tasks and specify their cooperation
- task = part of computation that can be performed in parallel to other tasks runtime system maps tasks to processes / threads
- pros and cons: + productivity, + adaptivity, +/- performance, - usage
- design options:
- level: library, pragmas, language, language extension (often C/C++) target architectures: shared memory, distributed memory, PGAS
- task generation: static, dynamic
- task cooperation: independent, fork/join, side effects, dataflow, message-driven
- task mapping: unbound, place-bound, data distribution-controlled, object-controlled
- features: cancellation, priorities, grain size control, reduction, resilience, ...

TaPP Examples

- shared-memory task models:
  - Cilk: fork-join
  - Cpp-Taskflow: explicit dataflow
  - OpenMP tasks: fork-join, implicit dataflow, side effects, independent
  - TBB: explicit dataflow, fork-join, side effects, independent

- distributed-memory task models:
  - Chapel: side effects, place-bound
  - Legion: dataflow, data distribution-controlled
  - HPX: dataflow, object-controlled
  - Charm++: message-driven, object-controlled

OpenMP Tasks

- specification for set of compiler directives, library functions, environment variables
- for C/C++ and Fortran
- for shared-memory architectures (incl. NUMA) and accelerators

History:

- OpenMP: 1998
- tasks: 2008
- task cancellation, task dependences: 2013
- task priorities: 2015
- task reduction, affinity, dynamic dependences: 2018

- integration:
  - programmer sees both threads and tasks
  - thread = implicit task

OpenMP Tasks – Example

```
int fib(int n) {
    if (n<2) return n;
    int x, y;
    #pragma omp task shared(x) // firstprivate(n)
        x = fib(n-1);
        y = fib(n-2);
    #pragma omp taskwait
    return x + y;
}

#pragma omp parallel
{
    #pragma omp single
    {
        result = fib(n);
    }
}
```

OpenMP Tasks – Task Model

- task = code block
- data sharing between parent and child: firstprivate, shared
  - result return via side effect
  - requires synchronization (atomic, critical, locks)
- taskwait: wait for all children
- taskgroup: at end of block wait for all descendants
- tasks can be interrupted at task scheduling points
  - e.g. task, taskwait, taskyield
  - tied tasks are resumed by same thread
  - untied tasks are resumed by any thread
  - implications for threadlocal variables

OpenMP Tasks – Features

- grain size control: if, final
- taskloop: incl. grain size control, reduction
- reduction: control of scope and participants
- task cancellation:
  - deferred cancellation of taskgroup
- task dependencies: in, out, mutexinout, dynamic dependencies, ...

```
...
#pragma omp task shared(x) depend(out:x) x
= …;
#pragma omp task shared(x) depend(in:x)
… = x;
```

- task priorities: programmer hints on scheduling order
- task affinities: place close to given data

TBB

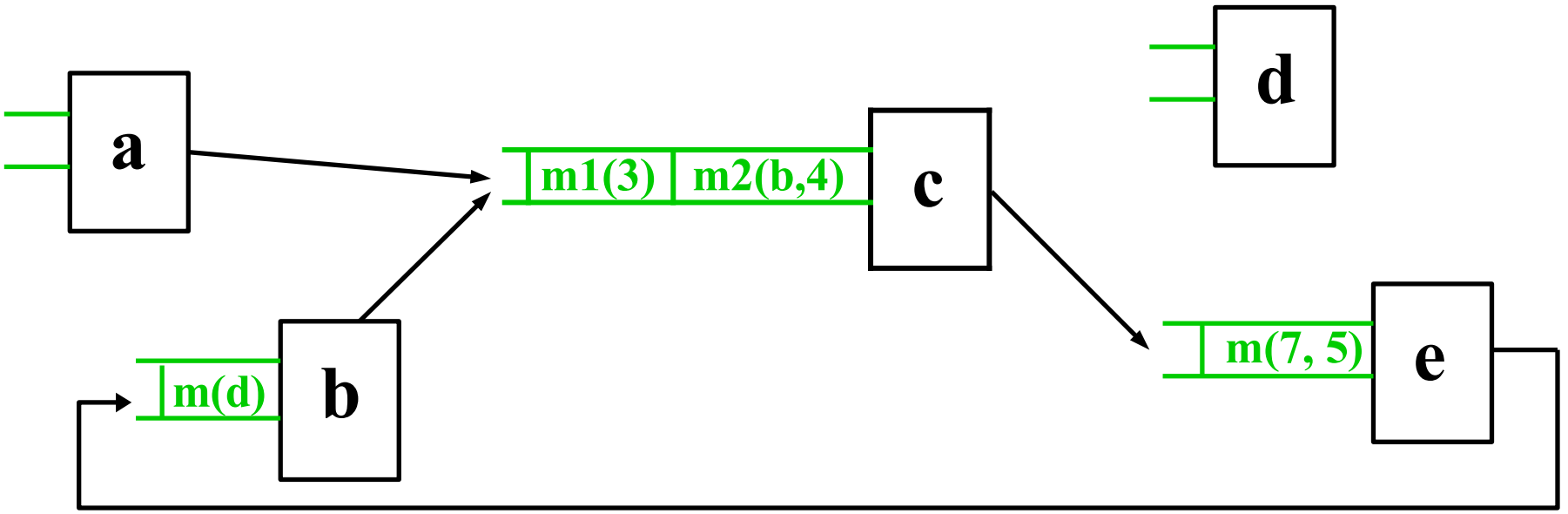
- TBB =Intel Threading Building Blocks
- C++ template library for shared-memory architectures
- features:
  - explicit task-based programming:
    - high-level: task_group class with run, wait, cancel
    - low-level: scheduler control e.g. cancellation, affinity
  - flow graph: user specifies task graph
  - functions for parallel patterns: parallel_for / reduce / pipeline ...
  - concurrent containers, e.g. concurrent_queue synchronization:
  - mutex, atomic operations
  - ...

TBB Tasks – Example

```
int Fib(int n) {
    if( n<2 ) {
        return n;
    } else {
        int x, y;
        task_group g;
        g.run([&]{x=Fib(n-1);});
        g.run([&]{y=Fib(n-2);});
        g.wait();
        return x+y;
    }
}
```

Message-Driven Actor Cooperation

- related to actor model
- tasks are methods of distributed objects
- invoked on message receipt
  - subject to scheduling from incoming message queue
- tasks may modify object data
  - sequential message receipt ⟶synchronization



General Variants of Task-based implementations

Features:

- *Task spawning:* any function can spawn a task, i.e. call a function that will be executed as a task. Dependencies are implicitly given by the order in which tasks are spawned, e.g. Cilk, OpenMP 4.0.
- *Dependency deduction*: tasks and the data they operate on are specified explicitly, dependencies are deduced from the data and the order in which the tasks are created, e.g. QUARK, OmpSs, StarPU.
- *Explicit task graph construction:* tasks and dependencies are explicitly specified by the user, e.g. Intel TBB, QuickSched.

Intel® oneAPI

ZAHA DIANA-NICOLETA

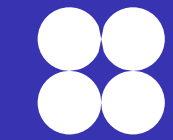
High Performance Computing
and Big Data Analysis



Presented on 14.05.2026 at MPP Course

In a nutshell…

Intel® oneAPI is a cross-industry, open, standards-based unified programming model that delivers a common developer experience across processor and accelerator architectures.

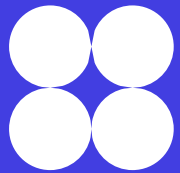






What is the problem?

- Modern computing requires using CPUs, GPUs, FPGAs, and other accelerators.
- Each device traditionally requires a separate programming language or model, such as OpenMP for CPUs, CUDA for NVIDIA GPUs, and OpenCL for FPGAs.
- **This fragmentation makes it difficult to write portable and maintainable code across architectures.**



Intel’s solution: oneApi



Intel oneAPI is a cross-architecture programming model that enables developers to write and run code across multiple types of hardware using a single language.

- It is built on open standards, primarily SYCL, and supports CPUs, GPUs, and FPGAs.
- Its goal is to simplify heterogeneous programming while maintaining high performance.

intel Software

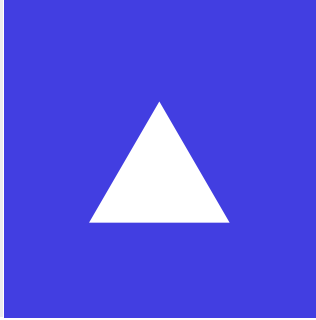
oneAPI

UNIFIED PROGRAMMING ACROSS MULTIPLE ARCHITECTURES

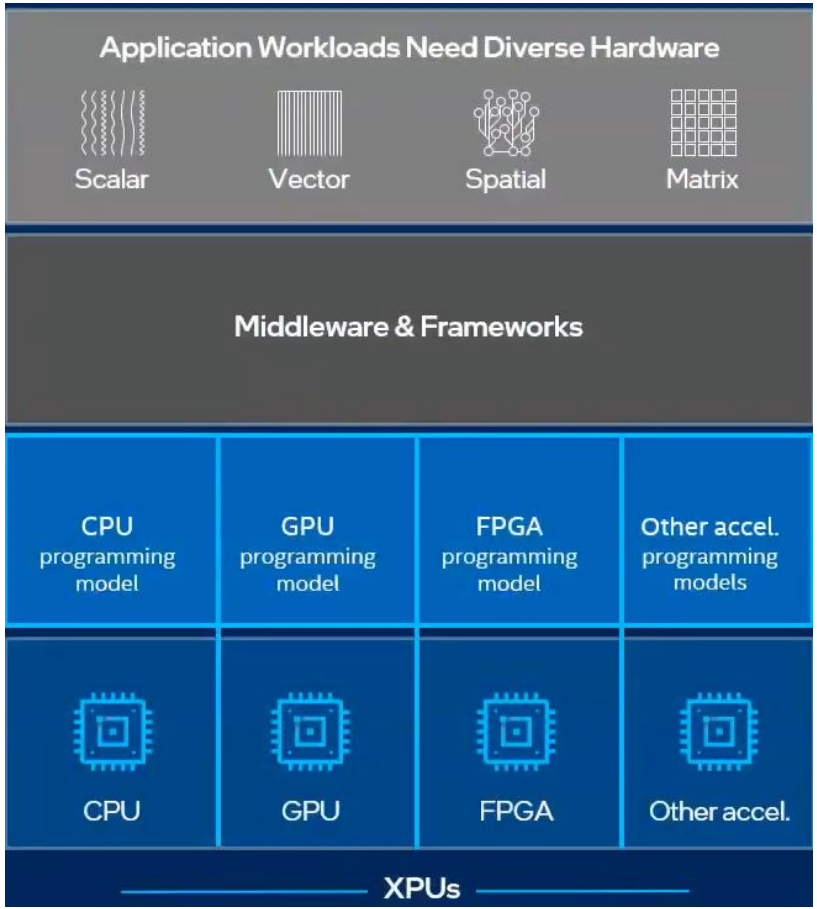


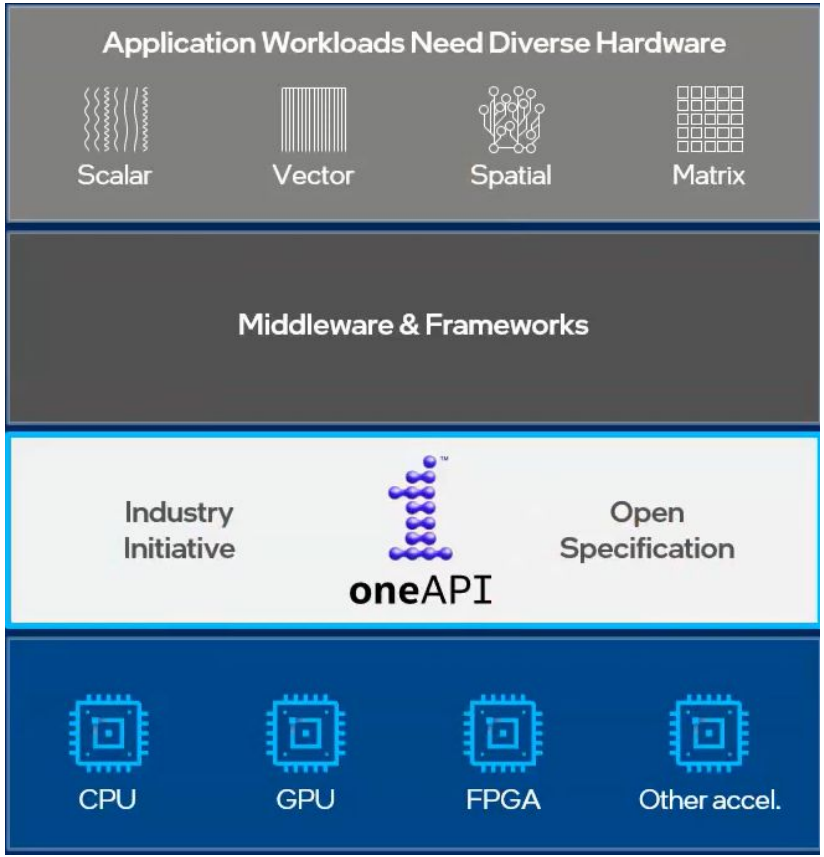
As the world's data-centric workloads become more specialized,
so do the architectures that best process that data.

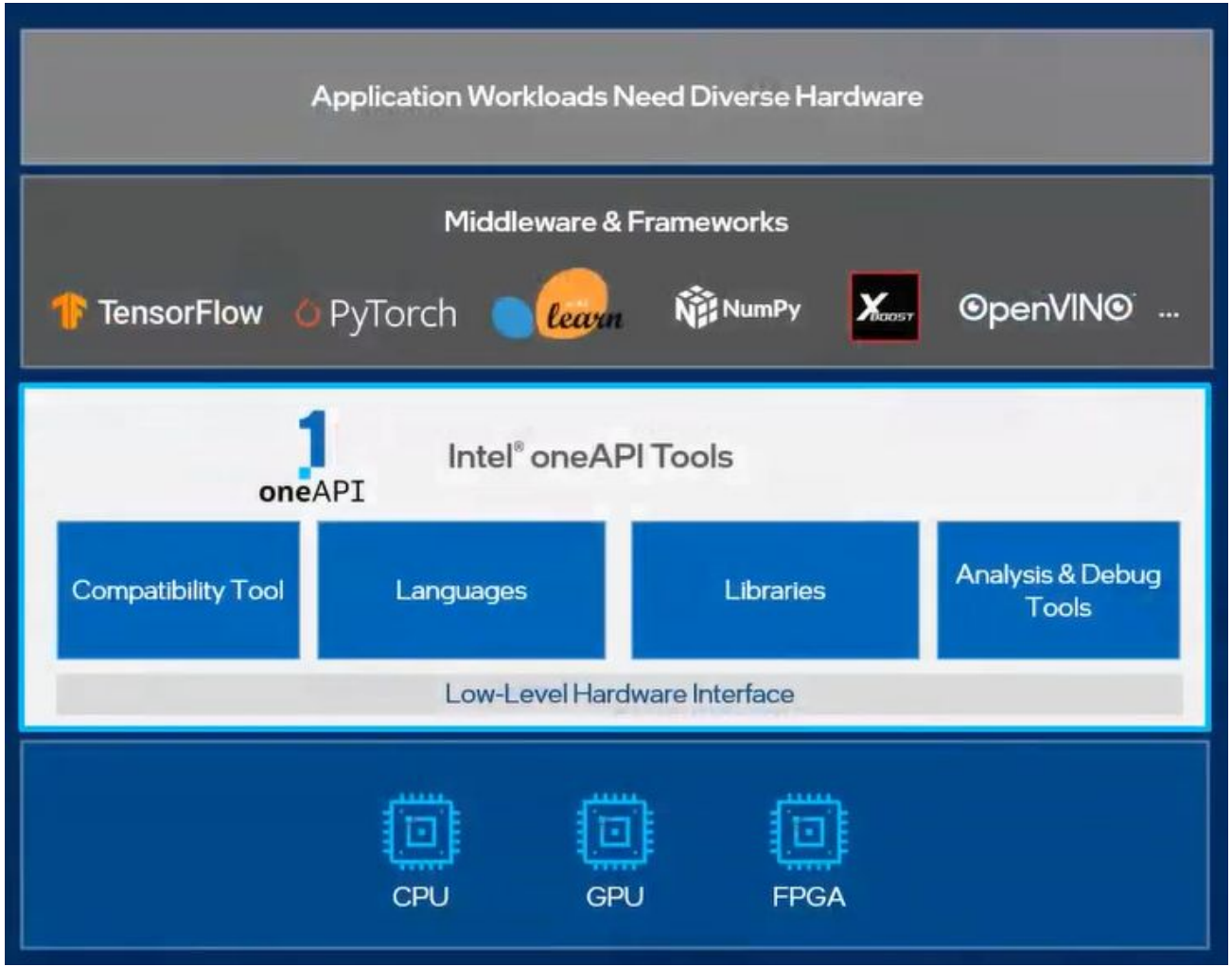
What are oneAPI goals?

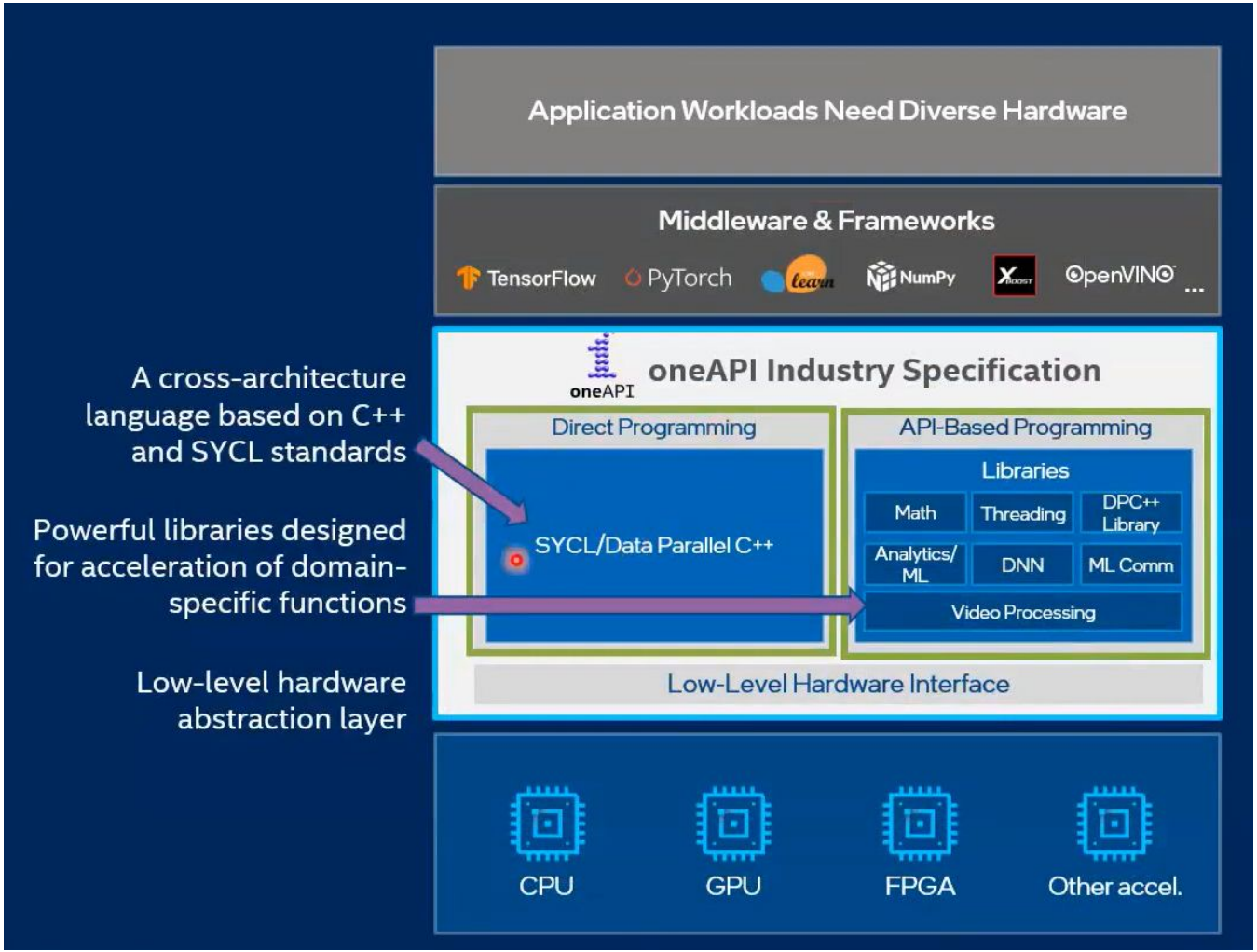


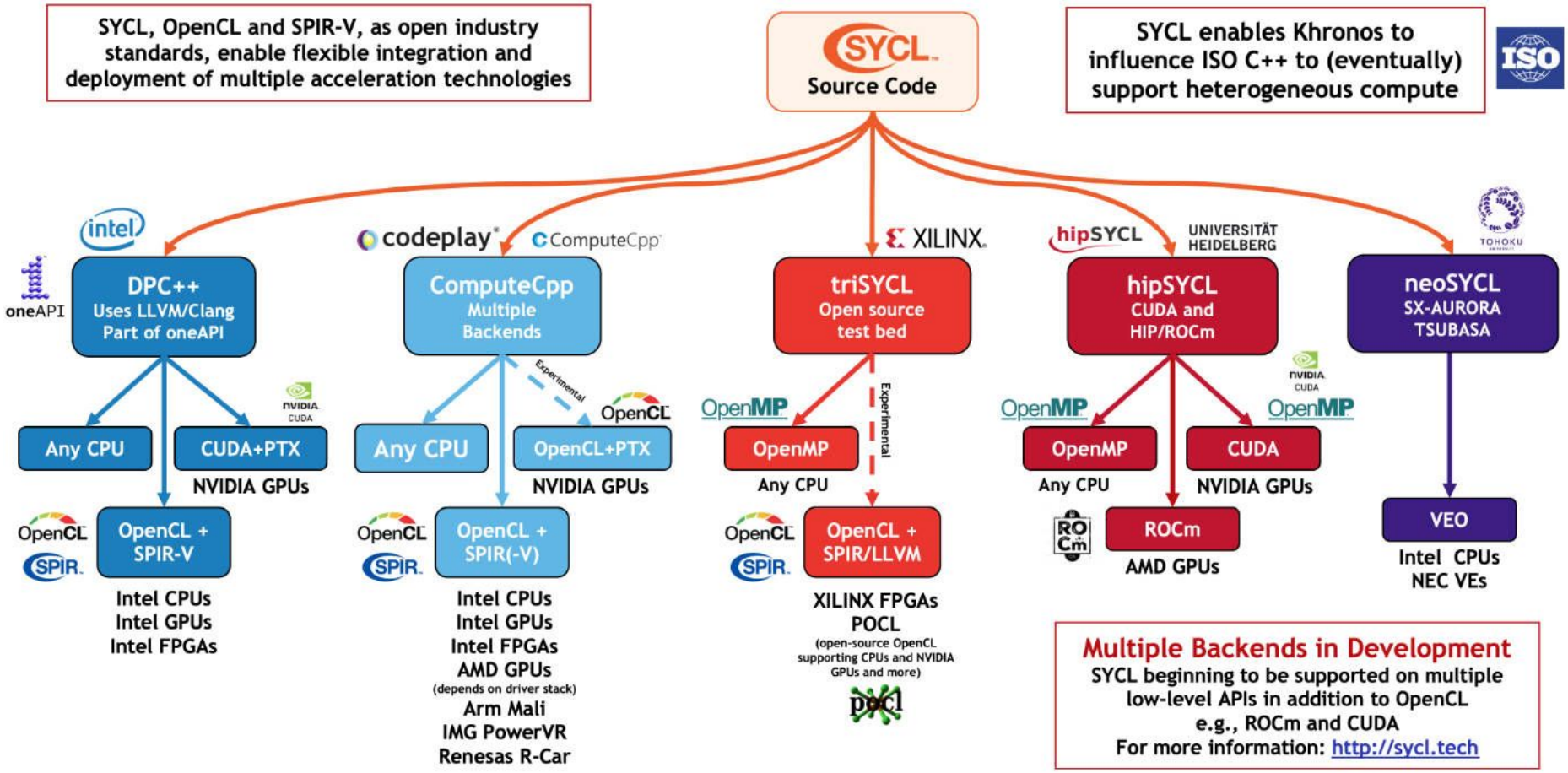
- The main objectives of oneAPI are to provide performance portability, meaning code runs efficiently across different architectures.
- It promotes open standards to avoid vendor lock-in.
- It also aims to unify development using a single toolchain for all hardware targets and to scale seamlessly across systems and accelerators.
- Be compatible with existing languages and programming models including C++, Python, SYCL, OpenMP, Fortran, and MPI


Application Workloads Need Diverse Hardware
Scalar
Vector
Spatial
Matrix
Middleware & Frameworks
CPU programming model
GPU programming model
FPGA programming model
Other accel. programming models
CPU
GPU
FPGA
Other accel.
XPUs


Application Workloads Need Diverse Hardware
Scalar
Vector
Spatial
Matrix
Middleware & Frameworks
Industry Initiative
oneAPI
Open Specification
CPU
GPU
FPGA
Other accel.


Application Workloads Need Diverse Hardware
Middleware & Frameworks
TensorFlow
PyTorch
learn
NumPy
XGBOOST
OpenVINO
...
oneAPI
Intel® oneAPI Tools
Compatibility Tool
Languages
Libraries
Analysis & Debug Tools
Low-Level Hardware Interface
CPU
GPU
FPGA


Application Workloads Need Diverse Hardware
Middleware & Frameworks
TensorFlow
PyTorch
NumPy
OpenVINO
...
A cross-architecture language based on C++ and SYCL standards
Powerful libraries designed for acceleration of domain-specific functions
Low-level hardware abstraction layer
oneAPI
oneAPI Industry Specification
Direct Programming
SYCL/Data Parallel C++
API-Based Programming
Libraries
Math
Threading
DPC++ Library
Analytics/ ML
DNN
ML Comm
Video Processing
Low-Level Hardware Interface
CPU
GPU
FPGA
Other accel.


SYCL, OpenCL and SPIR-V, as open industry standards, enable flexible integration and deployment of multiple acceleration technologies
SYCL
Source Code
SYCL enables Khronos to influence ISO C++ to (eventually) support heterogeneous compute
ISO
intel
oneAPI
DPC++
Uses LLVM/Clang
Part of oneAPI
Any CPU
CUDA+PTX
NVIDIA GPUs
OpenCL
SPIR
OpenCL + SPIR-V
Intel CPUs
Intel GPUs
Intel FPGAs
codeplay
ComputeCpp
ComputeCpp
Multiple Backends
Experimental
Any CPU
OpenCL+PTX
NVIDIA GPUs
OpenCL + SPIR(-V)
Intel CPUs
Intel GPUs
Intel FPGAs
AMD GPUs
(depends on driver stack)
Arm Mali
IMG PowerVR
Renesas R-Car
XILINX
triSYCL
Open source test bed
OpenMP
OpenMP
Any CPU
Experimental
OpenCL + SPIR/LLVM
XILINX FPGAs
POCL
(open-source OpenCL supporting CPUs and NVIDIA GPUs and more)
pocl
hipSYCL
UNIVERSITÄT HEIDELBERG
hipSYCL
CUDA and HIP/ROCm
OpenMP
OpenMP
Any CPU
CUDA
NVIDIA GPUs
ROCm
AMD GPUs
TOHOKU
neoSYCL
SX-AURORA TSUBASA
VEO
Intel CPUs
NEC VEs
Multiple Backends in Development
SYCL beginning to be supported on multiple low-level APIs in addition to OpenCL
e.g., ROCm and CUDA
For more information: http://sycl.tech

SYCL Programming Model

- oneAPI is based on SYCL, a modern C++ abstraction for heterogeneous programming.
- In SYCL, the host code runs on the CPU and launches kernels to run on accelerators via a queue.
- Data can be managed using buffers and accessors, or through Unified Shared Memory.
- The **parallel_for** construct is used to run device kernels in parallel.
- You can find out more about SYCL here : https://www.khronos.org/sycl/







Memory Models in oneAPI

SYCL supports two memory models:



Buffer and Accessor model

Higher-level abstraction that automatically manages memory and synchronization. Good for safety and portability



Unified Shared Memory (USM)

Gives developers direct pointer access to shared memory across host and device. Provides more control and resembles CUDA/OpenCL style.

Both models allow memory to be shared between host and device, with trade-offs between simplicity and flexibility.

```
std::vector<float> A(1024), B(1024), C(1024);
    {
      buffer bufA {A}, bufB {B}, bufC {C};
      queue q;
      q.submit([&](handler &h) {
          auto A = bufA.get_access(h, read_only);
          auto B = bufB.get_access(h, read_only);
          auto C = bufC.get_access(h, write_only);
          h.parallel_for(1024, [=](auto i){
              C[i] = A[i] + B[i];
          });
      });
    }
 for (int i = 0; i < 1024; i++)

       std::cout << "C[" << i << "] = " << C[i] << std::endl;

 }
```

Buffers creation via host vectors/pointers

Buffers encapsulate data in a SYCL application

- Across both devices and host!

```
std::vector<float> A(1024), B(1024), C(1024);
    {
      buffer bufA {A}, bufB {B}, bufC {C};
      queue q;
      q.submit([&](handler &h) {
          auto A = bufA.get_access(h, read_only);
          auto B = bufB.get_access(h, read_only);
          auto C = bufC.get_access(h, write_only);
          h.parallel_for(1024, [=](auto i){
              C[i] = A[i] + B[i];
          });
      });
    }
 for (int i = 0; i < 1024; i++)

      std::cout << "C[" << i << "] = " << C[i] << std::endl;

 }
```

- A queue submits command groups to be executed by the SYCL runtime
- Queue is a mechanism where work is submitted to a device.

queue q

Get a device (any device):	`queue q(); // default_selector{}`

Create queue targeting a pre-configured classes of devices:	`queue q(cpu_selector{});`
	`queue q(gpu_selector{});`
	`queue q(intel::fpga_selector{});`
	`queue q(accelerator_selector{});`
	`queue q(host_selector{});` SYCL 1.2.1

Create queue targeting specific device (custom criteria):	`class custom_selector : public device_selector {`
	`  int operator()(…… // Any logic you want!`
	
	`…`
	`queue q(custom_selector{});`

default_selector

- DPC++ runtime scores all devices and picks one with highest compute power
- Environment variable

export SYCL_DEVICE_TYPE=GPU | CPU | HOST

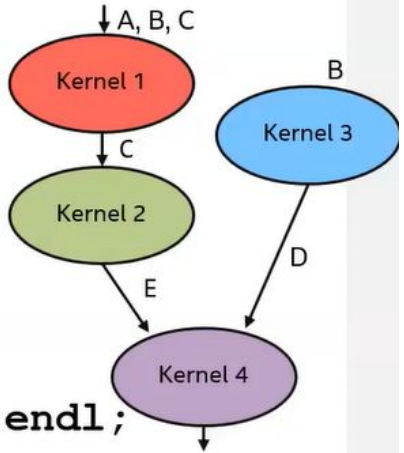
export SYCL_DEVICE_FILTER={backend:device_type:device_num}

```
std::vector<float> A(1024), B(1024), C(1024);
    {
      buffer bufA {A}, bufB {B}, bufC {C};
      queue q;
      q.submit([&](handler &h) {
          auto A = bufA.get_access(h, read_only);
          auto B = bufB.get_access(h, read_only);
          auto C = bufC.get_access(h, write_only);
          h.parallel_for(1024, [=](auto i){
              C[i] = A[i] + B[i];
          });
      });
    }
 for (int i = 0; i < 1024; i++)

       std::cout << "C[" << i << "] = " << C[i] << std::endl;

 }
```

- Mechanism to access buffer data
- Create data dependencies in the SYCL graph that order kernel executions



```
std::vector<float> A(1024), B(1024), C(1024);
    {
      buffer bufA {A}, bufB {B}, bufC {C};
      queue q;
      q.submit([&](handler &h) {
          auto A = bufA.get_access(h, read_only);
          auto B = bufB.get_access(h, read_only);
          auto C = bufC.get_access(h, write_only);
          h.parallel_for(1024, [=](auto i){
              C[i] = A[i] + B[i];
          });
      });
    }
 for (int i = 0; i < 1024; i++)

     std::cout << "C[" << i << "] = " << C[i] << std::endl;

 }
```

range<1>{1024} id<1>

- Vector addition kernel enqueues a parallel_for task.
- Pass a function object/lambda to be executed by each work-item

The functionality of basic parallel kernels is exposed via range, id and item classes:

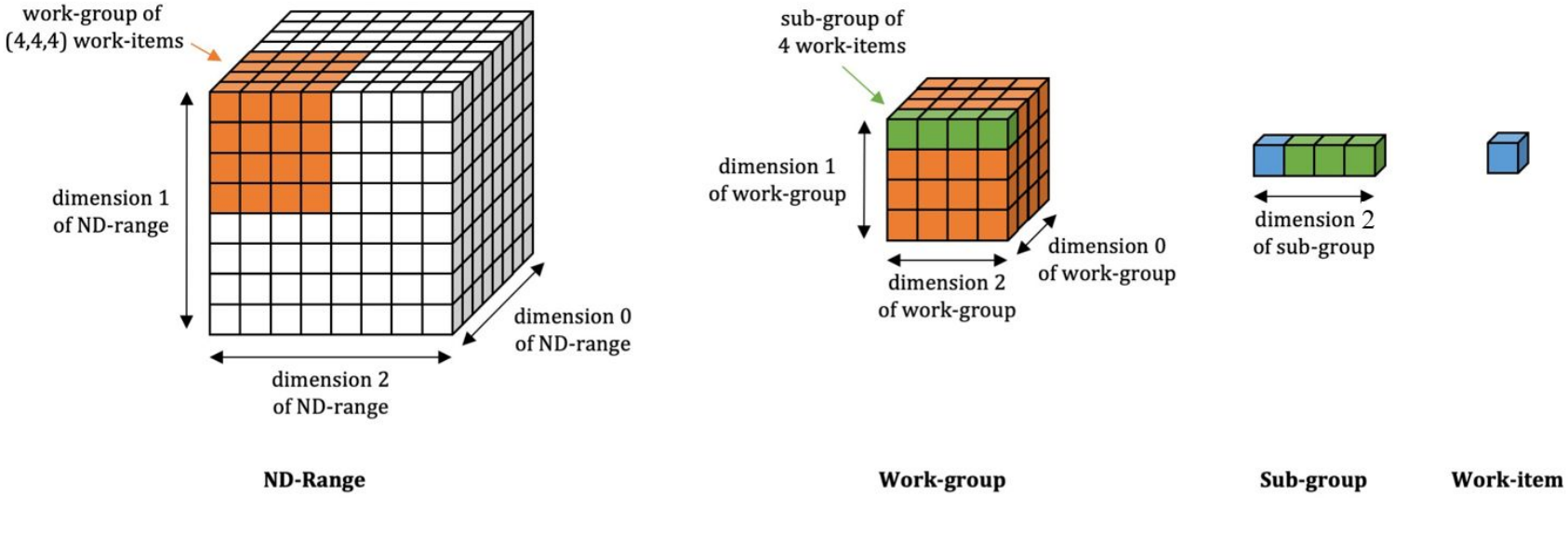
- range class is used to describe the iteration space of parallel execution
- id class is used to index an individual instance of a kernel in a parallel execution
- item class represents an individual instance of a kernel function, exposes additional functions to query properties of the execution range

```
h.parallel_for(range<1>(1024), [=](id<1> idx){
          // CODE THAT RUNS ON DEVICE
});
```

```
h.parallel_for(range<1>(1024), [=](item<1> item){
          auto idx = item.get_id();
          auto R = item.get_range();
          // CODE THAT RUNS ON DEVICE
});
```

Execution Hierarchy

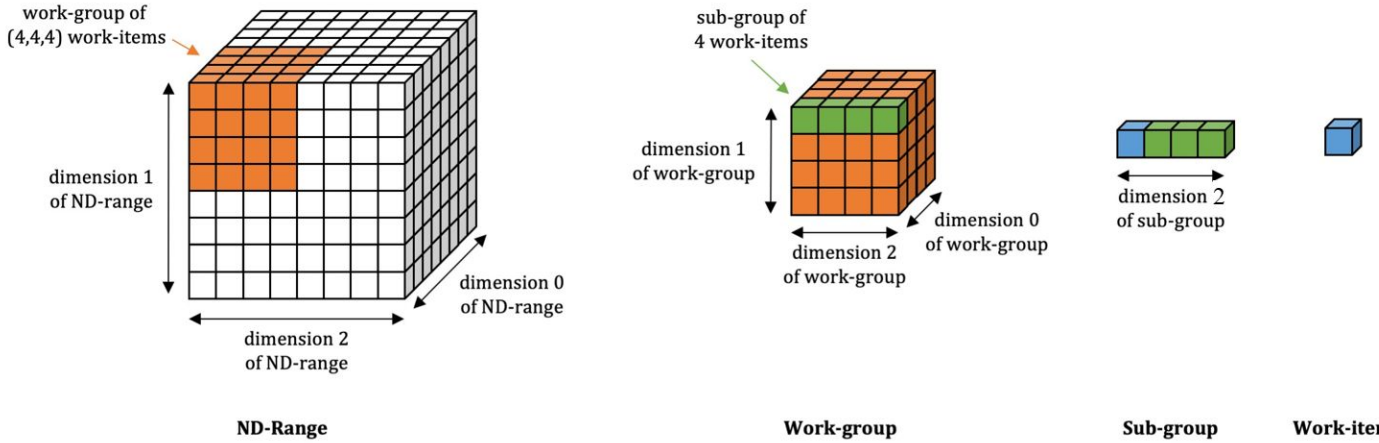
- SYCL defines a hierarchy for parallel execution: Work-items represent individual threads of computation.
- These are grouped into work-groups, and further into sub-groups for fine-grained control.
- This structure allows SYCL to express parallelism in a scalable and hardware-friendly way.



Basic Parallel Kernels are easy way to parallelize a for-loop but does not allow performance optimization at hardware level.

ND-range kernel is another way to express parallelism which enable low level performance tuning by providing access to local memory and mapping executions to compute units on hardware.

- The entire iteration space is divided into smaller groups called work-groups, work-items within a work-group are scheduled on a single compute unit on hardware.
- The grouping of kernel executions into work-groups will allow control of resource usage and load balance work distribution.



The functionality of nd_range kernels is exposed via nd_range and nd_item classes

- nd_range class represents a grouped execution range using global execution range and the local execution range of each work-group.
- nd_item class represents an individual instance of a kernel function and allows to query for work-group range and index.

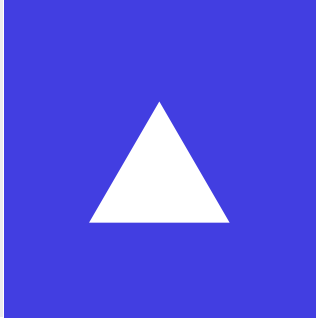
```
h.parallel_for(nd_range<1>(range<1>(1024),range<1>(64)), [=](nd_item<1> item){
    auto idx = item.get_global_id();
    auto local_id = item.get_local_id();
    // CODE THAT RUNS ON DEVICE
});
```

```
std::vector<float> A(1024), B(1024), C(1024);
    {
      buffer bufA {A}, bufB {B}, bufC {C};
      queue q;
      q.submit([&](handler &h) {
          auto A = bufA.get_access(h, read_only);
          auto B = bufB.get_access(h, read_only);
          auto C = bufC.get_access(h, write_only);
          h.parallel_for(1024, [=](auto i){
              C[i] = A[i] + B[i];
          });
      });
    }
 for (int i = 0; i < 1024; i++)

       std::cout << "C[" << i << "] = " << C[i] << std::endl;

 }
```

Synchronization



Synchronization within kernel function

- Barriers for synchronizing work items within a workgroup
- No synchronization primitives across workgroups

Synchronization between host and device

- Call to wait() member function of device queue
- Buffer destruction will synchronize the data with host memory
- Host accessor constructor is a blocked call and returns only after all enqueued kernels operating on this buffer finishes execution
- DAG construction from command group function objects enqueued into the device queue

Host Accessors

- Buffer takes ownership of the data stored in vector.
- Creating host accessor is a blocking call and will only return after all enqueued DPC++ kernels that modify the same buffer in any queue completes execution and the data is available to the host via this host accessor.

```
int main() {
  constexpr int N = 100;
  auto R = range<1>(N);
  std::vector<double> v(N, 10);
  queue q;

  buffer buf(v);
  q.submit([&](handler& h) {
  accessor a(buf, h)
  h.parallel_for(R, [=](auto i) {
    a[i] -= 2;
    });
  });

  host_accessor b(buf, read_only);
  for (int i = 0; i < N; i++)
    std::cout << b[i] << "\n";
  return 0;
}
```

```
#include <CL/sycl.hpp>
#include <iostream>
constexpr int num=16;
using namespace sycl;
int main() {

  auto r = range{num};
  buffer<int> a{r};

  queue{}.submit([&](handler& h) {
    accessor out{a, h};
    h.parallel_for(r, [=](item<1> idx) {
        out[idx] = idx;
      });
  });

  host_accessor result{a};
  for (int i=0; i<num; ++i)
    std::cout << result[i] << "\n";
}
```

Error Handling

Synchronous exceptions

- Detected immediately (Failure to construct an object, e.g. can't create buffer)
- => Use try...catch block

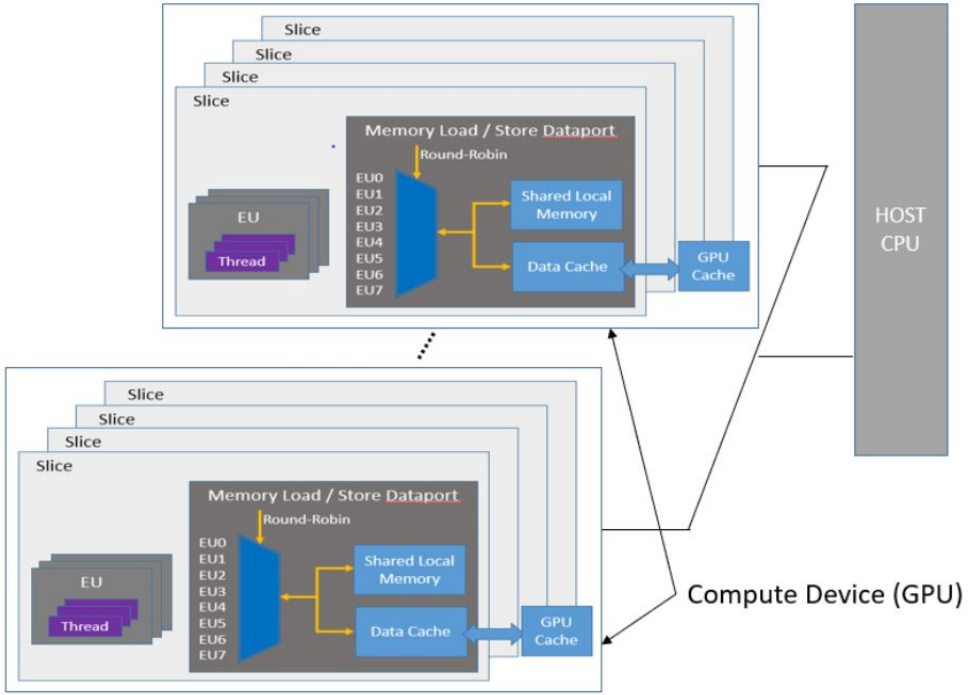
```
try {
  device_queue.reset(new queue(device_selector));
}
catch (exception const& e) {
std::cout << "Caught a synchronous SYCL exception:" << e.what();
return;
}
```

Asynchronous exceptions

- Caused by a future failure (E.g. error occurring during execution of a kernel on a device)
- Host program has already moved on to new things!
- => Programmer provides processing function, and says when to process
    - queue:wait _and_throw(),
    - queue:throw_asynchronous(),
    - event:wait_and_throw()

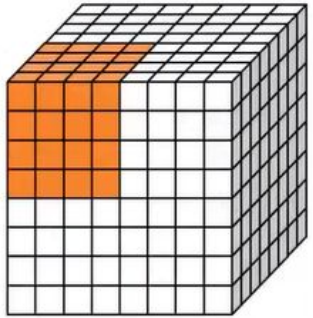
```
auto async_exception_handler = [](exception_list exceptions) {
  for (std::exception_ptr const& e : exceptions) {
    try {
      std::rethrow_exception(e);
    }
    catch (exception const& e) {
      std::cout << "Caught the Asynchronous SYCL exception"
                << e.what() << std::endl;
    }
  }
};
```

Memory Hierarchy

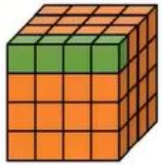


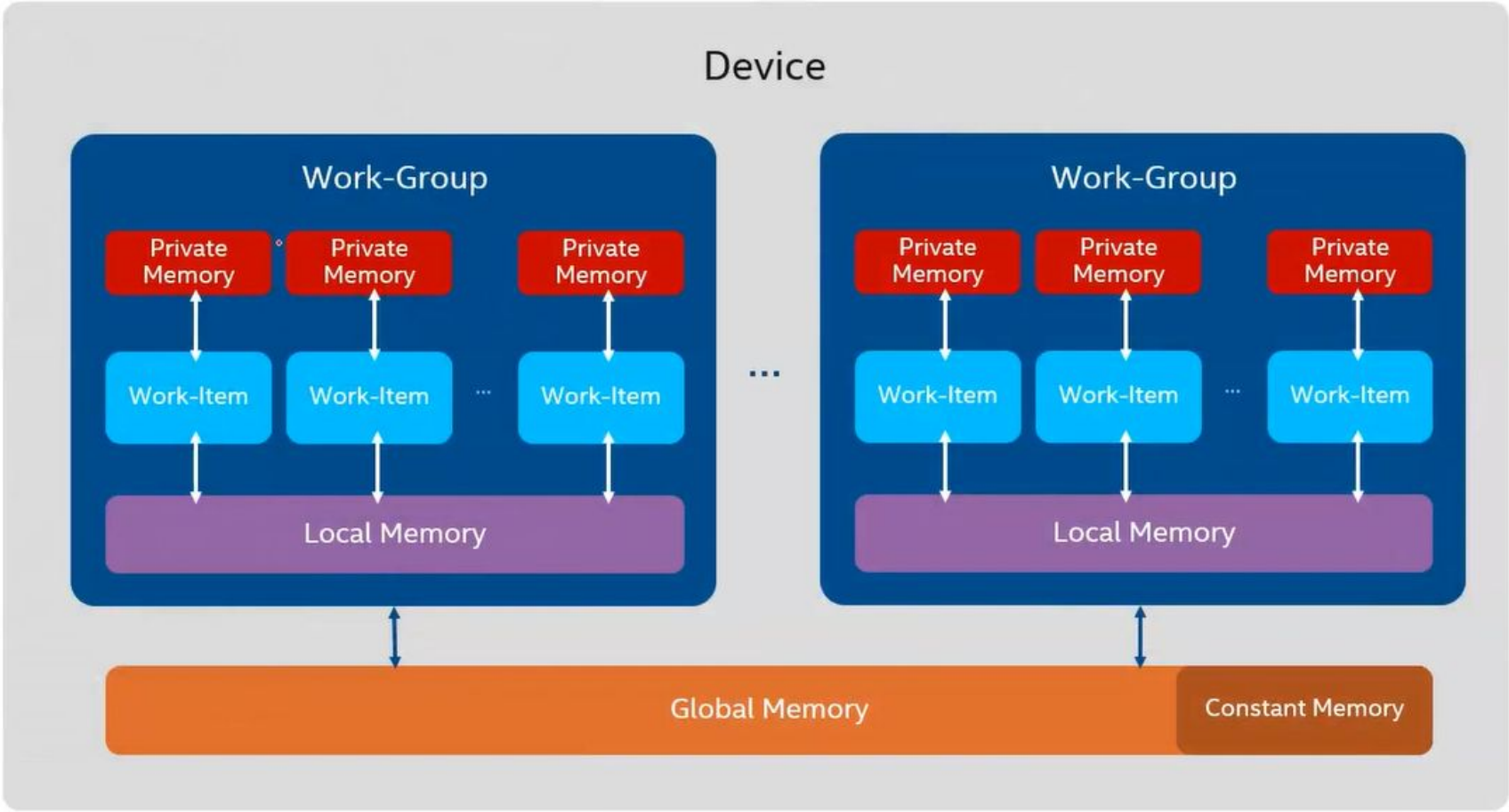


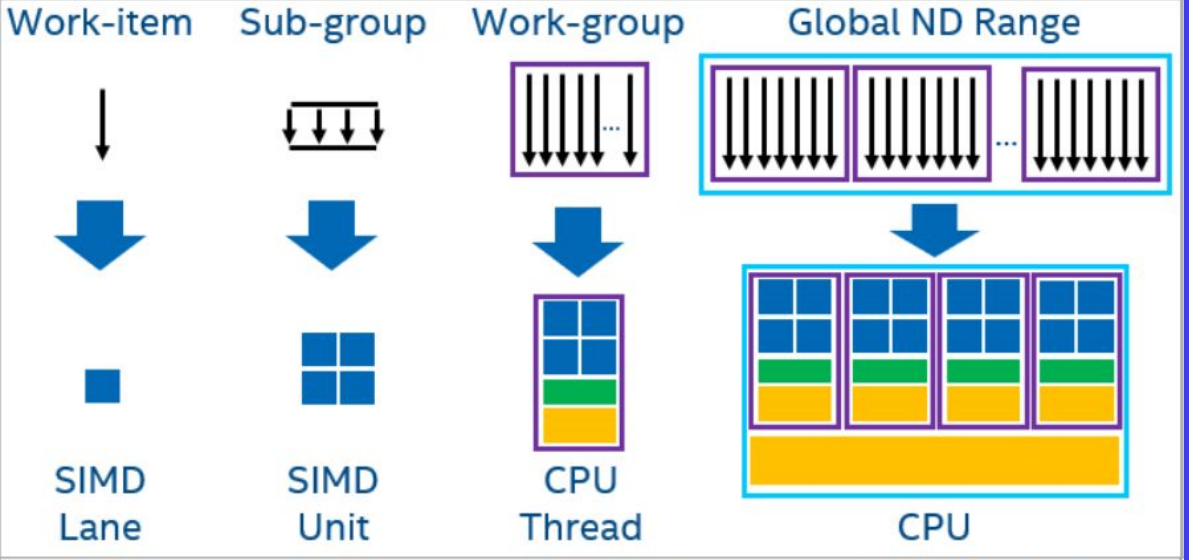
All work-items in a **work-group** are scheduled on one Compute Unit, which has its own local memory

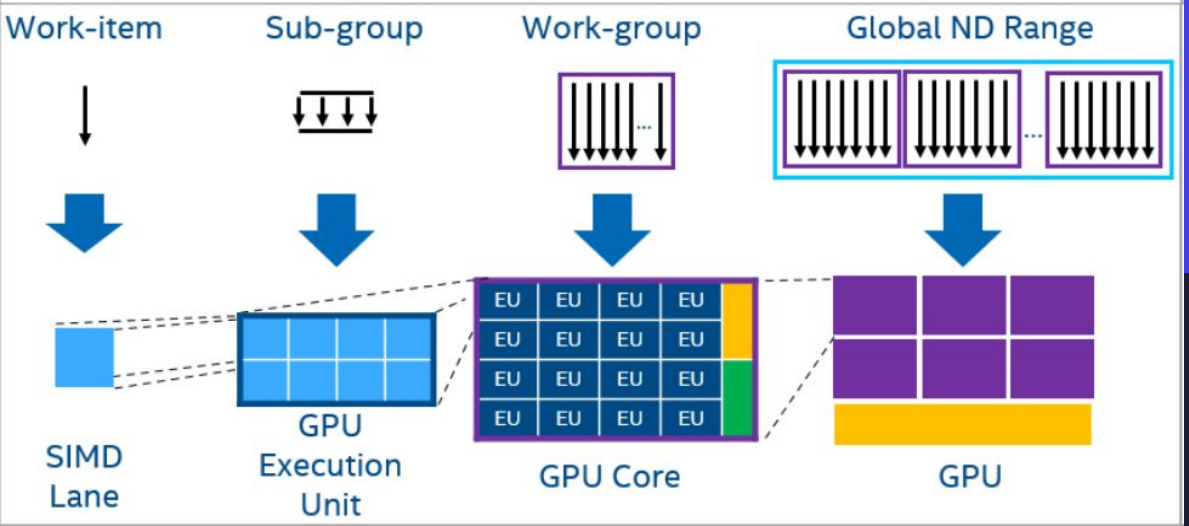


All work-items in a **sub-group** are mapped to vector hardware




Device
Work-Group
Private Memory
Private Memory
Private Memory
Work-Item
Work-Item
Work-Item
Local Memory
Work-Group
Private Memory
Private Memory
Private Memory
Work-Item
Work-Item
Work-Item
Local Memory
Global Memory
Constant Memory

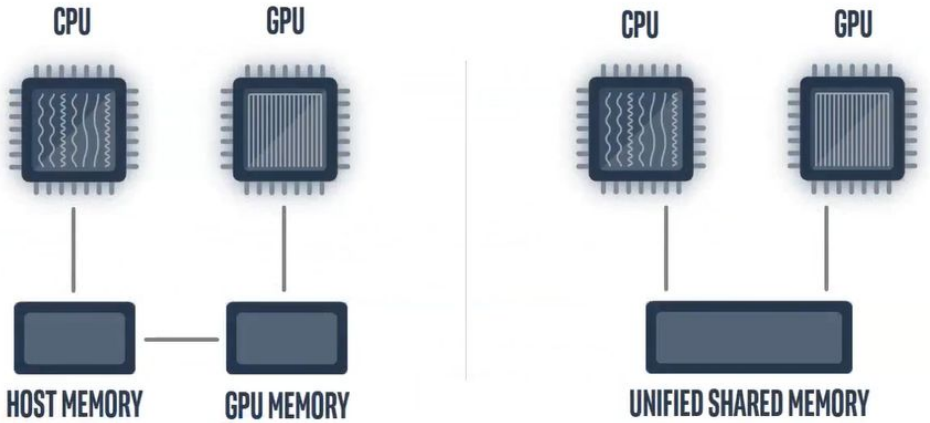




Offloading to CPU vs GPU

Unified Shared Memory

Developers can reference same memory object in host and device code with Unified Shared Memory



Unified Shared Memory

Unified Shared Memory provides both explicit and implicit models for managing memory.

Automatic data accessibility and explicit data movement supported

Allocation Type	Description	Accessible on HOST	Accessible on DEVICE
device	Allocations in device memory (**explicit**)	NO	YES
host	Allocations in host memory (**implicit**)	YES	YES
shared	Allocations can migrate between host and device memory (**implicit**)	YES	YES

Explicit Data Movement

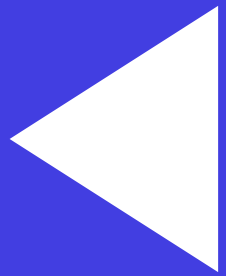
```
queue q;
int hostArray[42];
int *deviceArray = (int*) malloc_device(42 * sizeof(int), q);

for (int i = 0; i < 42; i++) hostArray[i] = 42;
// copy hostArray to deviceArray
q.memcpy(deviceArray, &hostArray[0], 42 * sizeof(int));
q.wait();
q.submit([&](handler& h){
  h.parallel_for(42, [=](auto ID) {
    deviceArray[ID]++;
  });
});
q.wait();
// copy deviceArray back to hostArray
q.memcpy(&hostArray[0], deviceArray, 42 * sizeof(int));
q.wait();
free(deviceArray, q);
```

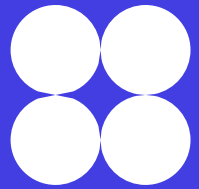
Implicit Data Movement

```
queue q;
int *hostArray = (int*) malloc_host(42 * sizeof(int), q);
int *sharedArray = (int*) malloc_shared(42 * sizeof(int), q);

for (int i = 0; i < 42; i++) hostArray[i] = 1234;
q.submit([&](handler& h){
  h.parallel_for(42, [=](auto ID) {
    // access sharedArray and hostArray on device
    sharedArray[ID] = hostArray[ID] + 1;
  });
});
q.wait();
for (int i = 0; i < 42; i++) hostArray[i] = sharedArray[i];
free(sharedArray, q);
free(hostArray, q);
```



Unified Shared Memory



No accessors in USM
Dependences must be specified explicitly using events

- queue.wait()
- wait on event objects
- use the depends_on method inside a command group

Explicit wait() used to ensure data dependency is maintained

wait() will block execution on host




```
queue q;
int* data = malloc_shared<int>(N, q);
for(int i=0;i<N;i++) data[i] = 10;
q.submit([&] (handler &h){
    h.parallel_for<class taskA>(range<1>(N), [=](id<1> i){
        data[i] += 2;
    });
}).wait();
q.submit([&] (handler &h){
    h.parallel_for<class taskB>(range<1>(N), [=](id<1> i){
        data[i] += 3;
    });
}).wait();
q.submit([&] (handler &h){
    h.parallel_for<class taskC>(range<1>(N), [=](id<1> i){
        data[i] += 5;
    });
}).wait();
for(int i=0;i<N;i++) std::cout << data[i] << " ";
free(data, q);
```

Use depends_on() method to let command group handler know that specified event should be complete before specified task can execute.

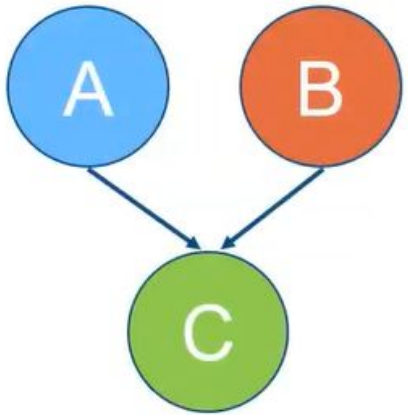



```
queue q;
int* data =  malloc_shared<int>(N, q);
for(int i=0;i<N;i++) data[i] = 10;
auto e1 = q.submit([&] (handler &h){
    h.parallel_for<class taskA>(range<1>(N), [=](id<1> i){
        data[i] += 2;
    });
});
auto e2 = q.submit([&] (handler &h){
    h.depends_on(e1);
    h.parallel_for<class taskB>(range<1>(N), [=](id<1> i){
        data[i] += 3;
    });
});

// non-blocking; execution of host code is possible

q.submit([&] (handler &h){
    h.depends_on(e2);
    h.parallel_for<class taskC> (range<1>(N), [=](id<1> i){
        data[i] += 5;
    });
}).wait();
for(int i=0;i<N;i++) std::cout << data[i] << " ";
free(data, q);
```

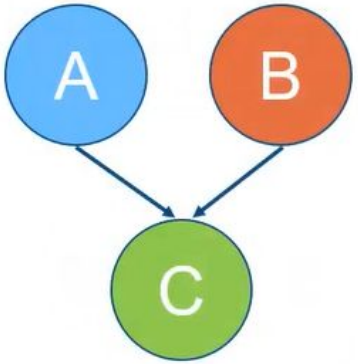
Use depends_on() method to let command group handler know that specified event should be complete before specified task can execute.




```
queue q;
int* data1 =  malloc_shared<int>(N, q);
int* data2 =  malloc_shared<int>(N, q);
for(int i=0;i<N;i++) {data1[i] = 10; data2[i] = 10;}
auto e1 = q.submit([&] (handler &h){
    h.parallel_for<class taskA>(range<1>(N), [=](id<1> i){
        data1[i] += 2;
    });
});
auto e2 = q.submit([&] (handler &h){
    h.parallel_for<class taskB>(range<1>(N), [=](id<1> i){
        data2[i] += 3;
    });
});
q.submit([&] (handler &h){
    h.depends_on({e1,e2});
    h.parallel_for<class taskC>(range<1>(N), [=](id<1> i){
        data1[i] += data2[i];
    });
}).wait();
for(int i=0;i<N;i++) std::cout << data[i] << " ";
free(data1, q); free(data2, q);
```

SYCL_PRINT_EXECUTION_GRAPH

A more simplified way of specifying dependency as parameter of parallel_for




```
queue q;
int* data1 =  malloc_shared<int>(N, q);
int* data2 =  malloc_shared<int>(N, q);
for(int i=0;i<N;i++) {data1[i] = 10; data2[i] = 10;}
auto e1 = q.parallel_for <class taskA>(range<1>(N), [=](id<1> i){
  data1[i] += 2;
});
auto e2 = q.parallel_for <class taskB>(range<1>(N), [=](id<1> i){
  data2[i] += 3;
});
q.parallel_for <class taskC>(range<1>(N), {e1, e2}, [=](id<1> i){
  data1[i] += data2[i];
}).wait();

for(int i=0;i<N;i++) std::cout << data[i] << " ";
free(data1, q); free(data2, q);
```

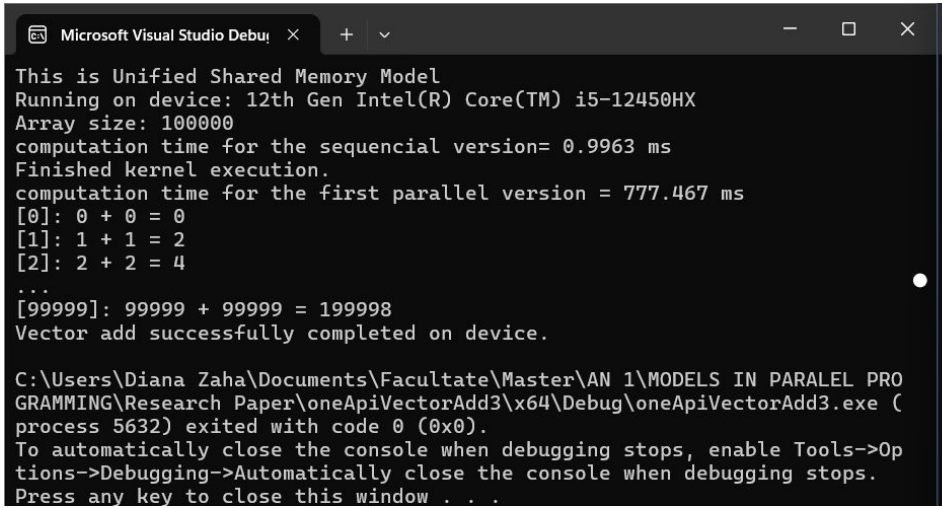
Demo Time

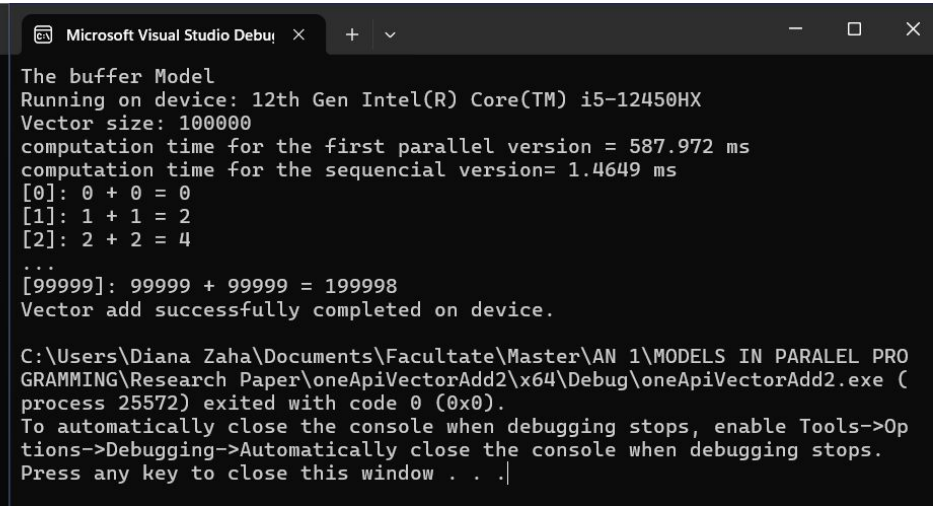
The “Hello world” of oneAPI

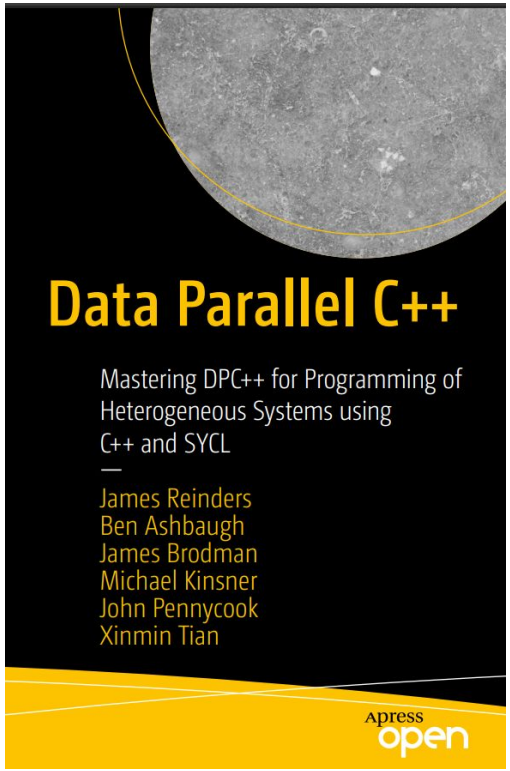
-> vector addition

with buffer model and USM




Microsoft Visual Studio Debu
This is Unified Shared Memory Model
Running on device: 12th Gen Intel(R) Core(TM) i5-12450HX
Array size: 100000
computation time for the sequencial version= 0.9963 ms
Finished kernel execution.
computation time for the first parallel version = 777.467 ms
[0]: 0 + 0 = 0
[1]: 1 + 1 = 2
[2]: 2 + 2 = 4
...
[99999]: 99999 + 99999 = 199998
Vector add successfully completed on device.
C:\Users\Diana Zaha\Documents\Facultate\Master\AN 1\MODELS IN PARALEL PROGRAMMING\Research Paper\oneApiVectorAdd3\x64\Debug\oneApiVectorAdd3.exe (process 5632) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .


Microsoft Visual Studio Debu
The buffer Model
Running on device: 12th Gen Intel(R) Core(TM) i5-12450HX
Vector size: 100000
computation time for the first parallel version = 587.972 ms
computation time for the sequencial version= 1.4649 ms
[0]: 0 + 0 = 0
[1]: 1 + 1 = 2
[2]: 2 + 2 = 4
...
[99999]: 99999 + 99999 = 199998
Vector add successfully completed on device.
C:\Users\Diana Zaha\Documents\Facultate\Master\AN 1\MODELS IN PARALEL PROGRAMMING\Research Paper\oneApiVectorAdd2\x64\Debug\oneApiVectorAdd2.exe (process 25572) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .



Copyright © 2021 by Intel Corporation

"SYCL 2020's primary goal is to achieve closer convergence with ISO C++, furthering our work to bring parallel heterogeneous programming to modern C++ thru open standards."
- Michael Wong

2019 - Intel announced the oneAPI initiative to move beyond proprietary programming models (like CUDA) and support an "xPU" (CPU, GPU, FPGA) strategy.

2020 - The first full versions of Intel® oneAPI Toolkits were released, succeeding Intel Parallel Studio XE.

2021-2025 - Regular updates added performance, broadened support for accelerators, and improved the DPC++ compiler (based on LLVM).

2026 - The 2026.0 release merged the Base and HPC toolkits into a single offering, enhancing support for new processors and GPUs.

oneAPI Libraries



oneDPL
Data
Parallel C++ Library



oneDNN
Deep Neural
Network Library



oneCCL
Collective
Communications Library



oneDAL
Data
Analytics Library



oneTBB
Threading
Building Blocks



oneMKL
Math
Kernel Library







Data Analytics at Scale:	MODIN	pandas	NumPy	SciPy		
DL Inference and Training:	TensorFlow	PyTorch	OpenVINO	Intel® Neural Compressor		
Classical ML:	scikit-learn	dmlc XGBoost	python			
	Intel® Fortran Compiler	Intel® MPI Library	Intel® SHMEM Library			
Tools	Intel® DPC++ Compatibility Tool	Intel® VTune™ Profiler	Intel® Advisor	Intel® Distribution for GDB	Intel® Distribution for Python	
Performance Libraries:	oneMKL	oneDNN	oneDAL	oneCCL	oneTBB	oneDPL	Intel® Cryptography Primitives Library
Direct Programming:	C++ with SYCL	C++	Python	OpenMP		
Compilers:	Intel® oneAPI DPC++/C++ Compiler					
	Hardware Interface – oneAPI Level Zero & OpenCL™ API					

Comparing oneAPI with Other Models

- Compared to CUDA, OpenMP, and OpenCL, oneAPI offers greater portability and developer productivity.
- It uses open standards, supports multiple vendors, and avoids vendor lock-in.
- Its syntax is modern and familiar to C++ developers, while maintaining performance and control.



Pros vs Cons

- oneAPI enables developers to write high-performance code that runs across CPUs, GPUs, and FPGAs.
- It simplifies development with a single-source model and unified memory.
- Its ecosystem includes tools, libraries, and a modern compiler, making it a powerful environment for parallel computing.

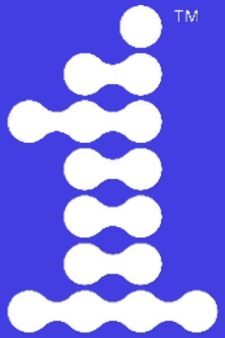
- While promising, oneAPI is still maturing.
- Support for non-Intel hardware is evolving, and SYCL itself has a learning curve.
- Some tooling and debugging features may not yet be as robust as those for CUDA or OpenMP. However, its adoption and support are growing steadily.

oneAPI Ecosystem Support



These organizations support the oneAPI initiative 'concept' for a single, unified programming model for cross-architecture development.
It does not indicate any agreement to purchase or use of Intel's products. *Other names and brands may be claimed as the property of others.

Conclusion



oneAPI

- Intel oneAPI represents a major step forward in simplifying heterogeneous programming.
- By combining performance portability, open standards, and modern C++ development, it addresses key challenges in cross-architecture computing.
- As workloads in AI and HPC grow more complex, tools like oneAPI will become increasingly essential.

Thank you!!!



Lecture 11

Coarse grain parallel computation models

BSP Model of Parallel Computation
LogP Model of Parallel Computation

Models of PP

- Main goal of parallel computation models is to provide a realistic representation of the costs of programming.
- Model provides algorithm designers and programmers a measure of algorithm complexity which helps them decide what is “good” (i.e. performance-efficient)

- We need a model that takes into account
  - Latency
  - Bandwidth
  - Communication Delay
  - Efficiency from coupling communication with computation

Fine versus coarse granularity

- PRAM- The most widely used parallel model
  - Overly simplistic in that it assumes:
    - Processors work synchronously
    - Inter-processor communication is not considered (analyzed)

  (considering classical variant)

- This leads to the creation of overly **fine grained** algorithms that in practice perform poorly since they spend so much time communicating/synchronizing

- Coarse grain models should be analyzed also…

Bulk Synchronous Parallelism

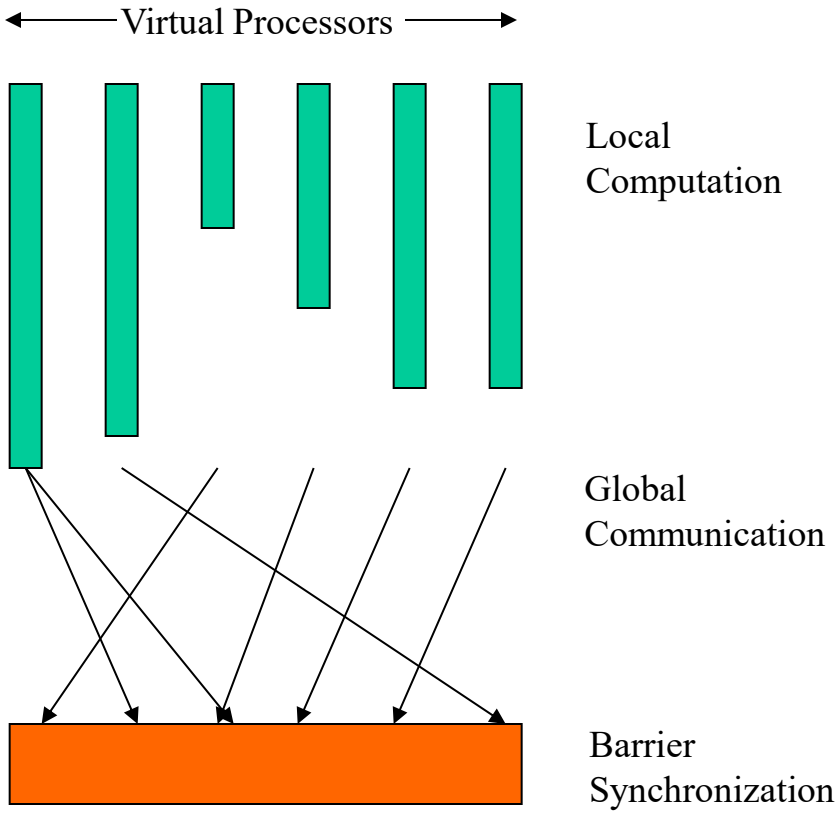
- BSP was proposed by L. Valiant as a ``bridging model'' in 1990 that provides a standard interface between parallel architectures and algorithms.

- BSP is a model of parallel computation
  - BSP computer
- BSP is a parallel programming model with coarse granularity

- BSP computer contains:
  - A set of processor-memory pairs.
  - A communications network that delivers messages in a point-to-point manner.
  - A mechanism for the efficient barrier synchronization for all or a subset of the processes.
  - There are no special combining, replicating, or broadcasting facilities.

BSP Programming Style

- Vertical Structure
  - Sequential composition of "supersteps".
    - Local computation
    - Process Communication
    - Barrier Synchronization
- Horizontal Structure
  - Concurrency among a fixed number of virtual processors.
  - Processes do not have a particular order.
  - Locality plays no role in the placement of processes on processors.
  - p = number of processors.



Locality – not very important

- the model only considers two levels of locality,
  - local (inside the processor) and
  - remote (outside a processor),

with remote access usually being more expensive than local ones.

when interconnection networks is considered into a model
the locality of a node is given by the set of its neighbors
- neighbors = nodes connected with maximum k edges (e.g. k=1)

efficient mappings should take into account locality

BSP Programming Style

- Properties:
  - Simple to write programs.
  - Independent of target architecture.
  - Performance of the model is predictable.

- Considers computation and communication at the level of the entire program instead of considering individual processes and individual communications.

- Renounces locality as a performance optimization.
  - Good and bad
  - BSP may not be the best choice for the programs/computation where the locality is critical – e,g. low-level image processing.

How Does Communication Work?

- BSP considers communication *en masse*.
  - Makes it possible to bound the time to deliver a whole set of data by considering all the communication actions of a superstep as a unit.

- If the maximum number of incoming or outgoing messages per processor is *h*, then such a communication pattern is called an *h-relation*.

- Parameter *g* measures the permeability of the network to continuous traffic addressed to uniformly random destinations.
  - Defined such that it takes time *hg* to deliver an *h-relation*.

- BSP does not distinguish between sending 1 message of length *m*, or *m* messages of length 1.
  - Cost is *mgh*

Cost in the BSP Model

- Characteristics:
  - p = number of processors
  - s = processor computation speed (flops/s) … used to calibrate g & l

  (could be ignored by expressing the remaining parameters in its units)
  - l = synchronization periodicity; the time required to synchronize all processors
  - g = total number of words delivered by the communications network in one unity of time
  - w_i = work executed by the processor i
  - h_i = number of incoming or outgoing messages of processor i

- Cost of a superstep (standard cost model):
  - $MAX(w_i) + MAX(h_i g) + l$ (or just $w + hg + l$)

- Cost of a superstep (overlapping cost model):
  - $MAX(w, hg) + l$

parameter g

- The parameter g is related to the *bisection bandwidth* (=bandwidth available between two partitions of the network topology) of the communication network but it is not equivalent.
- It also depends on other factors such as
  - the protocols used to interface with and within the communication network
  - the buffer management by both the processors and the communication network
  - the routing strategy used in the communication network and
  - the BSP runtime system

Barrier Synchronization – parameter *l*

- It has been considered that it is:
  "*Often expensive and should be used as sparingly as possible.*"

- Developers of BSP claim that barriers are not as expensive as they are believed to be in high performance computing folklore.

- The cost of a barrier synchronization has two parts:
  - The cost caused by the variation in the completion time of the computation steps that participate (=> idle times).
  - The cost of reaching a globally-consistent state in all processors.

- Cost is captured by parameter ***l*** (*parallel slackness*).
  - lower bound on l is the diameter of the network.

Predictability of the BSP Model

- Strategies used in writing efficient BSP programs:
  - Balance the computation in each superstep between processes.
    - "***w***" is a maximum of all computation times and the barrier synchronization must wait for the slowest process.
  - Balance the communication between processes.
    - "***h***" is a maximum of the fan-in and/or fan-out of data.
  - Minimize the number of supersteps.
    - Determines the number of times the parallel slackness appears in the final cost.

Advantages

- Rather than considering individual processes and individual communication actions BSP consider computation and communication at the level of the entire program and executing computer

- Considering communication actions en masse both simplifies their treatment and makes it possible to bound the time it takes to deliver a whole set of data BSP does this by considering all of the communication actions of a superstep as a unit

BSP vs. PRAM

- BSP can be regarded as a generalization of the PRAM model.
- If the BSP architecture has a small value of g (g=1), then it can be regarded as PRAM.
  - Use hashing to automatically achieve efficient memory management.
- The value of **l** determines the degree of parallel slackness required to achieve optimal efficiency.
  - If l = g = 1 … corresponds to idealized PRAM where no slackness is required.

Conclusions

- ***It is simple to write BSP programs*** they look much the same as sequential programs. Only a bare minimum of extra information needs to be supplied to describe the use of parallelism.

- ***It is independent of target architectures:*** Unlike many parallel programming systems BSP is designed to be architecture independent, so that programs run unchanged when they are moved from one architecture to another. Thus, BSP programs are portable in a strong sense.

- ***The performance of a program on a given architecture is predictable.*** The execution time of a BSP program can be computed from the text of the program and a few simple parameters of the target architecture. This makes design possible since the effect of a decision on performance can be determined at the time it is made.

BSP implementations

- BSP Worldwide is an association
  - interested in the development of the Bulk Synchronous Parallel (BSP) computing model for parallel programming.
  - https://bsp-worldwide.science.uu.nl
- Oxford BSPlib toolkit – not sustained but used as a model
- Bulk - a modern interface for writing BSP programs in C++
  https://jwbuurlage.github.io/Bulk/
- BSP Pro: a Java-based BSP performance profiling system

Weiqun Zheng ; Sch. of Inf. Technol., Murdoch Univ., WA, Australia ; Shamim Khan ; Hong Xie

- JBSP: A BSP Programming Library In Java

Yan Gu, Bu-Sung Lee, Wentong Cai , Johns Hopkins University

https://www.sciencedirect.com/science/article/abs/pii/S0743731501917356

Oxford BSPlib

- Supports a SPMD style of programming.
- Library is available in C and FORTRAN.
- Implementations were available for:
  - Cray T3E
  - IBM SP2
  - SGI PowerChallenge
  - Convex Exemplar
  - Hitachi SR2001
  - Various Workstation Clusters
- Allows for ***direct remote memory access*** or ***message passing.***
- Includes support for unbuffered messages for high performance computing.

BSPlib

- Initialization Functions
  - `bsp_init()`
    - Simulate dynamic processes
  - `bsp_begin()`
    - Start of SPMD code
  - `bsp_end()`
    - End of SPMD code
- Enquiry Functions
  - `bsp_pid()`
    - find my process id
  - `bsp_nprocs()`
    - number of processes
  - `bsp_time()`
    - local time
- Synchronization Functions
  - `bsp_sync()`
    - barrier synchronization
- DRMA Functions
  - `bsp_pushregister()`
    - make region globally visible
  - `bsp_popregister()`
    - remove global visibility
  - `bsp_put()`
    - push to remote memory
  - `bsp_get()`
    - pull from remote memory

BSPlib

- BSMP Functions
  - `bsp_set_tag_size()`
    - choose tag size
  - `bsp_send()`
    - send to remote queue
  - `bsp_get_tag()`
    - match tag with message
  - `bsp_move()`
    - fetch from queue
- Halt Functions
  - `bsp_abort()`
    - one process halts all

- High Performance Functions
  - `bsp_hpput()`
  - `bsp_hpget()`
  - `bsp_hpmove()`
  - These are unbuffered versions of communication primitives

BSPlib Examples

- Static Hello World

```
void main( void )
{
  bsp_begin(bsp_nprocs());

  printf( "Hello BSP from %d of %d\n",
          bsp_pid(), bsp_nprocs());

  bsp_end();
}
```

- Dynamic Hello World

```
int nprocs;   /* global variable */
void spmd_part( void )
{
  bsp_begin( nprocs );
  printf( "Hello BSP from %d of %d\n",
          bsp_pid(), bsp_nprocs());
  bsp_end()
}
void main( void )
{
  bsp_init( spmd_part, argc, argv );
  nprocs = ReadInteger();
  spmd_part();
}
```

BSPlib Examples

- Serialize Printing of Hello World (shows synchronization)

```
void main( void )
{
  int ii;
  bsp_begin( bsp_nprocs());
  for( ii=0; ii<bsp_nprocs(); ii++ )
  {
    if( bsp_pid() == ii )
      printf( “Hello BSP from %d of %d\n”, bsp_pid(), bsp_nprocs());
    fflush( stdout );
    bsp_sync();
  }
  bsp_end();
}
```

AllSums: collective communication (i.e., all processes have to call the function), such that when process *i* calls the function with an array xs containing $nelem_i$ elements, then the result on *all* the processes will be the sum of all the arrays from all the processes.

```
int bsp_sum(int *xs, int nelem) {
  int *local_sums,i,j,result=0;
  for(j=0;j<nelem;j++) result += xs[j];  //local sum
  bsp_push_reg(&result, sizeof(int));
  bsp_sync();
  local_sums = calloc(bsp_nprocs(), sizeof(int));
  if (local_sums==NULL)
    bsp_abort("{bsp_sum} no memory for %d int",bsp_nprocs());
  for(i=0;i<bsp_nprocs();i++)
    bsp_hpget(i,&result,0,&local_sums[i],sizeof(int));
//the result is taken from each proc
  bsp_sync();
  result=0;
  for(i=0;i<bsp_nprocs();i++) result += local_sums[i];
  bsp_pop_reg(&result);
  free(local_sums);
  return result;
}
```

BSPlib vs. MPI

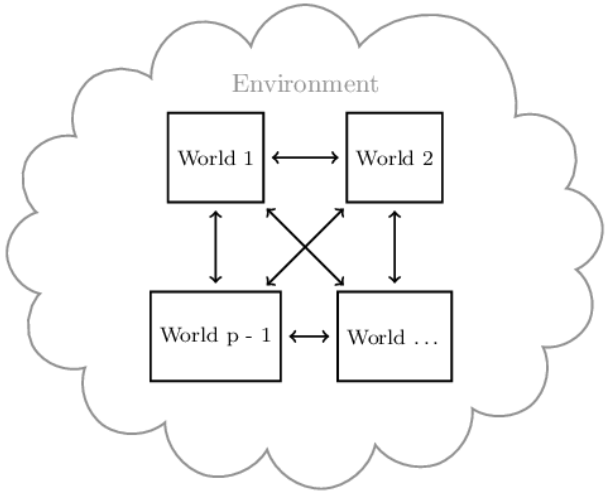
- MPI is widely implemented and widely used.
  - May be inefficient on (distributed-)shared memory systems. where the communication and synchronization are decoupled.
    - True for DSM machines with one sided communication.
  - Based on pairwise synchronization, rather than barrier synchronization.
    - No simple cost model for performance prediction.
    - No simple means of examining the global state.

- *BSP could be implemented using a small, carefully chosen subset of MPI subroutines.*

Bulk – C library

```
#include <bulk/bulk.hpp>
#include <bulk/backends/mpi/mpi.hpp>

int main() {
    bulk::mpi::environment env;
    env.spawn(env.available_processors(), [](auto& world) {
                auto s = world.rank();
                auto p = world.active_processors();
                world.log("Hello world from processor %d / %d!", s, p);
                }
   );
}
```



The environment could be:

- an MPI cluster,
- a many-core co-processor, or
- simply threads on a multi-core computer.

The **bulk::environment** is specialized for each *backend*, which is an implementation of the lower-level communication that reflects the actual environment.

,,

Bulk

- Bulk should work across all major Linux distributions.
- It requires an up-to-date compiler that supports C++20, e.g. GCC >= 11.0, or Clang >= 12.0.
- Backends
  - thread for multi-core systems using standard C++ <thread> threading support
  - mpi for distributed environments using MPI

BSP in distributed graph systems

- The reason BSP works well on graph algorithms.
  - graph algorithms naturally iterate,
  - deterministic synchronization simplifies fault tolerance,
  - message passing maps well to distributed clusters.
- Google's Pregel framework uses the BSP model to provide a structured, scalable way to process large-scale graphs.
- By adopting BSP, Pregel simplifies the complexity of distributed computing by replacing asynchronous, event-driven communication (which is prone to deadlocks and race conditions) with a highly organized, synchronous execution flow.

Hybrid BSP models

These preserves BSP semantics externally while relaxing barriers internally.

Partial-synchronous BSP pattern is now common in:

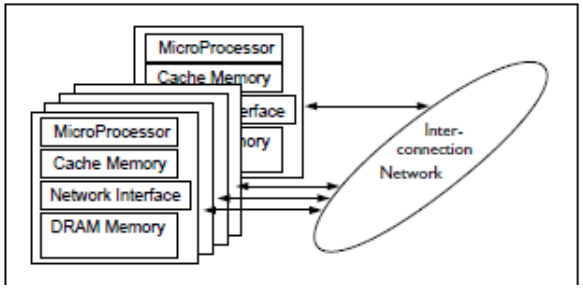
- graph analytics,
- distributed ML,
- sparse iterative solvers.

LogP – a similar model for parallel computation

chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.cs.umd.edu/class/fall2019/cmsc714/readings/Culler-LogP.pdf

LogP machine model

- Model of distributed memory multicomputer
- Developed by [Culler, Karp, Patterson, &all.]
- Authors tried to model prevailing parallel architectures.
- David Culler, Richard Karp, David Patterson, Abhijit Sahay, Klaus Erik Schauser, Eunice Santos, Ramesh Subramonian, and Thorsten von Eicken. 1993. LogP: towards a realistic model of parallel computation. In Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming (PPOPP '93). Association for Computing Machinery, New York, NY, USA, 1–12. https://doi.org/10.1145/155332.155333
- Machine model represents prevalent MPP(massive parallel programming) organization:
  - machine constructed with a few thousand nodes -
  - each node contains a powerful processor
  - each node contains substantial memory
  - interconnection structure has limited bandwidth
  - interconnection structure has significant latency



LogP – main parameters

- The main parameters of the model are:
  - L: Upper bound for the latency, or delay, incurred in communicating an empty message
  - o: Overhead, defined as the time the processor is engaged in sending or receiving a message, during which time it cannot do anything else
  - g: Gap, the minimum time interval between consecutive messages (so at most L/g messages can be in transit from any process or to any process at any time)
  - P: Number of processors

- The basic model assumes all messages are of small sizes.

Advantages of LogP

- LogP addresses Loop holes found in other models:
  - Interprocess communication: unlike PRAM, LogP does not hide the cost
  - Multithreading: LogP is able to express the upper bound of this technique (max L/g virtual processors)

- Encourages good practices like:
  - Coordinating work assignment (such that to reduce the communication bandwith)
  - Overlapping the computation and communication!!!

General analysis of Communication Closeup

Four main parts common to all communication:

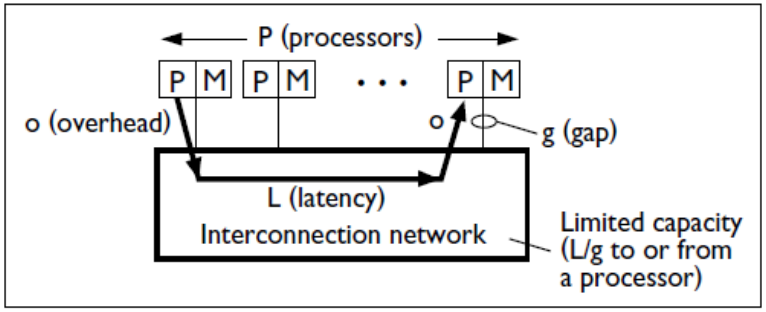
1. Tsnd = *Send Overhead*, time spent before the first bit can leave
2. Transition, time needed to get data onto the network
3. Transmission, time required for the bits to travel the network
4. Trcv=*Receive Overhead*, time after last bit arrives until the processor is free to work on something else

T(M, H) = Tsnd + M/w + Hr + Trcv

- M = size of message
- w = width of the network channel
- H = distance of the taken route
- r = delay through each node

Equation assumes lightly loaded networks

LogP – parameters



T(M, H) = Tsnd + M/w + Hr + Trcv

- M = size of message
- w = width of the network channel
- H = distance of the route taken
- r = delay through each node

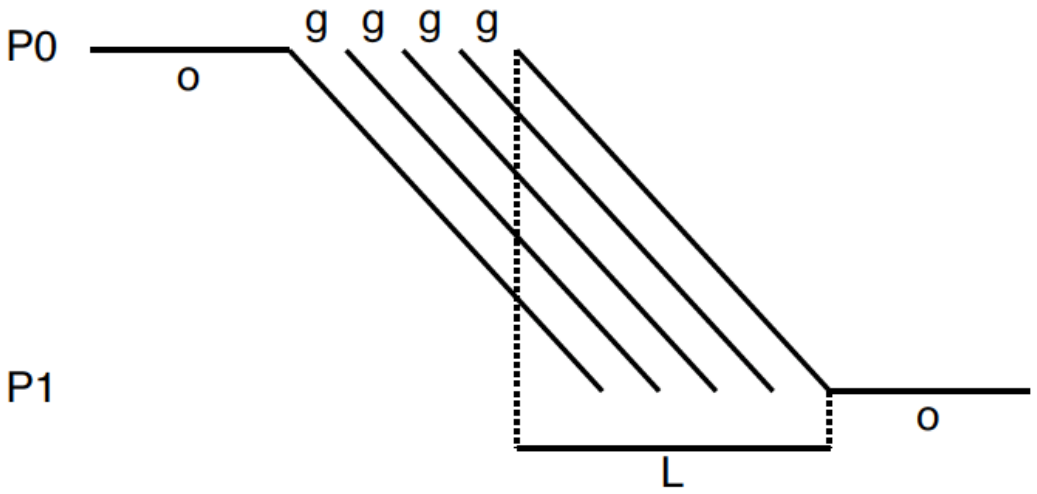
Equation assumes lightly loaded networks

Approx:

- o = *(Tsnd* + *Trcv*) / 2
- L = Hr + (M/w)
- g = M / (per processor bisection bandwith)
- P = Number of physical processors

Bisection width = minimum number of links cut to divide the network into two halves

Visualization



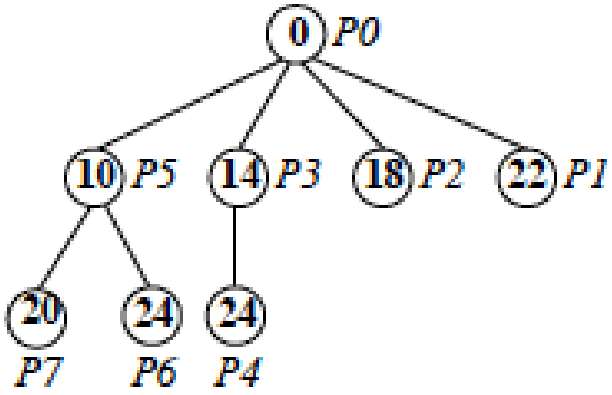
broadcasting a single datum from one processor to p-1 others

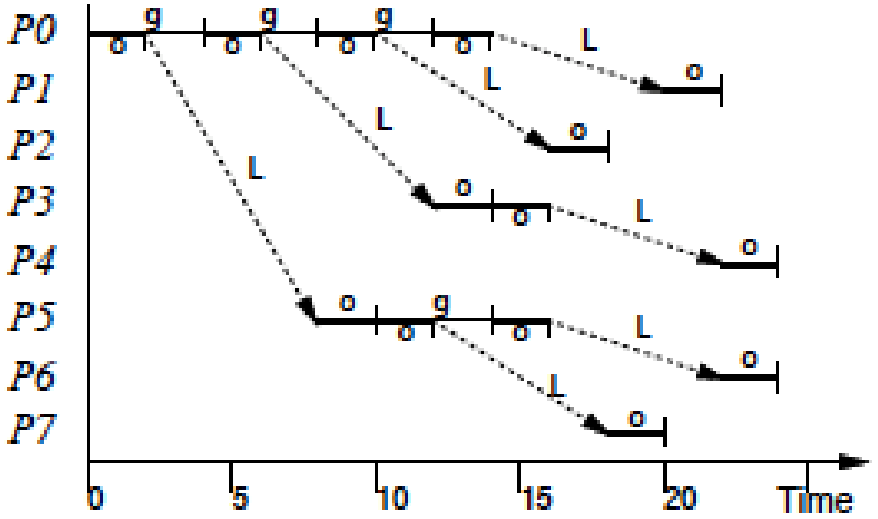
- Main idea:
  - all processors that have received the datum transmit it as quickly as possible, while ensuring that no processor receives more than one message.
- The source of the broadcast begins transmitting the datum at time 0.
- The first datum enters the network at time o , takes L cycles to arrive at the destination, and is received by the node at time L+ 2g
- Meanwhile, the source will have initiated transmission to other processors at time g, 2g, …

assuming g>=o, each of which acts as the root of a smaller broadcast tree.

Concrete example: which is the optimal broadcast tree for P=8 L= 6 g= 4 o= 2, and the activity of each processor over time

The number shown for each node is the time at which it has received the datum and can begin sending it on. The last value is received at time 24.





Notes:

- the optimal broadcast tree for processors is an unbalanced tree with the fan-out at each node determined by the relative values of L, o, g .
- The processor overhead of successive transmissions overlaps the delivery of previous messages.
- Nodes may experience idle cycles at the end of the algorithm while the last few messages are in transit.

Working with LogP

- **Short messages** (single message packet):

♦ 2o+L

- **Finite capacity of network**

♦ Ceil(L/g) messages in transit between any pair of nodes

- **Long messages**

♦ Pipeline of depth L with rate g and overhead o
(at each end)

• Depth L because it takes L units of time for message to travel through network and one message every g units of time. It is preferable that g = 1, but it might not.

Why Separate Latency and Overhead?

- Latency is Hardware – including time for data to traverse network

- Overhead is involvement of CPU
  - Significant difference between message passing (matching) and put/get (e.g., PGAS)
  - Message passing: receiver must find matching receive in a queue of posted but unmatched receives or save information on the message in a queue of unexpected messages
    - Overhead typically scales linearly with the number of messages in the queue

Linear algorithms fastest when queues nearly empty

What is the difference in distance (measured in clock cycles) between close and far nodes in large machines?

- **Nearby nodes** are less than 15cm apart
- for 2GHz clock, that is 1 clock cycle
- **Far away nodes** may be

  2*100*30cm = 6000cm
- 6000cm/15cm/clock = 400 clock cycles
- Only 0.2 usec

The LogGP model introduces an additional parameter G used for long messages

Remark: speed of signal in wire < speed of light;
distance is minimum possible rather than typical

BSP vs. LogP

- BSP differs from LogP in three ways:
  - LogP uses a form of message passing based on ***pairwise synchronization.***
  - LogP adds an extra parameter representing the overhead involved in sending a message. Applies to every communication!
  - LogP defines g in local terms. It regards the network as having a finite capacity and treats g as the minimal permissible *gap* between message sends from a single process.
  - The parameter g in both cases is the reciprocal of the available per-processor network bandwidth:
    - BSP takes a global view of g,
    - LogP takes a local view of g.

BSP vs. LogP

- When analyzing the performance of LogP model, it is often necessary (or convenient) to use barriers.
- Message overhead is present but decreasing…
  - Only overhead is from transferring the message from user space to a system buffer.
- LogP + barriers - overhead = BSP
- Both models can efficiently simulate the other.

Conclusion

- BSP is a computational model of parallel computing based on the concept of supersteps.
- BSP does not use locality of reference for the assignment of processes to processors.
- Predictability is defined in terms of three parameters.
- BSP is a generalization of PRAM.
- BSP = LogP + barriers - overhead
- BSPlib has a much smaller API as compared to MPI

References

- Valiant, Leslie G., "A Bridging Model for Parallel Computation", Communications of the ACM, Aug., 1990, Vol. 33, No. 8, pp. 103-111
- "BSP: A New Industry Standard for Scalable Parallel Computing", http://www.comlab.ox.ac.uk/oucl/users/bill.mccoll/oparl.html
- Hill, J. M. D., and W. F. McColl, "Questions and Answers About BSP", http://www.comlab.ox.ac.uk/oucl/users/bill.mccoll/oparl.html
- Hill, J. M. D., et. al, "BSPlib: The BSP Programming Library", http://www.comlab.ox.ac.uk/oucl/users/bill.mccoll/oparl.html
- McColl, W. F., "Bulk Synchronous Parallel Computing", Abstract Machine Models for Highly Parallel Computers, John R. Davy and Peter M. Dew eds., Oxford Science Publications, Oxford, Great Brittain, 1995, pp. 41-63.
- McColl, W. F., "Scalable Computing", http://www.comlab.ox.ac.uk/oucl/users/bill.mccoll/oparl.html
- The BSP Worldwide organization website is http://www.bsp-worldwide.org and an excellent Ohio Supercomputer Center tutorial is available at www.osc.org.
- "LogP: Towards a Realistic Model of Parallel Computation", PPOPP, May 1993

BSP vs LogP

- BSP emphasizes:
  - structured synchronization.
- LogP emphasizes:
  - realistic communication costs and asynchronous execution.
- Modern distributed systems increasingly resemble:
  - LogP internally (asynchronous execution engines)
  - BSP externally (periodic synchronization for consistency or optimization)

Presentation ref.

- Overview of BSP Model of Parallel Computation,

Michael C. Scherger,

  - Kent State University

- LogP: Towards a Realistic Model of Parallel Computation ☐

David Culler, Richard Karp, David Patterson,

Abhijit Sahay, Klaus Erik Schauser, Eunice Santos,

Ramesh Subramonian, and Thorsten von Eicken

  - Computer Science Division, University of California, Berkeley

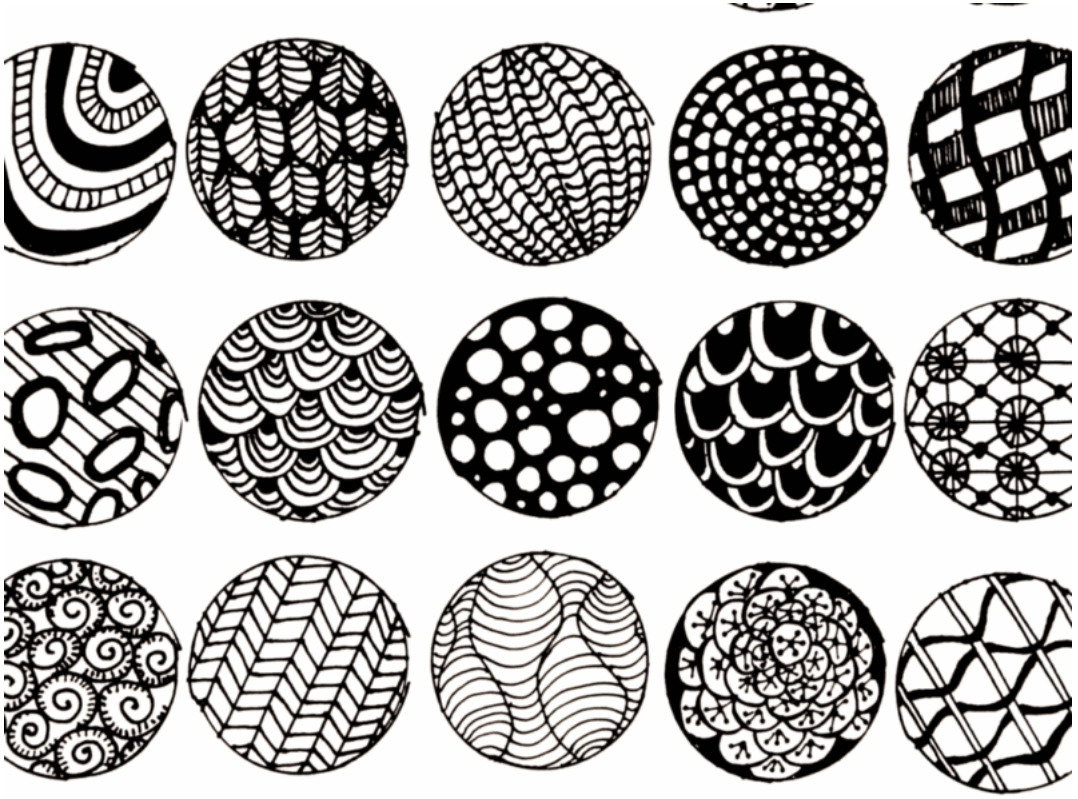
Lecture 12

Parallel Patterns

Pattern Languages

Skeletons

PATTERNS



Parallel Design Patterns

Software engineering setting:

- A **design pattern** is a general repeatable solution to a commonly occurring problem in software design.
  - A design pattern isn't considered a finished design that can be transformed directly into code.
  - It is a description or template for how to solve a problem that can be used in many different situations.

Parallel setting:

Generic patterns of computation and interaction.

- different classifications
- different names

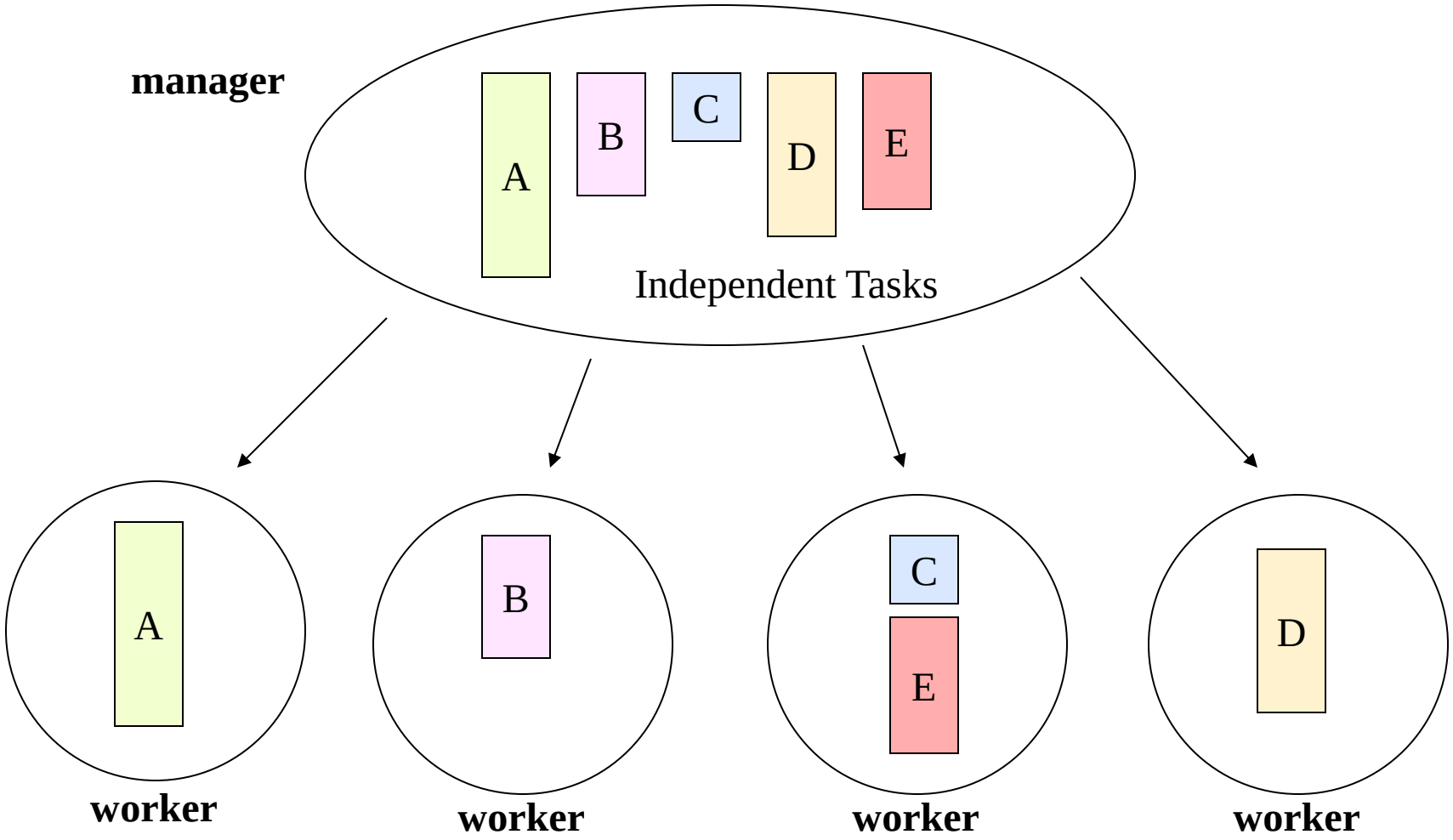
Common Parallel Patterns

- Embarassingly Parallel
- Replicable
- Repository
- Divide&Conquer
- Pipeline
- Recursive Data
- Geometric
- IrregularMesh

Parallel Programming Patterns
Eun-Gyu Kim
2004

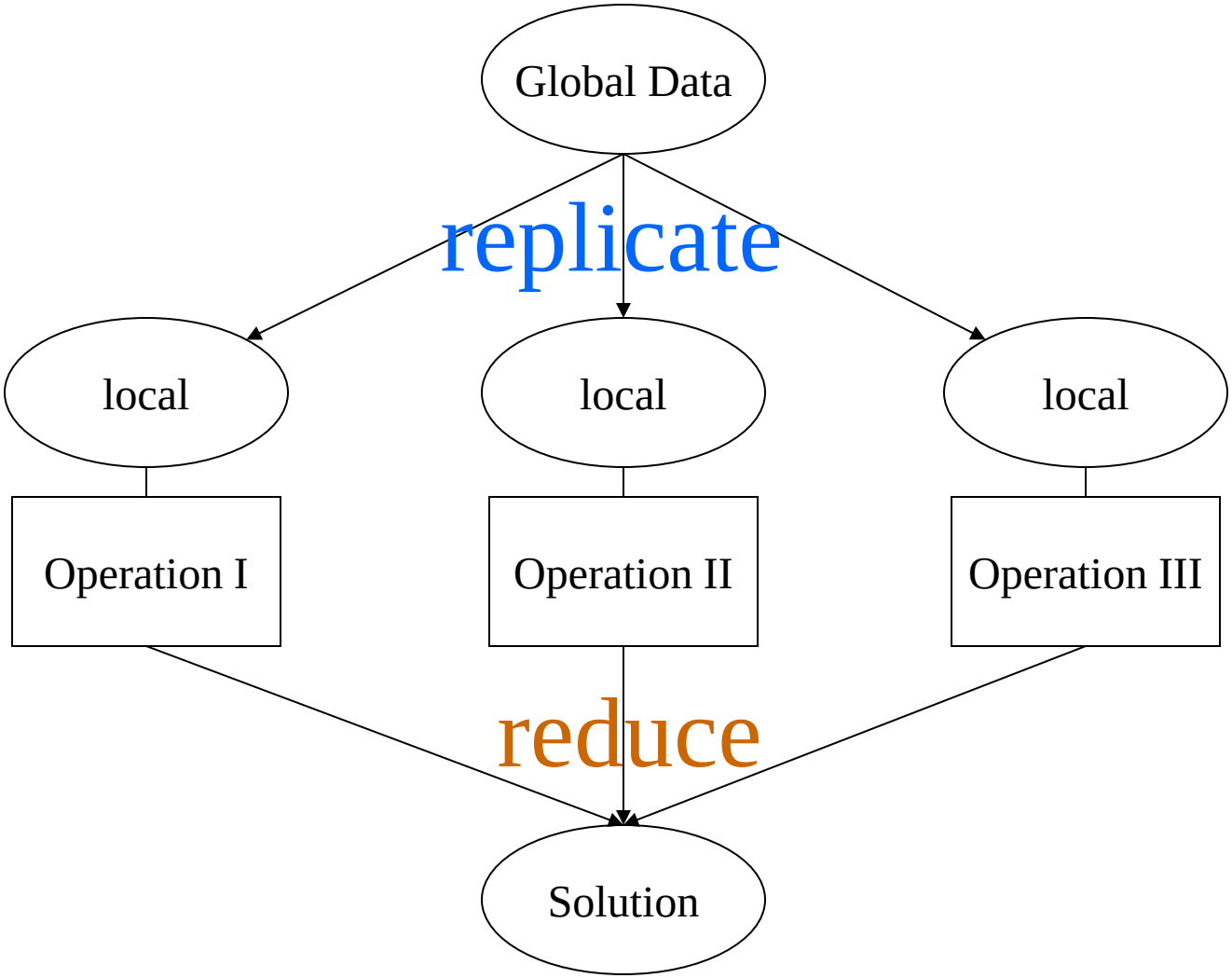
Embarassingly Parallel

Problem: Need to perform same operations to tasks that are independent



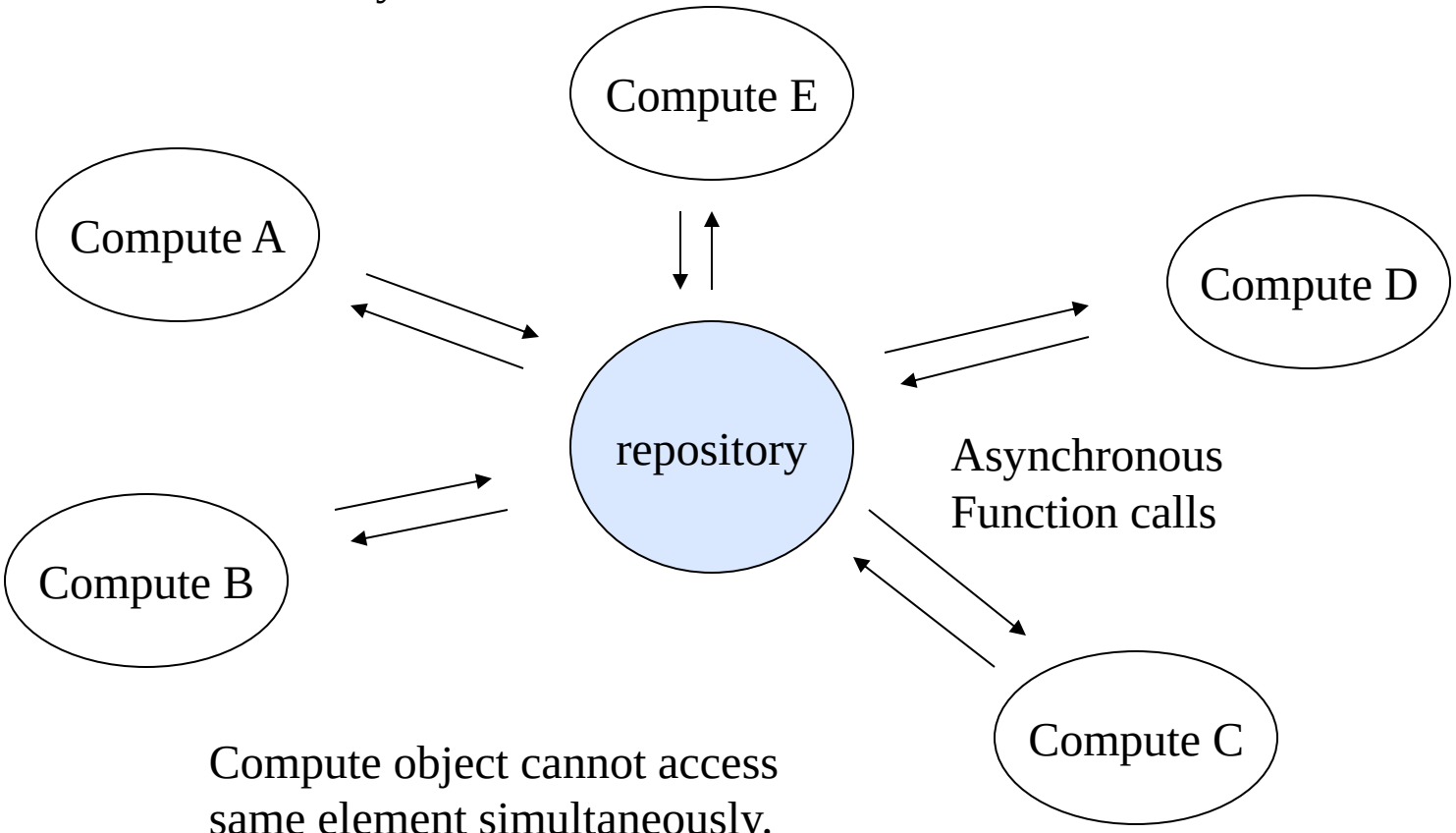
Replicable

Sets of operations need to be performed using global data structure, causing dependency.



Repository

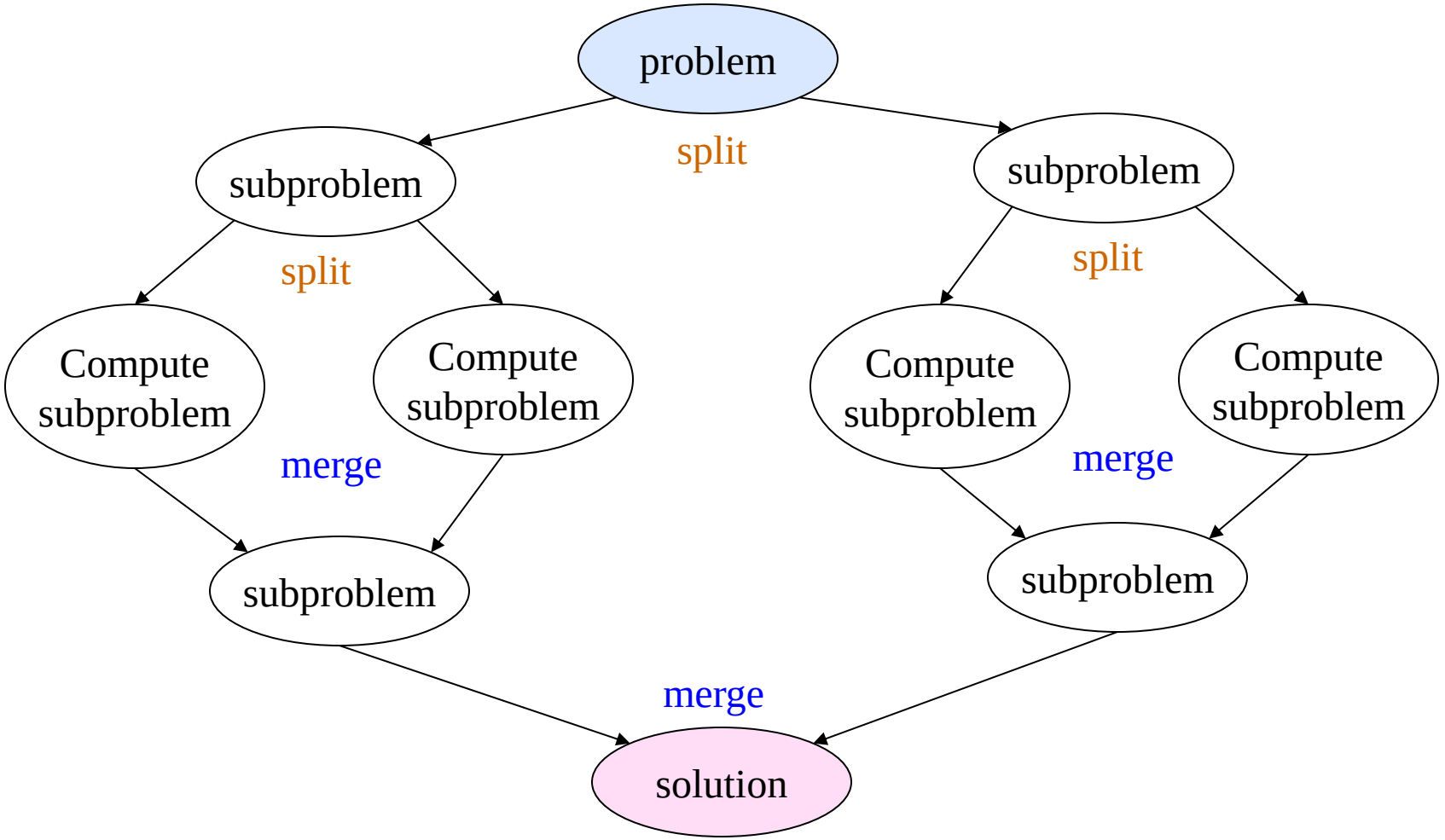
Independent computations needs to applied to centralized data structure in non-deterministic way.



Compute object cannot access same element simultaneously. (Repository controls access)

Divide & Conquer

A problem is structured to be solved in sub-problems independently, and merging them later.



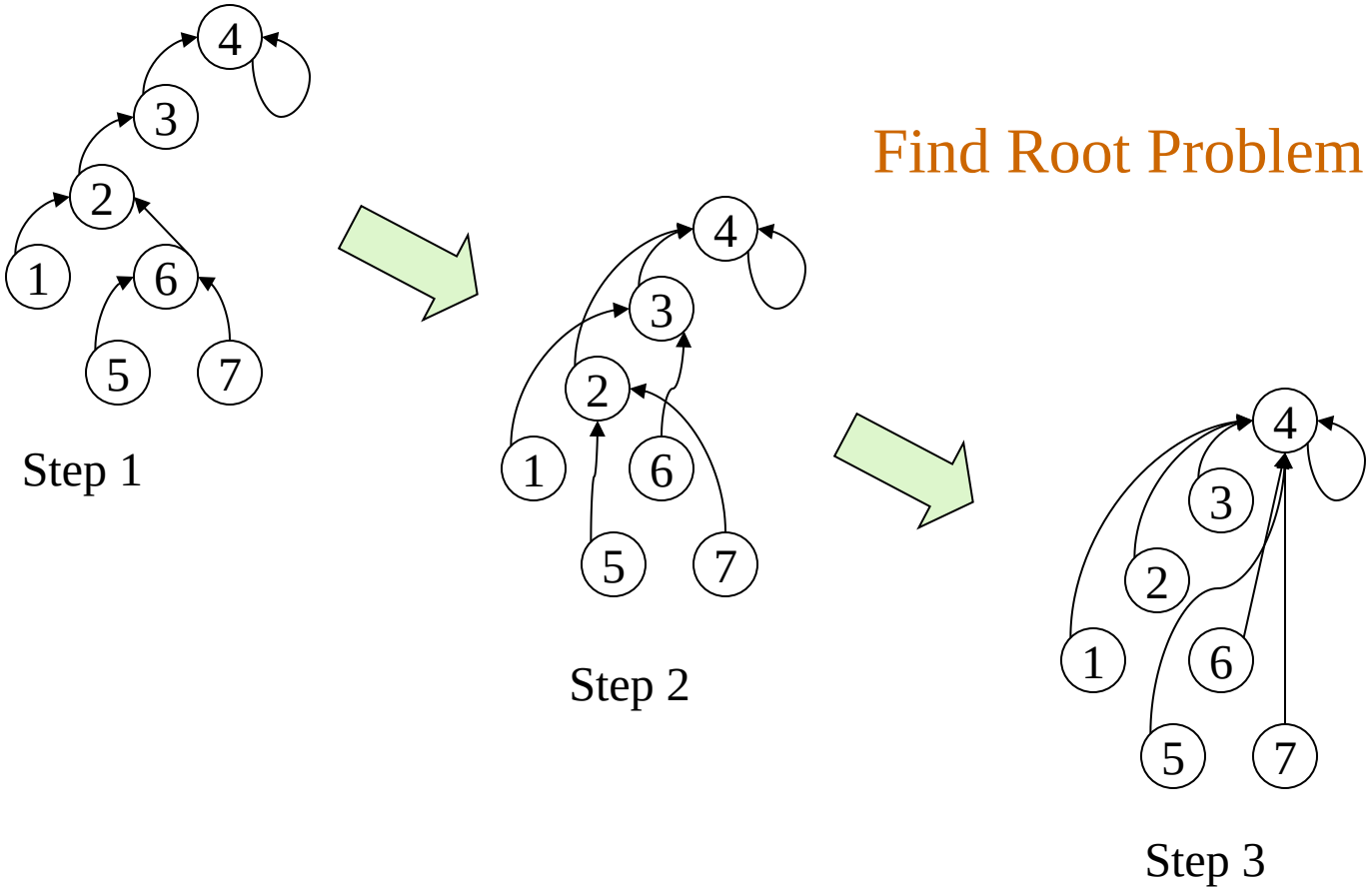
* Split level needs to be adjusted appropriately.

Example: Merge-sort

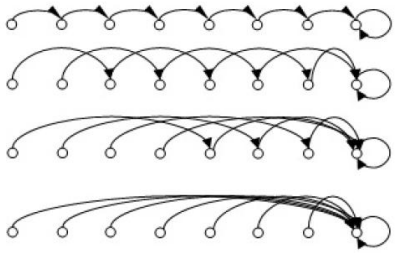
```
PROCEDURE interSort(A, n)
  if (n > 1) then
    imparte(A, n, A_0, n_0, A_1, n_1);
    in parallel
      interSort(A_0, n_0),
      interSort(A_1, n_1)
    end in parallel
    combina(A_0, n_0, A_1, n_1, A, n);
  end if
```

Recursive Data

Recursive data structures seem to have little exploitable concurrency.
But in some cases, the structure can be transformed.



Pointer jumping – recursive doubling
example: tree -> all path length

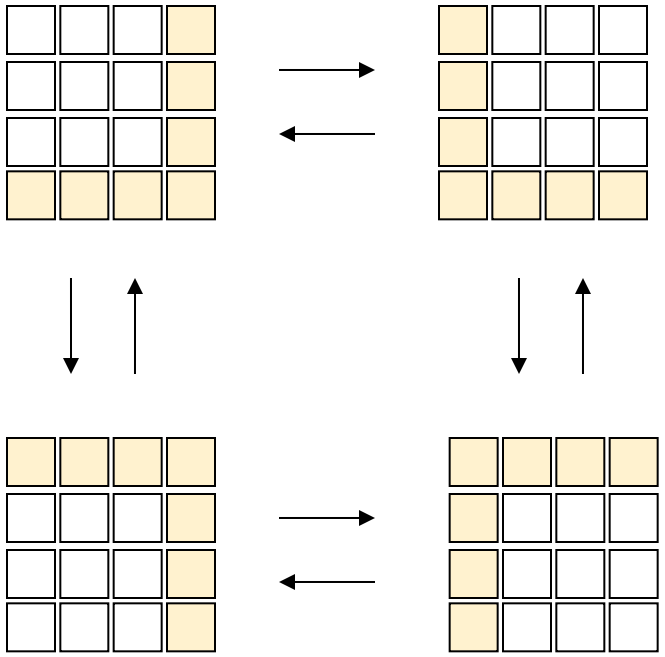


ALGORITHM LUNGIME-DRUM$< n, parinte[0..n-1], distanta[0..n-1] >$

for $i = 0, n-1$ **in parallel do**
  $distanta[i] \leftarrow 1;$
  while $(parinte[i] \neq -1)$ **do**
    $distanta[i] \leftarrow distanta[i] + distanta[parinte[i]];$
    $parinte[i] \leftarrow parinte[parinte[i]];$
  end while
end for

Geometric decomposition

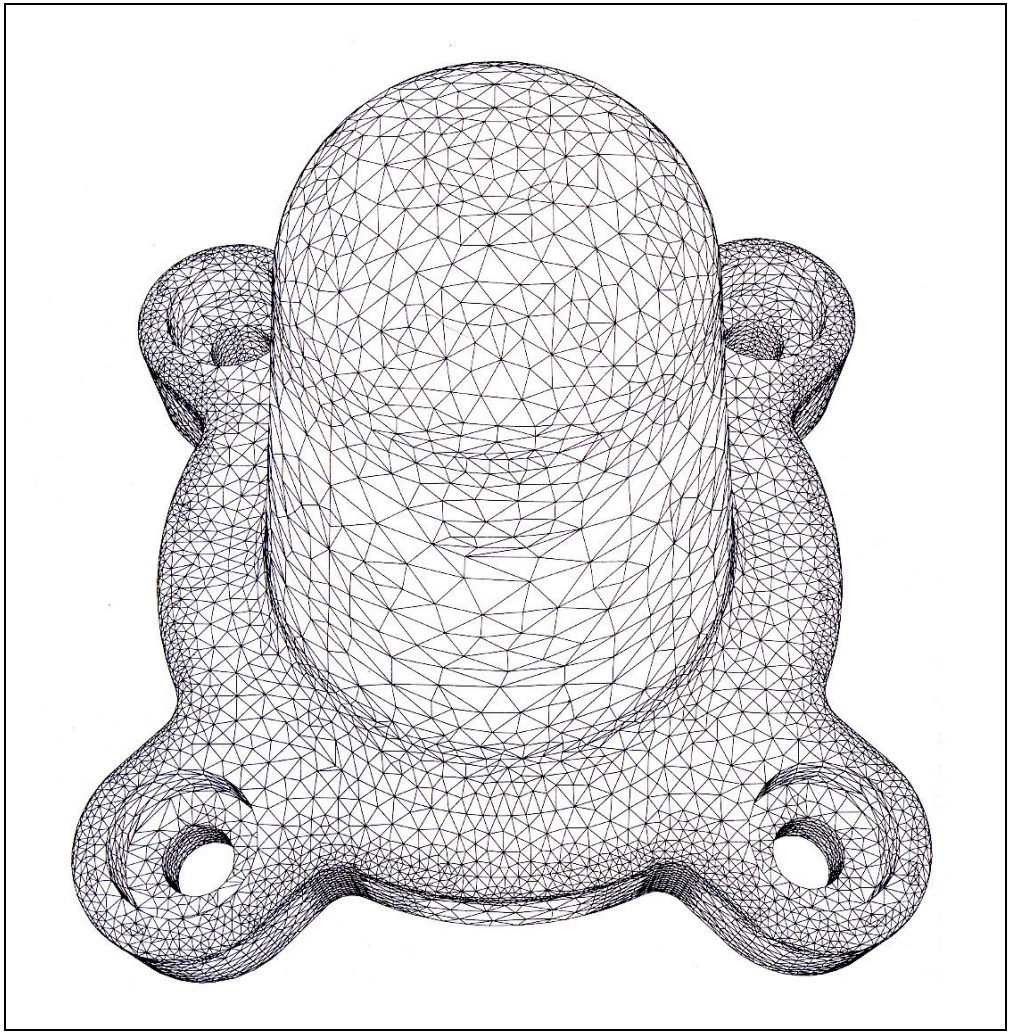
Dependencies exist but the communication is done
in predictable (geometric) neighbor-to-neighbor paths.



Neighbor-To-Neighbor communication

Irregular Mesh

Communication in non-predictable paths in mesh topology.



Hard to define due to varying communication patterns.

Start point :
Pattern that constructed this mesh.

all starts from ...

Decomposing a sequential problem to expose concurrency.

- **Data decomposition**
  - Concurrency comes from working on different data elements simultaneously.
- **Functional decomposition**
  - Concurrency comes from working on independent functional tasks simultaneously.
- **Data flow**
  - Concurrency comes from working simultaneously on different streaming data on different stages of computation.

Decomposition Techniques

How does one decompose a task into various subtasks?

- there is no single recipe that works for all problems,
- a set of commonly used techniques that apply to broad classes of problems.

These include:

- recursive decomposition
- data decomposition (geometric decomposition)
- exploratory decomposition
- speculative decomposition

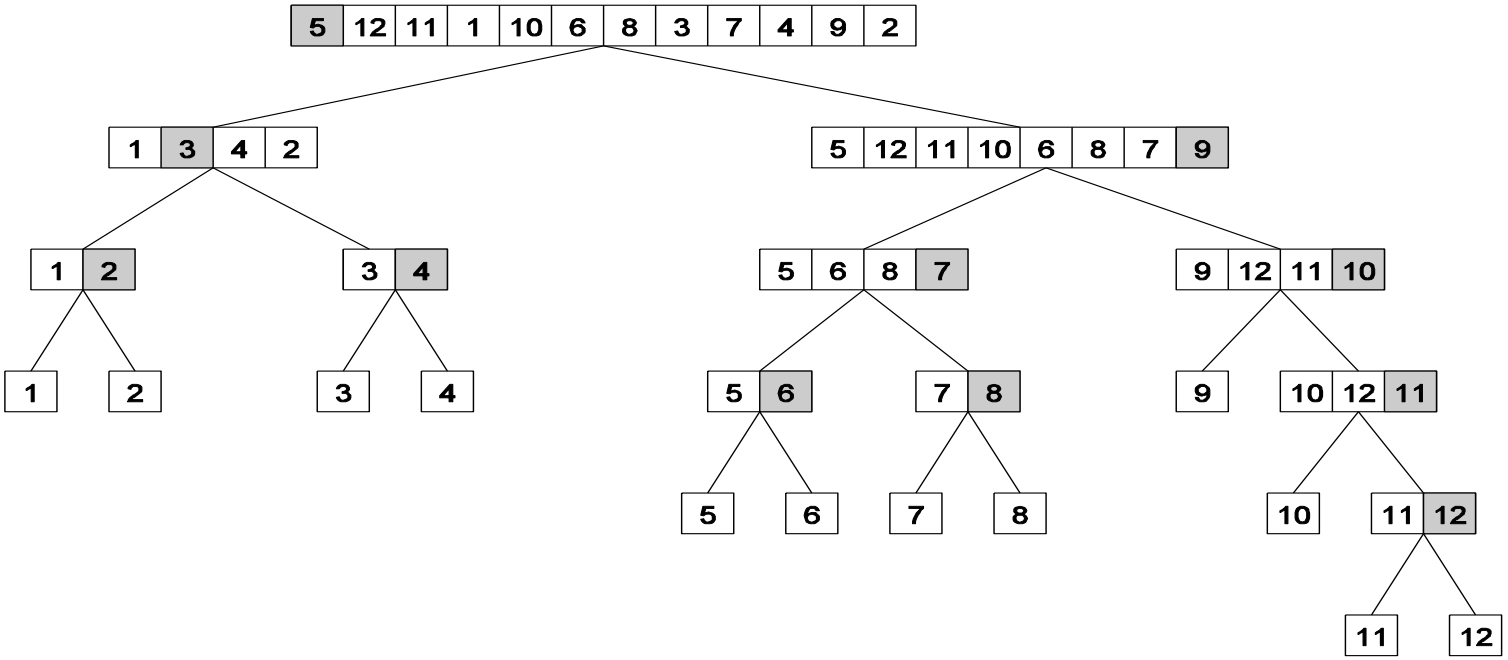
- Pipelines…

Recursive Decomposition

- Generally suited to problems that are solved using the **divide-and-conquer** strategy.
- A given problem is first decomposed into a set of sub-problems.
- These sub-problems are recursively decomposed further until a desired granularity is reached.

Recursive Decomposition: Example 1

A classic example of a divide-and-conquer algorithm on which we can apply recursive decomposition is Quicksort.



In this example, once the list has been partitioned around the pivot, each sublist can be processed concurrently (i.e., each sublist represents an independent subtask). This can be repeated recursively.

Recursive Decomposition: Example 2

The problem of finding the minimum number in a given list (or indeed any other associative operation such as sum, AND, etc.) can be fashioned as a divide-and-conquer algorithm. The following algorithm illustrates this.

We first start with a simple serial loop for computing the minimum entry in a given list:

```
1. procedure SERIAL_MIN (A, n)
2. begin
3. min = A[0];
4. for i := 1 to n − 1 do
5.                 if (A[i] < min) min := A[i];
6. endfor;
7. return min;
8. end SERIAL_MIN
```

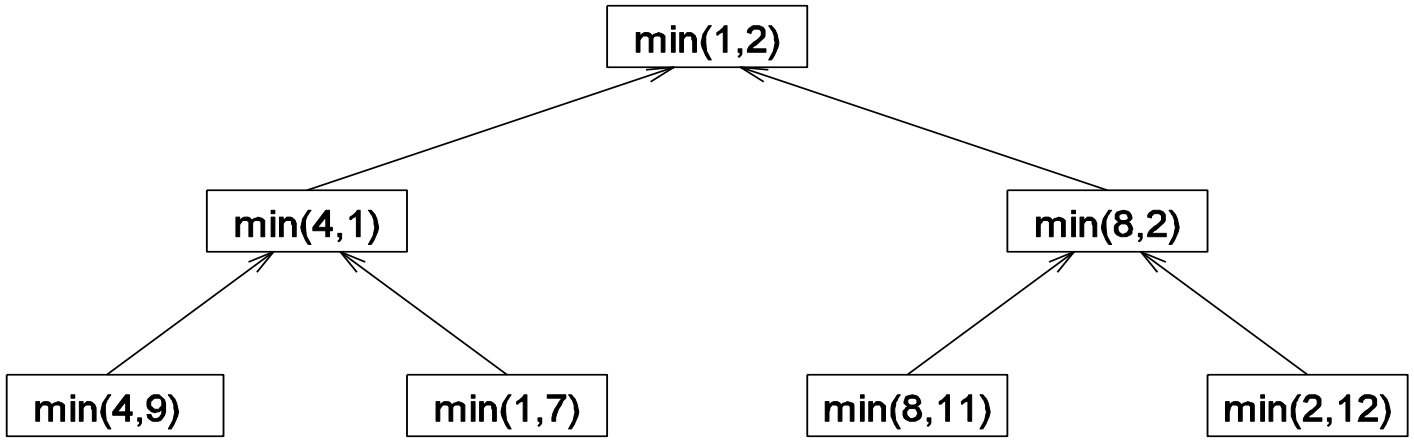
Recursive Decomposition: Example

We can rewrite the loop as follows:

```
1. procedure RECURSIVE_MIN (A, n)
2. begin
3. if ( n = 1 ) then
4.     min := A [0]  ;
5. else
6.     lmin := RECURSIVE_MIN ( A, n/2 );
7.     rmin := RECURSIVE_MIN (  &(A[n/2]), n - n/2 );
8.     if (lmin  < rmin) then
9.                 min := lmin;
10.    else
11.                min := rmin;
12.    endelse;
13. endelse;
14. return min;
15. end RECURSIVE_MIN
```

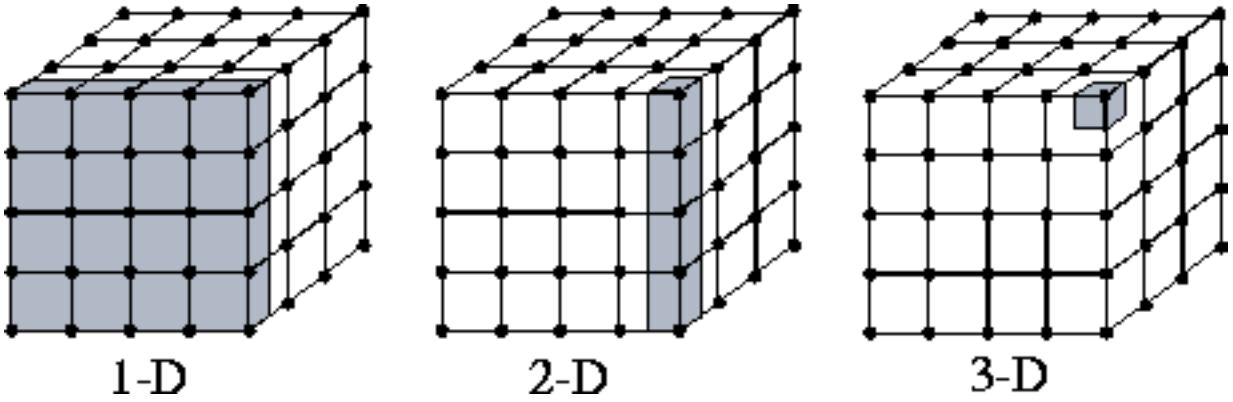
Recursive Decomposition: Example

The code in the previous slide can be decomposed naturally using a recursive decomposition strategy. We illustrate this with the following example of finding the minimum number in the set {4, 9, 1, 7, 8, 11, 2, 12}. The task dependency graph associated with this computation is as follows:

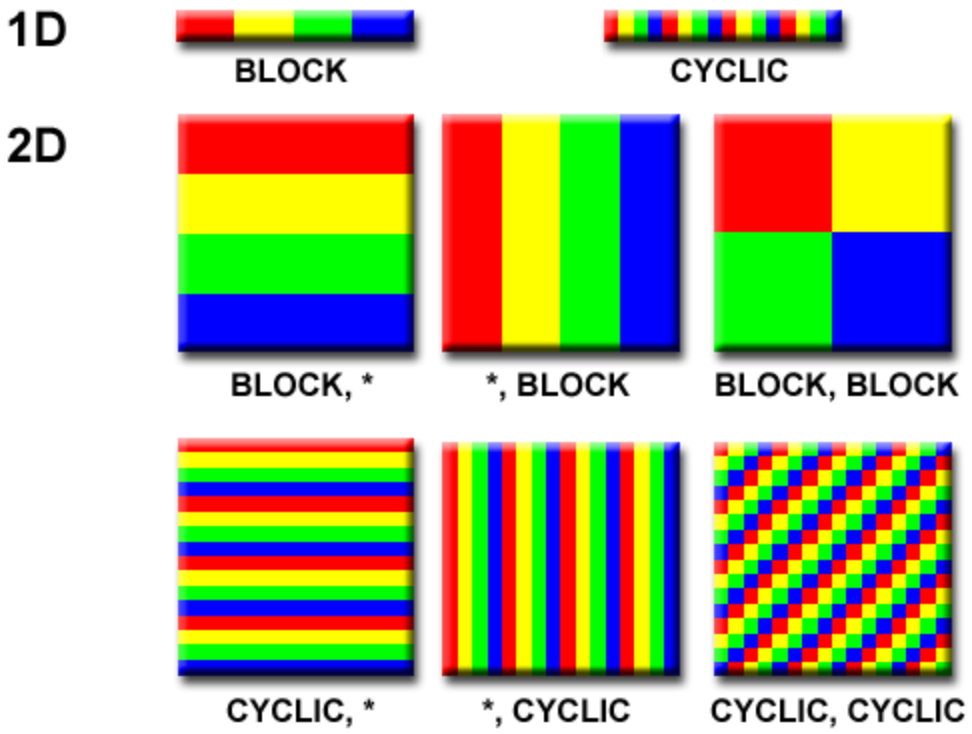


Data Decomposition (geometric)

- Identify the data on which computations are performed.
- Partition this data across various tasks.
- This partitioning induces a decomposition of the problem.
- Data can be partitioned in various ways - this critically impacts performance of a parallel algorithm.



Variants


1D
BLOCK
CYCLIC
2D
BLOCK, *
*, BLOCK
BLOCK, BLOCK
CYCLIC, *
*, CYCLIC
CYCLIC, CYCLIC

Data Decomposition: Output Data Decomposition

- Often, each element of the output can be computed independently of others (but simply as a function of the input).
- A partition of the output across tasks decomposes the problem naturally.

Output Data Decomposition: Example

Consider the problem of multiplying two ***n*** x ***n*** matrices ***A*** and ***B*** to yield matrix ***C***. The output matrix ***C*** can be partitioned into four tasks as follows:

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

Task 1: $C_{1,1} = A_{1,1}B_{1,1} + A_{1,2}B_{2,1}$

Task 2: $C_{1,2} = A_{1,1}B_{1,2} + A_{1,2}B_{2,2}$

Task 3: $C_{2,1} = A_{2,1}B_{1,1} + A_{2,2}B_{2,1}$

Task 4: $C_{2,2} = A_{2,1}B_{1,2} + A_{2,2}B_{2,2}$

Output Data Decomposition: Example

A partitioning of output data does not result in a unique decomposition into tasks. For example, for the same problem as in previous slide, with identical output data distribution, we can derive the following two (other) decompositions:

Decomposition I	**Decomposition II**
Task 1: $\boldsymbol{C_{1,1}} = \boldsymbol{A_{1,1}}\,\boldsymbol{B_{1,1}}$	Task 1: $\boldsymbol{C_{1,1}} = \boldsymbol{A_{1,1}}\,\boldsymbol{B_{1,1}}$
Task 2: $\boldsymbol{C_{1,1}} = \boldsymbol{C_{1,1}} + \boldsymbol{A_{1,2}}\,\boldsymbol{B_{2,1}}$	Task 2: $\boldsymbol{C_{1,1}} = \boldsymbol{C_{1,1}} + \boldsymbol{A_{1,2}}\,\boldsymbol{B_{2,1}}$
Task 3: $\boldsymbol{C_{1,2}} = \boldsymbol{A_{1,1}}\,\boldsymbol{B_{1,2}}$	Task 3: $\boldsymbol{C_{1,2}} = \boldsymbol{A_{1,2}}\,\boldsymbol{B_{2,2}}$
Task 4: $\boldsymbol{C_{1,2}} = \boldsymbol{C_{1,2}} + \boldsymbol{A_{1,2}}\,\boldsymbol{B_{2,2}}$	Task 4: $\boldsymbol{C_{1,2}} = \boldsymbol{C_{1,2}} + \boldsymbol{A_{1,1}}\,\boldsymbol{B_{1,2}}$
Task 5: $\boldsymbol{C_{2,1}} = \boldsymbol{A_{2,1}}\,\boldsymbol{B_{1,1}}$	Task 5: $\boldsymbol{C_{2,1}} = \boldsymbol{A_{2,2}}\,\boldsymbol{B_{2,1}}$
Task 6: $\boldsymbol{C_{2,1}} = \boldsymbol{C_{2,1}} + \boldsymbol{A_{2,2}}\,\boldsymbol{B_{2,1}}$	Task 6: $\boldsymbol{C_{2,1}} = \boldsymbol{C_{2,1}} + \boldsymbol{A_{2,1}}\,\boldsymbol{B_{1,1}}$
Task 7: $\boldsymbol{C_{2,2}} = \boldsymbol{A_{2,1}}\,\boldsymbol{B_{1,2}}$	Task 7: $\boldsymbol{C_{2,2}} = \boldsymbol{A_{2,1}}\,\boldsymbol{B_{1,2}}$
Task 8: $\boldsymbol{C_{2,2}} = \boldsymbol{C_{2,2}} + \boldsymbol{A_{2,2}}\,\boldsymbol{B_{2,2}}$	Task 8: $\boldsymbol{C_{2,2}} = \boldsymbol{C_{2,2}} + \boldsymbol{A_{2,2}}\,\boldsymbol{B_{2,2}}$

Output Data Decomposition: Example

Consider the problem of counting the instances of given itemsets in a database of transactions.

In this case, the output (itemset frequencies) can be partitioned across tasks.

(a) Transactions (input), itemsets (input), and frequencies (output)

Database Transactions	Itemsets	Itemset Frequency
A, B, C, E, G, H	A, B, C	1
B, D, E, F, K, L	D, E	3
A, B, F, H, L	C, F, G	0
D, E, F, H	A, E	2
F, G, H, K,	C, D	1
A, E, F, K, L	D, K	2
B, C, D, G, H, L	B, C, F	0
G, H, L	C, D, K	0
D, E, F, K, L		
F, G, H, L		

(b) Partitioning the frequencies (and itemsets) among the tasks

task 1

Database Transactions	Itemsets	Itemset Frequency
A, B, C, E, G, H	A, B, C	1
B, D, E, F, K, L	D, E	3
A, B, F, H, L	C, F, G	0
D, E, F, H	A, E	2
F, G, H, K,		
A, E, F, K, L		
B, C, D, G, H, L		
G, H, L		
D, E, F, K, L		
F, G, H, L		

task 2

Database Transactions	Itemsets	Itemset Frequency
A, B, C, E, G, H	C, D	1
B, D, E, F, K, L	D, K	2
A, B, F, H, L	B, C, F	0
D, E, F, H	C, D, K	0
F, G, H, K,		
A, E, F, K, L		
B, C, D, G, H, L		
G, H, L		
D, E, F, K, L		
F, G, H, L		

Output Data Decomposition: Example

From the previous example, the following observations can be made:

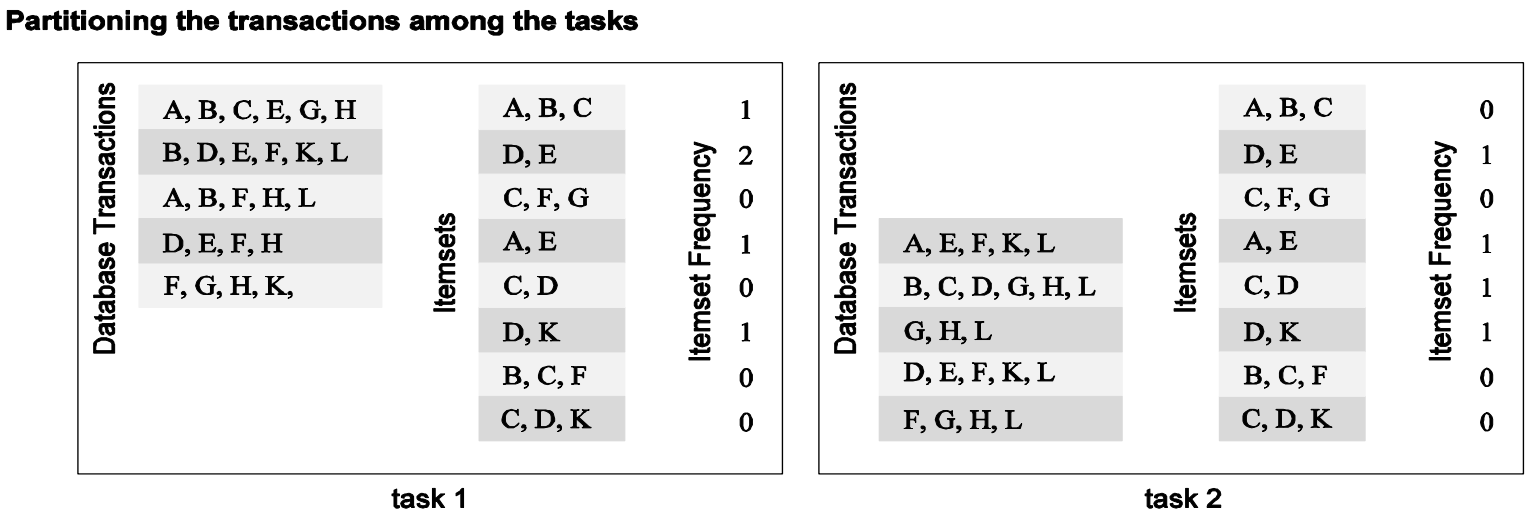
- If the database of transactions is replicated across the processes (or there is a shared access), each task can be independently accomplished with no communication.
- If the database is partitioned across processes as well (for reasons of memory utilization), each task first computes partial counts. These counts are then aggregated at the appropriate task.

Input Data Partitioning

- Generally applicable if each output can be naturally computed as a function of the input.
- In many cases, this is the only natural decomposition because the output is not clearly known a-priori (e.g., the problem of finding the minimum in a list, sorting a given list, etc.).
- A task is associated with each input data partition. The task performs as much of the computation with its part of the data. Subsequent processing combines these partial results.

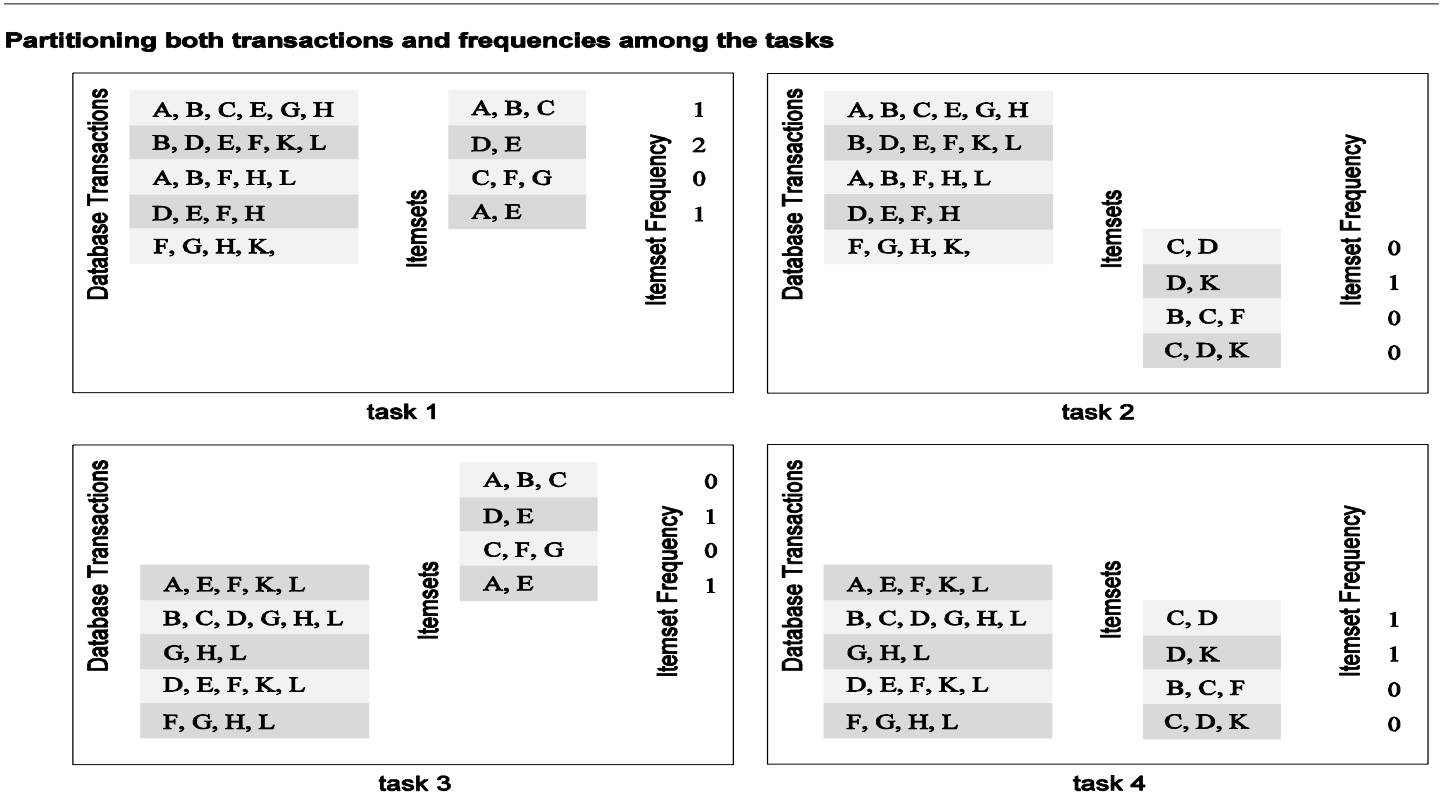
Input Data Partitioning: Example

In the database counting example, the input (i.e., the transaction set) can be partitioned. This induces a task decomposition in which each task generates partial counts for all itemsets. These are combined subsequently for aggregate counts.



Partitioning Input *and* Output Data

Often input and output data decomposition can be combined for a higher degree of concurrency. For the itemset counting example, the transaction set (input) and itemset counts (output) can both be decomposed as follows:



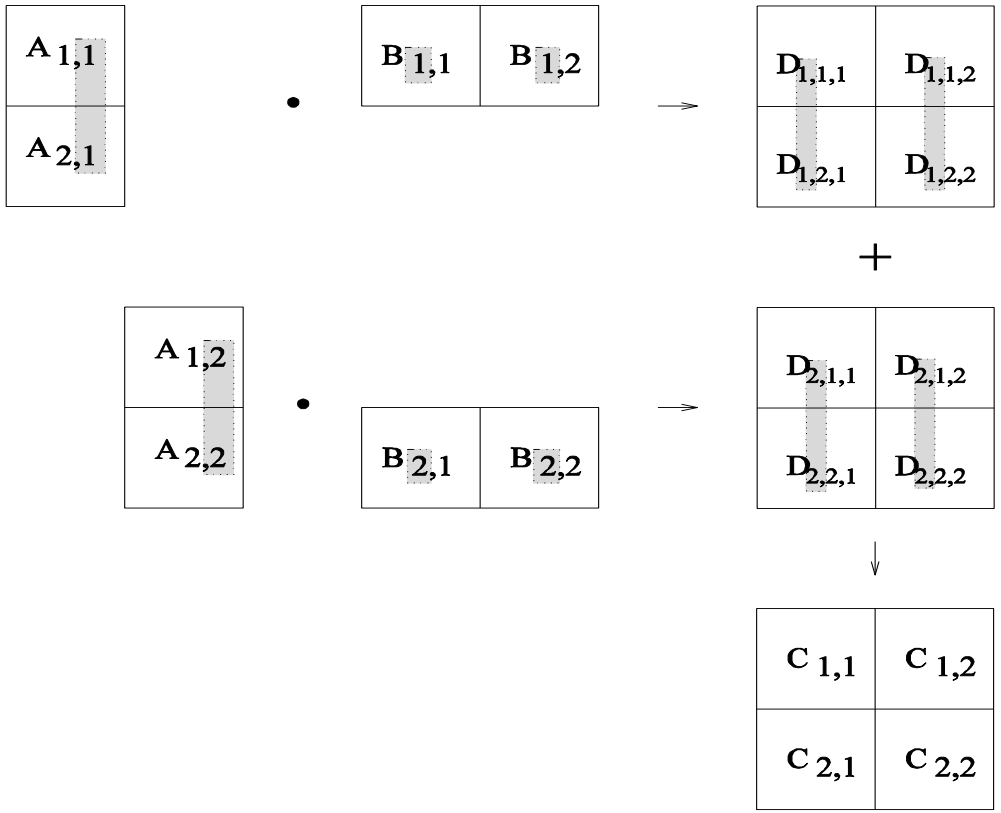
Intermediate Data Partitioning

- Computation can often be viewed as a sequence of transformation from the input to the output data.
- In these cases, it is often beneficial to use one of the intermediate stages as a basis for decomposition.

Intermediate Data Partitioning: Example

Dense matrix multiplication => variant:

-first show how we can visualize this computation in terms of intermediate matrices ***D***.



Intermediate Data Partitioning: Example

A decomposition of intermediate data structure leads to the following decomposition into 8 + 4 tasks:

Stage I

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} \begin{pmatrix} D_{1,1,1} & D_{1,1,2} \\ D_{1,2,2} & D_{1,2,2} \end{pmatrix} \\ \begin{pmatrix} D_{2,1,1} & D_{2,1,2} \\ D_{2,2,2} & D_{2,2,2} \end{pmatrix} \end{pmatrix}$$

Stage II

$$\begin{pmatrix} D_{1,1,1} & D_{1,1,2} \\ D_{1,2,2} & D_{1,2,2} \end{pmatrix} + \begin{pmatrix} D_{2,1,1} & D_{2,1,2} \\ D_{2,2,2} & D_{2,2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

Task 01: $\boldsymbol{D}_{1,1,1} = \boldsymbol{A}_{1,1}\boldsymbol{B}_{1,1}$

Task 02: $\boldsymbol{D}_{2,1,1} = \boldsymbol{A}_{1,2}\boldsymbol{B}_{2,1}$

Task 03: $\boldsymbol{D}_{1,1,2} = \boldsymbol{A}_{1,1}\boldsymbol{B}_{1,2}$

Task 04: $\boldsymbol{D}_{2,1,2} = \boldsymbol{A}_{1,2}\boldsymbol{B}_{2,2}$

Task 05: $\boldsymbol{D}_{1,2,1} = \boldsymbol{A}_{2,1}\boldsymbol{B}_{1,1}$

Task 06: $\boldsymbol{D}_{2,2,1} = \boldsymbol{A}_{2,2}\boldsymbol{B}_{2,1}$

Task 07: $\boldsymbol{D}_{1,2,2} = \boldsymbol{A}_{2,1}\boldsymbol{B}_{1,2}$

Task 08: $\boldsymbol{D}_{2,2,2} = \boldsymbol{A}_{2,2}\boldsymbol{B}_{2,2}$

Task 09: $\boldsymbol{C}_{1,1} = \boldsymbol{D}_{1,1,1} + \boldsymbol{D}_{2,1,1}$

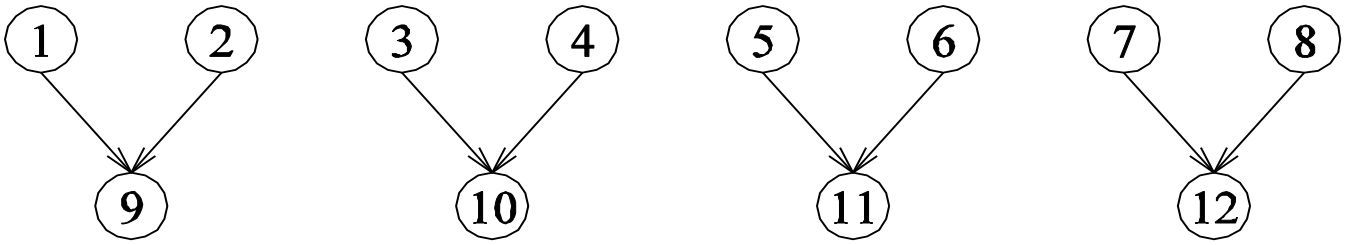
Task 10: $\boldsymbol{C}_{1,2} = \boldsymbol{D}_{1,1,2} + \boldsymbol{D}_{2,1,2}$

Task 11: $\boldsymbol{C}_{2,1} = \boldsymbol{D}_{1,2,1} + \boldsymbol{D}_{2,2,1}$

Task 12: $\boldsymbol{C}_{2,,2} = \boldsymbol{D}_{1,2,2} + \boldsymbol{D}_{2,2,2}$

Intermediate Data Partitioning: Example

The *task dependency graph* for the decomposition (shown in previous foil) into 12 tasks is as follows:



The Owner Computes Rule

- The *Owner Computes Rule* generally states that the process assigned a particular data item is responsible for all computation associated with it.

- In the case of input data decomposition, the owner computes rule implies that all computations that use the input data are performed by the process.

- In the case of output data decomposition, the owner computes rule implies that the output is computed by the process to which the output data is assigned.

Data Replication

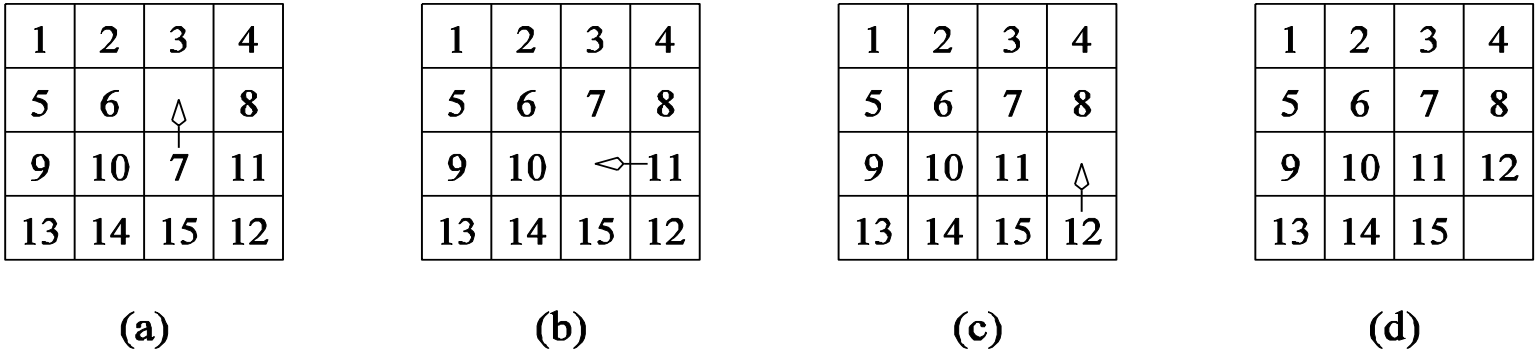
- Replicate data – in order to create independent tasks
- For example, for matrix multiplication: replication on third dimension.

Exploratory Decomposition

- In many cases, the decomposition of the problem goes hand-in-hand with its execution.
- These problems typically involve the exploration (search) of a state space of solutions.
- Problems in this class include a variety of discrete optimization problems
  - 0/1 integer programming,
  - search algorithms for finding approximate solutions to hard optimization problems
  - theorem proving,
  - game playing, etc.

Exploratory Decomposition: Example

A simple application of exploratory decomposition is in the solution to a 15 puzzle (a tile puzzle).

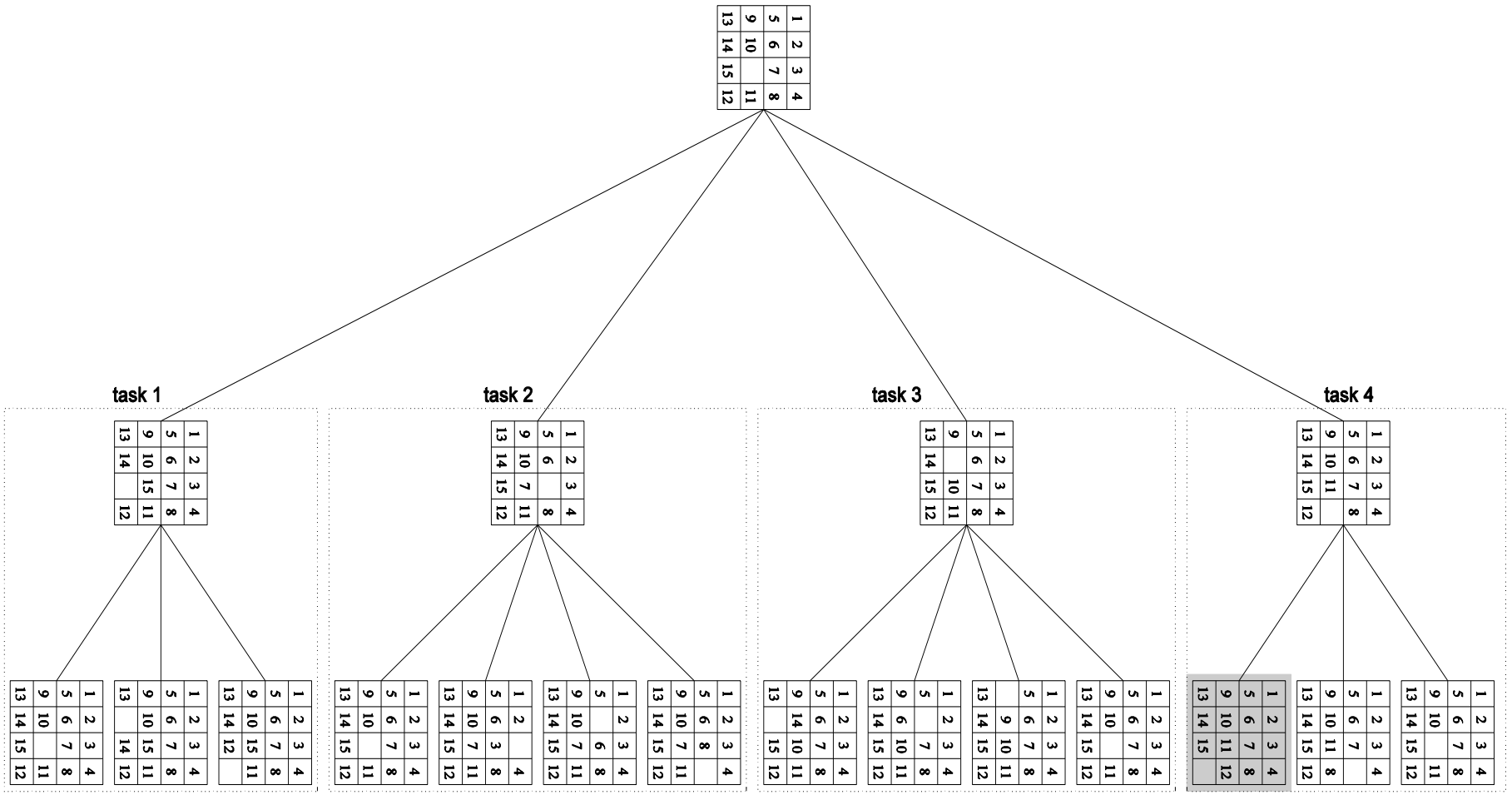


a sequence of three moves that transform a given initial state (a) to desired final state (d).

But the problem of computing the solution, in general, is much more difficult than in this simple example.

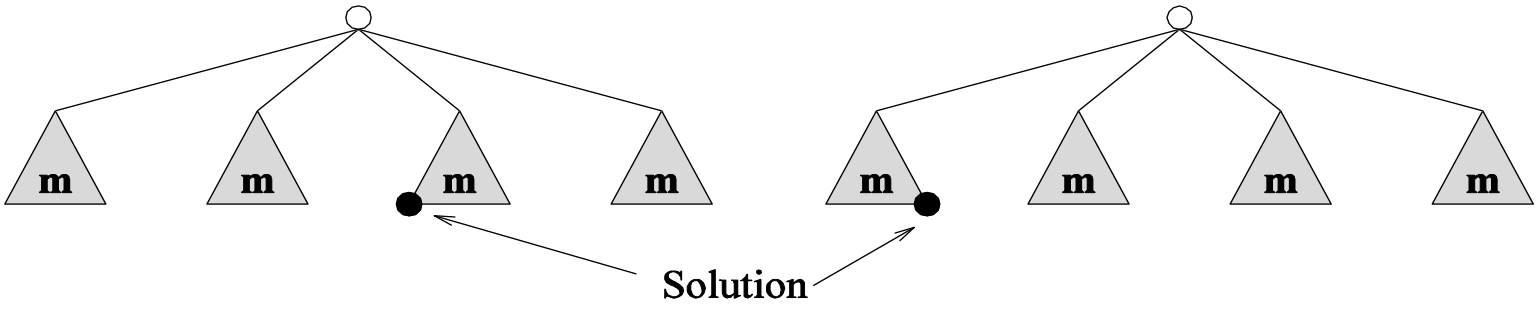
Exploratory Decomposition: Example

The state space can be explored by generating various successor states of the current state and to view them as independent tasks.



Exploratory Decomposition: Anomalous Computations

- In many instances of exploratory decomposition, the decomposition technique may change the amount of work done by the parallel formulation.
- This change results in super- or sub-linear speedups.



Total serial work: 2m+1
Total parallel work: 1

(a)

Total serial work: m
Total parallel work: 4m

(b)

Speculative Decomposition

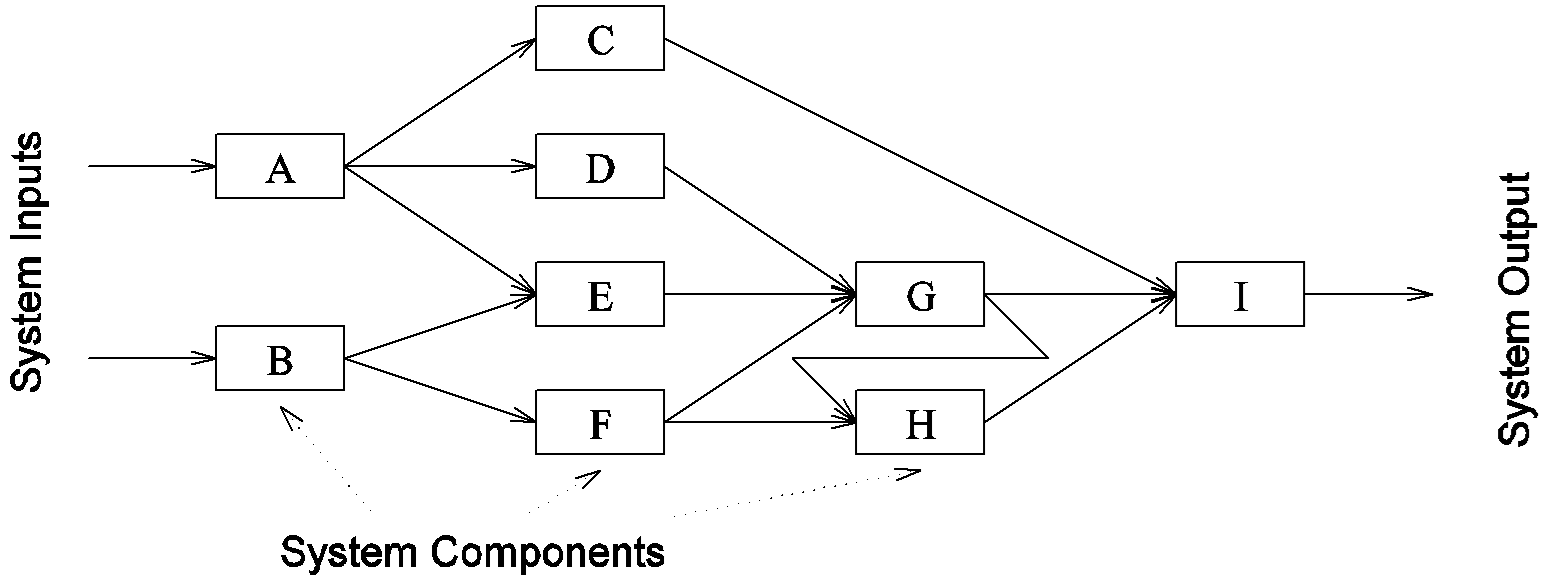
- In some applications, dependencies between tasks are not known a-priori.
- For such applications, it is impossible to identify independent tasks.
- There are generally two approaches to dealing with such applications:
  - *conservative* approaches, which identify independent tasks only when they are guaranteed to not have dependencies, and,
  - *optimistic* approaches, which schedule tasks even when they may potentially be erroneous.
- Conservative approaches may yield little concurrency and
- Optimistic approaches may require *roll-back* mechanism in the case of an error.

Classical example: ***discrete event simulation***

- The central data structure in a discrete event simulation is a time-ordered event list

Speculative Decomposition: Example

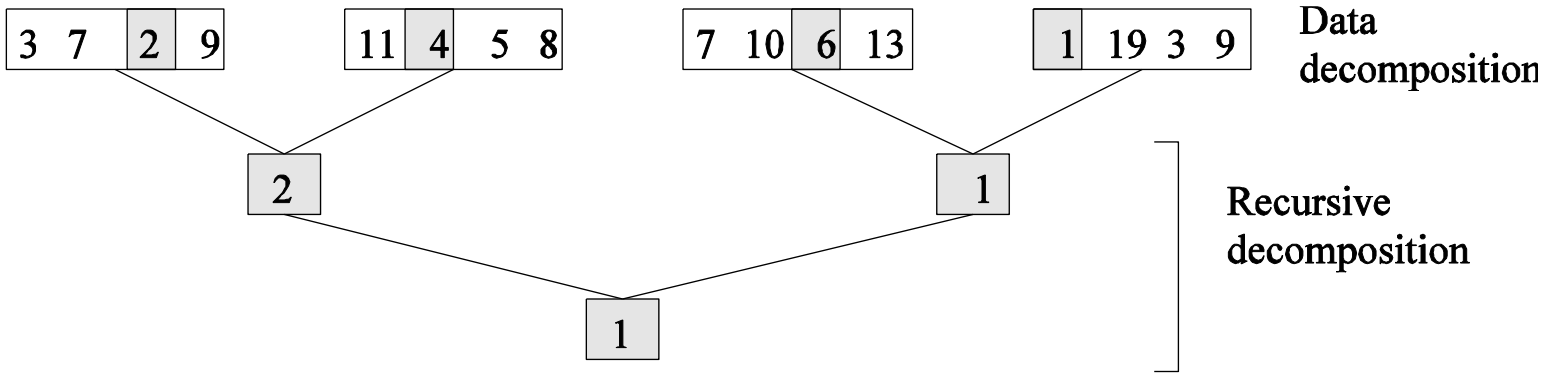
- The simulation of a network of nodes (for instance, an assembly line or a computer network through which packets pass).
- The task is to simulate the behavior of this network for various inputs and node delay parameters (note that networks may become unstable for certain values of service rates, queue sizes, etc.).



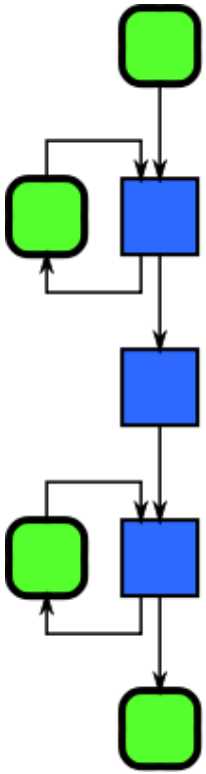
Hybrid Decompositions

Often, a mix of decomposition techniques is necessary for decomposing a problem. Consider the following examples:

- In quicksort, recursive decomposition alone limits concurrency (Why?). A mix of data and recursive decompositions is more desirable.
- In discrete event simulation, there might be concurrency in task processing. A mix of speculative decomposition and data decomposition may work well.
- Even for simple problems like finding a minimum of a list of numbers, a mix of data and recursive decomposition works well.

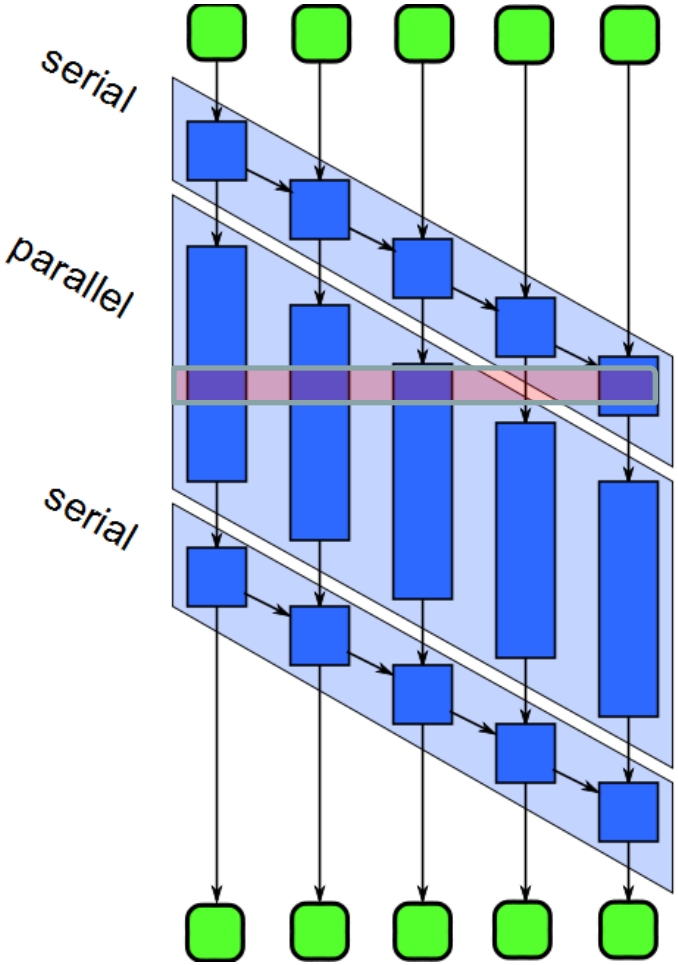


PIPELINE



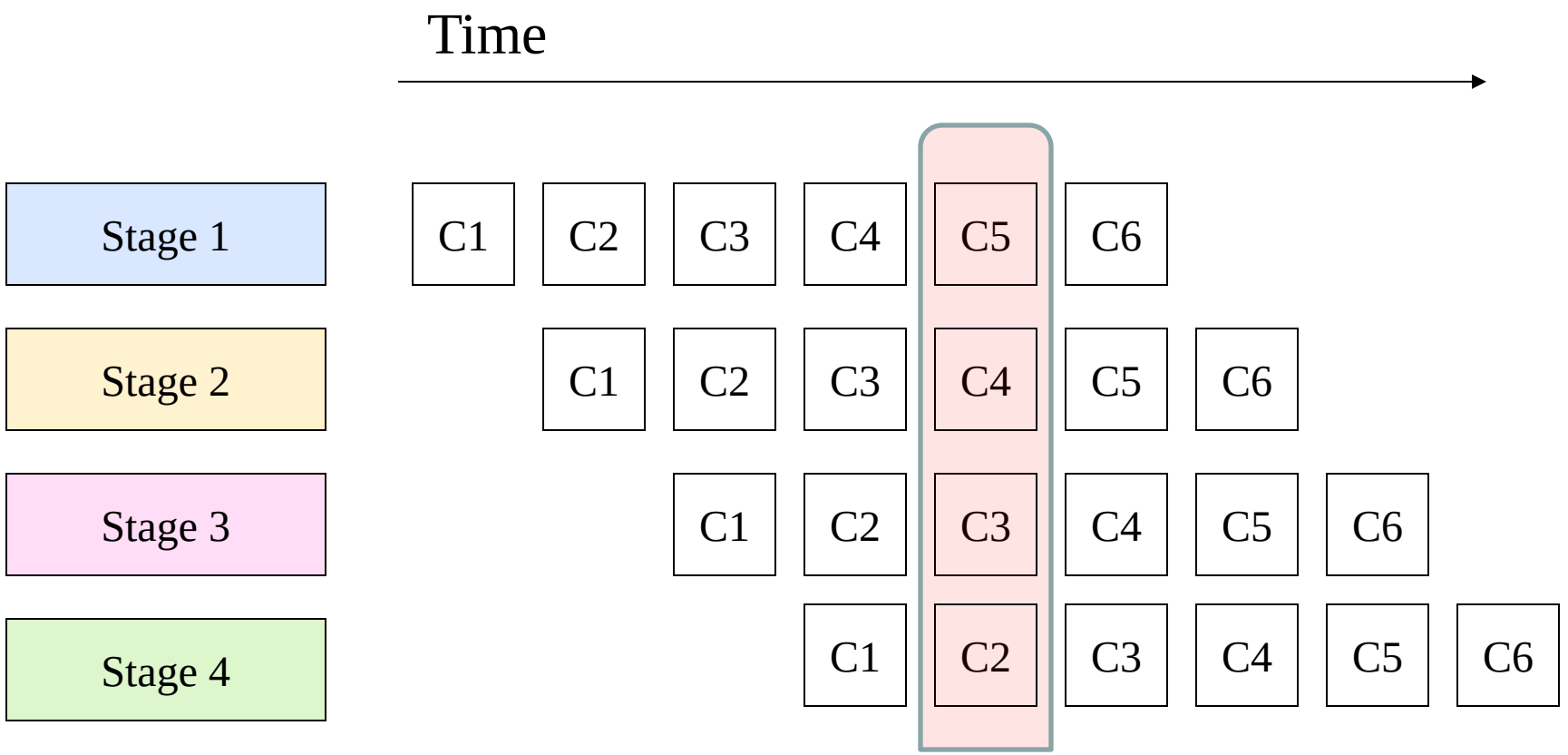
A series of ordered but independent computation stages need to be applied on data,
where each output of a computation becomes input of subsequent computation.

Pipeline

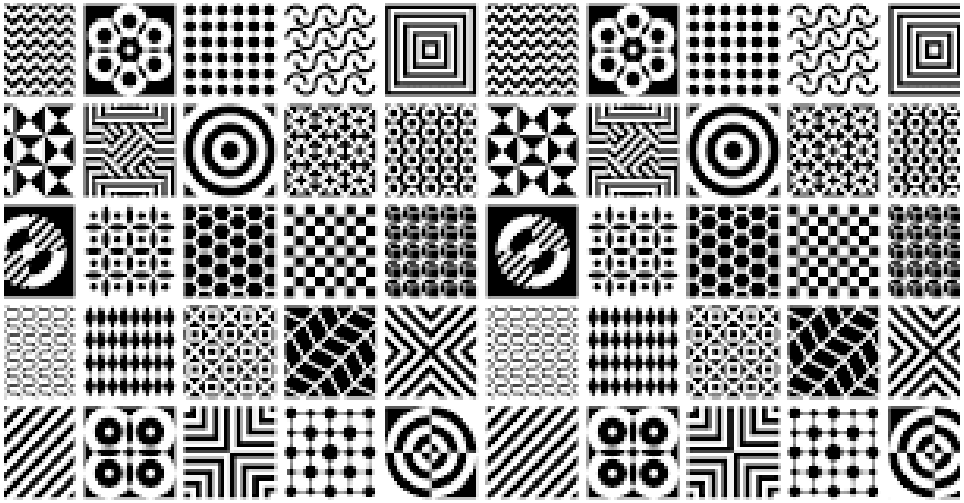


Different stages are executed in parallel!

Pipeline



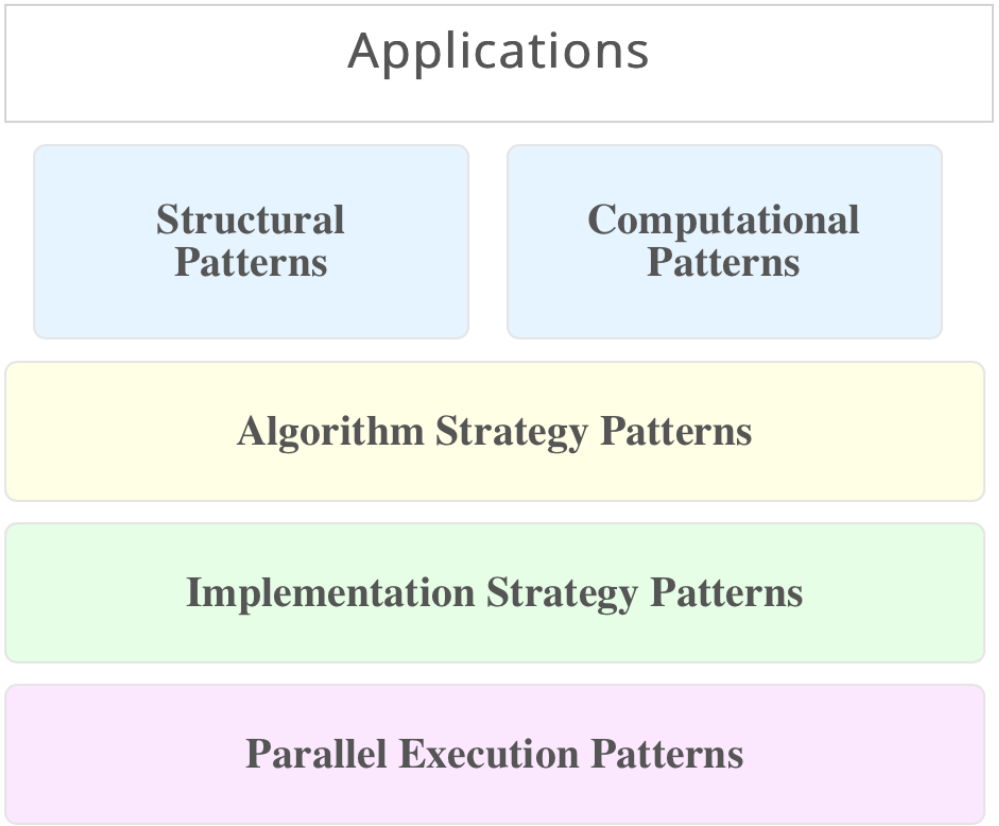
A more general perspective
=
Pattern Languages



A Design Pattern Language for Engineering (Parallel) Software

- https://patterns.eecs.berkeley.edu

Parallel and Distributed together!



Structural Patterns

- Model-View-Controller
- Map Reduce
- Layered systems
- Event-based, implicit invocation
- Iterative Refinement
- Process Control
- Arbitrary Static Task Graph
- Pipe and Filter
- Agent and Repository

Computational Patterns

- Backtracking & Branch and Bound
- Monte Carlo Methods
- Circuits
- N-Body Methods
- Dense Linear Algebra
- Sparse Linear Algebra
- Dynamic Programming
- Spectral Methods
- Finite State Machine
- Structured Grids
- Graph Algorithms
- Unstructured Grids
- Graphical Models
- Sorting

Parallel Algorithm Strategy Patterns

- Task Parallelism
- Recursive Splitting
- Discrete Event
- Pipeline
- Geometric Decomposition
- Data Parallelism
- Non-work-efficient Parallelism
- Speculation

Implementation Strategy Patterns

Program Structure

- SPMD
- Master-Worker
- Strict Data Parallelism
- Loop Parallelism
- Fork-Join
- BSP
- Actors
- Graph Partitioning

Data Structure

- Shared Queue
- Shared Hash Table
- Distributed Array
- Shared Data
- Memory Parallelism

Parallel Execution Patterns

Advancing Program Counters

- MIMD
- Task Graph
- SIMD
- Digital Circuits
- Thread Pool
- Speculation
- Data Flow

Coordination

- Message Passing
- Collective Communication
- Mutual Exclusion
- P2P Sync
- Collective Synchronization
- Transactional Memory

MODELS

- DIFFERENT LEVELS OF ABSTRACTION

SKELETONS



History

- In late '80s, Murray Cole introduced the **algorithmic skeleton concept**
- A research community was developed around this (mainly build by researchers coming from High Performance Computing community)
- Goal: *the development of programming frameworks suitable to support efficient parallel programming, in particular with respect to the performance achieved in parallel programs.*
- The community evolved and the concept of algorithmic skeleton evolved as well.
- Nowadays there are several research groups around the world, building parallel frameworks based on this concept.

The link with patterns

- In the '00s, software engineering researchers oriented the

design pattern concept onto

parallel programming.

- The parallel design patterns has been and are currently investigated by a community which is fairly disjoint from the one investigating algorithmic skeletons.

- The research on parallel design patterns produced (and it is currently producing) results that are similar to those produced in the algorithmic skeleton framework.

- Sometimes the results only differ in the “jargon” used to illustrate them.

- In more interesting cases they are similar but still
  - more software engineering oriented perspective used in investigation may provide useful hints to algorithmic skeleton designers as well.

General properties

- **Algorithmic skeletons** = pre-defined patterns encapsulating the structure of a parallel computation
- They are provided to user as ***building blocks*** to be used to write applications.
- ***Each skeleton corresponded to a single parallelism exploitation pattern.***
- A skeleton based programming framework was defined as a programming framework providing programmers with algorithmic skeletons that may be used to model the parallel structure of the applications at hand.
- In a skeleton based programming framework the programmer has no other way (but instantiating skeletons) to structure his/her parallel computation.
- Any skeleton application was the result of the instantiation of a single algorithmic skeleton and thus, in turn, of a single parallelism exploitation pattern.

First Definition *[M. Cole 1988]*

"The new system presents the user with a selection of independent 'algorithmic skeleton',

each of which describes the structure of a particular style of algorithm,

in the way in which higher order functions represent general computational frameworks in the context of functional programming languages.

The user must describe a solution to a problem as an instance of the appropriate skeleton."

A business logic connection

- The term `business logic' usually denotes the functional code of the application, distinct from the non-functional code modeling.

- Example:
  - an application processing a set of images to apply to each one of the images a filter f
    - the code for the filter is business logic code,
    - the code needed to set up a number of parallel/concurrent/distributed agents that altogether contribute to the parallel computation of the filtered images forms a lower layer.

M. Cole “skeleton manifesto"

- Many parallel algorithms can be characterized and classified by their adherence to one or more of a number of ***generic patterns of computation and interaction***.
  - For example, many diverse applications share the underlying control and data flow of the pipeline paradigm.

- Skeletal programming proposes that such patterns be ***abstracted*** and provided as ***a programmer's toolkit***, with :
  - specifications that transcend architectural variations,
  - implementations which recognize these architectural variations to enhance performance.

New definition

An algorithmic skeleton is a parametric, reusable and portable programming abstraction that

models

- a known, common and efficient parallelism exploitation pattern.

Characteristics:

- it models a known, **common** parallelism exploitation **pattern**.
  - (in this context) no interest in patterns that are not commonly found in working parallel applications.
  - un- common patterns could be anyway used with those skeleton frameworks that provide some degree of extensibility
- it is **portable**, that means
  - an efficient implementation of the skeleton should exists on a range of different architectures.
- it models **efficient** parallelism exploitation patterns.
  - Skeletons only model those patterns that have some known, efficient implementation on a variety of target architectures.
- it is **reusable**, that means
  - it could be used in different applications/contexts without actually requiring modifications
- it is **parametric** in that
  - it accepts parameters that can be used to specialize ***what is computed*** by the skeleton, rather than how the parallel computation is performed.

Advantages

- ***the abstraction level*** presented to the programmer by the programming framework ***is higher*** than the one presented by traditional parallel programming frameworks. This in turn has two consequences:
  - The whole process of parallel programming is simplified.
    - Parallelism exploitation mainly consist in properly ***instantiating*** the skeleton abstractions provided by the framework rather than programming your own parallel patterns using the concurrency, communication and synchronization mechanisms available.
  - Application programmers may experiment rapid ***prototyping*** of parallel applications.
    - Different application prototypes may be obtained by instantiating (that is by providing the proper functional/business logic code) ***different skeletons*** among those available or, in case this is allowed, different ***compositions of existing skeletons***.

… advantages

- The **correctness** and the **efficiency** of the parallelism exploitation is not a concern of the application programmers.
  - Rather, the ***system programmers*** that designed and implemented the skeleton framework guarantee both correctness and efficiency.

- Portability on different target architectures is guaranteed by the framework.
  - An algorithmic skeleton application can be moved to a different architecture just by recompiling.
  - Portability has to be intended in this case not only as
    - "***functional portability***", but also as
    - "***performance portability***" as the skeleton framework usually exploits best implementations of the skeletons on the different target architectures supported.

… advantages

- Application debugging is simplified.
  - Only sequential code (that is business logic code) has to be debugged by the application programmers.

- Possibilities for static and dynamic optimizations are much more consistent than in case of traditional, unstructured parallel programming frameworks as the skeleton structure completely exposes to the programming tools the parallel structure of the application.

- Application deployment and (parallel) run, that represent complex activities is completely in charge of the skeleton framework

disadvantages

- In case none of the skeletons provided to the programmer (or none of the possible skeleton nestings) suits to model the parallel structure of the application the application could not be run through the skeleton framework.

- Functional and performance portability across different target architectures requires a huge effort by the skeleton framework designers and programmers.
  - The effort required may be partially reduced by properly choosing the architecture targeted by the skeleton framework, e.g. by considering some “higher level" target to produce the “object" code of the skeleton framework.

Skeletons as higher order functions with associated parallel semantics

- Higher order functions

Stream

```
type 'a stream = EmptyStream | Stream of 'a * 'a stream;;
```

- Pipeline

$$pipeline :: (\alpha \rightarrow \beta) \rightarrow (\beta \rightarrow \gamma) \rightarrow \alpha\ stream \rightarrow \gamma\ stream$$

```
let rec pipeline f g =
  function
    EmptyStream -> EmptyStream
  | Stream(x,y) -> Stream((g(f x)),(pipeline f g y));;
```

- Parallel semantics = informal text

the pipeline stages are computed in parallel onto different items of the input stream. If x_i and x_{i+1} happen to be consecutive items of the input stream, than $f(x_{i+1})$ and $g(f(x_i))$ will be computed in parallel, for any i

Pipeline application

- an application to filter images and then recognize characters string appearing in the images, provided it has available two functions *filter :: image -> image* and *recognize :: image -> string list*, he can simply write a program such as:

let main = pipeline filter recognize;;

- *The parameters of the skeleton defines only the business logic – e.g. pipeline stages*

- *There could be also non-functional parameters = e.g. parallelism degree*

- The parallelism degree that may be conveniently used was evident in pipeline:
  - Having two stages, a parallelism degree equal to 2 may be used to exploit all the parallelism intrinsic to the skeletons.

the farm skeleton

- the farm skeleton models embarrassingly parallel stream parallelism

$$farm :: (\alpha \rightarrow \beta) \rightarrow \alpha\ stream \rightarrow \beta\ stream$$

```
let rec farm f =
  function
    EmptyStream -> EmptyStream
  | Stream(x,y) -> Stream((f x),(farm f y));;
```

- Parallelism semantics:

the computation of any items appearing on the input stream is performed in parallel

  - according to the parallel semantics a number of parallel agents computing function f onto input data items equal to the number of items appearing onto the input stream could be used.

Problems:

1. items of the item stream do not exist all at the same time. A stream is not a vector.

- Items of the stream may appear a different times.
- when we talk of consecutive items x_i and x_{i+1} of the stream we refer to items appearing onto the stream at times t_i and t_{i+1} with $t_i < t_{i+1}$.

=> *it makes no sense to have a distinct parallel agent for all the items of the input stream, as at any given time only a fraction of the input stream will be available*.

2. if we use an agent to compute item x_i, presumably the computation will end at some time t_k. If item x_j appears onto the input stream at a time $t_j > t_k$ this same agent

can be used to compute item x_j rather than picking up a new agent.

Parallelism degree

- Type:

$$farm :: (\alpha \rightarrow \beta) \rightarrow int \rightarrow \alpha\ stream \rightarrow \beta\ stream$$

```
let rec farm f n:int =
function
EmptyStream -> EmptyStream
|
Stream(x,y) -> Stream((f x),(farm f n-1 y));;
```

- parallel semantics
  - the computation of consecutive items of the input stream is performed in parallel on n parallel agents.

- Debate:
  - Include or not non-functional parameters

Alternative definitions

assume pipeline and farm just operate on a global input stream of data to produce a global stream of results

- pipeline

```
let pipeline f g = function x -> (g (f x));;
```

- task farm skeleton corresponds to identity:

```
let farm f = function x -> (f x);;
```

- The higher order function taking care of exploiting parallelism can then be defined as follows:

```
let rec streamer f x =
    match x with
    EmptyStream -> EmptyStream
    | Stream (v,s) -> Stream ((f v), (streamer f s));;
```

- Example

```
let main =
        streamer (pipeline filter recognize);;
```

parallel semantics is only associated to the streamer function as:

the processing of independent input stream data items is performed in parallel

refinement

- This way of modelling stream parallelism corresponds to the idea of structuring
  - all stream parallel applications as farms with more coarse grain computations as functional parameter.

Example: Let's suppose that the `recognize` phase is much more computationally expensive with respect to the `filter` phase.

Using the explicit stream handling versions of pipeline and farm skeletons, we would write the program as

```
let main =
        pipeline (filter) (farm recognize);;
```

analysis

- Recalling the parallel semantics of these skeletons we can easily understand that given a stream with data items x_1; …; x_m (with xi appearing on the stream after x_{i-k} for any k) the computations of

recognize(y_i) recognize(y_{i+k})

(being y_i = filter(x_i)) happen in parallel, as well as the computations of

recognize(yi) filter(x_{i+k})

- If we use the alternative modelling of stream parallelism the program instead is written as

```
let main =
        streamer (pipeline filter recognize);;
```

- and in this case the only computations happening in parallel are the

recognize(filter(x_i)) recognize(filter(x_{i+k}))

Skeleton framework basics

Common subset contains skeletons from three different classes of skeletons:

- Stream parallel skeletons
- Data parallel skeletons
- Control parallel skeletons

Stream parallel skeletons

- Stream parallel skeletons are those exploiting parallelism among computations relative to different, independent data items appearing on the program input stream.
- Each independent computation end with the delivery of one single item on the program output stream.
- Common stream parallel skeletons are the pipeline and task farm skeletons

Data parallel skeletons

- Data parallel skeletons are those exploiting parallelism in the computation of ***different sub-tasks*** derived from the ***same input task***.

- the data parallel skeletons are used to speedup a single computation by splitting the computation into parallel sub-tasks.

- Data parallel computation do not concern, *per se*, stream parallelism. But may be the case that a data parallel computation is used to implement the computation relative to a **single stream item** (item could be a complex data structure) in a stream parallel program.

- Typical data parallel skeletons considered in the skeleton based programming frameworks include ***map, reduce*** and ***parallel prefix*** skeletons.

Map

Given a function f and a vector x the map skeleton computes a vector y whose elements are such that for any i: yi = f(xi). The map skeleton is defined as follows:

```
let map1 f x =
        let len = Array.length x in
        let res = Array.create len (f x.(0)) in
                for i=0 to len-1 do
                        res.(i) := (f x.(i))
                done;
        res;;
```

Similar reasoning to task farm.
Difference = the kind of input data passed to the parallel agents:
in the farm case=>the input stream;
in
the map case=>the single input data

and its type is therefore $map :: (\alpha \rightarrow \beta) \rightarrow \alpha\ array \rightarrow \beta\ array$

The parallel semantics of the map states that
each element of the resulting vector is computed in parallel.

Reduce

- The reduce skeleton computes the “sum" of all the elements of a vector with a function -> associative oper

```
let rec reduce f x =
let len = Array.length x in
        let res = ref x.(0) in
                for i=1 to len-1 do
                        res := (f !res x.(i))
                done;
                !res;;
```

- Type: $(\alpha \rightarrow \alpha \rightarrow \alpha) \rightarrow \alpha\ array \rightarrow \alpha$

- Parallel semantics:

the reduction is performed in parallel, using n agents organized in a tree. Each agent computes f over the results communicated by the son agents. Root agent delivers the reduce result. Leaf agents possibly compute locally a reduce over the assigned partition of the vector.

Parallel prefix

- The parallel prefix skeleton computes all the partial sums of a vector, using a function – associative operator
- given a vector x it computes the vector whose elements are:

x1; x1 + x2; x1+x2 +x3; … ; x1+ x2 +x3 … +xlen

```
let parallel_prefix f x =
        let len = Array.length x in
        let res = Array.create len x.(0) in
                res.(0) <- x.(0);
                        for i=1 to len-1 do
                                res.(i) := (f x.(i) res.(i-1))
                        done;
        res;;
```

Type:

$$(\alpha \rightarrow \alpha \rightarrow \alpha) \rightarrow \alpha\ array \rightarrow \alpha\ array$$

Parallel prefix

- The parallel semantics states that:

the parallel prefix is computed in parallel by n agents with a schema similar to the

one followed to compute the reduce. Partial results are logically written to the proper

locations in the resulting vector by intermediate nodes.

Remark: there are many possible implementation of parallel prefix – specialized parallel algorithms

Others skeletons

- Stencil (defined using a list of indices)

```
let stencil f stencil_indexes a =
        let n = (Array.length a) in
        let item a i = a.((i+n) mod n) in
        let rec sten a i =
                function
                        [] -> []
                        | j::rj -> (item a (i+j))::(sten a i rj) in
                        let res = Array.create n (f a.(0) (sten a 0 stencil_indexes))
in
                        for i=0 to n-1 do
                                res.(i) <- (f a.(i) (sten a i stencil_indexes))
                        done;
        res;;
```

Divide &Conquer

```
let rec divconq cs dc bc cc x =
        if(cs x) then (bc x)
        else (cc (List.map (divconq cs dc bc cc) (dc x)));;
```

Type:

$$(\alpha \rightarrow bool) \rightarrow (\alpha \rightarrow \alpha\ list) \rightarrow (\alpha \rightarrow \beta) \rightarrow (\beta\ list \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$$

- The parallel semantics associated to the divide&conquer skeleton states that:

computation of Sub problems deriving from the \divide" phase are computed in parallel.

Control parallel skeletons

Control parallel skeletons actually do not express parallel patterns. Rather, they express structured coordination patterns and model those parts of a parallel computation that are needed to support or coordinate parallel skeletons.

- Sequential skeleton
- Conditional skeleton
- Iterative skeleton

- Conditional skeleton which is used to model if-then-else computations. Such a skeleton can be expressed by the following higher order function:

```
let ifthenelse c t e x =
        match(c x) with
                true -> (t x)
                | false -> (e x);;
```

- type is $ifthenelse : (\alpha \rightarrow bool) \rightarrow (\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$

Iterative skeleton

- Iterative skeleton is used to model definite or indefinite iterations.

- As an example, a *skwhile* iterative skeleton may be represented through the higher order function

```
let rec skwhile c b x =
                    match(c x) with
                              true -> (skwhile c b (b x))
                              | false -> x;;
```

- type is

$$skwhile : (\alpha \rightarrow bool) \rightarrow (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$$

Skeleton framework

- Grammar for the skeletons:

```
Skel ::= StreamParallelSkel |
         DataParallelSkel | ControlParallelSkel
StreamParallelSkel ::= Farm | Pipe | ...
DataParallelSkel ::= Map | Reduce |
                     ParallelPrefix | ...
ControlParallelSkel ::= Seq | Conditional |
                        DefiniteIter |
                        IndefiniteIter | ...
Seq ::= seq(<sequential function code wrapping>)
Farm ::= farm(Skel)
Pipe ::= pipeline(Skel,Skel)
Map ::= map(Skel)
Reduce ::= reduce(Skel)
ParallelPrefix ::= parallel_prefix(Skel)
Conditional ::= ifthenelse(Skel,Skel,Skel)
DefiniteIter ::= ...
...
```

API

- Example: Muskel

```
Skeleton filter = new Filter(...);
Skeleton recon = new Recognize(...);
Skeleton main = Pipeline(filter, recon);

Manager mgr =
        new Manager(main,
                new FileInputStreamManager("datain.dat"),
                        new ConsoleOutputStreamManager());
mgr.start();
```

SkeTo

- the skeleton framework may provide library calls directly implementing the different skeletons.

- Example, in frameworks such as SkeTo the skeletons are declared and executed via suitable library calls: to use a map skeleton followed by a reduce skeleton the user may write a code such as

```
dist_matrix<double> dmat(&mat);
dist_matrix<double> *dmat2 =
matrix_skeletons::map(Square<double>(), &dmat);
double *ss =
matrix_skeletons::reduce(Add<double>(),
Add<double>(), dmat2);
```

Skeleton Nesting

- In the initial form proposed by Cole nesting was not possible

$\Rightarrow$ Lead CISC skeletons library

Alternative:

“RISC" skeletons

- skeleton sets with a small number of items.
  - These skeletons modelled very basic parallelism exploitation patterns.
  - Each of the “code" parameters of the skeletons, however, could be recursively provided as skeletons.
  - The envisioned scenario behind this design choice allowed complex parallelism exploitation patterns to be defined ***as composition of primitive parallelism exploitation patterns***.
- In this case, the application programmer were required to know only a small set of base skeletons plus another small number of \composition rules".
- **composition** rules play a significant role in this case.

Lecture 13

Matrix Multiplication

Announcements

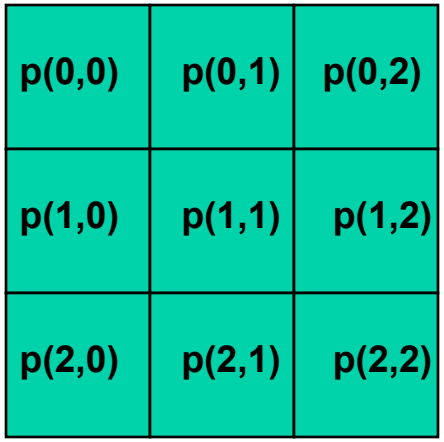
- Project Progress report, due next Weds 11/28

Today's lecture

- Cannon's Matrix Multiplication Algorithm
- 2.5D "Communication avoiding"
- SUMMA

Parallel matrix multiplication

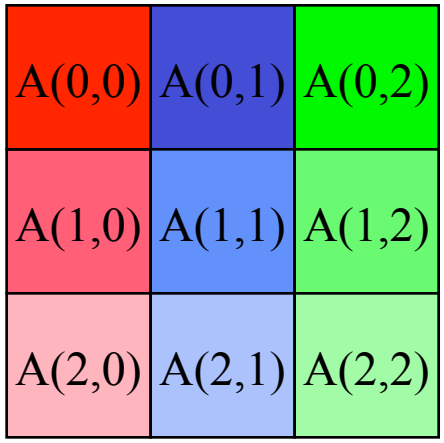
- Assume p is a perfect square
- Each processor gets an $n/\sqrt{p} \times n/\sqrt{p}$ chunk of data
- Organize processors into rows and columns
- Assume that we have an efficient serial matrix multiply (dgemm, sgemm)



Canon's algorithm

- Move data incrementally in √p phases
- Circulate each chunk of data among processors within a row or column
- In effect we are using a ring broadcast algorithm
- Consider iteration i=1, j=2:

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]



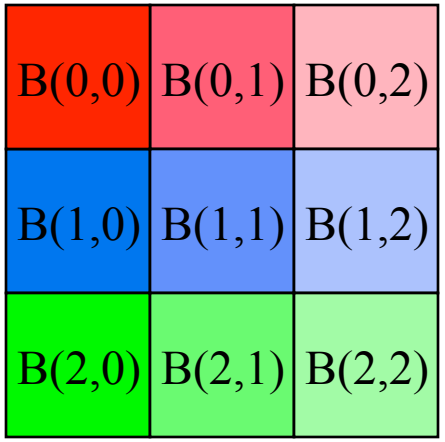
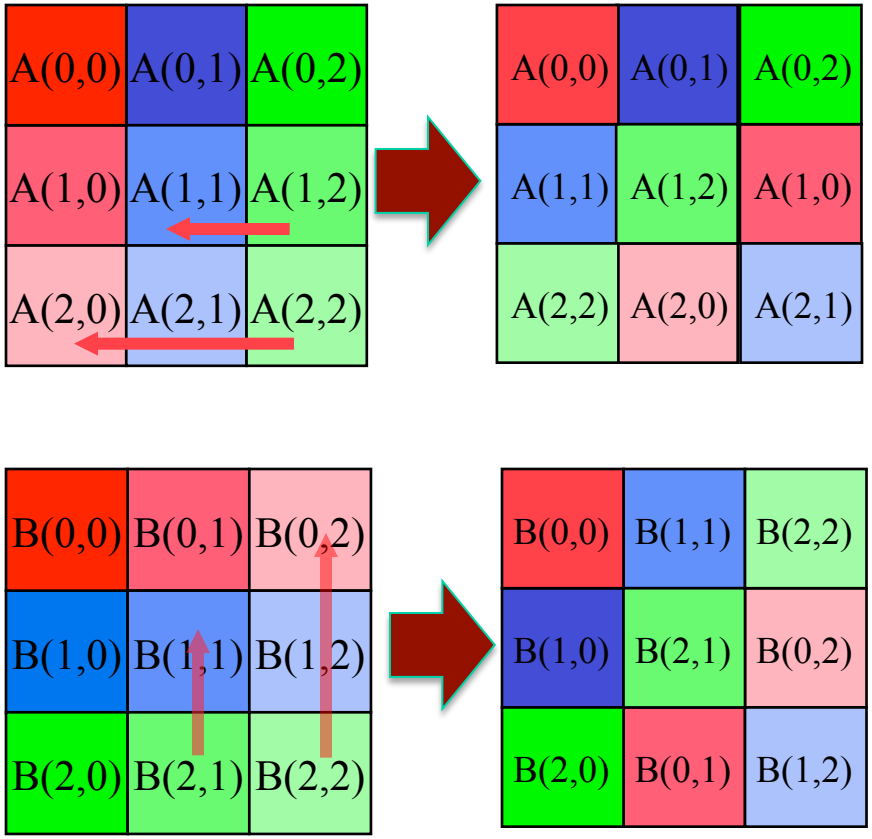


Image: Jim Demmel

Canon's algorithm

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

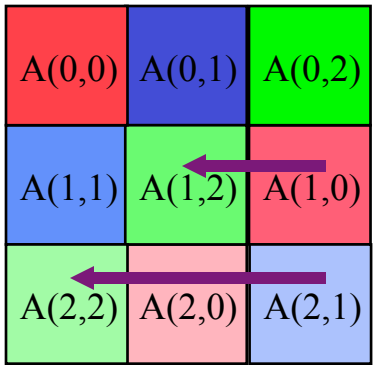
- We want A[1,0] and B[0,2] to reside on the same processor initially
- Shift rows and columns so the next pair of values A[1,1] and B[1,2] line up
- And so on with A[1,2] and B[2,2]

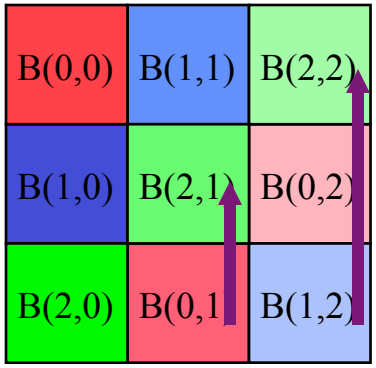


Skewing the matrices

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

- We first *skew* the matrices so that everything lines up
- Shift each row *i* by *i* columns to the left using sends and receives
- Communication wraps around
- Do the same for each column

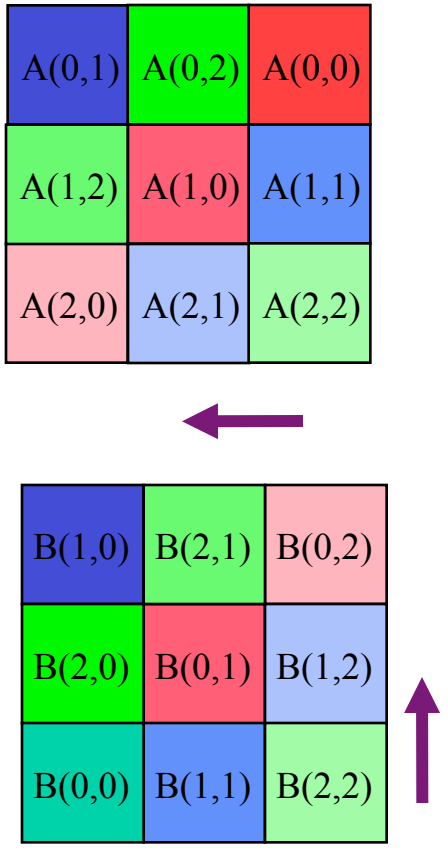




Shift and multiply

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

- Takes √p steps
- Circularly shift
  - each row by 1 column to the left
  - each column by 1 row to the left
- Each processor forms the product of the two local matrices adding into the accumulated sum



Cost of Cannon's Algorithm

```
forall  i=0 to √p -1
    CShift-left A[i; :] by i                    // T= α+βn²/p
forall  j=0 to √p -1
     Cshift-up B[: , j] by j                    // T= α+βn²/p
for k=0 to √p -1
     forall  i=0 to √p -1 and j=0 to √p -1
          C[i,j] += A[i,j]*B[i,j]               // T =  2*n³/p^(3/2)
          CShift-leftA[i; :] by 1               // T= α+βn²/p
          Cshift-up B[: , j] by 1               // T= α+βn²/p
      end forall
end for
```

$$T_P = 2n^3/p + 2(\alpha(1+\sqrt{p}) + \beta n^2/(1+\sqrt{p})/p)$$

$$E_P = T_1/(pT_P) = (1 + \alpha p^{3/2}/n^3 + \beta\sqrt{p}/n))^{-1}$$

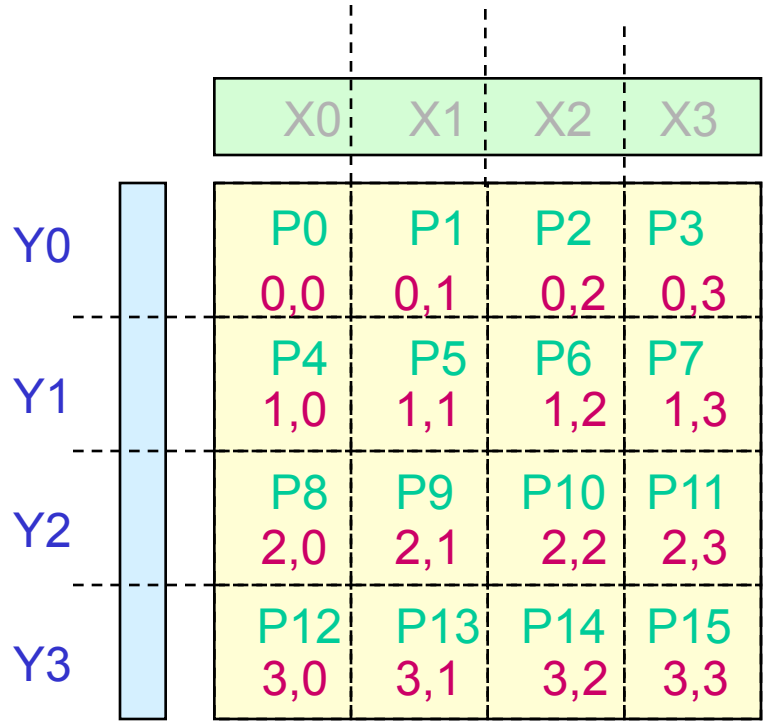
$$\approx (1 + O(\sqrt{p}/n))^{-1}$$

$E_P \rightarrow 1$ as $(n/\sqrt{p})$ grows [sqrt of data / processor]

Implementation

Communication domains

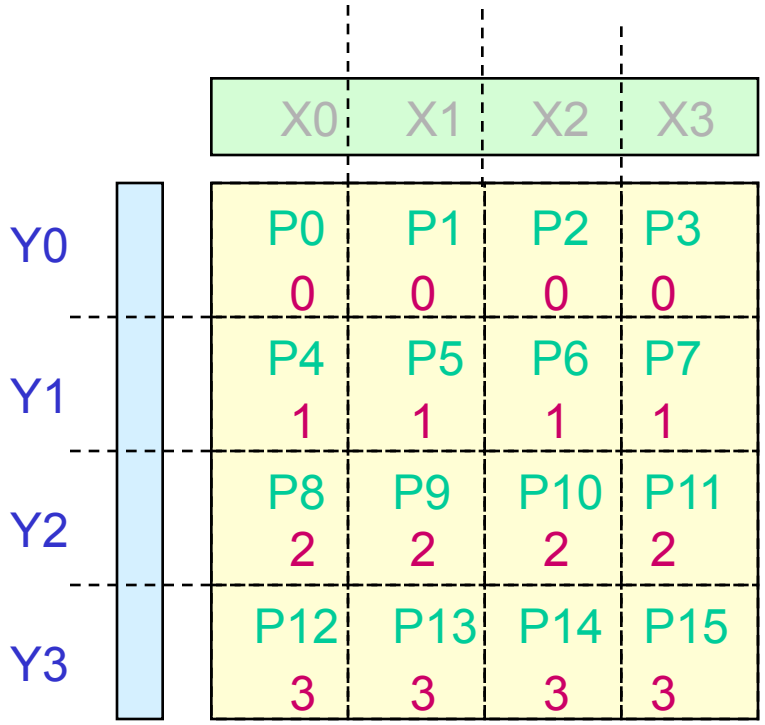
- Cannon's algorithm shifts data along rows and columns of processors
- MPI provides communicators for grouping processors, reflecting the communication structure of the algorithm
- An MPI communicator is a name space, a subset of processes that communicate
- Messages remain within their communicator
- A process may be a member of more than one communicator



Establishing row communicators

- Create a communicator for each row and column
- By Row

key = myRank div √P

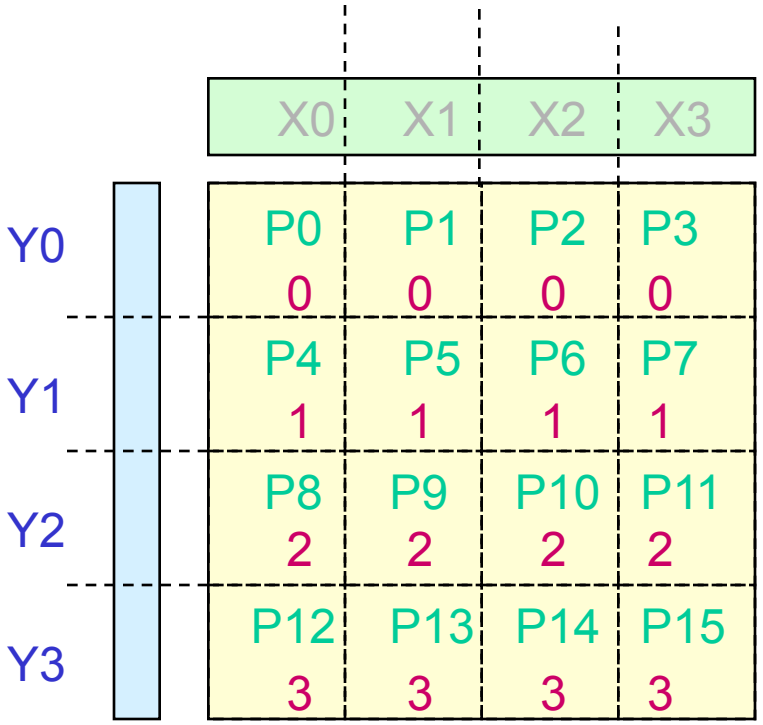


Creating the communicators

MPI_Comm rowComm;

MPI_Comm_split(MPI_COMM_WORLD,
myRank / √P, myRank, &rowComm);

MPI_Comm_rank(rowComm,&myRow);



- Each process obtains a new communicator
- Each process' rank relative to the new communicator
- Rank applies to the respective communicator only
- Ordered according to myRank

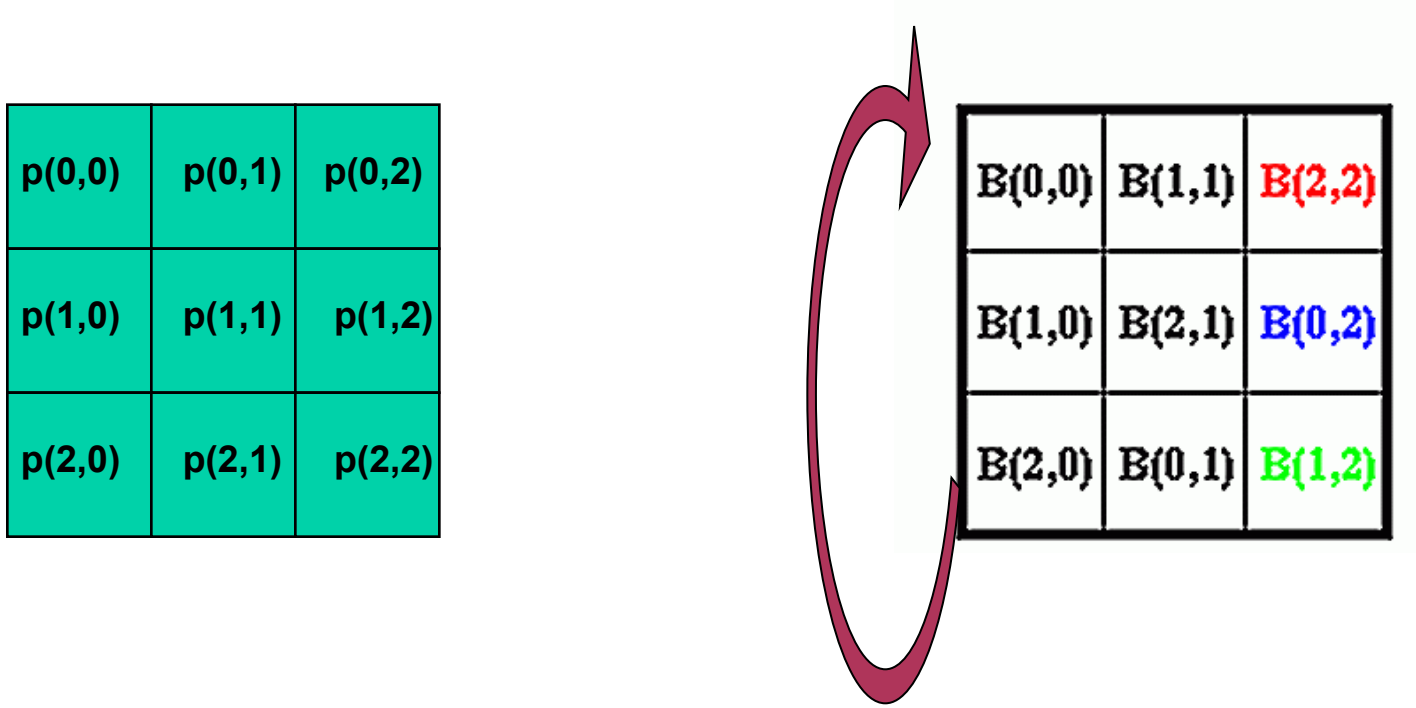
More on Comm_split

MPI_Comm_split(MPI_Comm comm, int splitKey,
int rankKey, MPI_Comm* newComm)

- Ranks assigned arbitrarily among processes sharing the same **rankKey** value
- May exclude a process by passing the constant **MPI_UNDEFINED** as the **splitKey**
- Return a special **MPI_COMM_NULL** communicator
- If a process is a member of several communicators, it will have a rank within each one

Circular shift

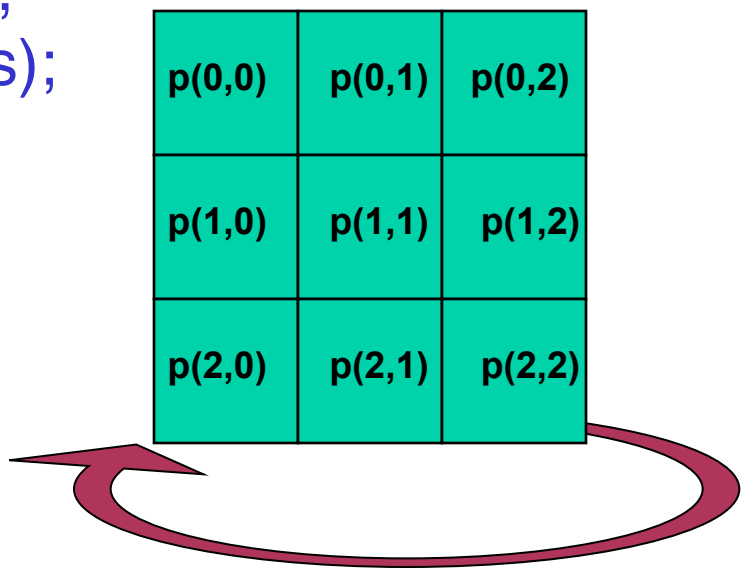
- Communication with columns (and rows



Circular shift

```
MPI_Comm_rank(rowComm,&myidRing);
MPI_Comm_size(rowComm,&nodesRing);
int next = (myidRng + 1 ) % nodesRing;
MPI_Send(&X,1,MPI_INT,next,0, rowComm);
MPI_Recv(&XR,1,MPI_INT,
                MPI_ANY_SOURCE,
                0, rowComm, &status);
```

Processes 0, 1, 2 are in one communicator because they share the same value of key (0)
Processes 3, 4, 5are in another (1), and so on



Today's lecture

- Cannon's Matrix Multiplication Algorithm
- 2.5D "Communication avoiding"
- SUMMA

Motivation

- Relative to arithmetic speeds, communication is becoming more costly with time
- Communication can be data motion on or off-chip, across address spaces
- We seek algorithms that increase the amount of work (flops) relative to the amount of data they move

Communication lower bounds on Matrix Multiplication

- Assume we are using an $O(n^3)$ algorithm
- Let M = Size fast memory (cache/local memory)
- Sequential case: # slow memory references

  $\Omega (n^3 / \sqrt{M})$ [Hong and Kung '81]
- Parallel, p = # processors,

  μ = Amount of memory needed to store matrices
  - Refs to remote memory

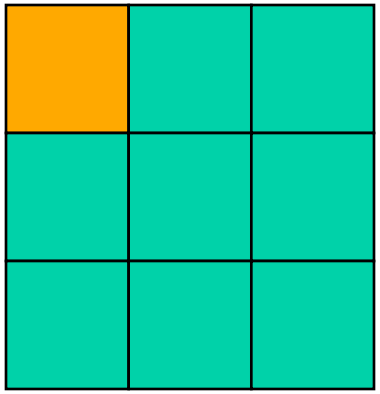
    $\Omega (n^3 / (p\sqrt{\mu}))$ [Irony, Tiskin, Toledo, '04]
  - If $\mu = 3n^2/p$ (one copy of A, B, C) ⇒ lower bound = $\Omega (n^2 / \sqrt{p})$ words
  - Achieved by Cannon's algorithm ("2D algorithm")
  - $T_P = 2n^3/p + 4\sqrt{p}(\alpha + \beta n^2/p)$

Canon's Algorithm - optimality

- General result
  - If each processor has M words of local memory …
  - … at least 1 processor must transmit Ω (# flops / $M^{1/2}$) words of data
- If local memory $M = O(n^2/p)$ …
  - at least 1 processor performs $f \geq n^3/p$ flops
  - … lower bound on number of words transmitted by at least 1 processor

$$\Omega\left((n^3/p) / \sqrt{(n^2/p)}\right) = \Omega\left((n^3/p) / \sqrt{M}\right)$$

$$= \Omega\left(n^2 / \sqrt{p}\right)$$



New communication lower bounds – direct linear algebra [Ballard &Demmel '11]

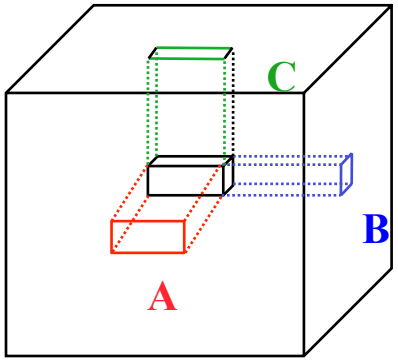
- Let M = amount of fast memory per processor
- Lower bounds
  - # words moved by at least 1 processor

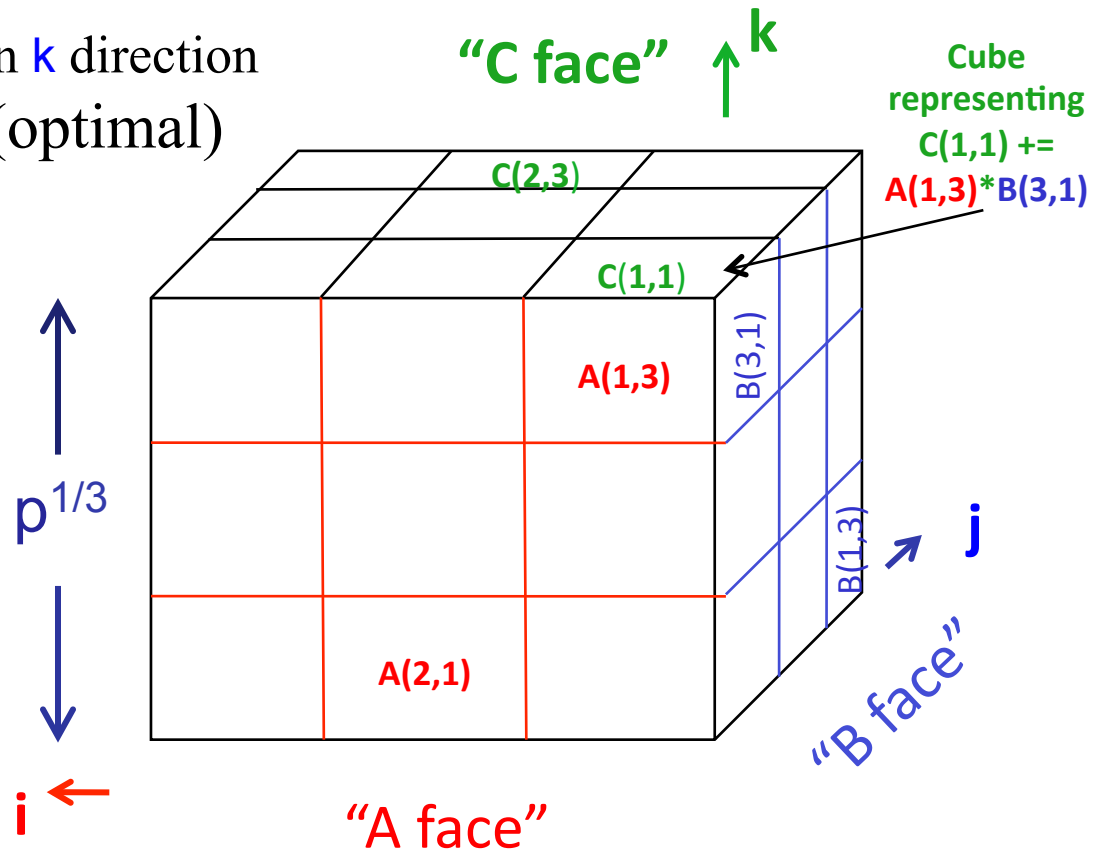
    Ω (# flops / $M^{1/2}$))
  - # messages sent by at least 1 processor

    Ω (# flops / $M^{3/2}$)
- Holds not only for Matrix Multiply but many other "direct" algorithms in linear algebra, sparse matrices, some graph theoretic algorithms
- Identify 3 values of M
  - 2D (Cannon's algorithm)
  - 3D (Johnson's algorithm)
  - 2.5D (Ballard and Demmel)

Johnson's 3D Algorithm

- 3D processor grid: $p^{1/3} \times p^{1/3} \times p^{1/3}$
  - Bcast A (B) in j (i) direction ($p^{1/3}$ redundant copies)
  - Local multiplications
  - Accumulate (Reduce) in k direction
- Communication costs (optimal)
  - Volume = $O(n^2/p^{2/3})$
  - Messages = $O(\log(p))$
- Assumes space for $p^{1/3}$ redundant copies
- Trade memory for communication



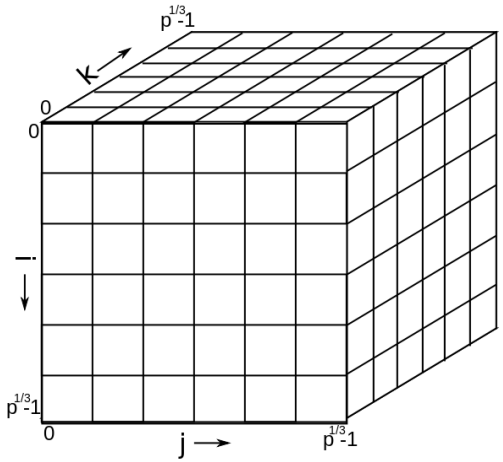


Source: Edgar Solomonik

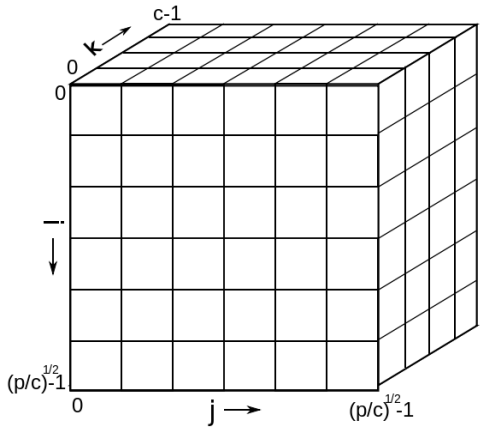
2.5D Algorithm

- What if we have space for only $1 \le c \le p^{1/3}$ copies ?
- $M = \Omega(c \cdot n^2/p)$
- Communication costs : lower bounds
  - Volume = $\Omega(n^2/(cp)^{1/2})$; Set $M = c \cdot n^2/p$ in Ω (# flops / $M^{1/2}$))
  - Messages = $\Omega(p^{1/2} / c^{3/2})$; Set $M = c \cdot n^2/p$ in Ω (# flops / $M^{3/2}$))
- 2.5D algorithm "interpolates" between 2D & 3D algorithms

3D



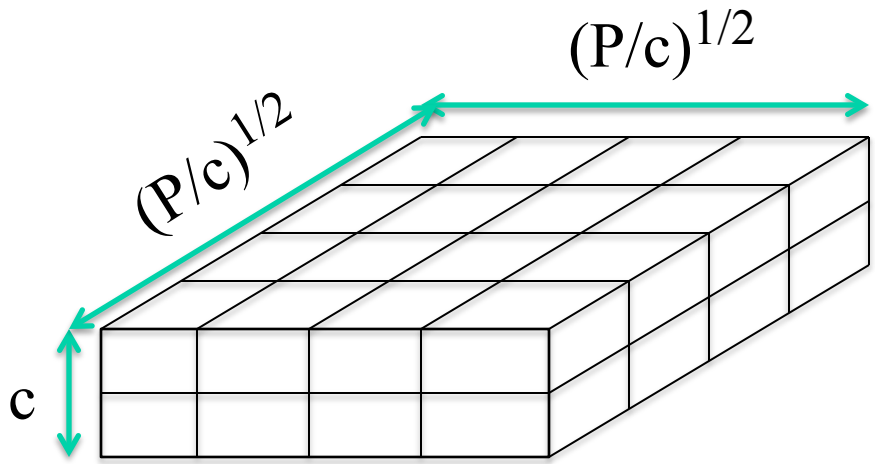
2.5D



Source: Edgar Solomonik

2.5D Algorithm

- Assume can fit cn^2/P data per processor, $c>1$
- Processors form $(P/c)^{1/2}$ x $(P/c)^{1/2}$ x c grid

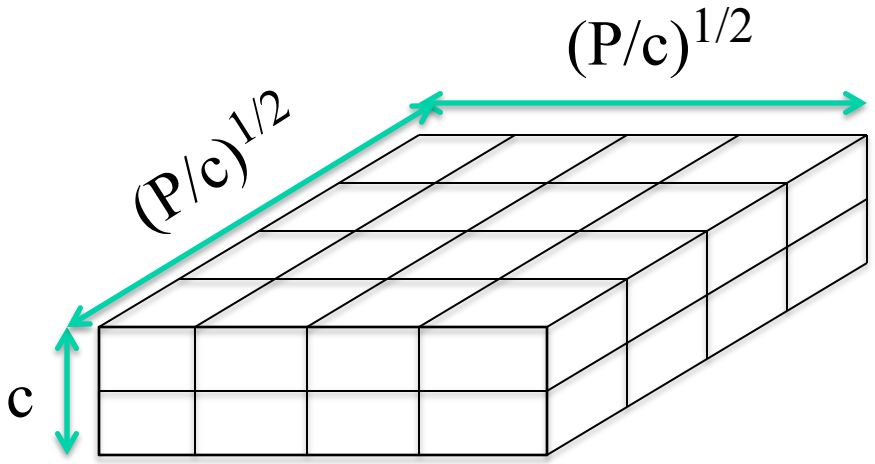


Example: P = 32, c = 2

Source Jim Demmel

2.5D Algorithm

- Assume can fit cn^2/P data per processor, $c>1$
- Processors form $(P/c)^{1/2}$ x $(P/c)^{1/2}$ x c grid

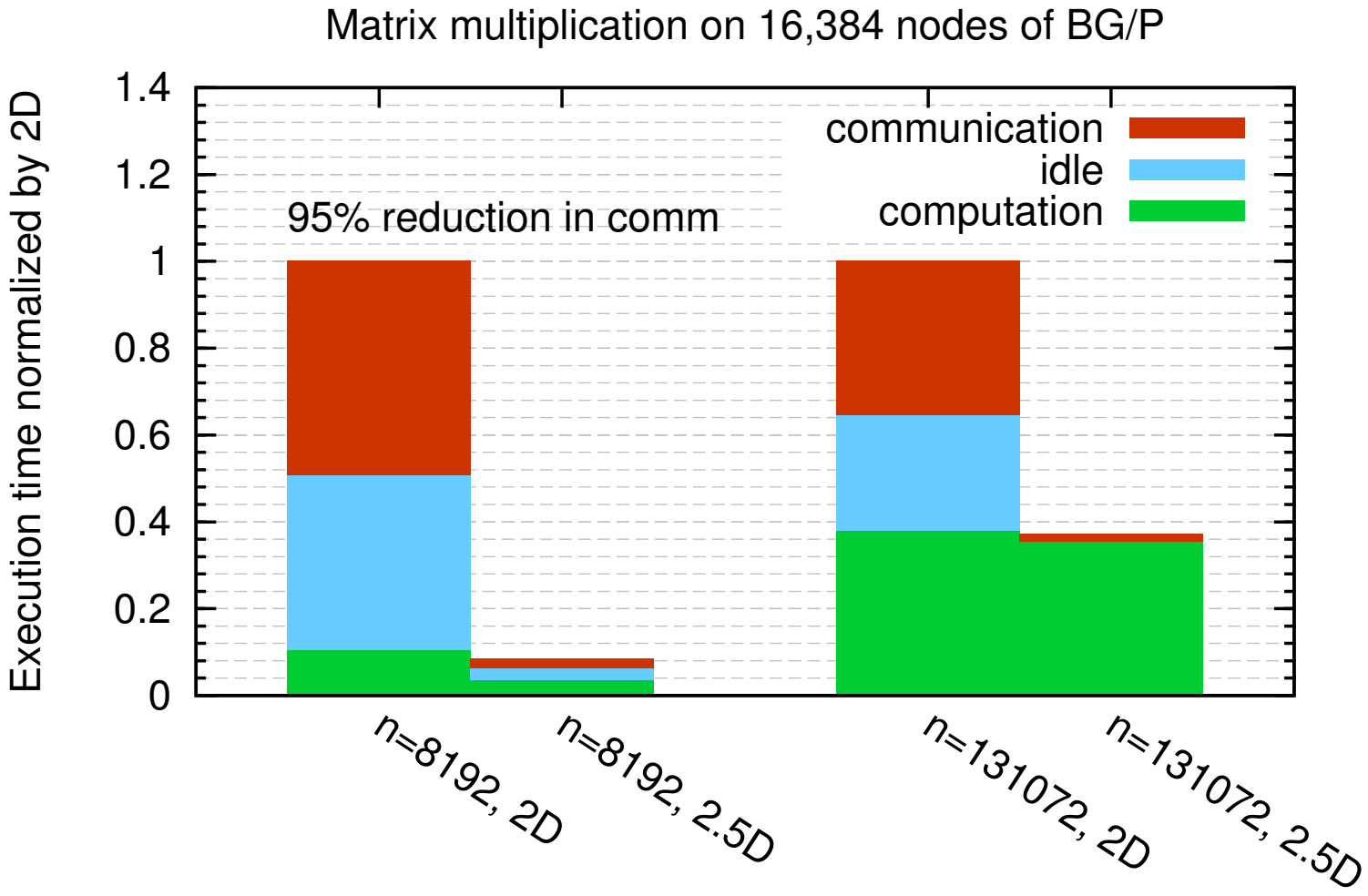


Initially P(i,j,0) owns A(i,j) &B(i,j)
each of size $n(c/P)^{1/2}$ x $n(c/P)^{1/2}$

(1) P(i,j,0) broadcasts A(i,j) and B(i,j) to P(i,j,k)
(2) Processors at level k perform 1/c-th of SUMMA,
i.e. 1/c-th of Σ_m A(i,m)*B(m,j)
(3) Sum-reduce partial sums Σ_m A(i,m)*B(m,j) along k-axis so that P(i,j,0) owns C(i,j)

Performance on Blue Gene P

C=16



Source Jim Demmel

2.5D Algorithm

- Interpolate between 2D (Cannon) and 3D
  - c copies of A & B
  - Perform $p^{1/2}/c^{3/2}$ Cannon steps on each copy of A&B
  - Sum contributions to C over all c layers
- Communication costs (not quite optimal, but not far off)
  - Volume:

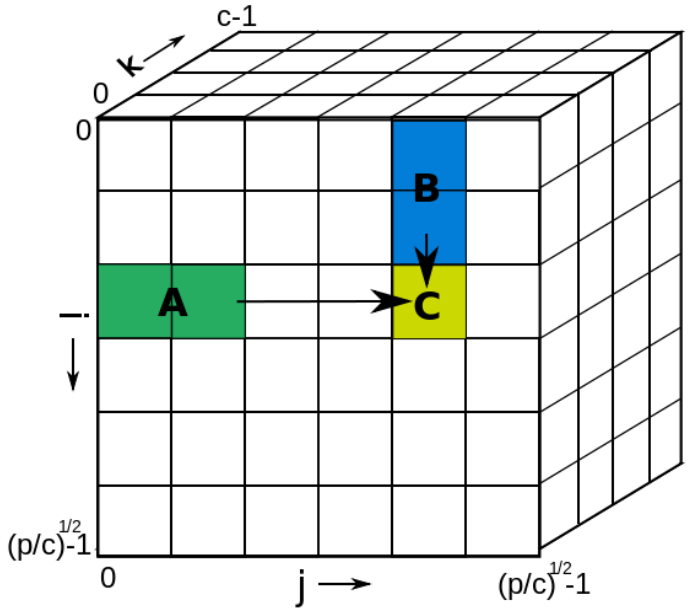
    $O(n^2/(cp)^{1/2})$

    $[\ \Omega(n^2/(cp)^{1/2})\ ]$

  - Messages:

    $O(p^{1/2}/c^{3/2} + \log(c))$

    $[\ \Omega(p^{1/2}/c^{3/2})\ ]$



Source: Edgar Solomonik

Today's lecture

- Cannon's Matrix Multiplication Algorithm
- 2.5D "Communication avoiding"
- SUMMA

Outer product formulation of matrix multiply

- Limitations of Cannon's Algorithm
  - P is must be a perfect square
  - A and B must be square, and evenly divisible by √p
- Interoperation with applications and other libraries difficult or expensive
- The SUMMA algorithm offers a practical alternative
  - Uses a shift algorithm to broadcast
  - A variant used in SCALAPACK by Van de Geign and Watts [1997]

Formulation

- The matrices may be non-square (kij formulation)

```
for k := 0 to n3-1
    for i := 0 to n1-1
        for j := 0 to n2-1
            C[i,j] += A[i,k] * B[k,j]          C[i,:] += A[i,k] * B[k,:]
```

- The two innermost loop nests compute

n3 *outer products*

```
for k := 0 to n3-1
        C[:,:] +=  A[:,k] • B[k,:]
```

where • is outer product

Outer product

- Recall that when we multiply
  an m×n matrix by an n × p matrix…
  we get an m × p matrix
- Outer product of *column vector* a^T and *vector* b = matrix C
  an m × 1 times a 1 × n

a[1,3] • x[3,1]

$$(a,b,c) * (x,y,z)^T \equiv \begin{pmatrix} ax & ay & az \\ bx & by & bz \\ cx & cy & cz \end{pmatrix}$$

*	1	2	3
10	11	20	30
20	20	40	60
30	30	60	90

Multiplication table with rows formed by a[:] and the columns by b[:]

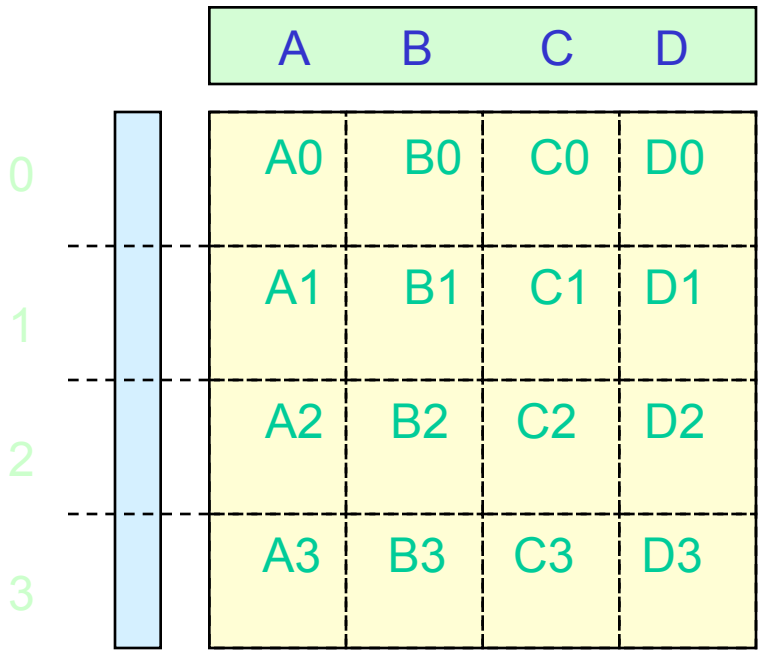
- The SUMMA algorithm computes n partial outer products:

```
for k := 0 to n-1
    C[:,:] +=  A[:,k] • B[k,:]
```

Outer Product Formulation

- The new algorithm computes n partial outer products:

```
for k := 0 to n-1
    C[:,:] +=  A[:,k] • B[k,:]
```



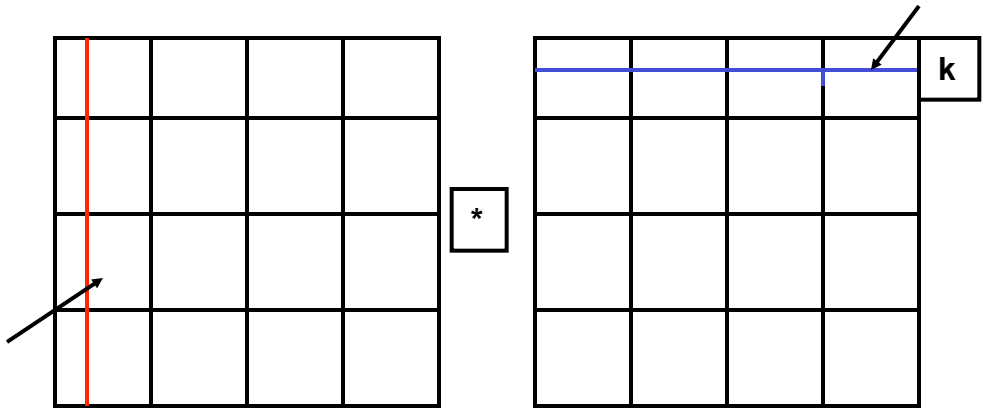
"Inner product" formulation:

```
for i:= 0 to n-1,  j:= 0 to n-1
        C[i,j] += A[i,:] * B[:,j]
```

Serial algorithm

- Each row k of B contributes to the n partial n partial outer products

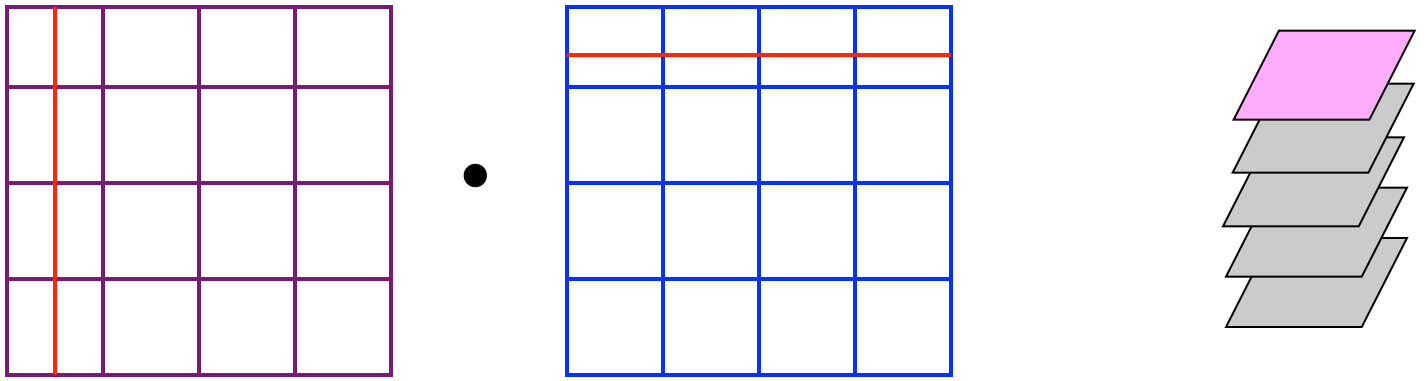
```
for k := 0 to n-1
    C[:,:] +=  A[:,k] • B[k,:]
```



Animation of SUMMA

- Compute the sum of **n** outer products
- Each row & column **(k)** of A & B generates a single outer product
  - Column vector **A[:,k]** (n × 1) & a vector **B[k,:]** (1 × n)

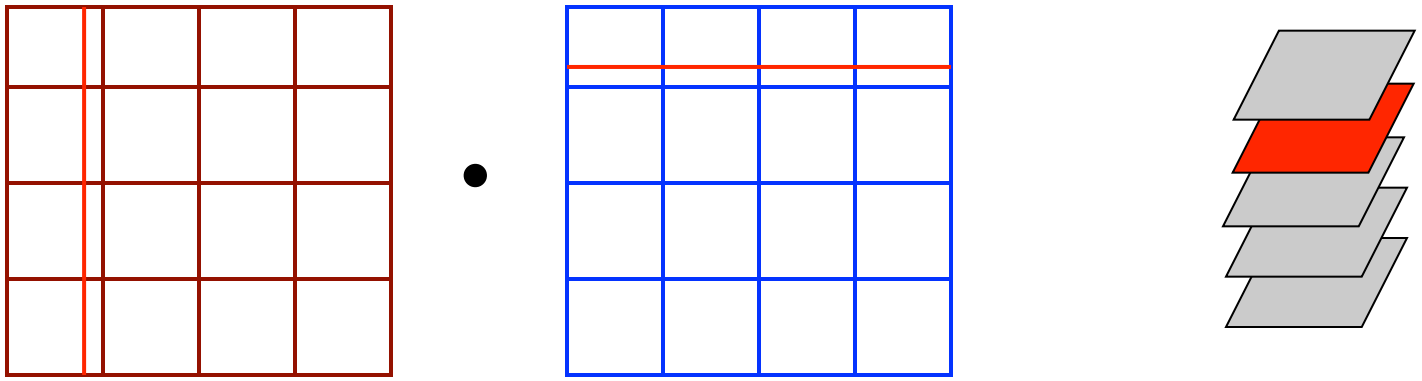
```
for k := 0 to n-1
        C[:,:] +=  A[:,k] • B[k,:]
```



Animation of SUMMA

- Compute the sum of **n** outer products
- Each row & column **(k)** of A & B generates a single outer product
  - **A[:,k+1] • B[k+1,:]**

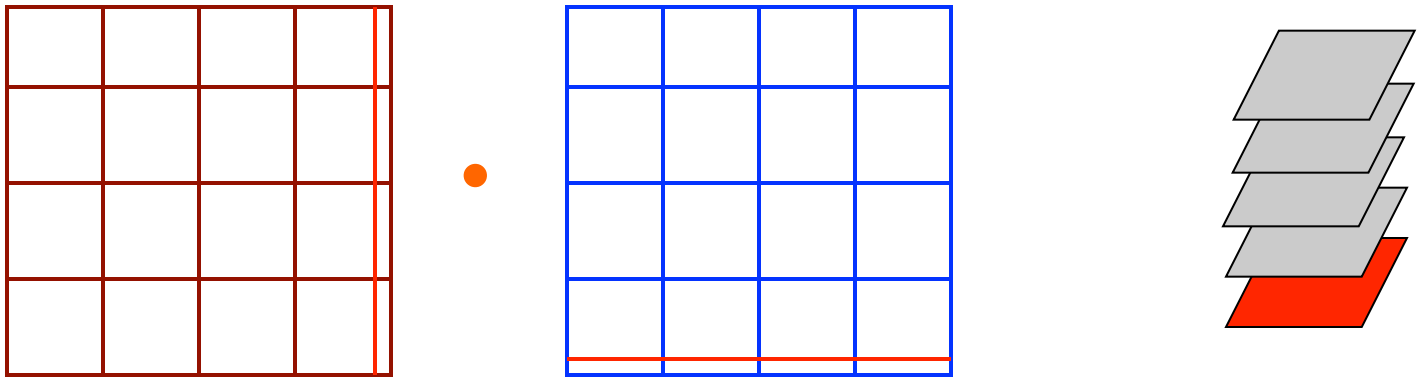
```
for k := 0 to n-1
        C[:,:] +=  A[:,k] • B[k,:]
```



Animation of SUMMA

- Compute the sum of **n** outer products
- Each row & column **(k)** of A & B generates a single outer product
  - **A[:,n-1] • B[n-1,:]**

```
for k := 0 to n-1
        C[:,:] +=  A[:,k] • B[k,:]
```

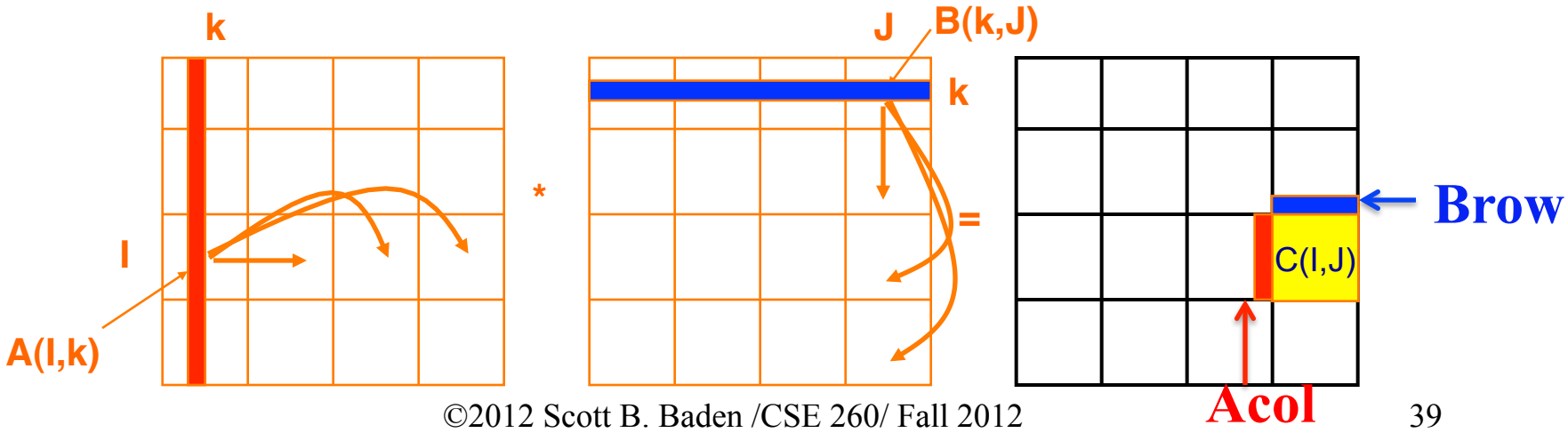


Parallel algorithm

- Processors organized into rows and columns, process rank an ordered pair
- Processor geometry P = **px × py**
- Blocked (serial) matrix multiply, panel size = **b << N/max(px,py)**

```
for k := 0 to n-1 by b
    Owner of A[:,k:k+b-1]  Bcasts to ACol    // Along processor rows
    Owner of B[k:k+b-1,:]  Bcasts BRow       // Along processor columns
    C += Serial Matrix Multiply(ACol,BRow )
```

- Each row and column of processors independently participate in a panel broadcast
- Owner of the panel (Broadcast root) changes with **k,** shifts across matrix



Fin